

\$ how to git

The git Best-Practices Book

Edition 2026.1

2026

Thomas Björk

Contents

What git really is	1
The problem version control solves	1
The object model	1
The three areas	1
A first walkthrough	2
What a branch really is	2
Local vs remote	3
See also	3
Installing and configuring git	4
Installing git	4
Configuration scopes	4
Setting your identity	4
Essential quality-of-life settings	5
Aliases	5
Global .gitignore	6
Best practices	6
See also	7
init	8
Mental model	8
Synopsis	8
Everyday usage	8
Key options	9
Best practices	9
Pitfalls & gotchas	10
Worked examples	10
Recovery	11
See also	12
clone	13
Mental model	13
Synopsis	13
Everyday usage	13
Key options	14
Best practices	15
Pitfalls & gotchas	16
Worked examples	16
Recovery	17
See also	18
add	19
Mental model	19
Synopsis	19
Everyday usage	19
Key options	20
Best practices	21
Pitfalls & gotchas	21
Worked examples	22
Recovery	24
See also	24

status	25
Mental model	25
Synopsis	25
Everyday usage	25
Key options	26
Best practices	27
Pitfalls & gotchas	28
Worked examples	28
Recovery	29
See also	29
diff	30
Mental model	30
Synopsis	30
Everyday usage	30
Key options	31
Best practices	32
Pitfalls & gotchas	33
Worked examples	33
Recovery	35
See also	35
commit	36
Mental model	36
Synopsis	36
Everyday usage	36
Key options	36
Best practices	37
Pitfalls & gotchas	38
Worked examples	38
Recovery	39
See also	39
restore	40
Mental model	40
Synopsis	40
Everyday usage	40
Key options	41
Best practices	42
Pitfalls & gotchas	42
Worked examples	43
Recovery	44
See also	45
reset	46
Mental model	46
Synopsis	46
Everyday usage	46
Key options	47
Best practices	48
Pitfalls & gotchas	48
Worked examples	49
Recovery	50

See also	51
rm	52
Mental model	52
Synopsis	52
Everyday usage	52
Key options	53
Best practices	53
Pitfalls & gotchas	54
Worked examples	54
Recovery	55
See also	56
mv	57
Mental model	57
Synopsis	57
Everyday usage	57
Key options	58
Best practices	58
Pitfalls & gotchas	59
Worked examples	59
Recovery	60
See also	61
clean	62
Mental model	62
Synopsis	62
Everyday usage	62
Key options	63
Best practices	63
Pitfalls & gotchas	64
Worked examples	64
Recovery	65
See also	65
stash	66
Mental model	66
Synopsis	66
Everyday usage	66
Key options	67
Best practices	68
Pitfalls & gotchas	68
Worked examples	69
Recovery	70
See also	71
branch	72
Mental model	72
Synopsis	72
Everyday usage	72
Key options	74
Best practices	75
Pitfalls & gotchas	76
Worked examples	77

Recovery	78
See also	78
checkout	80
Mental model	80
Synopsis	80
Everyday usage	80
Key options	81
Best practices	82
Pitfalls & gotchas	83
Worked examples	83
Recovery	84
See also	85
switch	86
Mental model	86
Synopsis	86
Everyday usage	86
Key options	87
Best practices	88
Pitfalls & gotchas	88
Worked examples	89
Recovery	90
See also	90
merge	92
Mental model	92
Synopsis	92
Everyday usage	92
Key options	93
Best practices	94
Pitfalls & gotchas	95
Worked examples	95
Recovery	97
See also	97
mergetool	98
Mental model	98
Synopsis	98
Everyday usage	98
Key options	99
Best practices	99
Pitfalls & gotchas	100
Worked examples	101
Recovery	102
See also	103
rebase	104
Mental model	104
Synopsis	104
Everyday usage	104
Key options	105
Best practices	106
Pitfalls & gotchas	107

Worked examples	107
Recovery	109
See also	110
cherry-pick	111
Mental model	111
Synopsis	111
Everyday usage	111
Key options	112
Best practices	113
Pitfalls & gotchas	113
Worked examples	114
Recovery	116
See also	116
revert	117
Mental model	117
Synopsis	117
Everyday usage	117
Key options	118
Best practices	119
Pitfalls & gotchas	119
Worked examples	120
Recovery	121
See also	122
tag	123
Mental model	123
Synopsis	123
Everyday usage	123
Key options	124
Best practices	125
Pitfalls & gotchas	126
Worked examples	127
Recovery	128
See also	129
bisect	130
Mental model	130
Synopsis	130
Everyday usage	130
Key options	131
Best practices	132
Pitfalls & gotchas	133
Worked examples	133
Recovery	134
See also	135
rerere	136
Mental model	136
Synopsis	136
Everyday usage	136
Key options	138
Best practices	139

Pitfalls & gotchas	139
Worked examples	140
Recovery	141
See also	142
log	143
Mental model	143
Synopsis	143
Everyday usage	143
Key options	144
Best practices	146
Pitfalls & gotchas	147
Worked examples	148
Recovery	149
See also	149
show	150
Mental model	150
Synopsis	150
Everyday usage	150
Key options	151
Best practices	152
Pitfalls & gotchas	152
Worked examples	153
Recovery	154
See also	154
blame	155
Mental model	155
Synopsis	155
Everyday usage	155
Key options	156
Best practices	157
Pitfalls & gotchas	158
Worked examples	159
Recovery	160
See also	160
shortlog	161
Mental model	161
Synopsis	161
Everyday usage	161
Key options	162
Best practices	163
Pitfalls & gotchas	163
Worked examples	164
Recovery	165
See also	165
describe	166
Mental model	166
Synopsis	166
Everyday usage	166
Key options	167

Best practices	168
Pitfalls & gotchas	168
Worked examples	169
Recovery	170
See also	170
reflog	171
Mental model	171
Synopsis	171
Everyday usage	171
Key options	172
Best practices	173
Pitfalls & gotchas	174
Worked examples	174
Recovery	176
See also	176
grep	177
Mental model	177
Synopsis	177
Everyday usage	177
Key options	178
Best practices	179
Pitfalls & gotchas	180
Worked examples	181
Recovery	182
See also	182
range-diff	183
Mental model	183
Synopsis	183
Everyday usage	184
Key options	184
Best practices	185
Pitfalls & gotchas	185
Worked examples	186
Recovery	187
See also	187
remote	188
Mental model	188
Synopsis	188
Everyday usage	188
Key options	190
Best practices	190
Pitfalls & gotchas	191
Worked examples	192
Recovery	193
See also	194
fetch	195
Mental model	195
Synopsis	195
Everyday usage	195

Key options	196
Best practices	197
Pitfalls & gotchas	197
Worked examples	198
Recovery	199
See also	200
pull	201
Mental model	201
Synopsis	201
Everyday usage	201
Key options	202
Best practices	203
Pitfalls & gotchas	203
Worked examples	204
Recovery	205
See also	206
push	207
Mental model	207
Synopsis	207
Everyday usage	207
Key options	208
Best practices	209
Pitfalls & gotchas	210
Worked examples	210
Recovery	212
See also	212
submodule	213
Mental model	213
Synopsis	213
Everyday usage	213
Key options	214
Best practices	216
Pitfalls & gotchas	217
Worked examples	218
Recovery	220
See also	220
worktree	221
Mental model	221
Synopsis	221
Everyday usage	221
Key options	222
Best practices	223
Pitfalls & gotchas	223
Worked examples	224
Recovery	225
See also	225
apply	227
Mental model	227
Synopsis	227

Everyday usage	227
Key options	228
Best practices	229
Pitfalls & gotchas	230
Worked examples	230
Recovery	232
See also	232
am	233
Mental model	233
Synopsis	233
Everyday usage	233
Key options	234
Best practices	235
Pitfalls & gotchas	236
Worked examples	237
Recovery	238
See also	239
format-patch	240
Mental model	240
Synopsis	240
Everyday usage	240
Key options	241
Best practices	243
Pitfalls & gotchas	243
Worked examples	244
Recovery	245
See also	246
send-email	247
Mental model	247
Synopsis	247
Everyday usage	247
Key options	248
Best practices	250
Pitfalls & gotchas	251
Worked examples	251
Recovery	253
See also	253
request-pull	254
Mental model	254
Synopsis	254
Everyday usage	254
Key options	255
Best practices	255
Pitfalls & gotchas	255
Worked examples	256
Recovery	257
See also	257
gc	258
Mental model	258

Synopsis	258
Everyday usage	258
Key options	259
Best practices	259
Pitfalls & gotchas	260
Worked examples	260
Recovery	262
See also	262
fsck	263
Mental model	263
Synopsis	263
Everyday usage	263
Key options	264
Best practices	265
Pitfalls & gotchas	265
Worked examples	266
Recovery	267
See also	268
maintenance	269
Mental model	269
Synopsis	269
Everyday usage	269
Key options	270
Best practices	270
Pitfalls & gotchas	271
Worked examples	272
Recovery	273
See also	274
archive	275
Mental model	275
Synopsis	275
Everyday usage	275
Key options	276
Best practices	276
Pitfalls & gotchas	277
Worked examples	278
Recovery	279
See also	279
bundle	280
Mental model	280
Synopsis	280
Everyday usage	280
Key options	281
Best practices	281
Pitfalls & gotchas	282
Worked examples	282
Recovery	283
See also	284
notes	285

Mental model	285
Synopsis	285
Everyday usage	285
Key options	286
Best practices	287
Pitfalls & gotchas	288
Worked examples	289
Recovery	290
See also	290
sparse-checkout	291
Mental model	291
Synopsis	291
Everyday usage	291
Key options	292
Best practices	292
Pitfalls & gotchas	293
Worked examples	294
Recovery	295
See also	295
replace	296
Mental model	296
Synopsis	296
Everyday usage	296
Key options	297
Best practices	297
Pitfalls & gotchas	298
Worked examples	299
Recovery	300
See also	300
filter-branch	301
Mental model	301
Synopsis	301
Everyday usage	302
Key options	302
Best practices	303
Pitfalls & gotchas	304
Worked examples	306
Recovery	307
See also	308
hooks	309
Mental model	309
Synopsis	309
Everyday usage	310
Key options	311
Best practices	312
Pitfalls & gotchas	312
Worked examples	313
Recovery	314
See also	315

Getting out of jams: git	316
Detached HEAD	316
Merge conflicts and rebase conflicts	316
“Updates were rejected (non-fast-forward)” on push	317
Committed to the wrong branch	318
Undo the last commit — soft vs mixed vs hard reset vs revert	319
Recover deleted commits and branches via the reflog	320
Safely undo a commit that was already pushed	321
A secret got committed (and pushed)	321
“fatal: refusing to merge unrelated histories”	322
Line-ending / .gitattributes surprises (CRLF/LF)	322
Accidentally committed a huge file	323
“Unable to create ‘.git/index.lock’” / stale lock	324
Submodule out of sync / empty submodule directory	325
HTTPS vs SSH authentication failures	325
“detected dubious ownership in repository”	326

What git really is

Git is a distributed version control system that records the complete history of a project as a graph of snapshots. Every contributor has the entire history on their own machine. Nothing is lost unless you explicitly throw it away.

That single fact — the whole history, everywhere — changes how you work. You can experiment freely on a branch, travel back in time, and share selective pieces of work without ever needing a network connection.

The problem version control solves

Imagine editing a document and saving copies: `report.docx`, `report-v2.docx`, `report-FINAL.docx`, `report-FINAL-2.docx`. This breaks down immediately when a second person edits the same file.

Version control replaces the filename-based approach with a database that tracks every change: who made it, when, and why. You can inspect any past state, compare two versions, merge concurrent edits, and revert a mistake.

Git goes further than older centralised systems (CVS, Subversion) by giving each contributor a full copy of the database. There is no single server you depend on; collaboration happens by exchanging change sets between peers.

The object model

Everything git stores is one of four object types, identified by a SHA-1 (or SHA-256 in newer repositories) hash of its content.

Blob — the raw contents of a single file at a single point in time. No filename, no path, just bytes. Two files with identical content share one blob.

Tree — a directory listing. It maps filenames to blobs and to other trees (subdirectories). A tree snapshot for the project root describes the entire file system at a moment in time.

Commit — a pointer to one root tree plus metadata: author, committer, timestamp, and a human-readable message. Crucially, every commit also points to its parent commit(s). That chain of parents is the project history.

Tag — an *annotated tag object*: a permanent pointer to a commit (usually a release) together with its own metadata — the tagger, a date, a message, and an optional GPG signature. Like the other three, it lives in `.git/objects/`.

Those four — blob, tree, commit, tag — are the only things git stores as objects. **References** are a separate, simpler mechanism: a ref is just a small text file under `.git/refs/` (or an entry in `.git/packed-refs`) holding the hash of an object. The names you work with every day — `main`, `feature/login`, `v1.4.0` — are refs. A **branch** is a ref that advances automatically when you commit on it; a **lightweight tag** is a ref that never moves (no object behind it), while an annotated tag is a ref pointing at the tag object described above. Refs are how humans name commits; they are not objects themselves.

```
blob "fn main() {...}" ← tree "src/" ←
blob "README.md"      ← tree "/" ←—— commit C3 ← commit C4 ← HEAD
                                   (parent: C3)
```

The three areas

Understanding where your changes live at any moment is the key to not being surprised by git.

Working tree — the files you can see and edit in your project directory. Making changes here does not tell git anything.

Index (staging area) — a lightweight snapshot that accumulates the changes you have decided to include in the next commit. `git add` copies changes from the working tree into the index. Nothing is permanent until you commit.

Repository (`.git/` directory) — the object database and history. When you run `git commit`, git writes the index contents as a new tree object, wraps it in a commit object, and moves the current branch pointer forward.

Files move through these areas as follows:

```
Working tree  —git add—>  Index  —git commit—>  Repository
               <—git restore—      <—git restore --staged—
```

A first walkthrough

The following sequence shows how the three areas interact in practice.

```
git init my-project      # create a repository from scratch
cd my-project

echo "Hello, git" > README.md  # file exists only in the working tree

git add README.md         # move it into the index

git commit -m "Initial commit" # snapshot the index into the repository
```

After the commit, `git log` shows one entry:

```
commit 9fceb02... (HEAD -> main)
Author: Ada Lovelace <ada@example.com>
Date:   Tue Jun 17 09:00:00 2026 +0200

    Initial commit
```

What a branch really is

A branch is nothing more than a text file in `.git/refs/heads/` containing a 40-character commit hash. When you commit, git updates that file to point to the new commit. That is the entire mechanism.

```
cat .git/refs/heads/main
```

```
9fceb02d...
```

HEAD is a special ref that tells git which branch (or commit) is currently checked out. Normally HEAD contains the name of a branch — it is said to be “attached”. When HEAD contains a raw commit hash instead of a branch name, you are in “detached HEAD” state (useful for inspecting history, but do not commit work there without creating a branch first).

Creating a branch is instant and cheap because it only creates a new pointer to an existing commit — no data is copied.

```
git branch feature/login    # create pointer at current HEAD
git switch feature/login    # move HEAD to point at the new branch
```

Local vs remote

A remote is another copy of the same repository — on a server, a colleague’s machine, or elsewhere. The most common remote is named `origin` by convention.

Git tracks remote branches as read-only snapshots in your local repository under names like `origin/main`. They update only when you explicitly ask:

```
git fetch origin            # download new objects and update remote-tracking branches
git pull                   # fetch + merge (or fetch + rebase, depending on config)
git push origin main       # upload local main to origin
```

You always work locally. The network is involved only when you explicitly push or fetch. This makes most operations instant regardless of connectivity.

See also

- *Installing and configuring git* — set up your identity before your first commit.
- *commit* — how to write good commit messages and use the staging area well.
- *Getting out of jams* — what to do when history gets tangled.

Installing and configuring git

Before you can track a single change, git needs to know who you are. This chapter walks through installation, identity, and the configuration knobs that make everyday git noticeably more comfortable.

Installing git

macOS — Git ships with the Xcode Command Line Tools. Install them once:

```
xcode-select --install
```

For a newer version, use Homebrew: `brew install git`.

Linux (Debian / Ubuntu)

```
sudo apt update && sudo apt install git
```

Linux (Fedora / RHEL)

```
sudo dnf install git
```

Windows — Download the official installer from <https://git-scm.com>. It bundles Git Bash and a credential manager. WSL 2 users can also install git inside their Linux distribution with the commands above.

Verify the installation:

```
git --version
```

```
git version 2.47.0
```

Configuration scopes

Git stores configuration in three layered files. A more specific scope overrides a broader one.

Scope	File	Applies to
--system	/etc/gitconfig	Every user on the machine
--global	~/.gitconfig (or ~/.config/git/config)	Your user account
--local	.git/config inside the repo	That repository only

Read the effective value of any key:

```
git config --show-origin user.email
```

List everything with its origin:

```
git config --list --show-origin
```

Setting your identity

Git embeds author name and email in every commit. Set them globally:

```
git config --global user.name "Ada Lovelace"
git config --global user.email "ada@example.com"
```

Override per-repository when you work under a different identity (e.g., a work email in a company repo):

```
git config --local user.email "ada@company.com"
```

If you forget to set these, git will guess from the system hostname — almost always wrong.

Essential quality-of-life settings

Default branch name

New repositories default to `master` in older git versions. Change to `main` (or any name your team uses) before you ever run `git init`:

```
git config --global init.defaultBranch main
```

Rebase on pull

By default `git pull` creates a merge commit when the remote has moved ahead. Rebasing keeps a linear history:

```
git config --global pull.rebase true
```

If you are new to rebase, set this to `false` and revisit the decision after reading *Rewriting history with rebase*.

Editor

Git opens a text editor when you commit without `-m`, write a rebase plan, or edit a tag. Point it at your preferred tool:

```
git config --global core.editor "code --wait" # VS Code
git config --global core.editor nvim        # Neovim
git config --global core.editor nano        # nano (safe default for
beginners)
```

Automatic upstream tracking

Without this setting, the first push to a new branch requires the verbose form

`git push --set-upstream origin <branch>`. Enable auto-setup:

```
git config --global push.autoSetupRemote true
```

After this, a plain `git push` on a new branch sets the upstream and pushes in one step.

Aliases

Aliases let you create short names for long or frequently-typed commands. They live in `~/.gitconfig` under the `[alias]` section.

```
git config --global alias.st status
git config --global alias.co checkout
git config --global alias.br branch
git config --global alias.lg "log --oneline --graph --decorate --all"
```

Run them like any git command: `git st`, `git lg`.

Aliases can also wrap external shell commands by prefixing with `!`:

```
git config --global alias.root "!pwd"
```

Global .gitignore

Some files should never appear in any repository: OS metadata, editor swap files, compiled artifacts. Maintain a global ignore list so you never have to add them to every project:

```
git config --global core.excludesFile ~/.gitignore_global
```

A minimal `~/.gitignore_global`:

```
## macOS
.DS_Store
.AppleDouble

## Windows
Thumbs.db
Desktop.ini

## Editor swap / workspace files
*.swp
*.sw0
.idea/
.vscode/

## Compiled output
*.o
*.pyc
__pycache__/
```

Best practices

Separate personal and work identities at the directory level. Use `includeIf` in `~/.gitconfig` to switch identity automatically based on where the repository lives:

```
[includeIf "gitdir:~/work/"]
  path = ~/.gitconfig-work
```

`~/.gitconfig-work` then sets `user.email = ada@company.com`. Git picks the right identity without any manual switching.

Pin `pull.rebase` to a value explicitly. Leaving it unset means git will warn on every pull once it detects diverged branches. Decide your policy upfront.

Avoid global `core.autocrlf` unless everyone on the project is on Windows. Let the repository's `.gitattributes` handle line-ending normalisation instead; that policy travels with the repo.

Use `--global` for identity and preferences; use `--local` for per-project overrides. Never use `--system` in day-to-day work — it requires root and affects every user on the machine.

See also

- *What git really is* — the object model and the three areas.
- *commit* — how git uses your identity in each commit object.

init

Create an empty Git repository in the current directory, or reinitialize an existing one.

Mental model

When you run `git init`, Git creates a single hidden directory named `.git` at the root of your project. That directory is the entire repository: it holds the object database (every version of every file you will ever commit), the ref store (branches and tags), configuration, and hooks. The working tree — the files you actually edit — lives beside `.git` and is separate from it.

```
my-project/
├── .git/                ← the repository (object DB, refs, config, hooks)
│   ├── objects/
│   ├── refs/
│   │   ├── heads/
│   │   └── tags/
│   ├── HEAD
│   └── config
├── src/                ← your working tree
└── README.md
```

Nothing is tracked yet after `git init`. Git knows the directory exists but has made no commits and recorded no files. The repository is a blank slate waiting for `git add` and `git commit`.

Running `git init` in a directory that already has a `.git` folder is safe: it does not overwrite existing objects, branches, or config. Its main use in that case is to apply a new template or relocate the git directory with `--separate-git-dir`.

Synopsis

```
git init [-q | --quiet] [--bare]
         [--template=<template-directory>]
         [--separate-git-dir <git-dir>]
         [--object-format=<format>]
         [--ref-format=<format>]
         [-b <branch-name> | --initial-branch=<branch-name>]
         [--shared[=<permissions>]]
         [<directory>]
```

Everyday usage

Initialize a repository in the current directory:

```
git init
```

Initialized empty Git repository in `/home/alice/my-project/.git/`

Initialize and name the initial branch `main` instead of the system default:

```
git init -b main
```

Initialize a new repository in a subdirectory that does not yet exist (Git creates it):

```
git init my-project
cd my-project
```

After any of the above, the typical next steps are:

```
git add .
git commit -m "Initial commit"
```

Key options

Option	What it does	When to use it
<code>-b <name> / --initial-branch=<name></code>	Sets the name of the first branch	Use <code>main</code> when team or host convention requires it
<code>--bare</code>	Creates a repository with no working tree	Central/server repos that only receive pushes
<code>--template=<dir></code>	Copies files from <code><dir></code> into <code>.git</code> after creation	Distribute standard hooks or config to new repos
<code>--separate-git-dir=<git-dir></code>	Stores the <code>.git</code> data at <code><git-dir></code> and leaves a pointer file in the working tree	Keep repo data on a different filesystem or hide it from tools that scan for <code>.git</code>
<code>--shared[=<permissions>]</code>	Sets group-writable permissions on <code>.git</code> contents	Shared bare repos on a multi-user server
<code>--object-format=<format></code>	Chooses the hash algorithm: <code>sha1</code> (default) or <code>sha256</code>	Future-proofing; SHA-256 repos are not yet interoperable with SHA-1 repos
<code>--ref-format=<format></code>	Chooses ref storage: <code>files</code> (default) or <code>reftable</code>	Experimental; <code>reftable</code> is faster for repos with millions of refs
<code>-q / --quiet</code>	Suppresses all output except errors and warnings	Scripts and automation

Best practices

Set `init.defaultBranch` once in your global config rather than typing `-b` every time. Agree on a name with your team (`main` is the current convention on GitHub and GitLab) and record it once:

```
git config --global init.defaultBranch main
```

Every subsequent `git init` will use that name automatically.

Add a `.gitignore` before your first commit. The first commit is the hardest to rewrite cleanly. A well-considered `.gitignore` prevents build artifacts, secrets, and editor metadata from entering the object database at all. Add it as part of the initial commit:

```
git init -b main
echo "node_modules/" >> .gitignore
git add .gitignore
git commit -m "Initial commit"
```

Use `--bare` only for server-side repositories. A bare repository has no working tree and cannot be used for day-to-day development. It exists solely to be the target of `git push` and `git fetch`. If you are setting up a shared repo on a server or NAS, `--bare` is correct. If you are starting a project on your laptop, it is not.

Distribute team hooks via `--template`. If your project requires a commit-msg linter or a pre-commit formatter, bake those hook scripts into a template directory and document that developers should run:

```
git init --template=/path/to/company-template
```

This ensures every fresh init picks up the same hooks without a separate installation step. You can also set `init.templateDir` in your global config so the template applies automatically.

Pitfalls & gotchas

Forgetting `-b main` when `init.defaultBranch` is not configured. Many platforms and teams default to `main`, but Git's own built-in default is still `master`. If your remote already has a `main` branch, the local `master` branch will not track it automatically. Set `init.defaultBranch` in your global config once and avoid this mismatch entirely.

Nesting a repository inside another repository. Running `git init` inside a directory that is already tracked by a parent repository creates a submodule-like situation — except without the submodule plumbing. The inner `.git` confuses `git add`, `git status`, and most GUI tools. If you meant to add a dependency as a submodule, use `git submodule add` instead. If you did it by accident, delete the inner `.git` directory before staging anything.

Running `git init` in the home directory. Running `git init` while `cd -d` to `~` makes your entire home directory a repository. Every file you own becomes potentially trackable, and `git add .` could stage credentials, private keys, or browser profiles. Always confirm your current directory before running `git init`.

SHA-256 repositories cannot exchange objects with SHA-1 repositories.

`--object-format=sha256` is forward-looking. There is currently no interoperability between SHA-256 and SHA-1 repositories. Do not use SHA-256 for a repo that needs to interact with GitHub, GitLab, or any SHA-1 remote until interoperability is officially supported.

`--shared` does not encrypt or authenticate. It adjusts Unix file permissions only. Anyone with filesystem access to the bare repository can read the full history. Use it on trusted servers behind proper access control, not as a substitute for it.

Worked examples

Starting a new project from scratch

```
mkdir payment-service
cd payment-service
git init -b main
```

```
Initialized empty Git repository in /home/alice/payment-service/.git/
```

Create a `.gitignore`, add files, and make the first commit:

```
echo "node_modules/" >> .gitignore
echo "*.log" >> .gitignore
git add .gitignore
git commit -m "Add .gitignore"
```

Add the remote and push:

```
git remote add origin git@github.com:alice/payment-service.git
git push -u origin main
```

Turning an existing directory into a repository

You have a directory full of scripts that you have been editing without version control. Convert it to a Git repository without losing anything:

```
cd /opt/scripts
git init -b main
```

Exclude files you do not want to track before staging anything:

```
echo "*.tmp" >> .gitignore
echo ".env" >> .gitignore
git add .gitignore
git add .
git status      # verify nothing unexpected is staged
git commit -m "Initial commit: import existing scripts"
```

All existing files are now in history. Future edits will be tracked incrementally.

Setting up a bare repository as a push target

On a shared server where several developers push code:

```
# on the server
mkdir /srv/git/widgets.git
cd /srv/git/widgets.git
git init --bare --shared=group
```

`--shared=group` makes all files under `.git` group-writable so every member of the Unix group can push. On each developer's machine:

```
git remote add origin git@server.example.com:/srv/git/widgets.git
git push -u origin main
```

Because the server-side repository is bare, developers never work in it directly — they only push to and fetch from it.

Recovery

If you ran `git init` in the wrong directory and have not yet committed anything, delete the `.git` directory to undo it entirely:

```
rm -rf .git
```


There are no commits to lose, so this is safe. If you have already made commits and want to abandon the repository, the same command applies — but any committed history will be gone permanently.

To move the `.git` directory to a different path after initialization, run

`git init --separate-git-dir=<new-path>` in the same working tree; Git will relocate the repository and leave a filesystem pointer in place.

See *Getting out of jams* for undoing commits made after initialization.

See also

- *add* — staging files before the first commit.
- *commit* — recording the staged snapshot into history.
- *clone* — an alternative to `git init` when copying an existing repository.
- *remote* — connecting a local repository to a remote after `git init`.
- *Getting out of jams* — recovering from mistakes made early in a repository's life.

clone

Copy a remote (or local) repository to your machine and wire up the connection so you can fetch and push changes going forward.

Mental model

When you clone a repository, Git does several things in one shot:

1. Creates a new directory (or uses the one you name) and initialises a fresh `.git` inside it.
2. Downloads every object — commits, trees, blobs, tags — from the source into that `.git/objects` store.
3. Creates a **remote** called `origin` pointing at the source URL, and records every remote branch as a remote-tracking branch under `refs/remotes/origin/`.
4. Checks out the branch the remote's `HEAD` points at (typically `main` or `master`) so you have a working tree immediately.

Remote repository	Local clone
<code>refs/heads/main</code>	<code>refs/remotes/origin/main</code>
<code>refs/heads/feature-x</code>	<code>refs/remotes/origin/feature-x</code>
	<code>refs/heads/main</code> (checked out)
	<code>.git/config:</code>
	<code>remote.origin.url = <url></code>
	<code>remote.origin.fetch = +refs/heads/*:refs/</code>
<code>remotes/origin/*</code>	

The remote-tracking branches are read-only snapshots. Your local `main` is a real branch that starts at the same commit; from that point on the two evolve independently until you `fetch`, `pull`, or `push`.

A shallow clone (`--depth`) truncates history at a given depth, giving you a working tree without downloading the entire project timeline. A partial clone (`--filter`) goes further and defers downloading file contents until they are actually needed.

Synopsis

```
git clone [<options>] <repository> [<directory>]

git clone <url>
git clone <url> <directory>
git clone --branch <name> <url>
git clone --depth <n> <url>
git clone --filter=blob:none <url>
git clone --recurse-submodules <url>
git clone --bare <url>
git clone --mirror <url>
```

Everyday usage

Clone a GitHub repository into a new directory named after the project:

```
git clone https://github.com/org/myproject.git
# result: ./myproject/
```

Clone into a specific directory name:

```
git clone https://github.com/org/myproject.git ~/code/proj
```

Clone and immediately check out a branch other than the default:

```
git clone --branch develop https://github.com/org/myproject.git
```

Shallow clone for a quick look or a CI job that does not need full history:

```
git clone --depth 1 https://github.com/org/myproject.git
```

Clone and initialise all submodules in one step:

```
git clone --recurse-submodules https://github.com/org/myproject.git
```

Partial clone — fetch commit and tree objects now, defer blob downloads until files are actually accessed (great for large monorepos):

```
git clone --filter=blob:none https://github.com/org/myproject.git
```

Key options

Option	What it does	When to use it
<code>--branch <name></code>	Check out <code><name></code> instead of the remote's default branch; also accepts tag names	Start work on a non-default branch or pin to a release tag
<code>--depth <n></code>	Shallow clone: fetch only the last <code><n></code> commits	CI pipelines, quick evaluations, saving bandwidth
<code>--single-branch</code>	Fetch only the one branch cloned; future fetches stay scoped to it	Paired with <code>--depth</code> to keep the clone permanently lean
<code>--no-tags</code>	Do not download tags	Lean CI clones that don't need release metadata
<code>--filter=<spec></code>	Partial clone: omit objects matching the filter until needed (<code>blob:none</code> , <code>blob:limit=<size></code>)	Large repos where you only touch a subset of files
<code>--recurse-submodules[=<pathspec>]</code>	Initialise and clone submodules after cloning the superproject	Repos that embed other repos as submodules
<code>--bare</code>	Create a bare repository (no working tree; the directory is <code>.git</code>)	Server-side mirrors, <code>git bundle</code> targets, deploy hooks
<code>--mirror</code>	Like <code>--bare</code> but copies all refs and sets up a refsPEC so <code>git remote update</code> overwrites them	Full mirrors: backup or read-only replica of a remote

<code>-o <name> /</code> <code>--origin <name></code>	Name the remote something other than <code>origin</code>	Disambiguate when you will later add a second remote also named <code>origin</code>
<code>--no-checkout / -n</code>	Skip the initial checkout	When you want to configure sparse-checkout or inspect the object store before materialising files
<code>--sparse</code>	Enable sparse-checkout, initially populating only top-level files	Monorepos where you need just one subtree
<code>-c <key>=<value></code>	Set a config variable in the new repo immediately after <code>init</code>	Override <code>core.autocrlf</code> , add extra fetch refsspecs, etc.
<code>--jobs <n> / -j <n></code>	Number of submodules to fetch in parallel	Speed up <code>--recurse-submodules</code> on repos with many submodules
<code>--dissociate</code>	After borrowing objects from a <code>--reference</code> repo, copy them locally so the clone is self-contained	Avoid a long-term dependency on the reference repo being present
<code>--no-local</code>	When cloning from a local path, use the network transport instead of hardlinks	When you want a fully independent copy, e.g. for backups

Best practices

Prefer HTTPS URLs for new clones unless you have SSH keys configured. HTTPS works through corporate proxies, requires no key setup, and prompts for credentials (or uses a credential helper). Switch to SSH later if you prefer:

```
git remote set-url origin git@github.com:org/myproject.git.
```

Use `--recurse-submodules` whenever the project has a `.gitmodules` file. Forgetting this leaves the submodule directories empty. You can repair it after the fact with

```
git submodule update --init --recursive,
```

but it is easier to get it right at clone time.

Use `--depth 1 --single-branch` in CI. A full clone on every pipeline run transfers the entire history. Pair `--depth 1` with `--single-branch` so that future `git fetch` calls inside the job also stay shallow. For release pipelines that need tags, add `--no-single-branch` instead.

Use `--filter=blob:none` for large monorepos. The partial clone filter defers downloading file contents until Git actually needs them. Commands like `git log`, `git diff`, and branch operations work on metadata without fetching blobs. Files are fetched transparently on first access. This is safer than a shallow clone because you still have the full history graph.

Give the directory an explicit name when the URL is ambiguous. URLs that end in `.git` give a sensible default name, but bare paths and some hosting services produce ugly directory names. Be explicit: `git clone https://example.com/repo.git clean-name`.

Do not clone into a non-empty directory. Git refuses unless the directory is empty. If you need to overlay a clone onto existing files, initialise with `git init` first, then add the remote and fetch manually.

Pitfalls & gotchas

Shallow clones break some Git operations. `git bisect`, `git merge-base`, and certain rebase operations need commit ancestry that a shallow clone may not have. Before running a bisect on a shallow clone, deepen it first: `git fetch --unshallow`.

--depth implies --single-branch by default. A `git fetch` in a shallow clone will only update the one branch unless you pass `--no-single-branch` at clone time or explicitly fetch other branches later. Surprise: `git fetch origin develop` on a `--depth 1` clone with the default `--single-branch` will fail with “fatal: couldn’t find remote ref develop” until you add the full refspec to `.git/config`.

Submodules are not cloned by default. If you see empty subdirectories after cloning, the project uses submodules and you forgot `--recurse-submodules`. Fix it without re-cloning:

```
git submodule update --init --recursive
```

--bare and --mirror clones have no working tree. You cannot check out files, make commits, or run most everyday commands. They are for hosting and mirroring, not day-to-day development.

--shared is dangerous on local clones. It sets up an alternates file so the new clone borrows objects directly from the source. If objects are later pruned from the source (e.g. after a `git gc` following branch deletion), the clone can become corrupt. Use `--no-hardlinks` for safe local backups instead.

Cloning over SSH fails if the agent is not running. If `git clone git@github.com:...` hangs or reports “Permission denied (publickey)”, verify that your SSH key is loaded: `ssh-add -l`. If it is empty, run `ssh-add ~/.ssh/id_ed25519` (or your key path).

The --branch flag accepts tag names but detaches HEAD. Cloning with `--branch v1.2.0` puts HEAD in detached state at that tag’s commit, which is correct for a pinned build but unexpected if you intend to start developing on a release branch.

Worked examples

Clone a GitHub project and start working

```
git clone https://github.com/cli/cli.git
cd cli
git log --oneline -5
```

```
al1b2c3d (HEAD -> trunk, origin/trunk, origin/HEAD) Add --json flag to pr list
...
```

Create a local branch from an existing remote branch:

```
git switch -c feature/my-change origin/main
```

Partial clone of a large monorepo

A monorepo with years of history and thousands of files. You only need recent commits and you will work in one subdirectory.

```
git clone --filter=blob:none --sparse https://github.com/org/monorepo.git
cd monorepo
git sparse-checkout set services/auth
```

Git fetches only commit and tree objects during the clone. Blobs under `services/auth/` are fetched on checkout; blobs elsewhere are never downloaded unless you explicitly need them. See the *sparse-checkout* chapter for more on narrowing the working tree.

Shallow clone for a CI pipeline

A GitHub Actions job that only needs to build from the tip of `main` :

```
git clone --depth 1 --single-branch --branch main \
https://github.com/org/myproject.git
```

This fetches exactly one commit's worth of objects, regardless of how long the project history is. The pipeline runs faster and uses less disk.

To later fetch tags without fetching all history (e.g. to run `git describe`) :

```
git fetch --tags --depth 1
```

Set up a bare mirror for backup

Create a local bare mirror that you can push to for backup:

```
git clone --mirror https://github.com/org/myproject.git \
/backups/myproject.git
```

Update the mirror later:

```
git -C /backups/myproject.git remote update
```

`--mirror` copies all refs (branches, tags, notes, remote-tracking branches) and writes a fetch refspec that overwrites them all on each update, making this a true read-only replica. See the *remote* chapter for managing remotes on an existing repository.

Clone with a custom remote name

When you plan to track two upstream forks (the original project and your own fork), name the remotes clearly from the start:

```
git clone --origin upstream https://github.com/original/project.git
cd project
git remote add origin https://github.com/yourname/project.git
```

Now `upstream` points at the canonical repo and `origin` points at your fork — the conventional arrangement for open-source contribution workflows.

Recovery

If you cloned with `--depth` and later find you need the full history:

```
git fetch --unshallow
```

If you forgot `--recurse-submodules` :

```
git submodule update --init --recursive
```

If you cloned the wrong URL, update it without re-cloning:

```
git remote set-url origin https://github.com/correct/repo.git
```

If a partial clone (`--filter`) left you missing objects that a command needs, refetch all objects from the remote:

```
git fetch --refetch origin
```

See *Getting out of jams* for broader recovery recipes including abandoned clones and corrupted object stores.

See also

- *init* — create a new empty repository from scratch instead of cloning.
- *remote* — inspect and modify the remotes configured by `clone` .
- *fetch* — update remote-tracking branches after the initial clone.
- *pull* — fetch and integrate upstream changes into the current branch.
- *sparse-checkout* — limit the working tree after cloning a large repo.
- *submodule* — manage nested repositories initialised with `--recurse-submodules` .
- *Getting out of jams* — fix shallow, partial, or broken clones.

add

Copy file contents from the working tree into the index so they are included in the next commit.

Mental model

Git tracks three distinct copies of every file: the committed version in the repository, the staged version in the index, and the version on disk in your working tree. The index is the crucial middle layer — it is the exact snapshot that `git commit` will turn into a commit object.

`git add` moves content from the working tree into the index. Until you run it, your edits exist only on disk and are invisible to the next commit. You can run `git add` as many times as you like before committing; each call updates the index to match the current disk state of the specified paths.

```
Working tree —git add—> Index —git commit—> Repository
      (your edits)           (staged snapshot)       (permanent history)
```

A common mental slip is to think of `git add` as “tell Git to track this file forever.” That is only true for new files. For already-tracked files, each invocation of `git add` means “record the current content of this file in the staging area right now.” If you edit the file again after staging it, the new edits are not staged; you must run `git add` again.

Synopsis

```
git add [--verbose | -v] [--dry-run | -n] [--force | -f]
        [--interactive | -i] [--patch | -p] [--edit | -e]
        [--all | -A | --update | -u | --no-all]
        [--intent-to-add | -N] [--refresh] [--ignore-errors]
        [--ignore-missing] [--renormalize] [--sparse]
        [--chmod=(+|-)x]
        [--pathspec-from-file=<file> [--pathspec-file-nul]]
        [--] [<pathspec>...]
```

Everyday usage

Stage a single file:

```
git add src/login.js
```

Stage several files at once:

```
git add src/login.js tests/login.test.js
```

Stage an entire directory (adds new, modified, and records deletions):

```
git add src/
```

Stage every change in the repository — new files, modifications, and deletions:

```
git add -A
```

Stage only modifications and deletions of already-tracked files (skips new untracked files):

```
git add -u
```


Interactively choose which hunks within a file to stage:

```
git add -p src/api.js
```

Git presents each changed block and asks: stage this hunk? The most useful responses are `y` (yes), `n` (no), `s` (split into smaller hunks), and `q` (quit). See the hunk keys table in the Worked examples section for the complete list.

Preview what would be staged without actually staging anything:

```
git add --dry-run .
git add -n .
```

Key options

Option	What it does	When to use it
<code><pathspec></code>	Stage the named files or directories	The everyday case — name what you want staged
<code>-A</code> / <code>--all</code>	Stage all changes: additions, modifications, deletions	Commit everything at once; be deliberate
<code>-u</code> / <code>--update</code>	Stage modifications and deletions of tracked files only	Exclude new untracked files from the stage
<code>--no-all</code> / <code>--ignore-removal</code>	Stage new and modified files, ignore deletions	Legacy-compatibility; rarely needed
<code>-p</code> / <code>--patch</code>	Interactively choose hunks to stage	Commit only part of a file's changes
<code>-i</code> / <code>--interactive</code>	Full interactive mode with a command menu	Explore staged/unstaged state interactively
<code>-e</code> / <code>--edit</code>	Open the diff in an editor; apply what you save	Fine-grained control when <code>-p</code> hunks are too coarse
<code>-n</code> / <code>--dry-run</code>	Show what would be added without staging it	Verify glob patterns or check ignore status
<code>-f</code> / <code>--force</code>	Stage files that would normally be ignored	Deliberately commit a file in <code>.gitignore</code>
<code>-N</code> / <code>--intent-to-add</code>	Register a path as “will be added” without staging content	Make a new file visible to <code>git diff</code> before staging
<code>--chmod=(+ -)x</code>	Set or clear the executable bit in the index only	Fix execute permissions without touching the filesystem
<code>--renormalize</code>	Re-apply line-ending normalization to all tracked files	Fix mixed CRLF/LF after changing <code>core.autocrlf</code>
<code>--refresh</code>	Update cached stat info without changing staged content	Silence false-positive dirty markers without restaging

<code>--ignore-errors</code>	Continue staging remaining files if some fail	Partially stage a large batch where a few files error
<code>--ignore-missing</code>	With <code>--dry-run</code> , check ignore status for absent files	Test whether a path would be ignored before it exists
<code>--sparse</code>	Allow staging paths outside the sparse-checkout cone	Advanced sparse-checkout workflows
<code>--pathspect-from-file=<file></code>	Read paths from a file instead of the command line	Script-generated file lists; very long path sets
<code>--pathspect-file-nul</code>	With <code>--pathspect-from-file</code> , use NUL as the separator	Safe handling of filenames that contain newlines

Best practices

Stage deliberately, not wholesale. `git add -A` is convenient but often stages more than you intended — debug output left in a test file, a half-finished change to an unrelated module, a build artifact that slipped past `.gitignore`. Stage specific files or directories by name, or use `-p` to review hunks. Each commit should represent one logical unit of work; that discipline starts at `git add`.

Use `-p` to split mixed edits. When a file contains two unrelated changes, `-p` lets you stage only the relevant hunks and leave the rest for a later commit. Reviewers and future `git bisect` users will thank you. See the *commit* chapter for how this pairs with `git commit`.

Re-run `git add` after editing a staged file. The index holds the content at the moment you staged, not a live reference. If you edit a file after staging it, `git status` will show it listed under both “Changes to be committed” and “Changes not staged for commit.” Run `git add` again to bring the staged version up to date — or stage only the first set of changes intentionally.

Confirm what you are about to stage with `--dry-run`. Before staging a broad glob like `git add '*.json'`, run `git add -n '*.json'` to see exactly which files match. This is especially valuable when your shell might expand the glob differently from Git.

Understand what `-A` and `-u` do differently. `-A` updates the entire index to match the working tree: new files, modifications, and deletions. `-u` updates only already-tracked paths: modifications and deletions, but not new untracked files. Neither option is “safer” in the abstract — the right one depends on your intent.

Use `-N` to make new files visible before staging them. `git diff` only shows tracked content by default, so a brand-new file is invisible until you stage it. `git add -N path/to/newfile.js` registers the path without staging any content; now `git diff` shows the full pending addition so you can review it before committing.

Pitfalls & gotchas

`.gitignore` only affects untracked files. If a file is already tracked in the repository, adding its pattern to `.gitignore` has no effect. Git continues to stage and commit changes to it. To stop tracking a file, use `git rm --cached` first; then add it to `.gitignore`.

`git add -A` without a pathspec operates on the entire repository. Running `git add -A` from a subdirectory stages changes across the whole working tree, not just the current directory. This changed from older versions of Git that scoped it to the current directory. If you want to limit scope, pass an explicit path: `git add -A src/`.

Shell glob expansion vs. Git glob expansion differ. When you write `git add *.js`, your shell expands the glob before Git sees it — so only files in the current directory are matched. To let Git expand the glob recursively, quote it: `git add '*.js'`. The `--dry-run` flag is the fastest way to verify which form you need.

Staging a file twice is harmless but the second stage silently wins. If you stage a file, edit it, then stage it again, the index holds only the most recent staged content. There is no warning. If you want to preserve the first staged version, commit before editing further.

--edit patches the index, not the working tree. When you modify the diff in your editor with `git add -e`, the index diverges from both HEAD and your working tree. `git diff --cached` shows what is staged; `git diff` shows what is unstaged. The divergence is intentional and powerful but confusing until you understand it.

Embedded repositories trigger a warning. If you run `git add` on a directory that contains its own `.git` folder (a nested repository not managed as a submodule), Git warns you and registers only an empty tree. To add a nested repo properly, use `git submodule add`.

Worked examples

Staging only the bug fix from a file with mixed changes

You edited `api.js` in two places: a one-line bug fix and a larger refactor. You want separate, focused commits.

```
git add -p api.js
```

Git presents each hunk:

```
@@ -42,7 +42,7 @@ function parseToken(token) {
-  if (token == null) return false;
+  if (token == null) throw new Error('token required');
...
Stage this hunk [y,n,q,a,d,g,/ ,j,J,k,K,s,e,p,]?
```

Hunk keys:

Key	Action
y	Stage this hunk
n	Do not stage this hunk
q	Quit; do not stage this hunk or any of the remaining ones
a	Stage this hunk and all later hunks in the file
d	Do not stage this hunk or any of the later hunks in the file
g	Select a hunk to go to
/	Search for a hunk matching the given regex
j	Leave this hunk undecided, see next undecided hunk
J	Leave this hunk undecided, see next hunk
k	Leave this hunk undecided, see previous undecided hunk
K	Leave this hunk undecided, see previous hunk
s	Split the current hunk into smaller hunks

e	Manually edit the current hunk
p	Print the current hunk
?	Print help

Stage the bug-fix hunk (`y`) and skip the refactor hunk (`n`). Then commit just the fix:

```
git commit -m "Throw on null token instead of silently returning false"
```

Go back and stage the refactor:

```
git add api.js
git commit -m "Extract token validation into parseToken helper"
```

Staging all changes then reviewing before committing

You have been working across several files and want to commit everything, but you want to double-check what is staged first.

```
git add -A
git diff --cached          # review the full staged diff
git status                 # confirm staged vs. unstaged summary
git commit -m "Implement user profile page"
```

If you spot something you do not want in this commit, unstage it:

```
git restore --staged src/debug-helpers.js
```

Then commit the rest and stage the debug file separately when it is ready.

Fixing execute permissions in the index

A deployment script `deploy.sh` needs to be executable, but the filesystem bit was not set when it was committed. Fix it in the index without touching the file on disk:

```
git add --chmod=+x deploy.sh
git commit -m "Mark deploy.sh as executable"
```

Verify the change took effect:

```
git ls-files --stage deploy.sh
```

The mode field in the output should read `100755` (executable) instead of `100644` .

Staging files listed by a script

A code-generation step produces a list of changed files. Pass them to `git add` without hitting shell argument-length limits:

```
./generate.sh | git add --pathspec-from-file=- --pathspec-file-nul
```

The `--pathspec-file-nul` flag tells Git to split on NUL bytes, which is safe for filenames containing spaces or special characters.

Recovery

To unstage a file that you added by mistake (put it back to “not staged for commit” while keeping your working-tree edits):

```
git restore --staged path/to/file.js
```

To unstage everything at once:

```
git restore --staged .
```

If you want to discard both the staged version and the working-tree edits entirely, see the *restore* chapter and the *reset* chapter.

See *Getting out of jams* for recipes covering accidentally staged secrets, staged deletions you did not intend, and recovering from a `git add -A` that swept up files you wanted to keep unstaged.

See also

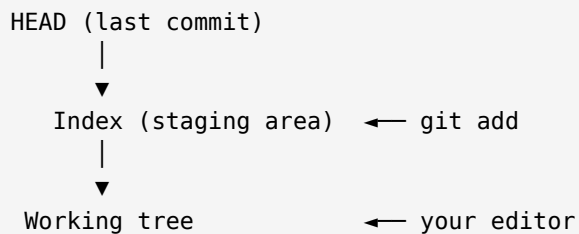
- *commit* — freezes the staged index into a permanent commit object.
- *status* — shows which files are staged, unstaged, or untracked.
- *diff* — use `git diff --cached` to review exactly what is staged.
- *restore* — unstage files with `git restore --staged`.
- *reset* — `git reset HEAD <file>` is an older equivalent of `git restore --staged`.
- *Getting out of jams* — recovering from accidental stages and other index mishaps.

status

Show what has changed between your working tree, the index, and HEAD — without modifying anything.

Mental model

Git tracks three distinct layers for every file in your repository:



`git status` compares all three layers at once and reports:

1. **Staged changes** — differences between HEAD and the index. These go into your next commit.
2. **Unstaged changes** — differences between the index and the working tree. These will *not* go into your next commit unless you stage them first.
3. **Untracked files** — files in the working tree that Git has never seen.

Nothing is written, moved, or deleted. `git status` is always safe to run.

Synopsis

```
git status [<options>] [--] [<pathspec>...]
```

Everyday usage

Run a plain status to see everything at once:

```
git status
```

On branch feature/login

Your branch is up to date with 'origin/feature/login'.

Changes to be committed:

(use "git restore --staged <file>..." to unstage)
modified: src/auth.js

Changes not staged for commit:

(use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
modified: src/user.js

Untracked files:

(use "git add <file>..." to include in what will be committed)
tests/auth.test.js

Get a compact one-line-per-file summary — useful inside a shell prompt or a quick scan:

```
git status -s
```

```
M src/auth.js
M src/user.js
?? tests/auth.test.js
```

The left column is the index state; the right column is the working-tree state. `M` (left column only) means a staged modification; `M` (right column only) means an unstaged modification; `??` means untracked.

Show the branch and tracking information alongside the short format:

```
git status -sb
```

```
## feature/login...origin/feature/login
M src/auth.js
M src/user.js
?? tests/auth.test.js
```

Limit the report to a specific path or directory:

```
git status src/
```

Show ignored files (normally hidden):

```
git status --ignored
```

Key options

Option	What it does	When to use it
<code>-s</code> , <code>--short</code>	One-line-per-file output with two-letter status codes	Quick scans, shell prompts
<code>-b</code> , <code>--branch</code>	Prefix short output with branch and tracking info	Combine with <code>-s</code> for a dense summary
<code>--show-stash</code>	Print the count of stash entries if any exist	Reminder that stashed work is pending
<code>--porcelain[=<version>]</code>	Stable machine-readable output (v1 or v2)	Scripts and tools that parse status
<code>--long</code>	Human-readable long format (the default)	Normal interactive use
<code>-v</code> , <code>--verbose</code>	Also show the diff of staged changes; <code>-vv</code> adds unstaged diff	Review exactly what will be committed
<code>-u[<mode>]</code> , <code>--untracked-files[=<mode>]</code>	Control untracked-file reporting: <code>no</code> , <code>normal</code> , or <code>all</code>	<code>no</code> for speed in large trees; <code>all</code> to expand new directories
<code>--ignored[=<mode>]</code>	Show ignored files; modes: <code>traditional</code> , <code>no</code> , <code>matching</code>	Auditing <code>.gitignore</code> rules

<code>-z</code>	NUL-terminate entries instead of newline; implies <code>--porcelain=v1</code>	Robust script parsing when paths may contain spaces
<code>--ahead-behind / --no-ahead-behind</code>	Show or suppress the ahead/behind count vs upstream	<code>--no-ahead-behind</code> speeds up status on slow network mounts
<code>--renames / --no-renames</code>	Enable or disable rename detection	Override user config in scripts
<code>--find-renames[=<n>]</code>	Enable rename detection with an optional similarity threshold	Fine-tune when files are only partially renamed
<code>--ignore-submodules[=<when>]</code>	Hide submodule changes; values: <code>none</code> , <code>untracked</code> , <code>dirty</code> , <code>all</code>	Suppress noise from submodule churn

Best practices

Run `git status` before every commit. Glancing at the output catches forgotten files, accidental deletions, and leftover debug code before they become part of history. The habit costs two seconds and has saved many developers from embarrassing commits.

Use `-v` when the diff matters. The plain output tells you *which* files changed; `-v` shows *what* changed in staged files. Running `git status -v` before committing is a lightweight alternative to `git diff --cached` and keeps the review in one place.

Use `--porcelain` in scripts — never rely on the long format. The long-format output is documented as subject to change. `--porcelain=v1` (or `v2` for richer metadata including object names and file modes) is stable across Git versions and ignores user colour configuration. Combine with `-z` for safety when file names may contain spaces or newlines:

```
git status --porcelain -z | while IFS= read -r -d '' entry; do
    status="${entry:0:2}"
    file="${entry:3}"
    printf '%s -> %s\n' "$status" "$file"
done
```

Prefer `-sb` over bare `-s` in a shell prompt. The branch line tells you at a glance whether you are ahead or behind the remote, which matters before a push.

Run with `--no-optional-locks` in background jobs. When a CI script or file-watcher calls `git status` in the background, it holds a brief write lock that can block concurrent Git operations. Add the top-level flag to prevent it:

```
git --no-optional-locks status --porcelain
```

Enable the untracked cache on large repositories. In a tree with tens of thousands of files, scanning for untracked files is the main cost of `git status`. One-time configuration that pays off on every subsequent run:

```
git config core.untrackedCache true
git config core.fsmonitor true # optional: also skip unmodified directories
```


Pitfalls & gotchas

A clean status does not mean your branch matches the remote. `git status` compares against HEAD — your *local* last commit — not against `origin/main`. You can have a perfectly clean working tree and still be ten commits behind the remote. Run `git fetch` first to update the remote-tracking refs; the `git status` output will then reflect the true ahead/behind gap.

Untracked directories are shown as a single entry. By default (`-u normal`) Git shows `new-directory/` rather than every file inside it. Run `git status -u all` to expand them. This is easy to miss when you copy an entire directory into your repo and see only one `??` line.

The two-column short format is easy to misread. The first character is the *index* state; the second is the *working-tree* state. `MM` means a file is both staged *and* has additional unstaged changes — the staged and unstaged versions differ. Only the staged portion goes into the next commit.

Ignored files hide silently. Files matched by `.gitignore` do not appear in the default output. If a file is missing from the report and you are confused, run `git status --ignored` to reveal what is suppressed, then use `git check-ignore -v <file>` to find which rule matched.

Rename detection can produce false positives. When you delete one file and create another with similar content, Git may report `renamed: old -> new`. If this is not actually a rename, use `git status --no-renames` to disable the heuristic and see the true delete-plus-add picture.

Worked examples

Reviewing a full staging session before committing

You have been editing several files and want to be certain exactly what will go into the next commit.

```
git status -v
```

The verbose flag appends the full staged diff below the file list:

```
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    modified:   src/cart.js

diff --git a/src/cart.js b/src/cart.js
index 3a4b5c6..7d8e9f0 100644
--- a/src/cart.js
+++ b/src/cart.js
@@ -42,6 +42,7 @@ function addItem(cart, item) {
+  validateStock(item);
```

You spot that a `console.log` you added for debugging is still staged. Unstage the file, remove the log, re-stage, then commit:

```
git restore --staged src/cart.js
# remove the console.log in your editor
git add src/cart.js
git commit -m "Validate stock before adding item to cart"
```

Parsing status output in a deployment script

A deployment script should abort if any tracked file has uncommitted changes.

```
#!/bin/sh
if git --no-optional-locks status --porcelain | grep -qv '^??'; then
    echo "ERROR: uncommitted changes detected, aborting deploy" >&2
    exit 1
fi
```

`grep -v '^??'` filters out untracked files so the check targets only staged or unstaged modifications to tracked files. `--no-optional-locks` prevents the background call from blocking parallel Git operations.

Diagnosing a conflict state after a failed merge

After `git merge` reports conflicts, `git status` identifies the unmerged paths:

```
git status -s
```

```
UU src/config.js
M  src/main.js
```

`UU` means both sides modified the file and Git could not auto-resolve it. Open `src/config.js`, resolve the conflict markers, then stage the result:

```
# resolve conflicts in your editor
git add src/config.js
git status -s
```

```
M  src/config.js
M  src/main.js
```

Both entries now show `M` (left column) — fully staged, nothing unstaged. Complete the merge commit as described in the *commit* chapter.

Recovery

`git status` is read-only — it never modifies anything, so there is nothing to undo from running it.

If you acted on what `git status` reported and want to reverse those actions:

- To unstage a file you just staged: `git restore --staged <file>`
- To discard working-tree changes you just discarded: see *Getting out of jams* — once discarded, changes can sometimes be recovered from the object database if they were ever staged.

See also

- *add* — staging files that `git status` reports as unstaged or untracked.
- *commit* — recording the staged changes shown under “Changes to be committed”.
- *diff* — viewing the exact line-level changes that `git status` summarises by filename.
- *restore* — unstaging or discarding the changes `git status` reports.
- *Getting out of jams* — recovering from accidental discards and other mistakes discovered through `git status`.

diff

Show changes between the working tree, the index, commits, or arbitrary files.

Mental model

Git tracks three distinct “copies” of your project at any given time:

```
Working tree — what is on disk right now
Index       — the staged snapshot (draft of the next commit)
Repository  — the committed history of tree objects
```

`git diff` is a window between any two of those layers, or between any two commits in history. Depending on how you call it, it answers a different question:

Invocation	Question answered
<code>git diff</code>	What have I changed but not yet staged?
<code>git diff --cached</code>	What have I staged but not yet committed?
<code>git diff HEAD</code>	What have I changed since the last commit (staged + unstaged)?
<code>git diff A B</code>	How does commit B differ from commit A?
<code>git diff A...B</code>	What did branch B add since it diverged from branch A?

The output is a unified diff: lines starting with `-` existed in the “before” snapshot; lines starting with `+` exist in the “after” snapshot; lines starting with a space are unchanged context. Each changed region is called a hunk, introduced by a `@@` header that names the surrounding function or line numbers.

Synopsis

```
git diff [<options>] [<commit>] [--] [<path>...]
git diff [<options>] --cached [--merge-base] [<commit>] [--] [<path>...]
git diff [<options>] [--merge-base] <commit> [--] [<path>...]
git diff [<options>] [--merge-base] <commit> <commit> [--] [<path>...]
git diff [<options>] <commit>..<commit> [--] [<path>...]
git diff [<options>] <commit>...<commit> [--] [<path>...]
git diff [<options>] <blob> <blob>
git diff [<options>] --no-index [--] <path> <path> [<pathspect>...]
```

Everyday usage

See unstaged changes (working tree vs. index):

```
git diff
```

See staged changes (index vs. last commit — what `git commit` would record):

```
git diff --cached
# --staged is an exact synonym
git diff --staged
```

See everything changed since the last commit — staged and unstaged together:

```
git diff HEAD
```

Compare two commits by hash or branch name:

```
git diff main feature/auth
git diff abc1234 def5678
```

Use the triple-dot form to see only what a branch added since it diverged from another branch (equivalent to diffing from the common ancestor):

```
git diff main...feature/auth
```

Limit the diff to specific files or directories:

```
git diff -- src/auth.js
git diff HEAD -- src/
```

Get a summary of which files changed and by how many lines, without the full diff text:

```
git diff --stat
git diff --stat main..feature/auth
```

Show only the names of changed files:

```
git diff --name-only HEAD~3
```

Compare two arbitrary files on disk, outside any repository:

```
git diff --no-index old-version.js new-version.js
```

Key options

Option	What it does	When to use it
<code>--cached</code> / <code>--staged</code>	Compare index against a commit (default: HEAD)	Review exactly what will be committed
<code>--stat</code>	Show a diffstat summary instead of the full patch	Quick overview of scope
<code>--name-only</code>	List only the names of changed files	Scripting, feeding to <code>xargs</code>
<code>--name-status</code>	List names with A/M/D/R status letters	See the type of change at a glance
<code>-U<n></code>	Show <code><n></code> lines of context (default 3)	Narrow (<code>-U0</code>) for patch review; wider for more context
<code>-w</code> / <code>--ignore-all-space</code>	Ignore all whitespace differences	Ignore reformatting-only changes
<code>-b</code> / <code>--ignore-space-change</code>	Ignore changes in the amount of whitespace	Ignore indent normalization
<code>--word-diff</code>	Show differences word-by-word rather than line-by-line	Prose and config files

<code>--color-moved</code>	Color moved code blocks distinctly from added/removed lines	Spot refactors that shuffle code around
<code>--diff-algorithm=<algo></code>	Choose <code>myers</code> (default), <code>minimal</code> , <code>patience</code> , or <code>histogram</code>	<code>patience</code> and <code>histogram</code> often produce cleaner hunks
<code>-M / --find-renames</code>	Detect renamed files	See renames instead of delete + add pairs
<code>-C / --find-copies</code>	Detect copies as well as renames	Track file duplications
<code>--diff-filter=<letters></code>	Limit output to Added/Deleted/Modified/Renamed/etc.	Show only new files: <code>--diff-filter=A</code>
<code>-S<string></code>	Find hunks where the occurrence count of <code><string></code> changes	Locate when a specific symbol was introduced or removed
<code>-G<regex></code>	Find hunks whose patch text matches a regex	Search for any change touching a pattern
<code>--merge-base</code>	Use the merge base of the two endpoints as the “before” side	Explicit alternative to the <code>...</code> notation
<code>--no-index</code>	Compare two arbitrary paths on disk	Diff files outside a repository
<code>-R</code>	Swap the before and after sides	See the inverse of a change
<code>--exit-code</code>	Exit 1 if there are differences, 0 if not	Use in scripts and CI checks
<code>--quiet</code>	Suppress all output; implies <code>--exit-code</code>	Fastest way to test for any difference in a script
<code>-W / --function-context</code>	Show the whole surrounding function as context	Understand a change without opening an editor
<code>--check</code>	Warn about conflict markers and whitespace errors	Quick pre-commit sanity check

Best practices

Run `git diff --cached` before every commit. It shows exactly what `git commit` will freeze into history — not what you think you staged, but what Git actually has. Make it muscle memory.

Narrow the scope with paths. When a repo has many files in flight, `git diff -- path/to/file.js` filters noise immediately. Pass a directory to diff everything beneath it.

Prefer `...` (triple-dot) for branch comparisons. `git diff main..feature` includes any commits on `main` that are not yet on `feature`, which produces noise when `main` has moved on. `git diff main...feature` diffs from the point the branch diverged, so you see only what the `feature` branch contributed.

Use `--stat` first, then drill in. On large diffs, `--stat` gives you a map before you descend into patch text. Once you know which files matter, diff them individually.

Try `--histogram` for restructured code. The default Myers algorithm sometimes produces confusing hunks when functions are reordered. `--diff-algorithm=histogram` often produces cleaner, more readable output on real-world code.

Use `--word-diff` for prose and configuration. Line-based diffs on documentation or long configuration values show entire lines as changed when only one word moved. `--word-diff` makes the actual change obvious.

Enable rename detection globally. If your project renames files regularly, add `diff.renames = true` to your `~/.gitconfig` so `-M` is always active. It prevents noise from delete + add pairs that are really just moves.

Pitfalls & gotchas

`git diff` with no arguments shows nothing when everything is staged. If you ran `git add -A`, the working tree and the index are identical, so plain `git diff` produces no output. Use `git diff --cached` to see what is staged. New users often think nothing changed when in fact everything is staged and ready to commit.

`..` and `...` mean opposite things between `log` and `diff`. In `git log A..B`, the double-dot means “commits reachable from B but not A.” In `git diff A..B`, the double-dot means the same as `git diff A B` — the difference between the two tips. The triple-dot `git diff A...B` means “from the common ancestor to B,” which corresponds more closely to what `git log A..B` shows. This asymmetry trips up almost everyone at some point.

Renamed files appear as delete + add unless you ask for rename detection. By default `git diff` does not detect renames. A file moved from `old.js` to `new.js` shows up as a deletion of `old.js` and a creation of `new.js`. Add `-M` to detect renames, or set `diff.renames = true` in your git config to enable it permanently.

Whitespace-only changes hide in plain diffs but break patches. A diff that looks empty with `-w` may still apply differently than you expect. Trailing whitespace and mixed tabs/spaces are silent bugs in patch workflows. Use `--check` to surface them before they cause problems.

`--exit-code` is not the default. Scripts that do `if git diff ...; then` will always take the truthy branch because `git diff` exits 0 even when there are differences. You must add `--exit-code` (or `--quiet`, which implies it) for the exit status to reflect whether a difference was found.

Diffs against HEAD fail on an unborn branch. On a brand-new repository with no commits, `git diff HEAD` errors because HEAD does not yet point to a commit. Use `git diff --cached` instead, which can show staged changes even before the first commit.

Worked examples

Reviewing a staged change before committing

You have modified two files and staged both. Before committing, you want to confirm what will actually be recorded.

```
git diff --cached
```

```
diff --git a/src/auth.js b/src/auth.js
index 3a1c2d4..9e8f01b 100644
--- a/src/auth.js
```

```
+++ b/src/auth.js
@@ -42,7 +42,10 @@ function validateToken(token) {
    if (!token) {
        throw new Error('Token required');
    }
+   if (token.length < 32) {
+       throw new Error('Token too short');
+   }
    return jwt.verify(token, process.env.JWT_SECRET);
}
```

The `+` lines are what will be added. Satisfied, you commit:

```
git commit -m "Enforce minimum token length in validateToken"
```

Comparing a feature branch to main

You want to see everything your `feature/payments` branch added since it diverged from `main`, ignoring any commits that landed on `main` in the meantime:

```
git diff --stat main...feature/payments
```

```
src/payments/checkout.js | 84 +++++
src/payments/stripe.js   | 47 +++++
tests/payments.test.js   | 62 +++++
3 files changed, 193 insertions(+)
```

Then drill into the file you care about most:

```
git diff main...feature/payments -- src/payments/stripe.js
```

Spotting a moved block of code

A colleague refactored a module by extracting a helper function. The diff looks cluttered because the same lines appear as both removed and added. `--color-moved` paints moved blocks in a distinct color so you can immediately see the extraction instead of a phantom delete and insert:

```
git diff --color-moved=zebra HEAD~1
```

Checking for whitespace errors before pushing

`--check` surfaces trailing whitespace and conflict markers across all staged changes:

```
git diff --cached --check
```

If it exits non-zero, Git prints the offending lines with their line numbers. Fix them before committing to avoid noisy follow-up commits.

Using diff in a CI script

A CI step that fails if any source or test file changed on a supposed docs-only branch:

```
if ! git diff --quiet origin/main...HEAD -- src/ tests/; then
  echo "Non-docs files changed; failing CI."
  exit 1
fi
```

`--quiet` suppresses all output and implies `--exit-code`, so the exit status reflects whether a difference was found.

Recovery

`git diff` is a read-only operation — it never modifies the working tree, index, or history. There is nothing to undo.

If you ran `git diff` expecting to see changes and saw nothing, the most likely causes are:

- Everything is staged: run `git diff --cached`.
- You are on a detached HEAD or a new repo with no commits: compare explicitly, e.g. `git diff --cached` or `git diff <commit>`.
- The path argument did not match: check spelling and use `--` before paths to prevent ambiguous interpretation as a branch name.

See *Getting out of jams* for help recovering accidentally discarded working-tree changes.

See also

- *add* — staging the changes that `git diff --cached` will then show.
- *commit* — recording the staged snapshot into history.
- *status* — a compact summary of which files have changed, without the diff text.
- *log* — navigating commit history; combine with `-p` to show each commit's diff inline.
- *show* — display the diff introduced by a single commit.
- *stash* — set aside in-progress work so you can diff on a clean working tree.
- *Getting out of jams* — recovering accidentally discarded changes.

commit

Record a snapshot of the staging area into the repository as a permanent commit object.

Mental model

The index (staging area) is a draft of your next commit. You build that draft incrementally with `git add`, then freeze it with `git commit`. The commit command does three things in sequence:

1. Writes the current index contents as a tree object in the object database.
2. Creates a commit object pointing to that tree, the current HEAD as parent, and your identity and timestamp as metadata.
3. Advances the current branch pointer to the new commit.

Nothing in the working tree changes. Files you have not staged are untouched. Files you staged become part of history and are now safe to modify again.

```
Working tree —git add—> Index —git commit—> Repository
                                     |
                                     └─ branch pointer advances
```

Synopsis

```
git commit [-m <msg>] [-a] [--amend] [--fixup=<commit>] [--squash=<commit>]
           [-s] [-S[<key-id>]] [-p] [-C <commit>] [-c <commit>]
           [--allow-empty] [--no-verify] [--date=<date>]
           [--author=<author>] [-- <paths-spec>...]
```

Everyday usage

Create a commit with a message on the command line:

```
git add src/login.js tests/login.test.js
git commit -m "Add login validation with rate limiting"
```

Open the configured editor to write a multi-line message:

```
git add -p          # interactively stage hunks
git commit          # editor opens with diff summary at the bottom
git commit -p       # or skip the staging step: pick hunks and commit in one go
```

Stage all modified and deleted tracked files, then commit in one step:

```
git commit -a -m "Fix typo in error message"
```

Amend the most recent commit (message, contents, or both):

```
git add forgotten-file.js
git commit --amend --no-edit  # add the file, keep the existing message
git commit --amend           # rewrite the message in the editor
```

Key options

Option	What it does	When to use it
--------	--------------	----------------

<code>-m <msg></code>	Set the commit message from the command line	Short, single-line messages
<code>-a</code>	Auto-stage all tracked modified/deleted files	Skip <code>git add</code> for quick fixes
<code>--amend</code>	Replace the previous commit with a new one	Fix the last commit before pushing
<code>--no-edit</code>	Keep the existing message when amending	Add a forgotten file silently
<code>--fixup=<commit></code>	Create a <code>fixup!</code> commit targeting <code><commit></code>	Prep for <code>rebase --autosquash</code>
<code>--squash=<commit></code>	Create a <code>squash!</code> commit targeting <code><commit></code>	Like <code>--fixup</code> but lets you edit the combined message
<code>-s</code>	Append a <code>Signed-off-by</code> trailer	Projects requiring DCO sign-off
<code>-S[<key-id>]</code>	GPG-sign the commit	Verified commits on GitHub/GitLab
<code>-p / --patch</code>	Interactively choose which hunks to stage and commit	Commit only part of a file's changes
<code>-C <commit></code>	Reuse the message and authorship of <code><commit></code> exactly	Cherry-pick workflows
<code>-c <commit></code>	Like <code>-C</code> but opens the editor to let you modify	Edit a message from another commit
<code>--allow-empty</code>	Permit a commit that introduces no file changes	Trigger CI, mark a deployment point
<code>--no-verify</code>	Skip the pre-commit and commit-msg hooks	Emergency; understand why the hook failed first
<code>--date=<date></code>	Override the author date	Backfill work done at a known time
<code>--author=<author></code>	Override the commit author	Committing on behalf of someone else

Best practices

Write the subject line in the imperative mood, 50 characters or fewer. Think of it as completing the sentence “If applied, this commit will...”. “Add login validation” is right. “Added login validation” and “Adding login validation” are not. The 50-character limit keeps it readable in `git log --oneline`, GitHub pull-request lists, and email subjects.

Separate the subject from the body with a blank line. Many tools (GitHub, `git shortlog`, `git format-patch`) split on that blank line. Without it the entire message is treated as a subject.

Explain the *why*, not the *what*. The diff already shows what changed. The commit message is the only place to record the context: the bug being fixed, the constraint being respected, the decision that was made.

Throttle login attempts to 5 per minute per IP

Brute-force attempts were succeeding in staging because the previous rate limiter only counted by username, not source IP. CVE-2026-1234.

Refs: #412

Stage deliberately, not with `git commit -a`. The `-a` flag bypasses the staging area entirely. It is convenient for solo work but can accidentally include debug code, half-finished changes, or unrelated fixes in the same commit.

Keep commits small and focused. Each commit should represent one logical change. Reviewers can understand and approve small commits; large omnibus commits make `git bisect` and `git revert` painful.

Use `--fixup` to keep history clean during review cycles. When a reviewer asks for a change to commit `abc1234`, add a fixup commit and autosquash when the branch is merged rather than amending in place.

Pitfalls & gotchas

Amending after pushing rewrites history. `--amend` produces a new commit object with a different hash. Any remote that already has the old commit will diverge. Only amend commits that exist solely on your local machine. If you have already pushed, ask yourself whether a follow-up commit is acceptable instead.

`-a` does not stage new (untracked) files. It stages modifications and deletions of already-tracked files. A new file you created this session is invisible to `-a` you must `git add` it explicitly.

Empty commits are allowed with `--allow-empty` but confuse tools. `git bisect`, `git cherry-pick`, and some CI systems treat every commit as potentially meaningful. Use empty commits sparingly and comment why in the message.

Hooks run by default. The pre-commit hook runs linters, formatters, or tests. If it fails, the commit is aborted — this is intentional. Use `--no-verify` only as a last resort and fix the underlying issue as soon as possible.

The `--date` flag sets the author date, not the committer date. If you need to set both, use `GIT_COMMITTER_DATE` in the environment.

Worked examples

Staging only part of a file

You have made two unrelated edits to `api.js` — a bug fix and a refactor — and want them in separate commits.

```
git add -p api.js
```

Git presents each changed hunk and asks: stage this hunk? (`y` yes, `n` no, `s` split into smaller hunks, `e` edit manually). Stage just the bug-fix hunks and commit:

```
git commit -m "Fix null-pointer dereference in parseToken"
```

Then stage and commit the refactor:

```
git add api.js
git commit -m "Extract token validation into a helper"
```

Cleaning up a branch with fixup commits

Your branch has three commits. A reviewer asks you to fix a typo that was introduced in the first commit (`a3f9c1`).

```
git add src/config.js
git commit --fixup=a3f9c1
```

Git creates a commit titled `fixup! Add environment config loader`.

When the branch is approved and ready to merge, squash the fixup in:

```
git rebase -i --autosquash origin/main
```

Git moves the `fixup!` commit immediately after `a3f9c1` and marks it for squash.

The result is a clean history without the typo ever appearing.

Signing a commit with GPG

Assuming a GPG key is configured (`user.signingKey` in git config):

```
git commit -S -m "Release v1.4.0"
```

Verify the signature:

```
git log --show-signature -1
```

GitHub displays a “Verified” badge next to signed commits when the key is uploaded to your account.

Recovery

To undo the last commit while keeping the changes staged:

```
git reset --soft HEAD~1
```

To undo the last commit and unstage the changes (but keep them in the working tree):

```
git reset HEAD~1
```

See *Getting out of jams* for more undo recipes, including recovering commits that have been amended away.

See also

- *What git really is* — the index (staging area) explained.
- *add* — building the index before committing.
- *rebase* — `--autosquash` to collapse fixup commits.
- *Getting out of jams* — undoing commits safely.

restore

Restore working tree files or index entries to a known state without moving the current branch.

Mental model

Git tracks three places where a file's content can live:

1. **The repository** — committed history, addressed by commit hashes, branch names, or tags.
2. **The index (staging area)** — the draft of the next commit, built up with `git add`.
3. **The working tree** — the files you actually edit on disk.

`git restore` is a targeted copy operation between those layers. You name a source (a commit, a tree, or the index itself) and a destination (the working tree, the index, or both), and Git copies the content across. The branch pointer and commit history are never touched.

```
Repository ---source=<tree>--> Index (--staged) --> Working tree (default)
Index (default source) -----> Working tree (default)
```

Before `git restore` was introduced in Git 2.23, these operations were split across `git checkout` (working tree) and `git reset` (index). The dedicated command makes intent explicit: restoring content is a separate concern from switching branches or rewriting history.

Note: The man page still marks this command as experimental. The interface has been stable in practice since Git 2.23, but the `--help` text carries the caveat.

Synopsis

```
git restore [<options>] [--source=<tree>] [--staged] [--worktree] [--]
<pathspec>...
git restore [<options>] [--source=<tree>] [--staged] [--worktree]
--pathspec-from-file=<file> [--pathspec-file-nul]
git restore (-p|--patch) [<options>] [--source=<tree>] [--staged] [--worktree]
[--] [<pathspec>...]
```

Everyday usage

Discard uncommitted edits to a file in the working tree:

```
git restore README.md
```

Unstage a file (move it out of the index without touching the working tree):

```
git restore --staged README.md
```

Unstage and also discard the working-tree changes in one step:

```
git restore --staged --worktree README.md
```

Restore a file to its state in a specific commit:

```
git restore --source=HEAD~2 src/config.js
```

Discard all uncommitted changes in the current directory:

```
git restore .
```

Discard all uncommitted changes in the entire repository regardless of where you are in the tree:

```
git restore :/
```

Interactively choose which hunks to discard:

```
git restore -p src/api.js
```

Key options

Option	What it does	When to use it
<code>--staged / -S</code>	Restore the index (unstage)	Undo a <code>git add</code>
<code>--worktree / -W</code>	Restore the working tree (default when neither flag given)	Explicitly target the working tree alongside <code>--staged</code>
<code>--source=<tree> / -s <tree></code>	Use a commit, branch, or tag as the source instead of the index	Pull an older version of a file into the working tree or index
<code>--patch / -p</code>	Interactively select hunks to restore	Discard only part of a file's changes
<code>--ours</code>	When restoring from the index during a merge conflict, use the ours (stage 2) version	Resolve a conflict by keeping your branch's side
<code>--theirs</code>	Use the theirs (stage 3) version from the index	Resolve a conflict by accepting the incoming side
<code>--merge / -m</code>	Recreate the three-way conflict markers for unmerged paths	Re-open a conflict you accidentally resolved
<code>--conflict=<style></code>	Override the conflict presentation style (<code>merge</code> , <code>diff3</code> , <code>zdiff3</code>)	Prefer <code>diff3</code> for more context around conflict markers
<code>--ignore-unmerged</code>	Skip unmerged paths without aborting	Restore a subset of files when a merge is in progress
<code>--overlay</code>	Never remove files when restoring	Prevent <code>--source</code> from deleting files absent in the source tree
<code>--no-overlay</code>	Remove tracked files absent from the source tree (default)	Make the working tree exactly match a given tree
<code>--recurse-submodules</code>	Also restore submodule working trees	Update a submodule to the commit recorded in the superproject
<code>--pathspec-from-file=<file></code>	Read paths from a file instead of the command line	Automate restores with long or generated path lists
<code>--quiet / -q</code>	Suppress progress messages	Scripting

Best practices

Prefer `git restore` over `git checkout -- <file>`. The older form mixes path-restoring and branch-switching in one command, making the intent unclear to readers and error-prone in scripts. `git restore` says exactly what it does.

Use `--staged` first, then `--worktree` separately when unsure. Running `git restore --staged --worktree` discards both your staged and unstaged work in one shot — an operation you cannot undo. If you only want to unstage, leave `--worktree` off. You can always discard the working-tree changes afterward if you decide to.

Use `--source` to safely examine an old file version. When you need to compare or copy logic from a file at an older commit, restore it with `--source=<commit>` and then decide whether to keep the result or revert to the index. The old commit is untouched.

Pair `--patch` with careful review. Interactive hunk selection (the same UI used by `git add -p`) lets you discard only the noise in a file while keeping meaningful changes. Treat each hunk as a decision, not a formality.

Quote globs to let Git expand them. When restoring all files matching a pattern (e.g., `'*.c'`), quote the glob. If the shell expands it, only files already in the working tree are matched; if Git expands it, files that were deleted but still exist in the index are also restored.

```
# correct: Git resolves the glob against the index
git restore '*.c'

# risky: the shell may miss deleted files
git restore *.c
```

Pitfalls & gotchas

Restoring the working tree is destructive and usually unrecoverable. `git restore` (without `--staged`) overwrites your working-tree file. The content is not committed, not stashed, and not in any reflog. Verify with `git diff` before running it if there is any chance you want those changes.

`--staged` alone does not touch the working tree. After `git restore --staged README.md`, the file in your editor still shows the staged content. The index is reset to HEAD, but your working-tree file is unchanged. This is what you want when unstaging; remember it when you expected everything to revert.

Without `--source`, `--staged` restores from HEAD, not the previous index state. If you staged a change and then want the index to reflect an earlier commit rather than HEAD, you must specify `--source=<commit>` explicitly.

`--ours` and `--theirs` are swapped during a rebase. During `git rebase`, “ours” is the branch being rebased onto and “theirs” is the commit being replayed — the opposite of a plain merge. If a resolution looks backwards, check which operation is in progress.

`--source` with `--no-overlay` (the default) can delete files. If the source tree does not contain a path that currently exists in the working tree, Git removes it. Pass `--overlay` to suppress deletions and only update files that exist in the source.

The `--staged --worktree` combination has no safety net. Unlike `git stash`, there is no way to recover from `git restore --staged --worktree` after the fact. Use `git stash` instead when there is any uncertainty — see the *stash* chapter.

Worked examples

Discarding accidental edits before a commit

You opened `src/db.js` to read some logic and accidentally saved changes you do not want. The working tree is dirty but nothing is staged.

```
git diff src/db.js          # confirm the noise
```

```
diff --git a/src/db.js b/src/db.js
index 4a2f1b3..7c9e801 100644
--- a/src/db.js
+++ b/src/db.js
@@ -42,6 +42,7 @@ function connect(url) {
+  console.log('debug');    // accidentally saved
  return pool.acquire();
```

Discard the change:

```
git restore src/db.js
git diff src/db.js          # empty – working tree matches index
```

Unstaging a file added by mistake

You ran `git add .` and swept in a generated file you did not intend to commit.

```
git status
```

```
On branch feature/auth
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   dist/bundle.js
    modified:   src/auth.js
```

Unstage the generated file while keeping `src/auth.js` staged:

```
git restore --staged dist/bundle.js
git status
```

```
On branch feature/auth
Changes to be committed:
    modified:   src/auth.js

Untracked files:
  dist/bundle.js
```

`dist/bundle.js` is back to untracked. Add it to `.gitignore` to prevent a repeat.

Pulling an old version of a file for comparison

A performance regression was introduced somewhere in the last week. You want to restore the database module to an older state to run benchmarks.


```
# find the commit from around last week
git log --oneline --since="7 days ago" -- src/db.js
```

```
a3f9c12 Refactor connection pool initialisation
8b2e041 Add retry logic on transient errors
```

Restore the file as it was just before the refactor:

```
git restore --source=8b2e041 src/db.js
```

The working tree now contains the older version. The index is unchanged — run

`git diff --staged` to confirm. If the benchmarks confirm a regression, commit the restored file or use it to guide a targeted fix. If not, throw it away:

```
git restore src/db.js      # back to the version in the index
```

Resolving a merge conflict by choosing one side

During a merge, `CHANGELOG.md` has conflicting edits on both branches. You decide the incoming branch has the correct version.

```
git restore --theirs CHANGELOG.md
git add CHANGELOG.md
git merge --continue
```

To choose your branch's version instead:

```
git restore --ours CHANGELOG.md
git add CHANGELOG.md
git merge --continue
```

Recovery

`git restore` on the working tree cannot be undone through Git — there is no reflog for unstaged file content. Before running a broad restore, stash your changes as a safety net:

```
git stash push -u -m "before bulk restore"
git restore .
# if you change your mind:
git stash pop
```

If you have already run `git restore --staged`, the working-tree content is intact. Simply re-stage the file:

```
git add <file>
```

If you ran `git restore --staged --worktree` and the content is gone, check whether your editor has an undo history or whether the OS has a recent backup. Git has no record of content that was never committed.

See *Getting out of jams* for broader undo recipes, including recovering content from the stash and the reflog.

See also

- *add* — building the index that `--staged` restores from.
- *commit* — the snapshot that `--source=HEAD` and other refs point to.
- *stash* — a safe alternative when you want to set aside changes rather than discard them.
- *checkout* — the older command that `git restore` and `git switch` replaced for path-based and branch-switching operations respectively.
- *Getting out of jams* — recovering from destructive operations.

reset

Move the current branch pointer to a different commit, with optional control over whether the index and working tree follow along.

Mental model

Git tracks three distinct snapshots at any moment:

- **HEAD** — the commit the current branch points to (the repository).
- **Index** — the staging area; a draft of the next commit.
- **Working tree** — files as they exist on disk.

`git reset` is a dial that controls how far those three layers are rewound toward a target commit. The dial has three main positions:

		HEAD	Index	Working tree	
<code>--soft</code>	rewind	yes	no	no	
<code>--mixed</code>	rewind	yes	yes	no	(default)
<code>--hard</code>	rewind	yes	yes	yes	

Moving HEAD backward is always safe — the commits you pass over are not deleted, just no longer reachable from the branch tip. The index and working tree are where data can be lost: `--hard` discards changes in both without a second prompt.

A second, distinct use of `reset` targets individual paths rather than the whole tree.

`git reset <pathspec>` copies entries from HEAD back into the index without touching the working tree or moving HEAD at all. That is the opposite of `git add <pathspec>`, and is the canonical way to unstage a file.

```
Unstage:  git add README  —————>  index has README change
          git reset README <————  index reverts to HEAD's README
```

Before every whole-branch reset, Git writes the current HEAD into `ORIG_HEAD`, giving you a one-step escape hatch.

Synopsis

```
# Move branch pointer (whole-tree forms)
git reset [--soft | --mixed [-N] | --hard | --merge | --keep] [-q] [<commit>]

# Unstage paths (path forms — never moves HEAD)
git reset [-q] [<tree-ish>] [--] <pathspec>...
git reset [-q] [--pathspec-from-file=<file> [--pathspec-file-nul]] [<tree-ish>]

# Interactively unstage hunks
git reset (--patch | -p) [<tree-ish>] [--] [<pathspec>...]
```

`<commit>` defaults to HEAD when omitted.

Everyday usage

Unstage a file you added by mistake:

```
git add oops.log
git reset oops.log          # remove from index, keep on disk
```

Undo the last commit, keep changes staged:

```
git reset --soft HEAD~1
```

The files are still staged exactly as they were; you can edit and re-commit.

Undo the last commit, unstage the changes (but keep files on disk):

```
git reset HEAD~1          # --mixed is the default
```

Use this when you want to re-examine what you committed before deciding how to split it up.

Discard the last commit and all changes completely:

```
git reset --hard HEAD~1
```

This is destructive. Anything not committed is gone.

Unstage part of a commit interactively:

```
git reset -p              # hunk-by-hunk: remove hunks from the index
```

Each hunk is presented and you choose whether to unstage it. This is the mirror image of `git add -p`.

Abort a botched merge that has not been committed:

```
git reset --hard ORIG_HEAD
```

Key options

Option	What it does	When to use it
<code>--soft</code>	Moves HEAD only; index and working tree unchanged	Re-do the last commit: change the message or add more files
<code>--mixed</code>	Moves HEAD and resets the index; working tree unchanged	Default; unstage everything since the target commit
<code>--hard</code>	Moves HEAD, resets index, and overwrites working tree	Throw away a commit and all uncommitted changes completely
<code>--merge</code>	Resets the index and updates working-tree files that differ between <code><commit></code> and HEAD, but preserves files that differ between the index and working tree (i.e. unstaged changes); aborts if a file that differs between <code><commit></code> and the index also has unstaged changes	Back out of a failed merge while keeping your in-progress edits

<code>--keep</code>	Resets index and working-tree files that differ between HEAD and target; aborts if any such file has local changes	Remove recent commits while protecting local edits from being overwritten
<code>-N</code>	With <code>--mixed</code> : marks removed paths as intent-to-add instead of truly removing them	Split a commit while preserving newly added files for <code>git add -p</code>
<code>-p / --patch</code>	Interactively select hunks to remove from the index	Partially unstage a file, hunk by hunk
<code>-q / --quiet</code>	Suppress output; report errors only	Scripts and hooks
<code>--pathspec-from-file=<file></code>	Read path list from a file instead of the command line	Scripted bulk unstage from a generated file list

Best practices

Prefer `--soft` or `--mixed` for commits that have not been pushed. Both modes preserve your work in a recoverable state. Reserve `--hard` for situations where you are certain you want the working-tree changes gone.

Reset to a symbolic reference rather than a raw hash. `HEAD~1` , `HEAD~3` , and `ORIG_HEAD` are self-documenting. Raw hashes typed by hand are opaque and error-prone.

Check `git status` immediately after `--mixed`. After unwinding commits, the index is clean and the changes appear as unstaged modifications. Glancing at `git status` before re-staging confirms you understand exactly what was unwound.

Use `ORIG_HEAD` as your safety net. Every whole-tree `reset` records the previous HEAD in `ORIG_HEAD` . If you immediately regret the reset, `git reset --hard ORIG_HEAD` brings you back. This only works before you do another reset that overwrites `ORIG_HEAD` .

Do not reset commits that have been pushed to a shared branch. Resetting rewrites the branch tip. Any collaborator who already pulled those commits will have a diverged history. Use `git revert` instead when you need to undo work on a public branch — it adds a new commit that reverses the changes without rewriting history.

Prefer `git restore --staged <file>` for single-file unstaging. For modern Git (2.23+), `git restore --staged <file>` is more explicit than `git reset <file>` for the path-form use case. Both are correct; `restore` makes the intent harder to misread.

Pitfalls & gotchas

`--hard` permanently discards uncommitted changes. There is no undo for working-tree changes that were never committed or stashed. Before running `--hard` , verify with `git status` that you have nothing valuable unstaged or untracked.

`--hard` deletes untracked files that block writing a tracked file. If a tracked file was deleted in HEAD but you have an untracked file with the same name in the working tree, `--hard` removes the untracked file to restore the tracked version.

`--soft` on a merge commit leaves the index in a merged state. If HEAD is a merge commit and you reset `--soft HEAD~1` , you land on one parent while the index still contains the merged tree. That is usually not what you want; use `--mixed` instead.

Path-form reset does not move HEAD. Running `git reset HEAD~1 src/` copies `src/` from `HEAD~1` into the index but leaves the branch pointer where it is. It looks like `--mixed` but only for the specified paths.

--merge and --keep both abort on certain dirty states. Neither will proceed if the conditions for safely preserving local changes cannot be met. Read the error message — it tells you which file caused the abort.

ORIG_HEAD is overwritten by the next whole-tree reset. If you run two resets in a row, the first `ORIG_HEAD` is gone. Recover with `git reflog` instead.

Resetting in a detached HEAD state moves HEAD itself, not a branch. If you are not on a named branch, `reset` still works but the commits you leave behind are only reachable via reflog. Create a branch before resetting if you want to keep those commits.

Worked examples

Splitting an over-stuffed commit into two

You committed a bug fix and a refactor together. Now you want them as separate commits.

```
# Undo the last commit; keep all changes unstaged in the working tree
git reset HEAD~1
```

```
Unstaged changes after reset:
M   src/api.js
M   src/utils.js
```

Stage and commit just the bug fix:

```
git add -p src/api.js      # choose only the bug-fix hunks
git commit -m "Fix null-pointer dereference in parseToken"
```

Then commit the refactor:

```
git add src/api.js src/utils.js
git commit -m "Extract token validation into a helper"
```

Rescuing work after an accidental --hard reset

You ran `git reset --hard HEAD~2` and immediately realized you needed one of those commits.

```
# Find the lost commits in the reflog
git reflog
```

```
d4e8b3a HEAD@{0}: reset: moving to HEAD~2
a1c72f9 HEAD@{1}: commit: Add payment retry logic
7f301bb HEAD@{2}: commit: Fix cart total rounding
e9a0c44 HEAD@{3}: checkout: moving from develop to feature/cart
```

The two lost commits are at `HEAD@{1}` and `HEAD@{2}`. Restore the branch to where it was before the reset:

```
git reset --hard HEAD@{1}
```

Or recover just one specific commit:

```
git cherry-pick 7f301bb
```

Undoing a faulty merge while preserving local edits

You pulled from upstream and the merge introduced conflicts you want to discard, but you also have in-progress local edits that have not been staged.

```
git pull origin main
```

```
Auto-merging checkout.js
CONFLICT (content): Merge conflict in checkout.js
Automatic merge failed; fix conflicts and then commit the result.
```

You decide you are not ready to merge yet and want to return to your pre-pull state without losing your in-progress edits:

```
git reset --merge ORIG_HEAD
```

`--merge` resets the files involved in the merge back to their pre-pull state while leaving your other unstaged edits untouched. `--hard ORIG_HEAD` would have discarded those edits.

Snapshot-and-return workflow for interrupted work

You are mid-feature when an urgent fix is needed on `main`. Your working tree is not in a committable state, but you need to switch branches cleanly.

```
# Save a rough snapshot commit (message does not matter)
git commit -a -m "WIP snapshot"
git switch main

# ... apply the urgent fix, commit, switch back ...

git switch feature/my-work
# Dissolve the snapshot: puts HEAD back one commit, changes stay unstaged
git reset HEAD~1
```

Your working tree is exactly as you left it and there is no stray WIP commit polluting the branch history. See the *stash* chapter for a lighter-weight alternative that avoids the snapshot commit entirely.

Recovery

If you reset the branch pointer too far back, recover the original tip from the reflog:

```
git reflog          # find the entry just before the reset
git reset --hard HEAD@{1} # substitute the correct reflog index
```

If you used `--hard` and lost uncommitted working-tree changes, there is no general Git recovery path — those changes were never stored as objects. Check whether your editor has a local history buffer (VS Code's Timeline, JetBrains' Local History) before concluding they are gone.

See *Getting out of jams* for more undo recipes, including recovering commits that were amended or rebased away and untangling diverged branches after an accidental push of a rewritten branch.

See also

- *commit* — the command `reset --soft` most commonly precedes; understanding how commits are built makes reset behavior clearer.
- *add* — `git reset <pathspec>` is the exact inverse of `git add <pathspec>`.
- *restore* — the modern, more explicit alternative for the path-form of reset (`git restore --staged <file>`).
- *revert* — the safe alternative to `reset` on already-pushed commits; adds a new commit rather than rewriting history.
- *rebase* — interactive rebase with `edit` or `drop` achieves many of the same history-rewriting goals as reset, with a step-by-step interface.
- *stash* — a lighter-weight alternative to the snapshot-and-reset pattern when you need to park in-progress changes temporarily.
- *Getting out of jams* — step-by-step rescue procedures for the most common reset-related disasters.

rm

Remove files from the index, or from both the working tree and the index, and stage that removal for the next commit.

Mental model

When you delete a file with your operating system's `rm` command, the working tree copy disappears but Git does not know about it yet. The index still lists the file, so `git status` shows it as “deleted” (unstaged). You would then have to run `git add` on the deletion to stage it, or `git commit -a` to pick it up automatically.

`git rm` collapses those two steps into one: it removes the file from the working tree *and* stages the removal, so your next `git commit` will record it as gone.

```
File tracked by Git
├──
├── os rm → file gone from disk, deletion NOT staged
│   └── git add <path> → deletion staged → commit
├──
└── git rm → file gone from disk, deletion staged → commit
```

The `--cached` flag breaks the symmetry: it removes the file from the index only, leaving the working tree copy intact. This is the right move when you accidentally committed a file that should have been in `.gitignore`.

Git refuses to remove a file whose index entry differs from either the last commit or the current working tree, because that would discard unsaved work. Pass `-f` to override that safety check when you are sure.

Synopsis

```
git rm [-f | --force] [-n] [-r] [--cached] [--ignore-unmatch]
      [--quiet] [--pathspec-from-file=<file>] [--pathspec-file-nul]
      [--] [<pathspec>...]
```

Everyday usage

Remove a file from the working tree and stage the deletion:

```
git rm src/legacy.js
git commit -m "Remove legacy.js"
```

Remove a file from Git's tracking without deleting it from disk (the most common use after updating `.gitignore`):

```
git rm --cached config/secrets.yml
echo "config/secrets.yml" >> .gitignore
git commit -m "Stop tracking secrets.yml"
```

Remove an entire directory recursively:

```
git rm -r build/
git commit -m "Remove generated build artifacts"
```

Preview what would be removed without touching anything:

```
git rm -n -r dist/
```

Remove files whose names might look like options:

```
git rm -- -oddly-named-file.txt
```

Key options

Option	What it does	When to use it
<code>--cached</code>	Remove from the index only; leave the working tree file alone	Untrack a file without deleting it (e.g. after updating <code>.gitignore</code>)
<code>-f</code> , <code>--force</code>	Override the up-to-date check	Remove a file with uncommitted local changes you are sure you do not need
<code>-n</code> , <code>--dry-run</code>	Show what would be removed without removing anything	Verify a glob or directory removal before committing to it
<code>-r</code>	Allow recursive removal of a directory	Required whenever a directory name is given instead of individual files
<code>--ignore-unmatch</code>	Exit with status 0 even when no files match the pathspec	Scripts that should not fail when the file is already gone
<code>-q</code> , <code>--quiet</code>	Suppress the per-file <code>rm</code> lines on stdout	Batch operations where the output would be noisy
<code>--pathspec-from-file=<file></code>	Read pathspecs from a file instead of the command line	Large lists of files, or paths generated by another command
<code>--pathspec-file-nul</code>	Use NUL as the separator in <code>--pathspec-from-file</code>	Filenames that contain newlines or spaces
<code>--sparse</code>	Allow updating index entries outside the sparse-checkout cone	Sparse-checkout workflows

Best practices

Prefer `git rm` over `rm` + `git add` for tracked deletions. The two-step approach works, but `git rm` is shorter, harder to forget halfway, and makes your intent immediately visible in the shell history.

Use `--cached` whenever you update `.gitignore`. If a file is already tracked, adding its path to `.gitignore` does nothing — Git continues to track changes to it. You must untrack it with `git rm --cached <path>` first. The file stays on disk; only Git’s awareness of it is removed.

Always pass `-n` before a broad glob or directory removal. Globbing in Git can behave differently from the shell (see *Pitfalls & gotchas*). A quick dry run costs nothing and prevents accidentally removing files you meant to keep.

Commit the removal promptly. `git rm` stages a deletion. Until you commit, the removal is sitting in the index as an uncommitted change. Other commands such as `git checkout` or `git reset` can disturb it. Stage the removal and commit in the same logical step.

Use `--ignore-unmatch` in scripts and cleanup routines. When a cleanup script runs `git rm` on paths that may or may not exist, the default non-zero exit on “no match” will abort the script. `--ignore-unmatch` makes the operation idempotent.

Pitfalls & gotchas

Git globbing crosses directory boundaries; shell globbing does not. When you quote the glob (`git rm 'src/*.js'`), Git expands it and will match files in any subdirectory of `src`. When you leave it unquoted (`git rm src/*.js`), the shell expands it first and matches only the top-level `src/` entries. The manual example `git rm 'd*'` would also match a sibling directory `d2/` — a surprise if you only intended `d/`. Prefer explicit paths or use `-n` to preview.

`git rm` will not remove only from the working tree. There is no flag to delete a file from disk while leaving the index entry untouched. Use your system’s `rm` for that. `git rm` always touches the index.

Removing a file with uncommitted local changes fails by default. Git compares the working tree content against the index and the last commit. If either differs, `git rm` refuses with an error like:

```
error: the following file has local modifications:
  src/important.js
(use --cached to keep the file, or -f to force removal)
```

Read the hint: `--cached` keeps the file on disk, `-f` discards changes and removes it from disk. Think before using `-f`.

Removing a submodule is not the same as removing a regular directory. `git rm` handles submodules that use a gitfile (`.git` as a file, not a directory) by moving the submodule’s repository into the superproject’s `.git` directory and removing the entry from `.gitmodules`. Submodules that still have a `.git` directory instead of a gitfile will not have their working tree removed. Use `git submodule deinit` before `git rm` when in doubt.

Files unknown to Git are silently skipped. `git rm` only operates on paths known to Git. If you pass an untracked file, the command exits non-zero. This is usually what you want, but it can confuse scripts that manage a mixed set of tracked and untracked files.

Worked examples

Untracking a file that was accidentally committed

A developer committed `config/database.yml` before adding it to `.gitignore`. Other contributors have pulled the branch. The goal is to stop tracking the file without breaking anyone’s local copy.

```
# Verify the file is tracked
git ls-files config/database.yml

# Remove it from Git's index, keep the disk copy
git rm --cached config/database.yml

# Prevent future commits from picking it up
```

```
echo "config/database.yml" >> .gitignore
git add .gitignore

git commit -m "Stop tracking database.yml; add to .gitignore"
```

After others pull this commit, their local `config/database.yml` remains on disk but is no longer tracked. They should ensure they have their own local copy if they do not already.

Cleaning up a directory of generated files

A `dist/` directory was committed by mistake. It contains dozens of generated files across several subdirectories. Preview first, then remove.

```
# See what would go
git rm -n -r dist/
```

```
rm 'dist/app.js'
rm 'dist/app.css'
rm 'dist/vendor/react.min.js'
...
```

The output looks right. Proceed:

```
git rm -r dist/
git commit -m "Remove dist/ from version control"
```

Add `dist/` to `.gitignore` so it stays out of the repository:

```
echo "dist/" >> .gitignore
git add .gitignore
git commit -m "Ignore dist/ directory"
```

Removing files from a script idempotently

A release script cleans up old per-change log fragments after they have been folded into the main `CHANGELOG.md`. The fragments may or may not exist on a given run.

```
git rm --ignore-unmatch --quiet \
    changelogs/unreleased/*.md
```

With `--ignore-unmatch`, the script succeeds whether zero or fifty files matched. With `--quiet`, the per-file output is suppressed so only real errors surface.

Recovery

If you ran `git rm` by mistake but have not yet committed, restore the file to both the index and the working tree:

```
git restore --staged --worktree <path>
```

To restore an entire directory:

```
git restore --staged --worktree -- <directory>/
```

If you used `git rm --cached` (index-only removal), the working tree copy is still on disk. Restore just the index entry:

```
git restore --staged <path>
```

If you already committed the removal, bring the file back by checking it out from the previous commit and committing the restoration:

```
git checkout HEAD~1 -- <path>
git commit -m "Restore <path>"
```

See *Getting out of jams* for broader undo recipes, including recovering from accidental force-removals.

See also

- *add* — staging changes, including deletions via `git add -u`.
- *commit* — recording staged deletions as a permanent commit.
- *restore* — recovering files from the index or a commit without a full revert.
- *Getting out of jams* — undoing staged and committed removals.

mv

Move or rename a file, directory, or symlink and immediately stage the result.

Mental model

When you rename a file outside of Git — with your OS file manager or the shell `mv` command — Git sees two unrelated events: the old path was deleted and a new, unrelated path was created. It cannot know they are the same file. `git status` shows one deletion and one untracked addition, and the index records them as two separate changes. Worse, tools that display rename detection (like `git log --follow`) have to guess the relationship later from content similarity.

`git mv` solves this by combining the filesystem rename with an index update in one step:

1. Renames the file on disk (the same operation your shell `mv` would do).
2. Stages the deletion of the old path.
3. Stages the addition of the new path.

After `git mv`, `git status` shows the change as a single `renamed:` entry and the index already reflects the intended final state. You still need to `git commit` to record the rename in history.

Before:	After <code>git mv old.txt new.txt</code> :
working tree: old.txt	working tree: new.txt
index: old.txt	index: old.txt -> new.txt (staged)
rename)	
HEAD: old.txt	HEAD: old.txt (unchanged until
commit)	

Git does not actually store rename events in its object model — it reconstructs rename history from content similarity at display time. `git mv` just makes that reconstruction reliable by ensuring the old and new paths appear together in the same commit with identical content.

Synopsis

```
git mv [-v] [-f] [-n] [-k] <source> <destination>
git mv [-v] [-f] [-n] [-k] <source>... <destination-directory>
```

In the first form, `<source>` is renamed to `<destination>`. In the second form, `<destination-directory>` must already exist and each `<source>` is moved into it.

Everyday usage

Rename a single file:

```
git mv README.txt README.md
git commit -m "Rename README to Markdown"
```

Move a file into an existing subdirectory:

```
git mv logo.png assets/images/logo.png
git commit -m "Move logo into assets directory"
```

Move several files into a directory at once:

```
git mv auth.js session.js token.js src/auth/
git commit -m "Gather authentication modules under src/auth"
```

Rename a directory:

```
git mv lib/ src/
git commit -m "Rename lib to src"
```

Preview a move without touching anything:

```
git mv -n old-name.js new-name.js
```

```
Renaming old-name.js to new-name.js
```

Key options

Option	What it does	When to use it
<code>-f</code> , <code>--force</code>	Overwrite <code><destination></code> if it already exists	Replace a tracked file with a renamed one
<code>-k</code>	Skip sources that would cause an error rather than aborting	Batch moves where some sources may not exist or may not be tracked
<code>-n</code> , <code>--dry-run</code>	Show what would happen without making any changes	Verify paths before a destructive rename
<code>-v</code> , <code>--verbose</code>	Print each file name as it is moved	Confirm progress during a large batch move

Best practices

Commit renames in their own commit, separate from content edits. When a rename and a content change land in the same commit, `git diff` and code review tools have to do extra work to distinguish intent from implementation. A reviewer seeing `src/util.js` renamed to `src/helpers/format.js` plus a content change may miss a subtle bug. Put the rename commit first, then follow with the content change.

Check the destination does not already exist before moving. If `<destination>` is a tracked file and you run `git mv` without `-f`, Git refuses. Using `-f` silently discards the destination file. Use `git mv -n` first to confirm the target path is free.

Use `git mv` rather than shell `mv` plus manual staging. You can reach the same end state with `mv old.txt new.txt && git rm old.txt && git add new.txt`, but `git mv` is less error-prone and communicates intent clearly in your shell history. It also handles symlinks and the `.gitmodules` update for submodules automatically.

For large restructures, move in small batches. Moving a hundred files in one commit is hard to review. Break the restructure into logical groups — for example, one commit per top-level directory — so reviewers can follow the reasoning.

After moving a submodule, run `git submodule update`. Git updates `.gitmodules` and the submodule's `core.worktree` setting automatically, but does not repopulate the new location. Run `git submodule update` to check out the submodule content at the new path.

Pitfalls & gotchas

Moving an untracked file fails. `git mv` only operates on paths that are already tracked. If a file has never been staged, Git treats it as non-existent and refuses to move it. Add it first, then move it:

```
git add newfile.txt
git mv newfile.txt dest/newfile.txt
```

Alternatively, move it with the shell and `git add` the new path — since it was never tracked, there is no old index entry to clean up.

The destination directory must already exist. `git mv` will not create intermediate directories. If `src/auth/` does not exist yet, the move fails. Create it first:

```
mkdir -p src/auth
git mv auth.js src/auth/
```

Git does not track empty directories, so you may want to add a `.gitkeep` file if the directory needs to be preserved before other files arrive.

-k silently skips errors — verify the result. In a batch operation with `-k`, any source that is untracked or would overwrite an existing file is silently skipped rather than aborting. Always run `git status` after a `-k` batch to confirm every intended move happened.

Rename detection has limits. Git detects renames at display time when content similarity between the old and new path exceeds a threshold (50% by default). If you rename a file and make extensive edits in the same commit, Git may fail to recognise it as a rename in `git log --follow` or `git diff`. Keeping the rename commit free of content edits avoids this.

Case-only renames fail silently on case-insensitive filesystems. On macOS and Windows, renaming `Config.js` to `config.js` may appear to succeed but the OS never actually changes the filename on disk. Use a two-step rename through a temporary name:

```
git mv Config.js config.js.tmp
git mv config.js.tmp config.js
git commit -m "Lowercase Config.js filename"
```

Worked examples

Reorganising a flat source tree into subdirectories

All source files live in the project root. You want to move them into `src/`.

```
mkdir src
git mv app.js utils.js router.js src/
git status
```

```
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    renamed:    app.js -> src/app.js
    renamed:    router.js -> src/router.js
    renamed:    utils.js -> src/utils.js
```



```
git commit -m "Move source files into src directory"
```

Later, `git log --follow src/app.js` traces history back through the rename to the original `app.js` commits.

Renaming a file with a case-only change on a case-insensitive filesystem

The file is currently `UserModel.js` but the team convention is lowercase filenames. A direct rename will not work on macOS or Windows.

```
# Step 1: rename to a temporary name
git mv UserModel.js usermodel_tmp.js

# Step 2: rename from temp to the final lowercase name
git mv usermodel_tmp.js usermodel.js

git status
```

```
On branch main
Changes to be committed:
  renamed:   UserModel.js -> usermodel.js
```

```
git commit -m "Rename UserModel.js to usermodel.js (enforce lowercase convention)"
```

Recovering when you already used shell mv

You ran `mv src/old.js src/new.js` in the shell before thinking about Git. The index now shows a deletion and an untracked file. Fix it:

```
git rm src/old.js
git add src/new.js
git status
```

```
On branch main
Changes to be committed:
  renamed:   src/old.js -> src/new.js
```

Git's rename detection recognises the content match and promotes the pair to a `renamed:` entry. The effect is identical to having used `git mv` from the start. Commit normally.

Recovery

To undo a staged rename before committing, move the file back:

```
git mv src/new.js src/old.js
```

Or use `restore` to unstage both sides and return the working tree to the state at HEAD:

```
git restore --staged src/old.js src/new.js
git restore src/new.js    # remove the new path from the working tree
# src/old.js is back in place
```

If you have already committed a rename and want to reverse it, use *revert* to create a new commit that undoes it (see the *revert* chapter). If the commit exists only on your local machine, `git reset HEAD~1` brings it back to the staging area so you can re-rename before recommitting (see the *reset* chapter).

See *Getting out of jams* for broader undo recipes involving staged changes.

See also

- *add* — staging changes manually when you have already renamed with the shell.
- *rm* — the counterpart for removing tracked files from the index.
- *status* — verifying that renames appear as `renamed:` rather than separate delete/add pairs.
- *commit* — recording the staged rename into history.
- *restore* — unstaging a rename without losing working-tree changes.
- *reset* — rolling back a rename commit that has not yet been pushed.
- *revert* — safely undoing a rename that has already been shared.

clean

Remove untracked files from the working tree.

Mental model

Git tracks three kinds of files in your working tree: tracked files (Git knows about them), ignored files (listed in `.gitignore` or similar rules), and untracked files (everything else — new files you have created but never staged). `git clean` targets that third category.

Think of it as the complement to `git restore`. While `git restore` resets the content of tracked files back to what Git knows, `git clean` sweeps away the files Git does not know about at all.

Working tree files

├ tracked	← <code>git restore</code> handles these
├ ignored	← <code>git clean -x</code> or <code>-X</code> can reach these
└ untracked	← <code>git clean</code> targets these by default

Because deleted files cannot be recovered from Git's history (Git never recorded them), `git clean` is intentionally conservative: it requires an explicit force flag by default and will not descend into untracked subdirectories unless you ask. Always use `--dry-run` first to see exactly what would be removed.

Synopsis

```
git clean [-d] [-f] [-i] [-n] [-q] [-e <pattern>] [-x | -X] [--] [<pathspec>...]
```

Everyday usage

Preview what would be removed without actually deleting anything:

```
git clean -n
git clean -nd    # also show untracked directories
```

Remove untracked files (force is required by default):

```
git clean -f
```

Remove untracked files and untracked directories:

```
git clean -fd
```

Review interactively before committing to any deletion:

```
git clean -i
```

Remove everything that would be ignored by a fresh checkout — build artifacts, compiled output, editor caches:

```
git clean -fdx
```

Remove only ignored files, leaving your untracked source files alone:

```
git clean -fdX
```

Restrict cleaning to a specific path:

```
git clean -fd -- src/generated/
```

Key options

Option	What it does	When to use it
<code>-n</code> , <code>--dry-run</code>	Show what would be removed; delete nothing	Always run this first
<code>-f</code> , <code>--force</code>	Actually perform the deletion (required unless <code>clean.requireForce</code> is false)	Every real clean operation
<code>-d</code>	Recurse into untracked directories	When you also want to remove new folders
<code>-i</code> , <code>--interactive</code>	Show files and enter an interactive menu	When you want to approve or filter individual items
<code>-q</code> , <code>--quiet</code>	Suppress the list of removed files; report errors only	Scripts and CI where the output is noise
<code>-e <pattern></code> , <code>--exclude=<pattern></code>	Exclude additional paths matching <code><pattern></code> on top of standard ignore rules	Protect specific files during a broad clean
<code>-x</code>	Also remove files that are ignored by <code>.gitignore</code> rules	Wipe build products for a pristine rebuild
<code>-X</code>	Remove <i>only</i> ignored files, keep untracked source	Rebuild artifacts without touching hand-created files

`-x` and `-X` are mutually exclusive.

Best practices

Dry-run before every destructive clean. Make `git clean -nd` a habit. The two-second check has saved many developers from wiping a half-finished config file they forgot to stage.

Use `-i` when the file list is unfamiliar. The interactive mode lets you review each file, filter by pattern, or select by number before anything is deleted. It is the safest way to clean a directory you have not tidied recently.

Prefer `-X` over `-x` for rebuild workflows. If your goal is to ensure a fresh build, you almost always want to wipe build artifacts while keeping your untracked source files. `-X` does exactly that; `-x` is a broader sledgehammer that also removes untracked source you have not staged yet.

Protect files with `-e` when running broad cleans in CI. If your pipeline generates a credentials file or a local config that should survive the clean, pass `-e .env.local` (or the appropriate pattern) rather than relying on a `.gitignore` entry that might not be committed.

Leave `clean.requireForce = true` in place. The force requirement is a safety rail. Removing it in your global config means a mistyped command has no second chance.

Pitfalls & gotchas

Deleted files are gone for good. Unlike modifications to tracked files, there is no `git restore` that brings back a file Git never recorded. If you run `git clean -f` and remove a file you needed, your only options are your editor's undo history, a filesystem-level backup, or luck.

-d without a pathspec can be surprisingly aggressive. If you have created an entire feature directory without staging any of it, `git clean -fd` will delete every file in that directory. Always use `-n` first when `-d` is in play.

Nested Git repositories are protected. Git will refuse to remove a directory that contains its own `.git` subdirectory unless you pass `-f` twice (`-ff`). This protects accidentally vendored submodules and any directory that carries its own `.git` folder.

-x reaches everything your ignore rules cover, including directory-level patterns. A pattern like `build/` in `.gitignore` covers everything inside that directory. Running `git clean -fx` will remove the entire contents of `build/`. That is usually what you want, but be aware it reaches files you may not have listed individually.

Interactive mode ignores `clean.requireForce`. When you pass `-i`, the force requirement is waived because the interactive prompts provide their own safety guarantee. This is intentional behavior documented in the man page.

Worked examples

Preparing a pristine build environment

Your CI server shares a workspace between runs. Old object files and generated headers are causing spurious test failures. You want to remove every ignored artifact but leave any untracked source files the developer may have added.

```
# Preview what will be removed
git clean -ndX
```

```
Would remove build/
Would remove dist/
Would remove src/generated/schema.pb.h
Would remove .cache/
```

```
# Perform the clean
git clean -fdX
```

The build directory is now in the same state it would be after a fresh clone, minus only the ignored files.

Surgically removing untracked files with interactive mode

You have been experimenting and your working tree has several new files: a scratch script, a test fixture, and a notes file you want to keep.

```
git clean -i
```

```
Would remove scratch.py
Would remove tests/fixtures/sample_big.json
```

```
Would remove NOTES.md
*** Commands ***
    1: clean                2: filter by pattern    3: select by numbers
    4: ask each             5: quit                  6: help
What now>
```

Choose option 3 (select by numbers) and enter `1 2` to select only `scratch.py` and the fixture. Press Enter with an empty line to return to the menu, then choose `1` to clean. `NOTES.md` is left untouched.

Excluding a file during a broad clean

You want to wipe all untracked and ignored files but keep a local `.env.local` file that is not committed and not in `.gitignore`:

```
# Preview
git clean -ndx -e .env.local

# Execute
git clean -fdx -e .env.local
```

The `-e` pattern takes precedence over `-x`, so `.env.local` survives even though everything else that is untracked or ignored is removed.

Recovery

There is no Git-native recovery path for files removed by `git clean`. Git never recorded them, so there is nothing to restore from the object database.

Before running any destructive clean, stash untracked files to create a recoverable snapshot:

```
git stash --include-untracked
```

See *stash* for how to pop or drop the stash once you are done. See *Getting out of jams* for broader recovery recipes including situations where the stash itself has been dropped.

If you have already run `git clean -f` and realize you needed a file, check:

1. Your editor's local history (VS Code, JetBrains, and others keep per-file undo history on disk independently of Git).
2. Your operating system's trash or Time Machine snapshot.
3. Any open terminal buffer that may still show the file's contents.

See also

- *restore* — reset the content of tracked files to their last committed state.
- *stash* — temporarily shelve untracked and modified files before a clean.
- *Getting out of jams* — recovering from accidental deletions and other destructive operations.

stash

Shelve uncommitted changes so you can switch context, then restore them later.

Mental model

Git requires a clean working tree for many operations — switching branches, pulling upstream changes, applying a patch. `git stash` solves this without forcing you to make a half-baked commit.

Under the hood, a stash entry is two commit objects stored outside normal branch history:

```
      .---W   (working tree state)
     /   /
    -----H---I   (index / staging area state)
```

`H` is the HEAD commit at the time you stash. `I` captures the index. `W` captures the working tree. The latest stash is referenced by `refs/stash` older entries live in its reflog and are addressed as `stash@{0}` (most recent), `stash@{1}`, `stash@{2}`, and so on. The integer shorthand also works: `2` is equivalent to `stash@{2}`.

Running `git stash` with no arguments is identical to `git stash push`. The working tree and index are restored to HEAD, and your changes wait in the stash stack until you `pop` or `apply` them.

Synopsis

```
git stash push [-p | --patch] [-S | --staged] [-k | --[no-]keep-index]
               [-u | --include-untracked] [-a | --all] [-q | --quiet]
               [(-m | --message) <message>] [--] [<pathspec>...]
git stash list [<log-options>]
git stash show [-u | --include-untracked | --only-untracked] [<diff-options>]
               [<stash>]
git stash pop  [--index] [-q | --quiet] [<stash>]
git stash apply [--index] [-q | --quiet] [<stash>]
git stash drop [-q | --quiet] [<stash>]
git stash branch <branchname> [<stash>]
git stash clear
git stash create [<message>]
git stash store [(-m | --message) <message>] [-q | --quiet] <commit>
git stash export (--print | --to-ref <ref>) [<stash>...]
git stash import <commit>
```

Everyday usage

Save all tracked modifications and index changes, then return to a clean tree:

```
git stash
# shorthand for: git stash push
```

Give the stash a descriptive message so you can identify it later:

```
git stash push -m "half-finished login refactor"
```

List all stash entries:

```
git stash list
```

```
stash@{0}: On main: half-finished login refactor  
stash@{1}: WIP on feature/checkout: a3f9c12 Add cart total
```

See what a stash contains before restoring it:

```
git stash show -p stash@{1}
```

Restore the most recent stash and remove it from the stack:

```
git stash pop
```

Restore a specific stash without removing it from the stack:

```
git stash apply stash@{1}
```

Stash only the changes that are already staged:

```
git stash push --staged -m "unrelated hotfix staged for later"
```

Stash everything including untracked files:

```
git stash push --include-untracked -m "spike with new dependencies"
```

Delete a stash entry you no longer need:

```
git stash drop stash@{0}
```

Wipe the entire stash stack:

```
git stash clear
```

Key options

Option	Applies to	What it does	When to use it
<code>-m <msg> /</code> <code>--message <msg></code>	<code>push</code>	Label the stash entry	Any time; makes <code>list</code> readable
<code>-u /</code> <code>--include-untracked</code>	<code>push</code> , <code>show</code>	Include untracked files in the stash and clean them up	New files not yet added to the index
<code>--only-untracked</code>	<code>show</code>	Show only untracked files in the stash diff	Inspecting which new files were stashed
<code>-a / --all</code>	<code>push</code>	Include untracked and ignored files	Temporarily clearing build artefacts
<code>-k / --keep-index</code>	<code>push</code>	Leave already-staged changes intact; stash only unstaged changes	Test staged changes in isolation before committing

<code>--no-keep-index</code>	push	Override the implicit <code>--keep-index</code> set by <code>--patch</code>	Stash everything including what is staged
<code>-p / --patch</code>	push	Interactively choose hunks to stash	Stash only part of a file's changes
<code>-S / --staged</code>	push	Stash only staged changes	Park staged-but-not-yet-committed work for later
<code>--index</code>	pop , apply	Restore the index state in addition to the working tree	When staging state matters, e.g. after <code>git add -p</code>
<code>-q / --quiet</code>	most subcommands	Suppress informational messages	Scripts and aliases
<code>--print</code>	export	Print exported commit chain to stdout without storing it	Transferring stashes between machines via scripts
<code>--to-ref <ref></code>	export	Store the exported commit chain at a named ref	Sharing stashes via push/fetch

Best practices

Always give stashes a message. The default label “WIP on branchname” becomes meaningless once you have several entries. A two-second description (`git stash push -m "auth middleware – broken redirect"`) saves minutes of archaeology with `git stash show` later.

Prefer apply over pop when uncertain. `pop` removes the stash entry after applying it. If you hit conflicts, the entry is gone. Use `apply` to keep the safety net intact, inspect the result, then `drop` manually once you are satisfied.

Keep the stash stack short. Stashes are not branches. They do not track diverging work, have no relationship to each other, and fall off the reflog after 90 days by default. If you need to park work for more than a day or two, create a real branch with a `git commit`.

Use `--keep-index` to test staged changes in isolation. When you have staged some changes and want to verify they build or pass tests without unstaged noise, `git stash push --keep-index` shelves only the unstaged changes. The index is left intact for a clean test run, then `pop` brings back the rest.

Use `branch` to promote a long-lived stash. If a stash grows stale because the branch it came from has moved on, `git stash branch <name>` creates a new branch at the original HEAD, applies the stash without conflicts, and drops it. This is the safest way to revive an old stash.

Pitfalls & gotchas

Untracked files are not stashed by default. Running `git stash` with no flags only captures tracked, modified files and the index. New files you have not yet run `git add` on remain in the working tree. Pass `-u / --include-untracked` to capture them.

Ignored files are not stashed even with `-u`. The `-a` / `--all` flag is needed to also stash ignored files such as build output or `.env` files. Use this carefully — `--all` runs `git clean` on those files, which is destructive if the stash is later lost.

pop after conflicts leaves the stash entry on the stack. If `pop` encounters merge conflicts, the entry stays in the stash list so you can retry. Resolve the conflicts, then call `git stash drop` manually. Do not call `git stash pop` again — that would apply the next entry in the stack.

Stash entries expire. The reflog retains `refs/stash` entries for `gc.reflogExpire` days (default 90). After that, `git gc` can prune them. `git stash clear` makes them immediately eligible for pruning. Recover them quickly (see Recovery below) or convert them to real commits.

--index can fail if you have index conflicts. The `--index` flag to `pop` / `apply` tries to restore the staging-area state as well as the working tree. If the current index already has conflicts, Git cannot apply the saved index state on top and will report an error. Omit `--index` to restore only the working tree in that case.

git stash save is deprecated. Older documentation and tutorials use `git stash save <message>`. The modern equivalent is `git stash push -m <message>`. The `save` subcommand is still functional but does not support `<pathspec>` arguments.

Worked examples

Interrupting work to apply an urgent fix

You are mid-feature on `feature/payments` when you are asked to patch `main` immediately.

```
# Save in-progress work with a descriptive label
git stash push -m "WIP: payment retry logic"

# Switch to main and apply the fix
git switch main
git commit -a -m "Fix null-pointer in order totals"
git push

# Return to the feature branch and restore your work
git switch feature/payments
git stash pop
```

If the `pop` produces conflicts, resolve them in the editor, then clean up the stash entry that was left behind:

```
# resolve conflicts...
git add src/payment.ts
git stash drop
```

Testing staged changes in isolation

You have staged a refactor of `src/auth.ts` and also have unstaged experimental changes in the same file. You want to confirm the refactor builds and passes tests before committing it.

```
# Stage just the refactor hunks
git add -p src/auth.ts

# Stash the unstaged experimental changes; leave the staged index alone
```

```
git stash push --keep-index -m "experimental auth ideas"

# Only the refactor is present – run the test suite
npm test

# Commit the clean, verified refactor
git commit -m "Extract token validation into AuthService"

# Restore the experimental work
git stash pop
```

Pulling upstream changes into a dirty tree

Your local branch has uncommitted changes and `git pull` refuses to proceed because they would conflict with incoming commits.

```
git stash push -m "local config tweaks"
git pull --rebase origin main
git stash pop
```

If the pop conflicts with the rebased commits, resolve them, stage the resolved files, and then drop the now-applied stash entry:

```
# resolve conflicts in editor...
git add src/config.ts
git stash drop
```

Promoting a stale stash to a branch

A stash created three weeks ago on `main` no longer applies cleanly because `main` has moved on. Rather than fighting the conflicts manually, promote the stash to a branch rooted at the commit it was created from.

```
git stash list
# stash@{2}: WIP on main: 4d8b3a1 Initial API scaffold

git stash branch fix/old-api-work stash@{2}
# Creates a branch at 4d8b3a1, applies the stash cleanly, drops stash@{2}
```

You now have a normal branch to rebase onto the current `main`, review, and merge at your leisure. See the *rebase* chapter for how to bring it up to date.

Recovery

If you accidentally dropped a stash entry, it may still be reachable as an unreferenced commit object before garbage collection runs. Search for it:

```
git fsck --unreachable \
| grep commit \
| cut -d' ' -f3 \
| xargs git log --merges --no-walk --grep=WIP
```

Once you identify the commit hash, restore it to the stash stack:

```
git stash store -m "recovered stash" <commit-hash>
```

To undo a `git stash pop` that you did not intend (the stash was removed and the changes are now in the working tree), simply re-stash before doing anything else:

```
git stash push -m "re-stashed: accidentally popped"
```

To undo a `git stash apply` without losing the stash entry, discard the working-tree changes that were just applied:

```
git restore .  
# or, to also unstage anything that was restored to the index:  
git reset HEAD  
git restore .
```

See *Getting out of jams* for broader undo recipes involving lost commits and branch recovery.

See also

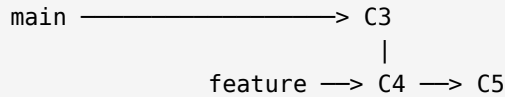
- *add* — building the index; relevant to `--keep-index` and `--staged` stash workflows.
- *commit* — when parked work should become a real snapshot instead of a stash.
- *branch* — `git stash branch` promotes a stash to a full branch.
- *rebase* — bringing a stash-derived branch up to date with the rest of history.
- *Getting out of jams* — recovering dropped stashes and resolving apply conflicts.

branch

List, create, rename, copy, or delete branch pointers in the repository.

Mental model

A branch in Git is nothing more than a named pointer to a single commit. When you add commits on a branch, Git advances that pointer forward automatically. The entire chain of history behind the pointer is what you think of as “the branch”, but the branch itself is only the tip.



Because branches are just pointers (a 41-byte file under `.git/refs/heads/`), creating and deleting them is essentially free. There is no copying of files, no reorganizing history — Git writes or removes a small ref file.

Every local branch can optionally track an *upstream branch*, usually a remote-tracking ref like `origin/main`. Git uses that relationship to report how far ahead or behind you are, and to know where to push or pull by default. `git branch` is the low-level tool for managing all of this: listing branches, creating them from a specific commit, renaming, copying, setting tracking, and deleting.

`git branch` creates a branch but does not switch to it. To create and switch in one step, use `git switch -c <name>` (see the *switch* chapter).

Synopsis

```
# List
git branch [-v | -vv] [-a | -r] [--merged | --no-merged] [--contains <commit>]
          [--sort=<key>] [--format=<format>] [--list] [<pattern>...]

# Create
git branch [--track[=(direct|inherit)] | --no-track] [-f]
          <branch-name> [<start-point>]

# Rename / copy
git branch (-m | -M) [<old-branch>] <new-branch>
git branch (-c | -C) [<old-branch>] <new-branch>

# Delete
git branch (-d | -D) [-r] <branch-name>...

# Tracking
git branch (-u | --set-upstream-to=<upstream>) [<branch-name>]
git branch --unset-upstream [<branch-name>]

# Miscellaneous
git branch --show-current
git branch --edit-description [<branch-name>]
```

Everyday usage

List all local branches (current branch is marked with `*`):

```
git branch
```

```
* feature/login
  fix/null-pointer
  main
```

List branches with their latest commit hash and subject:

```
git branch -v
```

```
* feature/login    3a1c9f2 Add JWT validation middleware
  fix/null-pointer 7f02d3c Fix null-pointer in parseToken
  main            a4e8b10 Release v2.1.0
```

Add a second `-v` to also show upstream tracking information:

```
git branch -vv
```

```
* feature/login    3a1c9f2 [origin/feature/login: ahead 2] Add JWT validation
middleware
  fix/null-pointer 7f02d3c [origin/fix/null-pointer] Fix null-pointer in
parseToken
  main            a4e8b10 [origin/main] Release v2.1.0
```

Show both local and remote-tracking branches:

```
git branch -a
```

Create a new branch pointing to the current HEAD :

```
git branch feature/search
```

Create a new branch from a specific commit, tag, or other branch:

```
git branch hotfix/login-crash v2.1.0
git branch experiment main~3
```

Delete a branch that has been fully merged:

```
git branch -d feature/login
```

Force-delete a branch regardless of merge status:

```
git branch -D experiment
```

Rename the current branch:

```
git branch -m feature/serach feature/search
```

Show just the name of the current branch (useful in scripts):

```
git branch --show-current
```

List branches already merged into `main` (candidates for deletion):

```
git branch --merged main
```

List branches not yet merged:

```
git branch --no-merged main
```

Key options

Option	What it does	When to use it
<code>-v</code> / <code>--vv</code>	Show commit hash, subject, and upstream relationship; <code>-vv</code> additionally shows the linked worktree path	Quickly audit branch state and divergence
<code>-a</code> / <code>--all</code>	List both local and remote-tracking branches	Get a complete picture of all refs
<code>-r</code> / <code>--remotes</code>	List (or delete) remote-tracking branches only	Inspect or clean up stale remote refs
<code>-d</code> / <code>--delete</code>	Delete a branch if fully merged into its upstream or <code>HEAD</code>	Safe cleanup after a merge
<code>-D</code>	Shortcut for <code>--delete --force</code> deletes regardless of merge status	Remove an abandoned or squash-merged branch
<code>-m</code> / <code>--move</code>	Rename a branch, carrying its reflog and config	Fix a typo; rename after a scope change
<code>-M</code>	Shortcut for <code>--move --force</code> overwrites the target if it exists	Force a rename
<code>-c</code> / <code>--copy</code>	Copy a branch along with its config and reflog	Experiment off a branch without destroying it
<code>-C</code>	Shortcut for <code>--copy --force</code> overwrites the target if it exists	Force a copy
<code>-f</code> / <code>--force</code>	Reset an existing branch pointer to a new start point	Move a branch tip without checking it out
<code>-t</code> / <code>--track</code>	Set up upstream tracking when creating a branch	Establish the push/pull relationship explicitly
<code>--no-track</code>	Do not set up upstream tracking	Suppress auto-tracking from <code>branch.autoSetupMerge</code>
<code>-u <upstream></code>	Change (or set) the upstream of an existing branch	Fix a mis-configured tracking relationship
<code>--unset-upstream</code>	Remove upstream configuration from a branch	Detach a local branch from its remote

<code>--show-current</code>	Print the current branch name; prints nothing in detached HEAD	Scripting and prompt integrations
<code>--merged [<commit>]</code>	Filter to branches whose tips are reachable from <code><commit></code>	Find branches safe to delete
<code>--no-merged [<commit>]</code>	Filter to branches not reachable from <code><commit></code>	Find branches that still need merging
<code>--contains [<commit>]</code>	Filter to branches that contain <code><commit></code> in their history	Find which branches carry a specific fix
<code>--no-contains [<commit>]</code>	Filter to branches that do not contain <code><commit></code>	Find branches that still need a patch applied
<code>--sort=<key></code>	Sort output by a ref field; prefix <code>-</code> for descending	Sort by <code>-committerdate</code> to see most-active branches first
<code>--format=<format></code>	Custom output using <code>%(fieldname)</code> placeholders	Build scripts or dashboards with precise output
<code>--list</code>	Treat the argument as a glob pattern, not a branch name	Filter: <code>git branch --list 'fix/*'</code>
<code>--edit-description</code>	Open an editor to write a description for the branch	Document purpose for <code>format-patch</code> cover letters

Best practices

Name branches with a prefix that signals intent. Common conventions are `feature/`, `fix/`, `hotfix/`, `chore/`, and `experiment/`. Prefixes keep `git branch` output scannable and enable glob filtering:

```
git branch --list 'fix/*'
```

Avoid spaces and special characters. Stick to alphanumerics, hyphens, and forward slashes.

Branch from an up-to-date starting point. A feature branch should start from the current tip of `main` (or whatever your integration branch is), not from a stale local snapshot. Fetch before creating:

```
git fetch origin
git branch feature/payments origin/main
```

Set upstream tracking so status, pull, and push work without arguments. When a branch has an upstream configured, `git status` shows how many commits you are ahead or behind, which is useful information every time you glance at the repo. Set it at push time:

```
git push -u origin feature/payments
```

Or after the fact:

```
git branch --set-upstream-to=origin/feature/payments
```

Keep branch names stable after sharing. As soon as you push a branch and others reference it, renaming it with `-m` orphans their remote-tracking refs. Rename only branches that exist solely on

your own machine. If you must rename a pushed branch, also delete the old remote branch and push the new name, then notify collaborators.

Prune stale remote-tracking refs regularly. Remote-tracking branches are not removed automatically when the remote deletes them. Use:

```
git fetch --prune
```

Or enable pruning permanently:

```
git config --global fetch.prune true
```

Confirm with `--merged` before a bulk cleanup. Before deleting a group of old branches, check that they are genuinely merged:

```
git branch --merged main
```

Delete only branches confirmed to be in that list. Use `-D` sparingly — force deletion discards any commits reachable only from that branch.

Pitfalls & gotchas

git branch <name> creates but does not switch. This trips up beginners who create a branch, keep committing on `main`, and then wonder why the branch is empty. Use `git switch -c <name>` when you mean to start work immediately. Check your current branch with `git branch --show-current` before committing.

-d refuses to delete an unmerged branch. Git protects you: if the branch contains commits not reachable from its upstream (or from `HEAD` when there is no upstream), `-d` fails with an error message. This is almost always the right behavior. Use `-D` only when you are certain the work is already elsewhere or truly abandoned.

Squash-merged branches appear unmerged. When a pull request is merged as a single squash commit, the original branch commits are not reachable from `main`. `git branch --merged` will not list the branch, and `-d` will refuse to delete it. Force-delete with `-D`. Projects that squash-merge routinely often automate branch cleanup in CI rather than relying on the safe-delete check.

-f silently resets a branch pointer with no confirmation. `git branch -f <name> <start-point>` moves an existing branch to a different commit immediately. There is no prompt and no undo other than the reflog. Verify the target commit before running this.

--set-upstream (without -to) is no longer supported. The old flag had confusing syntax and was removed. Always use `-u <upstream>` or `--set-upstream-to=<upstream>`.

Renaming the default branch affects all clones. If you rename `master` to `main`, every collaborator must update their local config:

```
git branch -m master main
git fetch origin
git branch --set-upstream-to=origin/main main
```

Co-ordinate with the team and update the remote's default branch setting before making the rename.

Worked examples

Cleaning up merged feature branches after a sprint

Several feature branches were merged into `main` during the sprint. Find and delete them:

```
git fetch --prune
git branch --merged main
```

```
feature/login
feature/search
fix/null-pointer
main
```

`main` always appears in its own `--merged` output, so skip it. Delete the rest:

```
git branch -d feature/login feature/search fix/null-pointer
```

```
Deleted branch feature/login (was 3a1c9f2).
Deleted branch feature/search (was 8b44a01).
Deleted branch fix/null-pointer (was 7f02d3c).
```

Sorting branches by most recent activity

On a long-lived repository with dozens of branches, find the most recently active ones:

```
git branch --sort=-committerdate -v
```

```
* feature/payments  a7d3e12 Add Stripe webhook handler
hotfix/session      b2c9100 Fix session expiry check
main                a4e8b10 Release v2.1.0
feature/search      0c3af77 Initial search scaffold
```

The `-` prefix on `committerdate` sorts newest first. Make it the default:

```
git config --global branch.sort -committerdate
```

Finding which branches contain a critical security fix

A fix for a vulnerability was committed as `d9f4a2b`. Before shipping, verify that all active long-term-support branches include it:

```
git branch --contains d9f4a2b
```

```
lts/2.1
lts/2.0
* main
```

`lts/1.9` is not in the list. It does not contain the fix and needs a cherry-pick before that line ships (see the *cherry-pick* chapter).

Fixing a mis-configured upstream

`git branch -vv` reveals the branch is tracking the wrong remote branch:

```
git branch -vv
```

```
* feature/payments  a7d3e12 [origin/feature/search: ahead 5] Add Stripe webhook handler
```

Fix it:

```
git branch --set-upstream-to=origin/feature/payments
```

```
Branch 'feature/payments' set up to track remote branch 'feature/payments' from 'origin'.
```

Confirm:

```
git branch -vv
```

```
* feature/payments  a7d3e12 [origin/feature/payments: ahead 5] Add Stripe webhook handler
```

Recovery

If you deleted a branch by mistake, its tip commit is still in the object database and tracked in the reflog. Find the hash and recreate the pointer:

```
# Show recent HEAD movements across all branches
git reflog --all | head -30

# Once you have the hash, recreate the branch
git branch recovered-branch <commit-hash>
```

If you moved a branch pointer with `-f` to the wrong commit, look up the previous tip in that branch's reflog:

```
git reflog show <branch-name>
```

```
a7d3e12 feature/payments@{0}: branch: Reset to origin/main
3alc9f2 feature/payments@{1}: commit: Add Stripe webhook handler
```

Restore the previous tip:

```
git branch -f feature/payments 3alc9f2
```

See *Getting out of jams* for broader undo recipes, including recovering commits after a force-push has overwritten a remote branch.

See also

- *switch* — create and switch to a branch in one step with `git switch -c`.

- *checkout* — the older combined command for switching branches and restoring files.
- *merge* — integrate a finished branch into another.
- *rebase* — replay branch commits onto a new base.
- *cherry-pick* — apply individual commits from one branch onto another.
- *fetch* — update remote-tracking branches from the network.
- *Getting out of jams* — recovering deleted or lost branches via the reflog.

checkout

Switch to a branch, create a new branch, restore working-tree files, or inspect a commit in detached-HEAD mode.

Mental model

`git checkout` is the Swiss-army knife of navigation. It does two conceptually different things depending on whether you pass it a branch/commit name or a path:

Branch/commit mode — moves `HEAD` to point at a different branch (or directly at a commit). Git then rewrites the index and working tree to match the target. Your uncommitted local modifications are carried along if they do not conflict.

Path mode — leaves `HEAD` exactly where it is and instead overwrites specific files in the working tree (and optionally the index) with a version from somewhere else: the index, a commit, or any tree-ish.

```
git checkout <target> {
  └─ branch mode → moves HEAD + rewrites tree
  └─ path mode  → overwrites files, HEAD stays put
```

A third mode — **detached HEAD** — puts `HEAD` directly on a commit object instead of on a branch. Any commits you make in this state are reachable only from `HEAD` once you leave, they become orphaned and will eventually be garbage-collected unless you give them a branch or tag.

Modern Git (2.23+) splits these responsibilities across two dedicated commands: *switch* for branch navigation and *restore* for file restoration. `git checkout` still works and is ubiquitous in scripts and documentation, but the newer commands are less ambiguous for interactive use.

Synopsis

```
git checkout [-q] [-f] [-m] [<branch>]
git checkout [-q] [-f] [-m] --detach [<branch>]
git checkout [-q] [-f] [-m] [--detach] <commit>
git checkout [-q] [-f] [-m] [--b|-B|--orphan] <new-branch> [<start-point>]
git checkout [-f] <tree-ish> [--] <pathspec>...
git checkout [-f|--ours|--theirs|-m|--conflict=<style>] [--] <pathspec>...
git checkout (-p|--patch) [<tree-ish>] [--] [<pathspec>...]
```

Everyday usage

Switch to an existing branch:

```
git checkout main
git checkout feature/login
```

Switch back to the previous branch (the `-` shorthand means “wherever I just was”):

```
git checkout -
```

Create a new branch and switch to it immediately:

```
git checkout -b feature/payments
git checkout -b hotfix/null-ptr origin/main # start from a specific point
```

Restore a single file to the version in the index (discard working-tree edits):

```
git checkout -- src/api.js
```

Restore a file to the version from a specific commit:

```
git checkout HEAD~2 -- src/api.js
git checkout a3f9c1 -- config/routes.rb
```

Check out a commit in detached-HEAD mode for read-only inspection:

```
git checkout v2.4.0 # tag
git checkout a3f9c1 # any commit hash
```

Key options

Option	What it does	When to use it
<code>-b <new-branch></code>	Create <code><new-branch></code> and switch to it	Start new work from the current or a named start point
<code>-B <new-branch></code>	Create <code><new-branch></code> , or reset it if it exists, then switch	Re-use a branch name during iterative experimentation
<code>-d / --detach</code>	Detach HEAD at the target commit rather than tracking a branch	Inspect a commit or run throwaway experiments
<code>--orphan <new-branch></code>	Create a new unborn branch; the first commit on it will have no parents and start a disconnected history	GitHub <code>gh-pages</code> branch; publishing a clean subtree
<code>-f / --force</code>	Proceed even when the index or working tree differs from HEAD, and even if there are untracked files in the way	Blow away local changes intentionally
<code>-m / --merge</code>	Three-way merge local modifications into the target branch instead of refusing	Carry in-progress work to a different branch
<code>--conflict=<style></code>	Set the conflict marker style (<code>merge</code> , <code>diff3</code> , <code>zdiff3</code>) when used with <code>-m</code>	Prefer <code>diff3</code> for more context in conflict hunks
<code>--ours</code>	During a merge conflict, check out stage 2 (the current branch's version)	Resolve a conflict by keeping your side entirely
<code>--theirs</code>	During a merge conflict, check out stage 3 (the incoming version)	Resolve a conflict by accepting the other side entirely

<code>-p / --patch</code>	Interactively pick hunks to restore from the index or a tree-ish	Selectively discard only some edits in a file
<code>-t / --track</code>	Set the upstream tracking reference when creating a branch	Wire a new branch to its remote counterpart for <code>git pull / push</code>
<code>--no-track</code>	Do not configure upstream even if <code>branch.autoSetupMerge</code> is set	Local-only branch that should never push
<code>-q / --quiet</code>	Suppress feedback messages	Scripting; reduce noise in CI logs
<code>--recurse-submodules</code>	Also update submodule working trees to match the superproject commit	Repos with submodules where you want everything in sync
<code>--no-overlay</code>	Remove files that appear in the index and working tree but not in the target <code><tree-ish></code>	Make a subdirectory match a commit exactly
<code>--pathspect-from-file=<file></code>	Read pathspects from a file rather than the command line	Scripted restores of a large or programmatically generated file list

Best practices

Prefer `-b` over a separate `git branch` + `git checkout` pair. The combined form is atomic: if the branch name is invalid or already exists the command fails before moving HEAD, leaving you on your original branch.

Always use `--` before pathspects. When a filename could be confused with a branch name, omit the separator and Git favors the branch interpretation. The `--` sentinel tells Git unambiguously that what follows is a path, not a branch or commit:

```
git checkout -- hotfix    # restores a file named "hotfix"
git checkout hotfix      # switches to the branch named "hotfix"
```

Name detached-HEAD work before leaving. When you check out a commit hash or tag directly, you are in detached-HEAD state. Any commits you make exist only through HEAD. Before switching away, give those commits a branch:

```
git checkout -b experiment/v2-prototype
```

If you forget, use `git reflog` to find the last commit hash and then create the branch retrospectively.

Use `-b` with an explicit start point instead of switching first. Rather than checking out `main` and then branching, pass the start point directly. This avoids accidentally picking up uncommitted changes from `main`:

```
git checkout -b fix/cache-bug origin/main
```

Use the `switch` and `restore` commands for new scripts. `git checkout` conflates two unrelated operations. In scripts written today, prefer *switch* for branch navigation and *restore* for file restoration — both are explicit about intent and less prone to ambiguous-argument errors.

Pitfalls & gotchas

--ours and --theirs flip meaning during a rebase. During `git rebase`, “ours” is the branch being rebased *onto* (typically `main`), not your feature branch. “Theirs” is your feature branch’s work. This is the opposite of the intuitive meaning during a plain `git merge`. When in doubt, inspect the conflict markers before reaching for `--ours` / `--theirs`.

-f silently discards uncommitted work. Force-switching a branch with `-f` will overwrite modified tracked files and delete untracked files that are in the way. There is no undo. Check `git status` before using `-f`.

Path-mode checkout does not stage the change. Running `git checkout HEAD~1 -- src/config.js` writes the old version into the working tree and into the index simultaneously. The file shows up in `git status` as “Changes to be committed”. This surprises people who expect only the working tree to be modified.

Switching branches with staged changes sometimes silently carries them. If your staged changes do not conflict with the target branch, Git moves the staging area along with you. Your staged diff on branch A will still be staged after switching to branch B. This is convenient but can cause an accidental commit on the wrong branch.

Detached-HEAD commits are easy to lose. Any commits made in detached-HEAD state exist only through the `HEAD` reference. The moment you run another `git checkout`, `HEAD` moves and those commits become unreachable. Git’s garbage collector will eventually delete them. Save them with a branch or tag before navigating away.

Argument ambiguity between branches and files. If a file and a branch share a name, `git checkout <name>` switches to the branch. Git warns about this situation but does not error. Always use `--` when operating on paths to eliminate the ambiguity.

Worked examples

Starting a feature branch from a remote base

You want to begin work on a payment integration, tracking the team’s `origin/main`:

```
git fetch origin
git checkout -b feature/stripe-payments origin/main
```

```
Switched to a new branch 'feature/stripe-payments'
Branch 'feature/stripe-payments' set up to track remote branch 'main' from
'origin'.
```

The new branch starts at the same commit as `origin/main` and is wired for tracking, so `git pull` and `git push` work without extra arguments.

Rescuing a file you accidentally deleted

You deleted `src/auth/token.js` and already staged the deletion:

```
git checkout HEAD -- src/auth/token.js
```


Git restores the file from the last commit into both the working tree and the index, cancelling the staged deletion. Confirm with `git status`.

Inspecting an old release without losing your work

You are debugging a regression and want to run the tests against the `v1.8.0` tag without touching your current branch:

```
git stash push -m "WIP: payment refactor"
git checkout v1.8.0
```

Note: switching to 'v1.8.0'.

You are in 'detached HEAD' state. You can look around, make experimental changes and commit them, and you can discard any commits you made in this state without impacting any branches by switching back to a branch.

Run your tests, then return to your branch and restore the stash:

```
git checkout feature/stripe-payments
git stash pop
```

Selectively discarding edits with `-patch`

You edited `services/email.js` in two unrelated ways and want to keep one change but throw away the other:

```
git checkout -p -- services/email.js
```

Git presents each hunk interactively (`y` accept, `n` skip, `s` split, `e` edit manually). Accepting a hunk restores that portion of the file from the index, discarding your working-tree edit for that hunk only.

Resolving a merge conflict by choosing one side entirely

You are in the middle of a merge and `config/database.yml` has a conflict. The current branch's version is correct:

```
git checkout --ours -- config/database.yml
git add config/database.yml
```

Or if the incoming branch is correct:

```
git checkout --theirs -- config/database.yml
git add config/database.yml
```

Mark the file resolved and continue as usual with `git commit`.

Recovery

Switched to the wrong branch and lost staged changes: staged changes that did not conflict were silently carried over. They are still in the index. Switch back and your staged diff will be there.

Made commits in detached-HEAD state and then navigated away: find the orphaned commit in the reflog and create a branch pointing at it:

```
git reflog
# find the hash, e.g. e7a3f12
git checkout -b rescue/detached-work e7a3f12
```

Restored a file from a commit with `git checkout <commit> -- <file>` by mistake: the original working-tree content is gone, but if you had not yet staged it the index still holds the HEAD version. Run:

```
git checkout HEAD -- <file>
```

to get back to the committed version, then re-apply your edits.

Accidentally discarded working-tree changes with `-f`: if the changes were never committed they are gone. If you had staged them at any point and Git's `gc` has not yet run, they may still exist as loose objects. See *Getting out of jams* for `git fsck --lost-found` techniques.

See *Getting out of jams* for broader undo recipes covering branch recovery and lost commits.

See also

- *switch* — the modern, unambiguous command for changing branches.
- *restore* — the modern, unambiguous command for restoring working-tree files.
- *branch* — creating, listing, and deleting branches without switching to them.
- *stash* — shelve work-in-progress before switching contexts.
- *reflog* — find and recover commits that became unreachable after detached-HEAD work.
- *rebase* — rewriting history; note the `--ours` / `--theirs` flip described above.
- *Getting out of jams* — recovering from force-checkouts and lost commits.

switch

Move HEAD (and your working tree) to a different branch, creating one first if needed.

Mental model

A Git repository has many branches, but your working tree shows only one at a time. HEAD is a pointer that says “I am here.” `git switch` moves that pointer to a different branch and updates every file in the working tree and index to match the tip of that branch.

Before:		After: <code>git switch feature</code>
main <-- HEAD		main
C3		C3
C2	feature	C2
		feature <-- HEAD
C1	Cf	C1
		Cf

Three things happen atomically:

1. HEAD is rewritten to point at the target branch.
2. The index is replaced with the tree at the tip of the target branch.
3. The working tree files are updated to match, file by file.

Git is cautious: if any of your local changes would be overwritten, the operation is aborted and your work stays intact. You must either commit, stash, or explicitly tell `git switch` what to do with the uncommitted changes before it will proceed.

`git switch` was introduced in Git 2.23 as a focused alternative to `git checkout` for branch-switching work. It has no staging or path-restore duties — those belong to the *restore* command. When you read older documentation or scripts using `git checkout <branch>`, the modern equivalent is `git switch <branch>`.

Synopsis

```
git switch [<options>] [--no-guess] <branch>
git switch [<options>] --detach [<start-point>]
git switch [<options>] (-c | -C) <new-branch> [<start-point>]
git switch [<options>] --orphan <new-branch>
```

Everyday usage

Switch to an existing local branch:

```
git switch feature/login
```

Jump back to the branch you were on before:

```
git switch -
```

Create a new branch and switch to it immediately:

```
git switch -c fix/null-check
```

Create a new branch starting from a specific commit or tag:

```
git switch -c release/2.1 v2.0.0
```

Check out a remote branch that does not exist locally yet (Git guesses the tracking relationship automatically):

```
git switch staging
# Branch `staging` set up to track remote branch `staging` from `origin`.
# Switched to a new branch 'staging'
```

Detach HEAD to inspect a historical commit without creating a branch:

```
git switch --detach HEAD~5
```

Key options

Option	What it does	When to use it
<code>-c <new-branch></code>	Create <code><new-branch></code> and switch to it	Start new work from the current or a named commit
<code>-C <new-branch></code>	Create <code><new-branch></code> , resetting it if it already exists	Re-point an existing branch to a new start-point
<code>-d / --detach</code>	Switch to a commit in detached-HEAD state	Inspect history or run throwaway experiments
<code>--orphan <new-branch></code>	Create a new branch with no history	Start a completely isolated history (e.g. <code>gh-pages</code>)
<code>--guess / --no-guess</code>	Auto-create a local tracking branch when the name matches a remote	<code>--guess</code> is the default; use <code>--no-guess</code> to suppress the behaviour when you do not want a tracking branch created automatically
<code>-t / --track</code>	Explicitly set the upstream tracking branch	When creating a branch that should track a non-default remote
<code>--no-track</code>	Do not set up upstream tracking	Suppress tracking even when <code>branch.autoSetupMerge</code> is true
<code>-m / --merge</code>	Three-way merge local changes into the target branch	Carry uncommitted edits across a branch switch
<code>--conflict=<style></code>	Override conflict-marker style (<code>merge</code> , <code>diff3</code> , <code>zdiff3</code>)	Prefer <code>diff3</code> when you want the common ancestor shown
<code>-f / --force / --discard-changes</code>	Throw away local changes and force the switch	You want to start over and do not care about uncommitted work
<code>-q / --quiet</code>	Suppress progress and informational messages	Scripting; reduces noise

<code>--recurse-submodules</code>	Also update submodule working trees	Repositories that use submodules
-----------------------------------	-------------------------------------	----------------------------------

Best practices

Prefer `git switch` over `git checkout` for branch operations. The commands overlap, but `git switch` has a single, clear responsibility. Code reviews and shell histories are easier to read when branch moves and file restores are separate operations.

Use `git switch -c` instead of `git branch + git switch`. The combined form is transactional: if the switch fails (because the branch is already checked out in another worktree, for example), the branch is not created either. With the two-command form the branch exists but you are still on the old one, which can confuse `git status`.

Let `--guess` do the remote-tracking wiring for you. When you type `git switch staging` and `staging` does not exist locally but `origin/staging` does, Git creates the local branch and sets its upstream automatically. There is no need to spell out `git switch -c staging --track origin/staging` by hand.

Commit or stash before switching. If your local changes conflict with the target branch, the switch is aborted. Running `git stash push -m "WIP: login refactor"` before `git switch` and `git stash pop` after is the safest and most explicit pattern. It preserves a clear record of what you set aside.

Use `-` liberally. `git switch -` is the fastest way to toggle between two branches. It behaves like `cd -` in the shell: it always switches to wherever you were before.

Name branches consistently before creating them. Use a `<type>/<short-description>` convention (e.g. `fix/`, `feat/`, `docs/`, `chore/`) so branches are sortable and identifiable in long `git branch -a` listings.

Pitfalls & gotchas

“THIS COMMAND IS EXPERIMENTAL” — the help page carries this warning because the interface may still change between Git versions. In practice `git switch` has been stable since Git 2.23, but be aware that very old installations may not have it at all.

Detached HEAD is not a branch. When you run `git switch --detach HEAD~5`, any commits you make are reachable only through their hash. If you switch away without first running `git switch -c <new-branch>`, those commits become unreachable and are eventually garbage-collected. Git warns you when you leave a detached HEAD that has new commits, but the warning is easy to overlook.

`-C` is destructive. `git switch -C existing-branch HEAD` resets `existing-branch` to point at HEAD without asking for confirmation. Use it only when you are certain you want to abandon the existing tip of that branch.

Stashed changes are not automatically reapplied. `git switch` does not touch the stash. After switching, you must run `git stash pop` yourself. If you forget and stash again on the new branch, you will have two stash entries to untangle.

`--merge` can leave conflicts. When `--merge` is used to carry local changes to the target branch and there are conflicts, the index is left in an unmerged state. You must resolve conflicts and `git add` each resolved file before continuing. Run `git status` immediately after a `--merge` switch to see what needs attention.

--discard-changes is silent and permanent. Unlike `git stash`, it destroys your uncommitted work with no recovery path. Run `git diff` and `git diff --cached` first so you know exactly what you would be discarding.

The `@{-N}` shorthand counts switch operations, not branch hops. If you run `git switch feature`, then `git switch -`, then `git switch -` again, you end up back on `feature`. Switching back to where you were with `-` is conceptually a single hop regardless of how many times you do it in a row.

Worked examples

Starting a feature branch and returning to main

```
# You are on main. Start a new feature.
git switch -c feat/user-preferences
# ... edit and commit work ...
git commit -m "Add user preference storage"

# Reviewer asks for a hotfix on main before the feature is done.
git switch main
git switch -c fix/typo-in-readme
git commit -m "Fix typo in README"

# Merge the hotfix, then return to feature work.
git switch main
git merge --no-ff fix/typo-in-readme
git switch feat/user-preferences
```

The `-` shorthand works perfectly here for the final step if `main` was the last branch:

```
git switch - # back to feat/user-preferences
```

Carrying uncommitted changes to the right branch

You edited `src/api.js` on `main` but realise the changes belong on `feat/api-refactor`.

```
# Attempt a normal switch – will fail if the changes conflict.
git switch feat/api-refactor
# error: Your local changes to the following files would be overwritten:
#     src/api.js
# Please commit your changes or stash them before you switch branches.

# Option A: stash, switch, and reapply.
git stash push -m "WIP: api refactor edits"
git switch feat/api-refactor
git stash pop

# Option B: let git do a three-way merge during the switch.
git switch -m feat/api-refactor
# Auto-merging src/api.js
```

Option A is safer when the branches are far apart. Option B is convenient when you know the branches share a recent common ancestor and the change is small.

Inspecting a release tag without creating a branch

```
git switch --detach v1.8.3
# HEAD is now at d3a91f2 Release 1.8.3

# Run tests against the old release.
make test

# Nothing worth keeping – return to your branch.
git switch main
# Warning: you are leaving 0 commits behind, not connected to any branch.
# Switched to branch 'main'
```

If you had made commits in detached HEAD state and wanted to keep them:

```
git switch -c investigation/v1.8.3-repro
```

Creating an orphan branch for GitHub Pages

```
git switch --orphan gh-pages
# Switched to a new branch 'gh-pages'
# All tracked files are removed from the working tree.

# Add the static site content and commit.
cp -r _site/* .
git add .
git commit -m "Publish docs site"
git push -u origin gh-pages
```

The `--orphan` branch starts with an empty index and no parent commit, so the documentation history is completely isolated from the source-code history.

Recovery

If you switched to the wrong branch by accident, switch back immediately:

```
git switch - # return to the previous branch
```

If you ran `--discard-changes` and lost uncommitted work, there is no direct undo — discarded working-tree changes are not stored by Git. Check whether your editor or IDE has a local history feature.

If you accidentally moved away from a detached HEAD that had new commits, the commits are still in the reflog for a while:

```
git reflog
# find the hash of the last commit you made in detached HEAD
git switch -c recovery/detached abc1234
```

See *Getting out of jams* for additional undo recipes covering lost commits and stash conflicts.

See also

- *checkout* — the older command that `switch` was split out from; still needed for checking out individual files.

- *restore* — the companion command for discarding file-level changes without switching branches.
- *branch* — create, list, rename, and delete branches.
- *stash* — save uncommitted changes before switching when `-m` is not appropriate.
- *worktree* — check out multiple branches simultaneously in separate directories, eliminating many context-switch round-trips.
- *Getting out of jams* — recovering lost commits and resolving post-switch conflicts.

merge

Integrate the history of one branch into another by creating a merge commit or advancing the branch pointer via fast-forward.

Mental model

Every branch in Git is just a pointer to a commit. When you merge, Git finds the common ancestor of the two branch tips, then combines the work done on each side since that ancestor into a unified result.

There are two outcomes depending on the shape of the history:

Fast-forward — when the current branch has not diverged at all from the branch you are merging in, Git simply moves the current branch pointer forward to the tip of the incoming branch. No new commit is created. The history remains a straight line.

```
Before:
main:   A---B---C
          \
feature:      D---E

After fast-forward merge of feature into main:
main:   A---B---C---D---E
```

True merge — when both branches have new commits since they diverged, Git performs a three-way merge using the common ancestor as the base. If the changes do not overlap, Git resolves them automatically and records the result in a new merge commit with two parents. If they do overlap, Git pauses with a conflict for you to resolve manually.

```
Before:
main:   A---B---C---G
          \
feature:      D---E---F

After true merge (H is the merge commit):
main:   A---B---C---G---H
          \           /
feature:      D---E---F
```

Before the operation begins, Git saves the current branch tip as `ORIG_HEAD` so you can undo the merge if needed.

Synopsis

```
git merge [--ff | --no-ff | --ff-only] [--squash] [--no-commit]
          [-m <msg>] [-F <file>] [-s <strategy>] [-X <option>]
          [--no-verify] [--allow-unrelated-histories]
          [--[no-]rerere-autoupdate] [--autostash]
          [-S[<keyid>]] [<commit>...]
git merge (--continue | --abort | --quit)
```

Everyday usage

Merge a feature branch into the current branch (typically `main`):

```
git switch main
git merge feature/login
```

Merge and always produce a merge commit, even if fast-forward is possible. This preserves the fact that the work was done on a separate branch:

```
git merge --no-ff feature/login
```

Merge only if a fast-forward is possible; refuse and exit non-zero otherwise:

```
git merge --ff-only origin/main
```

Squash all commits from the incoming branch into the index, then commit them as a single new commit on the current branch:

```
git merge --squash feature/login
git commit -m "Add login feature"
```

Abort an in-progress merge after conflicts arise:

```
git merge --abort
```

Resume after resolving conflicts manually:

```
# edit conflicting files, then:
git add src/auth.js
git merge --continue
```

Key options

Option	What it does	When to use it
<code>--ff</code>	Fast-forward when possible; create a merge commit otherwise (default)	Normal day-to-day merges
<code>--no-ff</code>	Always create a merge commit, even when fast-forward is possible	Preserve branch topology in history
<code>--ff-only</code>	Refuse to merge if it cannot be fast-forwarded	Keep a linear history; fail loudly if diverged
<code>--squash</code>	Apply the combined changes to the index without recording a merge commit	Deliver a branch as one clean commit
<code>--no-commit</code>	Perform the merge but stop before creating the commit	Inspect or tweak the merge result before recording it
<code>-m <msg></code>	Set the merge commit message	Customize the auto-generated message
<code>-F <file></code>	Read the merge commit message from a file	Multi-line messages prepared in advance

<code>-s <strategy></code>	Select the merge strategy (<code>ort</code> , <code>octopus</code> , <code>ours</code> , <code>resolve</code> , <code>subtree</code>)	Override the default strategy
<code>-X <option></code>	Pass a strategy-specific option (e.g. <code>-X ours</code> , <code>-X theirs</code>)	Auto-resolve conflicts by favoring one side
<code>--no-verify</code>	Skip the pre-merge and commit-msg hooks	Emergency bypass; fix the hook issue afterward
<code>--allow-unrelated- histories</code>	Merge histories with no common ancestor	Joining two independent repositories
<code>--autostash</code>	Automatically stash dirty working tree before merging and re-apply after	Merge without needing a clean working tree
<code>--abort</code>	Discard the in-progress merge and restore the pre-merge state	Give up after hitting conflicts
<code>--continue</code>	Conclude a paused merge after resolving conflicts	Resume after editing conflict markers
<code>--quit</code>	Forget the in-progress merge without touching the working tree	Abandon the state machine, keep your edits
<code>-S[<keyid>]</code>	GPG-sign the resulting merge commit	Verified merge commits on GitHub/GitLab
<code>--verify-signatures</code>	Abort if the incoming branch tip is not signed with a valid key	Enforce signed-commit policy

Best practices

Always merge into a clean working tree. Git warns you if you have uncommitted changes, and `--abort` may not fully restore them afterward. Run `git status` before merging; stash or commit anything in flight.

Prefer `--no-ff` when merging feature branches into long-lived branches. With the default fast-forward, the branch structure is lost from history. `--no-ff` leaves an explicit record that a set of commits was developed together and then integrated, which makes `git log --graph` and `git bisect` more informative.

Set `merge.ff=false` in your project config to make `--no-ff` the default for that repository:

```
git config merge.ff false
```

Use `--squash` only when the branch-internal history is noise. Squash merges collapse the entire branch into one commit, which is convenient for tiny spikes or work-in-progress branches but destroys the record of individual changes. Use it deliberately; do not squash feature branches that other people will rebase on.

Set `merge.conflictStyle = diff3` or `zdiff3` to improve conflict readability. The default style shows only your side and their side. `diff3` also shows the common ancestor, which makes it far easier to understand what each side actually changed:

```
git config --global merge.conflictStyle diff3
```

Write a meaningful merge message. The auto-generated “Merge branch X” message is the bare minimum. When merging a long-lived feature, add a line describing the scope of the work.

Pitfalls & gotchas

Fast-forward erases branch topology. After a fast-forward merge there is no commit in history that says “this is where the feature branch was merged.” `git log --oneline` shows a straight line with no indication that these commits were developed on a side branch. If traceability matters, use `--no-ff`.

--squash does not record a merge relationship. After `git merge --squash`, Git does not set `MERGE_HEAD`, so the next `git commit` creates an ordinary commit, not a merge commit. The source branch is not recorded as merged; `git branch --merged` will not list it. If you later try to merge the same branch again, Git will not know it has already been incorporated and may produce unexpected conflicts.

Merging with a dirty index is refused. If your index has staged changes relative to `HEAD`, Git will refuse to merge. Commit or unstage them first with `git restore --staged`.

--abort may not recover uncommitted working-tree changes. If you had dirty tracked files when you started the merge, `--abort` attempts to restore them, but complex cases can leave the working tree in a partial state. Start merges from a clean working tree.

Reverted commits re-appear after a merge. If commit X was made on branch A and then reverted on branch B, merging A into B will re-introduce the original change. Git’s three-way merge sees the revert as “no net change” on branch B and substitutes branch A’s version. The fix is to revert the revert before merging, or use `-X ours` to explicitly discard the incoming change.

Octopus merges refuse complex conflicts. Merging more than two branches at once uses the `octopus` strategy, which does not perform interactive conflict resolution. If any conflict requires manual intervention, the entire octopus merge is refused and you must merge branches one at a time.

Worked examples

Merging a feature branch with an explicit merge commit

You have finished work on `feature/checkout` and want to integrate it into `main` with a visible merge commit.

```
git switch main
git pull --ff-only origin main # get the latest upstream first
git merge --no-ff feature/checkout
```

Git opens the editor with an auto-generated message. Edit it to describe the feature, save, and close. The resulting graph:

```
* 9f3a2c1 (HEAD -> main) Merge branch 'feature/checkout'
|\
| * d4e8f12 Add order summary to checkout page
| * b7a1c09 Implement payment provider selection
```

```
|/  
* a3f0d88 (origin/main) Prepare release pipeline
```

Resolving a conflict manually

You merge `feature/nav` into `main` and Git stops with a conflict in `src/components/NavBar.jsx`.

```
$ git merge feature/nav  
Auto-merging src/components/NavBar.jsx  
CONFLICT (content): Merge conflict in src/components/NavBar.jsx  
Automatic merge failed; fix conflicts and then commit the result.
```

Open `src/components/NavBar.jsx`. The conflict markers look like this with `merge.conflictStyle = diff3` enabled:

```
<<<<<<< HEAD  
  <nav className="nav-dark">  
  ||||| base  
  <nav className="nav">  
  =====  
  <nav className="nav-light">  
>>>>>> feature/nav
```

The `|||||` section shows the original. `HEAD` changed it to dark; the feature branch changed it to light. Decide which version is correct, remove all conflict markers, and save. Then stage and continue:

```
git add src/components/NavBar.jsx  
git merge --continue
```

Git creates the merge commit. If other files have conflicts, repeat the edit-and-add cycle before running `--continue`.

Squash-merging a short-lived spike

A teammate ran a proof-of-concept on `spike/graphql` across five messy commits. You want the end result on `main` as a single, clean commit.

```
git switch main  
git merge --squash spike/graphql
```

Git applies the combined diff to the index but does not commit. Review the staged changes, then commit with a proper message:

```
git diff --staged      # review what will be committed  
git commit -m "Introduce GraphQL endpoint for product search  
  
Proof of concept extracted from spike/graphql. Supersedes the REST  
endpoint added in a3f9c1. See docs/spikes/graphql.md."
```

The `spike/graphql` branch is not marked as merged, so delete it explicitly when done:

```
git branch -D spike/graphql
```

Auto-resolving conflicts with a strategy option

You are merging a maintenance branch that contains only whitespace cleanups, and you know every conflict should favor the incoming branch:

```
git merge -X theirs maintenance/formatting
```

`-X theirs` passes the `theirs` option to the `ort` strategy, which resolves every conflicting hunk by accepting the incoming side. Non-conflicting changes are merged normally.

Recovery

To undo a completed merge commit before pushing, reset to `ORIG_HEAD` :

```
git reset --merge ORIG_HEAD
```

`--merge` undoes the merge commit and cleans up conflict-related index entries while preserving any uncommitted working-tree changes you had before the merge.

To abort a merge that has stopped mid-flight due to conflicts:

```
git merge --abort
```

If `--abort` leaves the working tree in an unexpected state, or if you need to undo a merge that has already been pushed, see *Getting out of jams* for more targeted reset and revert recipes.

See also

- *branch* — create and list branches before merging.
- *switch* — change to the target branch before running merge.
- *rebase* — an alternative to merging that rewrites commits onto a new base, producing a linear history.
- *cherry-pick* — integrate individual commits without merging an entire branch.
- *revert* — safely undo a merged commit by adding a new commit that cancels its changes.
- *mergetool* — launch a graphical three-way merge tool to resolve conflicts.
- *Getting out of jams* — undo a bad merge or recover from a conflict gone wrong.

mergetool

Launch a visual or text-based merge tool to resolve conflicts left behind by a failed merge, rebase, or cherry-pick.

Mental model

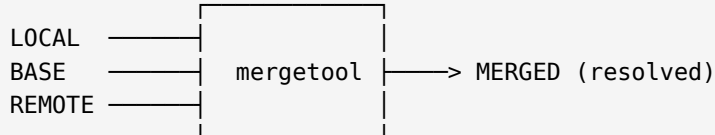
When Git cannot automatically reconcile two sides of a change it writes conflict markers into the affected file and stops:

```
<<<<<<< HEAD
result = a + b;
=====
result = a * b;
>>>>>>> feature/multiply
```

Your job is to edit every such file until it contains exactly the code you want, then tell Git the conflict is resolved with `git add`. Doing that by hand in a plain text editor works, but it is error-prone: it is easy to leave a stray `<<<<<<<` line or accidentally delete a closing `}`.

`git mergetool` automates the tedious parts. For each conflicted file it:

1. Extracts three versions into temporary files: `LOCAL` (your branch), `REMOTE` (the branch being merged in), and `BASE` (their common ancestor).
2. Opens the configured tool with those three inputs and the `MERGED` output file.
3. Waits for you to finish resolving, then marks the file as resolved (or asks you whether it was resolved, for tools that do not report success via their exit code).



Think of it as a structured wrapper around `git add`: the tool gives you a side-by-side view so you can make an informed decision on every conflict hunk, then `mergetool` stages the result for you.

Synopsis

```
git mergetool [--tool=<tool>] [-y | --[no-]prompt] [<file>...]
```

Everyday usage

Run the tool on every conflicted file (most common invocation):

```
git merge feature/multiply    # merge fails with conflicts
git mergetool                 # open tool for each conflicted file
git commit                    # record the resolved merge
```

Resolve a specific file rather than all conflicts at once:

```
git mergetool src/pricing.js
```

Resolve all conflicted files under a directory:

```
git mergetool src/
```

Ask which tools are available on your system:

```
git mergetool --tool-help
```

Use a specific tool for this one invocation without changing your config:

```
git mergetool --tool=meld
```

Skip the “was the merge successful?” prompt between files:

```
git mergetool --no-prompt
```

Key options

Option	What it does	When to use it
<code>-t <tool></code> , <code>--tool=<tool></code>	Use a specific merge tool (e.g. <code>meld</code> , <code>kdiff3</code> , <code>vimdiff</code>)	Override <code>merge.tool</code> for a single session
<code>--tool-help</code>	Print all tools recognised on this system	Discover what is available before configuring
<code>-y</code> , <code>--no-prompt</code>	Skip the per-file “was the merge successful?” prompt	Batch processing; tool reports success via exit code
<code>--prompt</code>	Always prompt before opening each file	Review each file name before the tool launches
<code>-g</code> , <code>--gui</code>	Use <code>merge.guitool</code> instead of <code>merge.tool</code>	Switch to a graphical tool when a display is available
<code>--no-gui</code>	Force <code>merge.tool</code> , ignoring <code>merge.guitool</code> and <code>mergetool.guiDefault</code>	Override a GUI default in a headless session
<code>-O<orderfile></code>	Process files in the order given by a glob-per-line file	Resolve the most critical files first

Best practices

Set `merge.tool` once in your global config. Typing `--tool=meld` every time is friction that leads people to skip the tool entirely. Pick the tool that matches your workflow and commit to it:

```
git config --global merge.tool meld
```

Disable backup files once you are comfortable. By default Git writes a `file.orig` backup beside every resolved file. That safety net is valuable when you are learning, but it pollutes the working tree and can confuse `git status`. Once you trust your tool, turn it off:

```
git config --global mergetool.keepBackup false
```


Resolve one file at a time when a conflict is complex. Pass the filename explicitly. It is easier to focus on a single file than to tab through ten in sequence. Come back to the remaining files with another `git mergetool` call:

```
git mergetool src/config.js    # focus here first
# ... later ...
git mergetool                  # handle the rest
```

Set `mergetool.<tool>.trustExitCode` for reliable tools. The interactive “Was the merge successful?” prompt exists because not every tool reports success via its exit code. If your tool does, suppress the prompt:

```
git config --global mergetool.meld.trustExitCode true
git mergetool --no-prompt
```

Use `mergetool.hideResolved` to reduce noise. By default `LOCAL` and `REMOTE` show the raw pre-resolution state. With `hideResolved` enabled Git overwrites those files so only the hunks that still have conflicts are visible — auto-resolved hunks disappear from the side panels:

```
git config --global mergetool.hideResolved true
```

Pair `mergetool` with `rerere` on long-lived branches. If you resolve the same conflict repeatedly (common on a branch that tracks a fast-moving main), enabling `rerere` lets Git remember your resolution and reapply it automatically. See the *rerere* chapter for setup details.

Pitfalls & gotchas

The `.orig` files land in version-controlled directories. If `mergetool.keepBackup` is `true` (the default), Git writes `*.orig` files next to every resolved file. Add `*.orig` to your global `.gitignore` or they will eventually slip into a commit:

```
echo '*.orig' >> ~/.gitignore
git config --global core.excludesFile ~/.gitignore
```

Closing the tool without saving does not automatically abort. Most tools exit with a non-zero code when you quit without writing, and `mergetool` will ask whether the merge was successful. Answer `n` to skip the file — Git leaves it conflicted so you can try again. If you answered `y` by mistake, the conflict markers may still be present in the staged file. Check with `git diff --check` before committing.

`git mergetool` does not commit for you. After the tool exits, the resolved files are staged, but you must still run `git commit` to record the merge. Forgetting this step leaves the repository in a mid-merge state that confuses later commands.

The continuation command differs depending on what caused the conflict. After a conflicted rebase you run `git rebase --continue` instead of `git commit`. After a conflicted cherry-pick you run `git cherry-pick --continue`. Running `git commit` directly in those contexts creates a stray commit rather than advancing the rebase or cherry-pick. See the *rebase* and *cherry-pick* chapters for details.

Custom tools must set `mergetool.<tool>.cmd` correctly. The four environment variables `BASE`, `LOCAL`, `REMOTE`, and `MERGED` are exported by Git before invoking the command. Getting the

argument order wrong (some tools expect `local remote base merged`) is a common source of confusing three-way diffs. Test with a simple conflict before relying on a custom tool in production.

--tool-help output varies by platform. The list is built from what Git finds in `$PATH` at the time you run it. A tool that is installed but not on `$PATH` will not appear. If your preferred tool is missing, verify its binary is reachable before debugging the Git configuration.

Worked examples

Resolving a merge conflict with meld

Your branch `feature/pricing` diverged from `main`. A colleague also touched `src/pricing.js` on `main` and the merge fails.

```
git switch feature/pricing
git merge main
```

```
Auto-merging src/pricing.js
CONFLICT (content): Merge conflict in src/pricing.js
Automatic merge failed; fix conflicts and then commit the result.
```

Launch meld (assuming `merge.tool = meld`):

```
git mergetool src/pricing.js
```

Meld opens with three panels: LOCAL (your version) on the left, BASE in the centre, REMOTE (the incoming side) on the right. Use the arrow buttons to pull hunks from either side, edit the output panel directly for anything that needs a custom blend, then save and close.

Git stages the result automatically. Finish the merge:

```
git commit
```

Git opens your editor with a pre-filled merge commit message. Save and close to record the commit.

Batch-resolving many files after a rebase

You are rebasing a long-running integration branch onto a new base commit and fifteen files have conflicts.

```
git rebase --onto main upstream/main integration
```

```
CONFLICT (content): Merge conflict in src/api/auth.js
CONFLICT (content): Merge conflict in src/api/users.js
... (13 more) ...
error: could not apply alb2c3d... Add user session handling
```

Resolve them all in one pass, skipping the per-file prompt:

```
git mergetool --no-prompt
```

The tool opens once per file. Resolve and save each one; the tool advances automatically. After the last file, continue the rebase:

```
git rebase --continue
```

If further commits in the rebase also have conflicts, the cycle repeats until the rebase is complete.

Setting up vimdiff with a custom layout

The default vimdiff layout opens four windows (LOCAL, BASE, REMOTE across the top, MERGED filling the bottom). If you prefer three columns without BASE you can configure a custom layout:

```
git config --global merge.tool vimdiff
git config --global mergetool.vimdiff.layout "LOCAL,MERGED,REMOTE"
```

Launch it:

```
git mergetool --tool=vimdiff
```

Vim opens with LOCAL on the left, MERGED (the editable buffer) in the centre, and REMOTE on the right. Edit MERGED, resolve each conflict, then save and quit:

```
:wq      " save and mark as resolved
:cq      " quit without saving – mergetool will ask whether to retry
```

To use Neovim instead, set `--tool=nvimdiff` and configure `mergetool.nvimdiff.layout` independently (it falls back to the vimdiff layout if not set).

Configuring a custom tool

Any editor or diff program can be wired in via `mergetool.<tool>.cmd`. The shell command has access to four variables set by Git: `$BASE`, `$LOCAL`, `$REMOTE`, and `$MERGED`.

Example using `kdiff3` at a non-standard path:

```
git config --global merge.tool kdiff3
git config --global mergetool.kdiff3.path /opt/local/bin/kdiff3
```

Example using an entirely custom invocation:

```
git config --global merge.tool myfancytool
git config --global mergetool.myfancytool.cmd \
  'myfancytool "$LOCAL" "$REMOTE" "$BASE" -o "$MERGED"'
git config --global mergetool.myfancytool.trustExitCode true
```

Verify the tool is recognised:

```
git mergetool --tool-help
```

Recovery

If you want to abandon the resolution and start the underlying operation over from scratch, abort it rather than trying to manually unwind staged files:

```
# If you are mid-merge:
git merge --abort
```

```
# If you are mid-rebase:
git rebase --abort

# If you are mid-cherry-pick:
git cherry-pick --abort
```

Each command restores the working tree and index to the state before the operation began. Any `.orig` backup files written by `mergetool` are left behind — remove them with:

```
git clean -f "*.orig"
```

If you already ran `git commit` on a bad resolution, see *Getting out of jams* for how to undo a merge commit or revert a botched resolution without losing history.

See also

- *merge* — the command that produces the conflicts `mergetool` resolves.
- *rebase* — conflicts during rebase are resolved the same way; continue with `git rebase --continue`.
- *cherry-pick* — conflicts during cherry-pick; continue with `git cherry-pick --continue`.
- *rerere* — record and replay conflict resolutions to avoid resolving the same conflict twice.
- *Getting out of jams* — how to abort a merge, undo a bad resolution, or recover a lost working-tree state.

rebase

Reapply a sequence of commits on top of a different base commit, rewriting history to produce a cleaner, linear chain.

Mental model

Every commit in Git points to a parent. A branch is just a pointer to the tip of a chain of commits.

When you rebase branch `topic` onto `main`, Git:

1. Finds the common ancestor of `topic` and `main`.
2. Takes each commit on `topic` that came after that ancestor and turns it into a patch.
3. Resets `topic` to point at the tip of `main`.
4. Replays those patches, one by one, producing new commit objects with new hashes — the same changes, but with `main`'s tip as their ancestry.

Before:	After git rebase main:
<pre> A---B---C topic / D---E---F---G main</pre>	<pre> A'--B'--C' topic / D---E---F---G main</pre>

The prime marks (`A'` , `B'` , `C'`) are brand-new commit objects. The original `A` , `B` , `C` still exist in the object database until garbage-collected — they are simply no longer reachable from any branch pointer.

Interactive rebase (`-i`) adds an editing step: Git opens a to-do list of the commits to be replayed and lets you reorder, squash, drop, or reword them before any replaying happens. This makes

`git rebase -i` the primary tool for polishing a branch's history before a code review or merge.

Synopsis

```
git rebase [<upstream> [<branch>]]
git rebase --onto <newbase> [<upstream> [<branch>]]
git rebase -i [--autosquash] [--exec <cmd>] <upstream>
git rebase (--continue | --skip | --abort | --quit | --edit-todo | --show-current-patch)
```

Everyday usage

Update a feature branch to include the latest work from `main`:

```
git switch feature/login
git rebase main
```

The same result, specifying the branch explicitly (Git will switch to it first):

```
git rebase main feature/login
```

Interactively clean up the last four commits before opening a pull request:

```
git rebase -i HEAD~4
```

Git opens an editor showing the to-do list:

```
pick alb2c3 Add login form
pick d4e5f6 WIP wiring
pick 7890ab Fix typo in label
pick cdef01 Add tests for login
```

Change `pick` to the desired action, save, and close the editor. Common actions:

Action	Effect
<code>pick</code>	Keep commit as-is (default)
<code>reword</code>	Keep commit, edit its message
<code>edit</code>	Pause after applying so you can amend
<code>squash</code>	Fold into previous commit, combine messages
<code>fixup</code>	Fold into previous commit, discard this message
<code>drop</code>	Remove the commit entirely
<code>exec</code>	Run a shell command after the preceding commit
<code>break</code>	Pause here without applying any commit

Collapse `fixup!` and `squash!` commits created with `git commit --fixup` or `--squash` (see the *commit* chapter):

```
git rebase -i --autosquash origin/main
```

Transplant a topic branch from one base to another (see Worked examples):

```
git rebase --onto main next topic
```

Key options

Option	What it does	When to use it
<code>--onto <newbase></code>	Set a different landing point for the replayed commits	Transplant a branch to a new base that differs from its upstream
<code>--keep-base</code>	Use the merge base as the landing point rather than the upstream tip	Sync with an upstream that has advanced without moving your commits' base
<code>-i</code> , <code>--interactive</code>	Open an editor to reorder, squash, drop, or reword commits	Polishing a branch before review
<code>--autosquash</code>	Automatically arrange <code>fixup!</code> / <code>squash!</code> commits in the to-do list	Used with <code>git commit --fixup</code> usually paired with <code>-i</code>
<code>--autostash</code>	Stash dirty working tree before rebasing, reapply after	Rebase without a clean working tree
<code>-x <cmd></code> , <code>--exec <cmd></code>	Insert an <code>exec</code> line after every commit in the to-do list	Run tests or builds at each step

<code>--continue</code>	Resume after resolving a conflict or finishing an <code>edit</code>	Conflict resolution workflow
<code>--skip</code>	Discard the current conflicting commit and move on	When a commit is already superseded upstream
<code>--abort</code>	Stop and reset the branch to its state before the rebase started	Bail out entirely
<code>--quit</code>	Stop without resetting HEAD	Keep partial progress, clean up manually
<code>--edit-todo</code>	Re-open the interactive to-do list mid-rebase	Correct a mistake in the plan without aborting
<code>--show-current-patch</code>	Show the patch that caused a conflict or pause	Understand what Git is trying to apply
<code>-f</code> , <code>--force-rebase</code>	Recreate all commits as new objects even if fast-forward is possible	After reverting a merge, to re-merge cleanly
<code>--rebase-merges</code>	Preserve merge commits in the to-do list	Rebasing a branch with intentional merge topology
<code>--signoff</code>	Append <code>Signed-off-by</code> to every rebased commit	Projects requiring DCO sign-off
<code>-S</code> , <code>--gpg-sign</code>	GPG-sign every rebased commit	Verified commits on GitHub/GitLab
<code>--no-verify</code>	Skip the pre-rebase hook	Emergency use only
<code>--update-refs</code>	Force-update intermediate branch pointers within the rebased range	Rebasing a stack of dependent branches at once
<code>--root</code>	Rebase all commits reachable from the branch, including the root	Rewriting the entire history of a repository
<code>-X <option></code> , <code>--strategy-option</code>	Pass an option to the merge strategy (e.g. <code>-X theirs</code>)	Resolve systematic conflicts automatically

Best practices

Rebase local, merge public. Rebase is safe on commits that exist only on your machine. The moment a commit has been pushed and others may have based work on it, rewriting it breaks their history. Use merge for integrating shared branches; use rebase for cleaning up your own work before it becomes shared.

Set `rebase.autoSquash = true` globally when using `--fixup` commits. If you regularly create `fixup!` commits with `git commit --fixup`, enabling this config key means `git rebase -i` will always arrange them for you without requiring the `--autosquash` flag every time.

```
git config --global rebase.autoSquash true
```

Keep interactive rebases small. Rebasing 50 commits interactively is error-prone. If a branch has drifted that far, consider squashing the whole branch into a handful of meaningful commits rather than carefully editing each one.

Run tests after rebasing with `--exec`. When reshaping a branch, it is easy to create intermediate states that do not build. The `--exec` flag inserts a command after every replayed commit:

```
git rebase -i --exec "make test" origin/main
```

If any step fails, the rebase pauses so you can fix it before continuing.

Prefer `--onto` over manual branch gymnastics when transplanting. When you need to move a set of commits to an entirely different base, `--onto` expresses the intent clearly and lets Git figure out which commits to carry.

Use `--update-refs` when working on a stack of branches. If `feature/b` is based on `feature/a` and `feature/a` is based on `main`, rebasing `feature/b` onto a new `main` with `--update-refs` will also slide `feature/a`'s pointer forward. Without it, `feature/a` would still point to the old commits.

Pitfalls & gotchas

Rebasing pushed commits causes divergence. After `git push`, your branch and the remote are in sync. If you rebase and push again, the hashes change and the remote will reject the push unless you use `--force-with-lease`. Anyone who has fetched the original commits now has a divergent history. Only force-push to feature branches you own and have confirmed no one else is working from.

“Ours” and “theirs” are swapped during rebase. During a normal merge, “ours” is your branch. During a rebase, Git replays your commits on top of the upstream, so “ours” is the upstream content and “theirs” is your incoming commit. This surprises people the first time they read a rebase conflict marker.

Commits that duplicate upstream changes are silently dropped. If a patch in your branch is identical in content to one already merged upstream, Git skips it during the rebase. This is usually the right behavior, but it can surprise you if you expected that commit to appear in the result.

Interactive rebase on a branch with merge commits drops those merges by default. A plain `git rebase -i` linearizes history and removes any merge commits on your branch. Use `--rebase-merges` if preserving the topology matters.

`--autostash` can produce conflicts on reapplication. The autostash mechanism stashes your working tree, completes the rebase, then pops the stash. If rebased code changed the same lines as your stashed work, the stash pop will produce a conflict. Ensure your working tree is as clean as practical before rebasing.

Deleting a line in the to-do list silently drops the commit. Unlike changing `pick` to `drop`, deleting a line in the interactive editor removes the commit without any confirmation (unless `rebase.missingCommitsCheck` is set to `warn` or `error` in config). If you accidentally delete a line, abort and start over.

Worked examples

Updating a feature branch to track main

Your `feature/payments` branch was cut from `main` three days ago. Other work has landed on `main` since then and you want those changes available in your branch.


```
git fetch origin
git switch feature/payments
git rebase origin/main
```

If there are no conflicts, the branch is silently updated. If a conflict arises, Git pauses:

```
CONFLICT (content): Merge conflict in src/checkout.js
error: could not apply d4e5f6... Add Stripe webhook handler
hint: Resolve all conflicts manually, mark them with `git add`, and then run
hint: `git rebase --continue`.
```

Resolve the conflict, stage the file, and continue:

```
# edit src/checkout.js to resolve conflict markers
git add src/checkout.js
git rebase --continue
```

Git opens your editor to confirm or adjust the commit message, then replays the remaining commits. Repeat for any further conflicts.

Polishing a branch before a pull request

You have been working on a five-commit feature branch. Two commits are “WIP” checkpoints that should be folded into their neighbors, and one commit message needs rewording.

```
git rebase -i origin/main
```

The editor opens:

```
pick 1a2b3c Add product search index
pick 4d5e6f WIP: search controller
pick 7a8b9c Wire up search controller to router
pick 0d1e2f WIP: missing pagination
pick 3a4b5c Add pagination to search results
```

Edit the to-do list:

```
pick 1a2b3c Add product search index
squash 4d5e6f WIP: search controller
reword 7a8b9c Wire up search controller to router
squash 0d1e2f WIP: missing pagination
pick 3a4b5c Add pagination to search results
```

Save and close. Git will:

1. Squash `4d5e6f` into `1a2b3c`, prompting you to edit the combined message.
2. Replay `7a8b9c` and open the editor for a message update.
3. Squash `0d1e2f` into the now-reworded commit above.
4. Replay `3a4b5c` unchanged.

The result is three clean, well-named commits ready for review.

Transplanting a topic branch with `--onto`

Your branch `feature/dark-mode` was accidentally branched from `staging` instead of `main`. You want to move only the commits that belong to `feature/dark-mode` — not those inherited from `staging` — onto `main`.

```
    A---B---C  feature/dark-mode
    /
    D---E---F  staging
    /
o---o---o  main
```

```
git rebase --onto main staging feature/dark-mode
```

Git finds commits reachable from `feature/dark-mode` but not from `staging` (that is, `A`, `B`, `C`) and replays them on top of `main`:

```
        A'--B'--C'  feature/dark-mode
        /
o---o---o  main

    D---E---F  staging (untouched)
```

The `staging`-only commits are not carried along.

Autosquashing fixup commits

You have a three-commit branch. During review, a teammate points out a bug in the first commit (`1a2b3c`). Create a fixup commit targeting it:

```
git add src/header.js
git commit --fixup=1a2b3c
```

Git creates a commit titled `fixup! Add responsive header component`. Collapse it into place:

```
git rebase -i --autosquash origin/main
```

The to-do list opens with the fixup already moved below its target and marked for folding:

```
pick 1a2b3c Add responsive header component
fixup f9e8d7 fixup! Add responsive header component
pick 2b3c4d Add header unit tests
pick 5e6f7a Update navigation styles
```

Save without changes. The fixup disappears into `1a2b3c` and history stays clean.

Recovery

To abort a rebase in progress and restore the branch to its pre-rebase state:

```
git rebase --abort
```

To undo a completed rebase on a branch that has not yet been pushed, use `ORIG_HEAD`, which rebase sets to the pre-rebase tip:

```
git reset --hard ORIG_HEAD
```

Note that `ORIG_HEAD` can be overwritten by subsequent reset-style operations. If that has happened, use the reflog instead:

```
git reflog
# find the entry just before the rebase started, e.g.:
# HEAD@{4}: commit: Add responsive header component
git reset --hard HEAD@{4}
```

To recover a branch that was rebased and force-pushed, and you need to get back to the remote's previous state:

```
git fetch origin
git reset --hard origin/feature/my-branch@{1} # the reflog of the remote ref
```

See *Getting out of jams* for additional undo recipes including recovering from a force-push that discarded others' work.

See also

- *branch* — creating and managing the branches that rebase rewrites.
- *merge* — the alternative to rebase for integrating branches; understand when to use each.
- *commit* — `--fixup` and `--squash` flags that feed into `rebase --autosquash`.
- *cherry-pick* — applying individual commits without a full rebase.
- *reflog* — finding and recovering commit hashes after a rebase rewrites them.
- *Getting out of jams* — recovering from a botched rebase.

cherry-pick

Apply the changes introduced by one or more existing commits onto the current branch, creating a new commit for each.

Mental model

Every commit in Git records a *diff* — the delta between its parent tree and its own tree.

`git cherry-pick` takes that diff, replays it on top of your current HEAD, and records the result as a brand-new commit. The new commit has a different hash (it has a different parent, a different timestamp, and possibly a different author date) but carries the same logical change.

Think of cherry-pick as a targeted copy, not a move. The source commit stays exactly where it was; you are grafting an equivalent patch onto a different branch.

```
main:    A—B—C—D          (D introduced the bug fix you need)

feature: E—F—G            (your current branch, before)

After: git cherry-pick D

feature: E—F—G—D'         (D' is the replayed patch; D remains on main)
```

Cherry-pick uses the same three-way merge machinery as `git merge`. If the context around the changed lines differs enough from what the patch expects, you get a conflict, and you resolve it the same way you would a merge conflict.

Synopsis

```
git cherry-pick [--edit] [-n] [-m <parent-number>] [-s] [-x] [--ff]
                  [-S[<keyid>]] <commit>...
git cherry-pick (--continue | --skip | --abort | --quit)
```

Everyday usage

Copy a single commit to the current branch:

```
git cherry-pick a3f9c1b
```

Copy a range of commits (picks all commits reachable from `feature` but not from the current HEAD):

```
git cherry-pick main..feature
```

Copy several non-contiguous commits in one command:

```
git cherry-pick a3f9c1b e72d44f 09ab231
```

Stage the changes without creating commits yet — useful when you want to combine multiple picks into a single commit:

```
git cherry-pick -n a3f9c1b e72d44f
git commit -m "Combine two upstream fixes"
```

Backport a fix to a maintenance branch and annotate the new commit with where it came from:

```
git switch release/2.x
git cherry-pick -x a3f9c1b
```

Edit the commit message before the new commit is recorded:

```
git cherry-pick --edit a3f9c1b
```

Key options

Option	What it does	When to use it
<code>-e</code> , <code>--edit</code>	Open the editor to modify the commit message before committing	Adapting an upstream message to a backport context
<code>-x</code>	Append (cherry picked from commit ...) to the commit message	Backporting between public branches so the provenance is traceable
<code>-n</code> , <code>--no-commit</code>	Apply changes to the index and working tree without creating a commit	Combining multiple picks into one commit, or inspecting before committing
<code>-m <n></code> , <code>--mainline <n></code>	Specify which parent (1-based) is the mainline when picking a merge commit	Required when the commit to pick is a merge commit
<code>-s</code> , <code>--signoff</code>	Append a Signed-off-by trailer to the commit message	Projects requiring DCO sign-off
<code>-S[<keyid>]</code> , <code>--gpg-sign</code>	GPG-sign the resulting commit	Maintaining a verified-commit policy on the target branch
<code>--ff</code>	Fast-forward HEAD if the current commit is the parent of the picked commit	Applying a linear series cleanly without creating redundant merge objects
<code>--allow-empty</code>	Preserve commits that were initially empty	Carrying intentional empty commits (e.g. CI trigger points) across branches
<code>--empty=(drop keep stop)</code>	Control what happens when a picked commit becomes empty due to prior changes	<code>drop</code> to silently skip redundant commits; <code>keep</code> to preserve them; <code>stop</code> (default) to pause and inspect
<code>--strategy=<strategy></code>	Choose a merge strategy (e.g. <code>ort</code> , <code>recursive</code>)	Switching strategies when the default produces avoidable conflicts
<code>-X<option></code> , <code>--strategy-option=<option></code>	Pass an option through to the merge strategy	Trying <code>-Xpatience</code> when context-line mismatches cause unnecessary conflicts

<code>--rerere-autoupdate</code>	Let rerere automatically stage its resolved hunks	Speeding up repetitive conflict resolution on a long backport series
<code>--continue</code>	Resume after resolving conflicts	Picking up a multi-commit sequence after fixing conflicts
<code>--skip</code>	Skip the current conflicting commit and continue the sequence	Intentionally omitting one commit from a range pick
<code>--abort</code>	Cancel the entire operation and restore pre-pick state	Abandoning a pick that is too conflicted to proceed
<code>--quit</code>	Clear the sequencer state without restoring the working tree	Keeping partial progress while discarding the sequencer bookkeeping

Best practices

Prefer merging or rebasing over cherry-picking for whole-branch integration. Cherry-pick duplicates commits — the same logical change exists twice in history under different hashes. If you later merge the original branch, Git has no way to know those patches are equivalent, and the same change can appear twice in the final log or cause spurious conflicts. Reserve cherry-pick for cases where you genuinely need only *some* commits from another branch, not the whole thing.

Use `-x` when backporting to shared branches. The annotation

(cherry picked from commit abc1234) lets anyone who bisects or audits the maintenance branch trace the fix back to its origin. Omit `-x` only when picking from a private or short-lived branch where the source hash is meaningless to future readers.

Pick the smallest meaningful unit. If the commit you need is entangled with unrelated changes, consider splitting it on a scratch branch first (see the *commit* chapter on `--patch` staging), then picking the focused result. A clean pick is easier to review and much easier to revert if something goes wrong.

Resolve conflicts promptly during a sequence. When picking a range and a conflict appears, Git halts with the sequencer state recorded in `.git/sequencer`. Fix the conflict, stage the resolved files, then run `git cherry-pick --continue`. Leaving the repository in a mid-pick state invites accidental edits that complicate resolution.

Test after picking, especially on maintenance branches. A patch that applies cleanly at the hunk level does not guarantee correct behavior in the target codebase. The surrounding code may have diverged since the original commit was written. Run at minimum the tests that exercise the changed area before pushing.

Pitfalls & gotchas

Picking a merge commit requires `-m`. A merge commit has two or more parents. Git cannot determine which side represents the mainline without being told. Omitting `-m` causes an immediate error. Pass `-m 1` to treat the first parent as the mainline, which is the branch that was active when the merge was made.

Duplicate commits after merging the source branch later. If you cherry-pick commit D from `feature` into `main`, and then later merge `feature` into `main`, the same change appears in history twice under different hashes. `git log` will show both; `git blame` will attribute lines to whichever

commit landed last. Use cherry-pick with the awareness that the source branch should not be merged back unless you are prepared to deal with this duplication.

Empty-commit behavior is not obvious. If the change you are picking has already been applied to the target branch by another path, the resulting commit is empty — no diff. By default (`--empty=stop`) Git halts and asks what to do. During a long range pick this can be surprising. Use `--empty=drop` to silently skip such commits when you know the target branch has overlapping history.

--no-commit picks up anything already staged. When you use `-n` and there are already staged changes in your index, those changes will be included in whatever commit you eventually run. This is intentional and useful, but if you are not expecting it, you can accidentally bundle unrelated work into one commit. Check `git diff --cached` before committing.

Conflict markers follow the merge layout. The `<<<<<<< HEAD` side is your current branch; the `>>>>>>> <hash>` side is the commit being picked. Read and resolve them exactly as you would a merge conflict.

--abort discards all in-progress picks. If you are in the middle of a ten-commit range pick and abort, Git restores the branch pointer to the state before the very first commit in the sequence. Any commits that had already been created during the sequence are still reachable through the reflog, but your branch pointer is rolled back.

Worked examples

Backporting a security fix to a maintenance branch

Your team has just merged a security fix (`e8b3f12`) into `main`. The same vulnerability exists in the `release/3.2` maintenance branch.

```
# Start from the maintenance branch
git switch release/3.2

# Pick the fix and annotate it with its origin
git cherry-pick -x e8b3f12
```

If it applies cleanly, Git creates a new commit on `release/3.2` with the message from `e8b3f12` plus the annotation line:

```
Fix SQL injection in user search endpoint

(cherry picked from commit e8b3f12a94c1d7b0e5f3a...)
```

Run your test suite, then push the maintenance branch.

Picking a range with a conflict mid-sequence

You want to port three commits from `experiment` to `main`:

```
git switch main
git cherry-pick experiment~3..experiment # picks the three newest commits
```

The second commit conflicts. Git halts:

```
error: could not apply 7a4c91b... Refactor query builder
hint: After resolving the conflicts, mark them with
hint: "git add/rm <paths>", then run
hint: "git cherry-pick --continue".
```

Open the conflicting file, resolve the conflict markers, then continue:

```
git add src/query.js
git cherry-pick --continue
```

Git applies the third commit automatically and finishes the sequence. To abandon the entire operation instead:

```
git cherry-pick --abort
```

Your branch returns to the state before the first pick.

Combining multiple upstream commits into one

You want to pull two small upstream refactoring commits into a feature branch as a single clean commit:

```
git switch my-feature

# Apply both sets of changes to the index without committing
git cherry-pick -n c3d1e2f 88fa012

# Verify what will be committed
git diff --cached

# Create one unified commit
git commit -m "Apply upstream query-builder refactors"
```

Picking a merge commit

A hotfix was delivered as a merge commit `m1a2b3c` on `main`, and you need it on `hotfix/4.1`. The merge's first parent is `main`, so pass `-m 1`:

```
git switch hotfix/4.1
git cherry-pick -m 1 m1a2b3c
```

Without `-m`, Git refuses: `is a merge but no -m option was given`.

Retrying a conflicted pick with a different strategy

The default merge strategy produces conflicts because the surrounding context lines have shifted significantly since the original commit. Try the `patience` diff algorithm:

```
# First attempt fails with conflicts; abort it
git cherry-pick a3f9c1b
git cherry-pick --abort

# Retry with a more careful context-matching strategy
git cherry-pick -Xpatience a3f9c1b
```


Recovery

To undo the most recent cherry-pick commit while keeping the changes staged:

```
git reset --soft HEAD~1
```

To discard both the commit and the changes:

```
git reset --hard HEAD~1
```

To create a proper inverse commit instead of erasing history — the right choice after pushing — see the *revert* chapter.

If you used `--abort` but want to recover commits that were created before the abort, they are still reachable through the reflog:

```
git reflog
# Identify the hash of the last commit made during the sequence, then
git cherry-pick <that-hash>
```

See *Getting out of jams* for broader undo recipes, including recovering from a hard reset.

See also

- *revert* — create an inverse commit to undo a change without rewriting history.
- *rebase* — replay an entire branch's commits onto a new base; prefer over cherry-pick when you want to move all commits from one branch to another.
- *commit* — details on commit messages, `--allow-empty`, and staging with `--patch`.
- *merge* — the three-way merge machinery that cherry-pick relies on for conflict resolution.
- *Getting out of jams* — undoing picks and resolving a stuck sequencer.

revert

Create a new commit that undoes the changes introduced by one or more earlier commits, leaving the full history intact.

Mental model

Every commit in Git is permanent — you cannot delete a commit that has already been shared.

`git revert` works around this by going forward, not backward: it reads the diff that a target commit introduced, flips it (additions become deletions, deletions become additions), and records the result as a brand-new commit on top of HEAD.

```
Before:
A — B — C — D   (HEAD)

After git revert C:
A — B — C — D — C' (HEAD)
                  C' undoes exactly what C added
```

The repository's object database is append-only. C is still there, still reachable, still auditable. The revert is itself a permanent record: it documents what was undone and when. This is the fundamental difference between `revert` and `reset`: `reset` moves a branch pointer to an earlier commit (rewriting or discarding history), while `revert` adds to history. Use `revert` whenever the commit you want to undo has already been pushed to a shared remote.

Internally, reverting is a three-way merge. Git takes the state at the target commit's parent as the “base”, the current HEAD as “ours”, and the inverse of the target diff as “theirs”. If other commits have touched the same lines since the target commit was created, a conflict can arise — the same way merging two branches can conflict.

Synopsis

```
git revert [--no-edit] [-n] [-m <parent-number>] [-s] [-S[<keyid>]] <commit>...
git revert (--continue | --skip | --abort | --quit)
```

Everyday usage

Revert the most recent commit:

```
git revert HEAD
# editor opens for the commit message; save and close to complete
```

Revert a specific earlier commit by its hash:

```
git revert a3f9c1b
```

Revert without opening the editor (accept the generated message as-is):

```
git revert --no-edit HEAD
```

Revert a range of commits, creating one revert commit per original commit:

```
git revert --no-edit HEAD~3..HEAD
```

Stage the revert changes but do not commit yet – useful when you want to combine several reverts into a single commit:

```
git revert -n HEAD~2 HEAD~1 HEAD
git commit -m "Revert the broken payment refactor"
```

Key options

Option	What it does	When to use it
<code>-e</code> , <code>--edit</code>	Open the commit-message editor before committing (default in a terminal)	Explain <i>why</i> the commit is being reverted
<code>--no-edit</code>	Skip the editor and use the generated message	Quick reverts in scripts or when the message is self-explanatory
<code>-n</code> , <code>--no-commit</code>	Apply the inverse diff to the index and working tree but do not commit	Combine several reverts into one commit, or inspect changes before committing
<code>-m <parent-number></code>	Specify which parent of a merge commit is the mainline (1-based)	Required when reverting a merge commit, which has two parents
<code>--cleanup=<mode></code>	Control how the commit message is cleaned up before storage	Use <code>scissors</code> to get a clear conflict-marker section in the message
<code>-s</code> , <code>--signoff</code>	Append a Signed-off-by trailer to the message	Projects requiring DCO sign-off
<code>-S[<keyid>]</code> , <code>--gpg-sign</code>	GPG-sign the revert commit	Maintaining a verified-commits policy
<code>--no-gpg-sign</code>	Override <code>commit.gpgSign</code> config for this operation	Suppress signing when it is not needed
<code>--strategy=<strategy></code>	Use a specific merge strategy	Rare; prefer the default strategy
<code>-X<option></code> , <code>--strategy-option</code>	Pass an option to the merge strategy	e.g. <code>-Xours</code> to auto-resolve conflicts in favour of HEAD
<code>--rerere-autoupdate</code>	Let rerere automatically stage its recorded resolution	Speed up repeated conflict resolution on long revert sequences
<code>--no-rerere-autoupdate</code>	Prevent rerere from auto-staging	Inspect rerere's suggestion before accepting it
<code>--reference</code>	Use a short <code>--pretty=reference</code> citation in the message body instead of the full object name	Cleaner, more readable revert messages
<code>--continue</code>	Resume a revert sequence after resolving conflicts	After <code>git add</code> on conflicted files

<code>--skip</code>	Skip the current commit in a multi-commit revert sequence	When the commit's changes are already absent and reverting would produce an empty diff
<code>--abort</code>	Cancel the entire revert sequence and restore HEAD to its pre-revert state	When you decide not to proceed
<code>--quit</code>	Clear the sequencer state without restoring HEAD	When you have already committed partial reverts and want to stop cleanly

Best practices

Always explain why the commit is being reverted. Git generates a subject line for you (Revert "Add payment gateway"), but the body is empty. Use the editor to record the defect number, the incident, or the reasoning. A future reader looking at `git log` deserves to understand what went wrong.

```
Revert "Add payment gateway"

This reverts commit a3f9c1b.

The gateway integration broke 3DS authentication for Visa cards
(incident INC-2847). Reverting until the provider fixes their SDK.
```

Use `--no-edit` only in automation, not in interactive work. The generated message is a bare minimum. In CI or scripted hotfix pipelines `--no-edit` is fine; when you are working interactively, open the editor and add context.

Combine related reverts with `-n`. When reverting a feature that spans several commits, revert them all with `--no-commit` first, then write a single cohesive commit message that explains the full picture. One revert commit is easier to read in `git log` than four separate ones.

Watch commit order when reverting a range. Git reverts commits in the range newest-first. If commit B depends on commit A, reverting B before A is the correct order — which is also the natural order Git uses for a range like `A..B`.

Avoid stacking “Reapply Reapply” subject lines. If you later need to re-apply a reverted commit, do not blindly revert the revert. Use `git cherry-pick` on the original commit, or revert the revert but immediately amend the message to something meaningful and shorter.

Pitfalls & gotchas

Reverting a merge commit requires `-m`. A merge commit has two parents. Git does not know which parent represents the mainline (the branch you merged into). Without `-m 1` (or `-m 2`) the command fails with an error. `-m 1` almost always means “the branch the merge landed on” — verify with `git log --oneline --graph` before deciding.

Reverting a merge does not prevent future merges from re-introducing the same changes. Once you revert a merge commit, Git considers those commits to have already been incorporated. If you later re-merge the same branch, only commits made *after* the original merge are brought in. To re-introduce the reverted work you must revert the revert commit first, or cherry-pick individual commits manually.

A clean working tree is required. `git revert` refuses to start if you have uncommitted modifications. Stash or commit your work-in-progress first.

```
git stash
git revert HEAD
git stash pop
```

Conflicts are possible. If later commits modified the same lines that the target commit introduced, the three-way merge will conflict. Resolve conflicts the same way you would during a merge: edit the conflicted files, `git add` them, then `git revert --continue`. To abandon the revert entirely, run `git revert --abort`.

-n leaves the index in a modified state. After `--no-commit`, your index contains staged revert changes. `git status` shows them as staged files. This is intentional — finish with `git commit` when you are satisfied with the combined diff.

Repeatedly reverting reverts produces unwieldy subject lines. Reverting a revert yields `Revert "Revert "Add payment gateway""`, and so on indefinitely. When re-applying previously reverted work, prefer `git cherry-pick` on the original commit, or rewrite the message after reverting the revert.

Worked examples

Reverting a bad hotfix on a shared branch

You merged a hotfix this morning. It broke production. The offending commit is `d4e9ab2`. The branch has been pushed and your teammates have pulled it, so rewriting history is off the table.

```
# confirm what d4e9ab2 changed before reverting
git show d4e9ab2

# revert it – explain the incident in the editor
git revert d4e9ab2
```

The editor opens with a pre-filled subject line. Add context in the body:

```
Revert "Fix null check in order processor"

This reverts commit d4e9ab2.

Caused NullPointerException for orders without a shipping address
(incident INC-3012). Reverting until a proper fix is in place.
```

Save, close, and push:

```
git push origin main
```

The bad code is gone from the live codebase. Both the original commit and the new revert commit remain in history, giving a full audit trail.

Reverting a multi-commit feature in one step

A three-commit feature (API layer, service layer, UI) must be pulled due to a compliance issue. The commits are `b1`, `b2`, `b3` (oldest to newest). You want to undo all three in a single, clearly-explained commit.

```
# stage the inverse of all three without committing
git revert -n b1 b2 b3
```

Git applies each inverse diff to the index. Review the staged changes:

```
git diff --staged
```

When satisfied, commit with a single informative message:

```
git commit -m "Revert feature/pqi-logging (compliance block)

Reverts commits b1, b2, b3.

PCI-DSS audit found that request-body logging in the API layer
captures raw card numbers. Reverting until a scrubbing layer
is in place. Tracked in JIRA-8821."
```

Reverting a merge commit

Your team merged `feature/dark-mode` into `main` three days ago. The feature ships a setting that corrupts user preferences on iOS. You need to revert the merge.

```
# find the merge commit
git log --oneline --graph main | head -15
```

```
* f7c3d01 Merge branch 'feature/dark-mode'
| \
| * 9a1b2c3 Add dark-mode toggle
| * 4d5e6f7 Persist theme preference
* | 8g9h0i1 Fix checkout button alignment
```

The merge commit is `f7c3d01`. Its first parent (`8g9h0i1`) is the main branch tip before the merge; its second parent (`9a1b2c3`) is the feature branch. Declaring parent 1 as the mainline means “keep what main had; undo what the feature added”:

```
git revert -m 1 f7c3d01
```

This produces a commit that restores `main` to its pre-merge state without touching the commits on `feature/dark-mode` themselves.

Recovery

To cancel a revert that is in progress (conflicts not yet resolved):

```
git revert --abort
```

If the revert has been committed but not yet pushed, roll back with `reset` :

```
git reset --soft HEAD~1 # uncommit, keep changes staged
git reset HEAD~1        # uncommit and unstage (working tree intact)
```

If the revert has already been pushed to a shared branch, undo it by reverting the revert:

```
git revert HEAD # creates a new commit that cancels the last revert
```

See *Getting out of jams* for more undo recipes and guidance on choosing between `reset`, `restore`, and `revert`.

See also

- `commit` — the `-m`, `--no-edit`, `-s`, and `-S` flags work the same way.
- `cherry-pick` — applying a specific commit's changes to a different branch; the complement of `revert`.
- `reset` — moving a branch pointer backward; the right tool when history has not yet been shared.
- *Getting out of jams* — choosing between `revert`, `reset`, and `restore`.

tag

Mark a specific commit with a permanent, human-readable name — typically to record a release point.

Mental model

Every commit has a hash: `a3f9c1d...`. Hashes are precise but meaningless to humans. A tag is a named pointer into the object database that says “this commit is `v2.4.0`”. Once created, the name never moves on its own (unlike a branch, which advances with every new commit).

Git has two kinds of tags:

- **Lightweight tag.** A bare ref — a file in `refs/tags/` that contains a commit hash and nothing more. Think of it as a sticky note.
- **Annotated tag.** A full object stored in the object database. It records the tagger name and email, a creation timestamp, a message, and optionally a GPG signature. Think of it as a signed certificate.

```
Lightweight:  refs/tags/v2.4.0 → commit a3f9c1d
```

```
Annotated:   refs/tags/v2.4.0 → tag object → commit a3f9c1d
                                     (tagger, date, message, sig)
```

The distinction matters: commands like `git describe` ignore lightweight tags by default, and many hosting platforms show the annotated tag message on the release page. Use annotated tags for anything you intend to share.

Synopsis

```
# Create
git tag [-a | -s | -u <key-id>] [-f] [-m <msg> | -F <file>] [-e]
      <tagname> [<commit> | <object>]

# List
git tag [-l | --list] [--sort=<key>] [-n<num>]] [--contains <commit>]
      [--no-contains <commit>] [--points-at <object>]
      [--merged <commit>] [--no-merged <commit>] [<pattern>...]

# Delete
git tag -d <tagname>...

# Verify
git tag -v <tagname>...
```

Everyday usage

Create an annotated tag at HEAD for a release:

```
git tag -a v1.0.0 -m "First stable release"
```

Create a lightweight tag (useful for private bookmarks):

```
git tag wip-before-refactor
```

Tag a specific earlier commit by its hash:


```
git tag -a v0.9.1 -m "Backfill release tag" 7c3e8a2
```

List all tags:

```
git tag
```

List tags matching a pattern (shell wildcard):

```
git tag -l 'v1.*'
```

List tags with the first line of their annotation:

```
git tag -n
```

List tags with up to three annotation lines:

```
git tag -n3
```

Push a single tag to the remote:

```
git push origin v1.0.0
```

Push all local tags that the remote does not yet have:

```
git push origin --tags
```

Delete a local tag:

```
git tag -d v0.9.0-rc1
```

Delete a remote tag:

```
git push origin --delete v0.9.0-rc1
```

Key options

Option	What it does	When to use it
<code>-a</code> , <code>--annotate</code>	Create an unsigned annotated tag object	All public/release tags
<code>-s</code> , <code>--sign</code>	Create a GPG-signed annotated tag using your default key	Release tags that need authenticity guarantees
<code>-u <key-id></code> , <code>--local-user=<key-id></code>	Sign with a specific GPG key	Multiple signing identities on one machine
<code>--no-sign</code>	Override <code>tag.gpgSign = true</code> in config	Create an unsigned tag when signing is globally enforced

<code>-f, --force</code>	Replace an existing tag rather than fail	Retag during local-only development; avoid after pushing
<code>-d, --delete</code>	Delete one or more tags by name	Removing a mistaken or superseded tag
<code>-v, --verify</code>	Verify the GPG signature of a tag	Confirming a signed release before using it
<code>-m <msg>, --message=<msg></code>	Set the tag message from the command line; implies <code>-a</code>	Short release notes
<code>-F <file>, --file=<file></code>	Read the tag message from a file; use <code>-</code> for stdin; implies <code>-a</code>	Long changelogs or templated messages
<code>-e, --edit</code>	Open the editor even when <code>-m</code> or <code>-F</code> supplied the message	Fine-tune a scripted or templated message before saving
<code>-l, --list</code>	List tags, optionally filtered by a shell-wildcard pattern	Browsing or scripting tag lists
<code>-n[<num>]</code>	Print <code><num></code> lines of the annotation alongside the tag name; implies <code>--list</code>	Quick overview of release notes
<code>--sort=<key></code>	Sort output by the given key; prefix <code>-</code> for descending; <code>version:refname</code> sorts semantically	Listing tags in version order
<code>--contains <commit></code>	List only tags that contain the given commit; implies <code>--list</code>	Finding which releases include a specific fix
<code>--no-contains <commit></code>	List only tags that do not contain the given commit	Finding releases before a particular change
<code>--points-at <object></code>	List only tags pointing at the given object; implies <code>--list</code>	Finding what a specific commit was tagged as
<code>--merged <commit></code>	List tags reachable from the given commit	Finding all releases included in a branch
<code>--no-merged <commit></code>	List tags not reachable from the given commit	Finding releases not yet integrated
<code>--format=<format></code>	Custom output format using <code>%(fieldname)</code> placeholders	Scripting and automation

Best practices

Always use annotated tags for releases. A lightweight tag is just a pointer; it carries no authorship, no timestamp, and no message. Tools like `git describe`, hosting platforms, and package managers expect annotated tags. Run `git tag -a v2.0.0 -m "Release v2.0.0"` — the three extra characters save you confusion later.

Follow a consistent naming scheme and stick to it. The most widely understood scheme is `v<major>.<minor>.<patch>` following Semantic Versioning (`v1.4.0`, `v1.4.1`, `v2.0.0-rc1`).

Consistency lets shell wildcards work cleanly (`git tag -l 'v1.4.*'`) and makes `--sort=version:refname` useful.

Tag the commit, not the branch tip. A branch tip changes with every push. Before tagging, confirm you are on the exact commit you intend to mark — check with `git log -1 --oneline`. If you tag HEAD immediately after merging and before anyone pushes again, this is usually fine; otherwise supply the commit hash explicitly.

Sign release tags when authenticity matters. If users download release tarballs or pull your tag by name, a GPG-signed tag lets them verify that the tag came from you and that the tagged commit has not been replaced. Configure `user.signingKey` in git config once, then use `git tag -s` for every release.

Push tags explicitly; do not rely on `--follow-tags` as your only path. Tags are not pushed with `git push` by default. A common workflow is to push the branch, then immediately push the tag:

```
git push origin main
git push origin v1.4.0
```

Using `git push --follow-tags` pushes only reachable annotated tags automatically, but it is easy to forget that the flag exists. An explicit push makes intent clear in CI logs and scripts.

Never move a published tag. Once a tag has been pushed and others may have fetched it, moving it with `-f` creates a silent divergence: your copy and their copy point to different commits under the same name. If you tagged the wrong commit, acknowledge the mistake publicly (see *Pitfalls & gotchas*) and publish a corrected tag under a new name.

Pitfalls & gotchas

Tags are not pushed automatically. `git push origin main` leaves all your local tags behind. Many developers have shipped a release and only discovered the tag was missing when the CI pipeline or package registry tried to read it. Always push the tag as a separate step.

Deleting a remote tag requires a separate command. `git tag -d v0.9.0` removes the tag locally but leaves it on the remote. To remove both:

```
git tag -d v0.9.0
git push origin --delete v0.9.0
```

Forgetting the second step means the tag lives on indefinitely for anyone who pulls from the remote.

-f after publishing is a trap. Force-retagging a published tag updates your local ref and updates the remote if you force-push the tag, but other people who already fetched the old tag will not receive the update on a normal `git pull` or `git fetch`. They silently keep the old commit under the same name. See the On Re-tagging discussion in `git tag --help` and the Recovery section below for the correct procedure.

Lightweight tags are invisible to `git describe`. `git describe` finds the nearest annotated tag reachable from a commit. If you used lightweight tags for releases, `git describe` will skip them and either report a much older annotated tag or fail entirely. Retrofit annotated tags with:

```
git tag -a -f v1.2.0 v1.2.0 -m "v1.2.0"
```

(This replaces the lightweight tag locally; push the new object with `git push origin --force v1.2.0` — after notifying anyone who uses the tag.)

Pattern-matching requires `-l`. Running `git tag v1.*` does not list tags matching `v1.*` it tries to create a tag named `v1.*`. You need `-l` (or `--list`) when supplying a pattern:

```
git tag -l 'v1.*'    # correct
git tag 'v1.*'       # creates a tag named literally "v1.*"
```

Tags name objects, not just commits. You can tag any Git object — a tree, a blob, even another tag. This is rarely useful in day-to-day work, but it means `git tag -a` on an arbitrary object hash works. Verify what you are tagging with `git cat-file -t <hash>` if you are unsure.

Worked examples

Tagging a release with a changelog message

Your team is cutting `v3.1.0`. You want the tag to contain a brief changelog so the release page on GitHub shows useful information.

```
# Confirm you're on the right commit
git log -1 --oneline
# a3f9c1d Merge pull request #198: Add retry logic for transient errors

# Write the message in an editor (opens $GIT_EDITOR)
git tag -a v3.1.0

# Or supply it on the command line
git tag -a v3.1.0 -m "$(cat <<'EOF'
Release v3.1.0

Changes since v3.0.2:
- Add retry logic for transient network errors (#198)
- Fix race condition in connection pool teardown (#201)
- Bump minimum Go version to 1.22
EOF
)"

# Push the tag
git push origin v3.1.0
```

On GitHub, the text between the tag name and the end of the message appears as the release description when you create a release from the tag.

Sorting tags by version number to find the latest release

Tags sorted lexicographically put `v1.10.0` before `v1.9.0`. Use semantic version sorting instead:

```
git tag -l --sort=-version:refname 'v*'
```

```
v3.1.0
v3.0.2
v3.0.1
v3.0.0
```

```
v2.4.0
...
```

The `-` prefix reverses the order so the newest tag appears first. This is useful in scripts that need to derive the previous release:

```
PREV=$(git tag -l --sort=-version:refname 'v*' | sed -n '2p')
echo "Comparing v3.1.0 against $PREV"
git log ${PREV}..v3.1.0 --oneline
```

Finding which release first included a specific commit

A bug was fixed in commit `7c3e8a2`. You want to know which release tag first contained that fix:

```
git tag -l --contains 7c3e8a2 --sort=version:refname | head -1
```

```
v2.3.1
```

The `--contains` filter lists every tag reachable from the given commit. Sorting by version and taking the first result gives you the earliest release.

Creating and verifying a signed release tag

Assuming a GPG key is configured (`user.signingKey` in git config):

```
git tag -s v4.0.0 -m "Release v4.0.0"
```

Verify the signature before distributing the tag:

```
git tag -v v4.0.0
```

```
object a3f9c1d...
type commit
tag v4.0.0
tagger Alice <alice@example.com> 1750000000 +0200

Release v4.0.0
gpg: Signature made ...
gpg: Good signature from "Alice <alice@example.com>"
```

A collaborator who has imported your public key can run the same `git tag -v` after fetching the tag to confirm its authenticity.

Recovery

Deleted a local tag by mistake. If the tag still exists on the remote, fetch it back:

```
git fetch origin tag v1.4.0
```

If the tag existed only locally and is gone, you can recreate it by pointing to the commit hash — find the hash in `git log` or `git reflog`, then:

```
git tag -a v1.4.0 <commit-hash> -m "Restore v1.4.0"
```

Tagged the wrong commit before pushing. Delete the local tag and recreate it on the correct commit:

```
git tag -d v1.4.0  
git tag -a v1.4.0 <correct-commit> -m "Release v1.4.0"
```

Tagged the wrong commit after pushing. This is the hard case. Moving a published tag will confuse anyone who already fetched it. The recommended approach is to publish a corrected tag under a new name (e.g. `v1.4.1` or `v1.4.0-fixed`) and announce the mistake. If you must reuse the same name, tell affected users to delete their local copy and re-fetch:

```
# On each affected machine:  
git tag -d v1.4.0  
git fetch origin tag v1.4.0
```

Then on your side, force-push the corrected tag:

```
git tag -f v1.4.0 <correct-commit>  
git push --force origin v1.4.0
```

See *Getting out of jams* for broader undo recipes involving published history.

See also

- *commit* — creating the commits that tags point to.
- *describe* — generating version strings from the nearest annotated tag.
- *push* — getting tags onto the remote with `--tags` or `--follow-tags`.
- *log* — `git log v1.3.0..v1.4.0` to list commits between two tags.
- *Getting out of jams* — recovering from published-history mistakes.

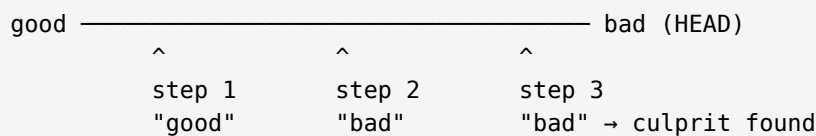
bisect

Use binary search across commit history to find the exact commit that introduced a bug or changed a property of the project.

Mental model

Imagine 1 000 commits between a version you know works and the HEAD you know is broken. Testing every commit by hand would take hours. Binary search cuts that to roughly ten steps: test the commit in the middle, decide “still broken” or “this one works”, and halve the remaining range. That is exactly what `git bisect` automates.

When you start a session you hand Git two anchor points: one commit where the behaviour is **bad** and one where it was **good**. Git checks out the midpoint, you test it and report back, and Git narrows the window. After $\log_2(N)$ rounds it prints the first bad commit and leaves `refs/bisect/bad` pointing at it.



The working tree is **checked out** at each candidate commit, so your build and test tooling runs against the real source code at that point in history. When the session is finished, `git bisect reset` returns you to where you started.

Git does not have to be searching for a regression. If the two poles are called **old** and **new** instead of good and bad, you can locate the first commit where any property appears — a fix, a naming change, a performance improvement.

Synopsis

```
git bisect start [--term-(bad|new)=<term> --term-(good|old)=<term>]
                  [--no-checkout] [--first-parent]
                  [<bad> [<good>...]] [--] [<pathspec>...]
git bisect (bad|new|<term-new>) [<rev>]
git bisect (good|old|<term-old>) [<rev>...]
git bisect skip [(<rev>|<range>)...]
git bisect reset [<commit>]
git bisect terms [--term-(good|old) | --term-(bad|new)]
git bisect (visualize|view)
git bisect log
git bisect replay <logfile>
git bisect run <cmd> [<arg>...]
git bisect help
```

Everyday usage

Manual bisect — three-step session

```
git bisect start
git bisect bad           # current HEAD is broken
git bisect good v1.8.0    # this tag was working

# Git checks out the midpoint.
```

```
# Build/test, then mark it:
git bisect good          # or: git bisect bad

# Repeat until Git prints the culprit:
# "abc1234 is the first bad commit"

git bisect reset          # restore HEAD and clean up
```

Supply anchors on the start line

```
git bisect start HEAD v1.8.0 -- src/
# HEAD is bad, v1.8.0 is good, only consider commits touching src/
```

Automated bisect with a test script

```
git bisect start HEAD v1.8.0 --
git bisect run ./scripts/check.sh
git bisect reset
```

`check.sh` exits 0 when the build/test passes (good) and 1 when it fails (bad). Git drives the whole search without further input.

Skip an untestable commit

```
git bisect skip          # skip the currently checked-out commit
git bisect skip v2.5..v2.6 # skip an entire range
```

Inspect progress

```
git bisect log           # show every decision made so far
git bisect visualize     # open gitk (or git log) showing candidates
git bisect visualize --oneline # compact list in the terminal
```

Key options

Option / subcommand	What it does	When to use it
<code>start</code>	Begin (or restart) a bisect session	Always the first step
<code>bad [<rev>]</code>	Mark a commit as bad (contains the bug)	After each checkout, or at start
<code>good [<rev>]</code>	Mark a commit as good (bug absent)	After each checkout, or at start
<code>skip [<rev>...]</code>	Tell Git to skip untestable commits	Build broken, flaky test, unrelated failure
<code>reset [<commit>]</code>	End the session and restore HEAD	When done or abandoning mid-session
<code>run <cmd></code>	Automate by delegating to a script	When you have a reliable pass/fail test
<code>log</code>	Print the record of all decisions	Reviewing progress; saving for replay

<code>replay <file></code>	Re-execute a saved log	Recovering from a wrong mark
<code>visualize / view</code>	Open <code>gitk</code> or <code>git log</code> on remaining suspects	Quick visual sanity-check
<code>terms</code>	Show which terms (good/bad or custom) are active	Mid-session reminder
<code>--no-checkout</code>	Update <code>BISECT_HEAD</code> but do not touch the working tree	Non-filesystem tests
<code>--first-parent</code>	Follow only first-parent on merge commits	Pinning regressions to a merge, ignoring branch history
<code>--term-bad=<term> /</code> <code>--term-new=<term></code>	Use a custom label instead of “bad”	Searching for a fix, a new feature, a performance change
<code>--term-good=<term> /</code> <code>--term-old=<term></code>	Use a custom label instead of “good”	Same as above, paired with <code>--term-bad / --term-new</code>

Best practices

Mark the anchors precisely before you start searching. The wider your good–bad range, the more steps bisect needs. If you have a rough idea which release introduced the problem, start from there rather than the project’s first commit. Every halving you do manually with a tighter range saves a round inside the session.

Automate whenever you have a reliable test. A hand-run bisect is error- prone: it is easy to mark a commit wrong after a distraction. `git bisect run` eliminates that risk. Even a one-liner is enough:

```
git bisect run sh -c "make -q && ./bin/test_case"
```

Use `skip` freely for build failures unrelated to the bug. If the midpoint commit simply does not compile due to an unrelated issue, skip it. Git will find another candidate nearby. Do not mark it bad — that would poison the search.

Restrict the search with a `paths`pec. If you know the bug lives in a subsystem, pass `-- <path>` to `start`. Git will only consider commits that touched those paths, dramatically reducing the search space.

```
git bisect start HEAD v2.0.0 -- lib/auth/
```

Save the log before resetting. If you might need to redo or audit the session, capture it first:

```
git bisect log > bisect-session.log
git bisect reset
```

Use custom terms when searching for a non-regression. Labelling commits “broken” and “fixed”, or “slow” and “fast”, makes the session self- documenting and avoids the confusion of calling a desired state “bad”.

```
git bisect start --term-old broken --term-new fixed
git bisect fixed           # HEAD has the fix
git bisect broken v3.0.0   # this tag is before the fix
```

Pitfalls & gotchas

Forgetting to call `reset` leaves your repo mid-session. If you close the terminal and come back, `git status` will show `(no branch, bisect in progress)`. Run `git bisect reset` to escape.

Marking a commit wrong invalidates all subsequent decisions. A single incorrect “good” or “bad” causes the algorithm to converge on the wrong commit. If you suspect a mistake, check `git bisect log`, fix the file, reset, and replay (see Recovery).

`skip` adjacent to the culprit produces an ambiguous result. If the commit immediately before or after the real culprit is skipped, Git will report a range of two commits rather than a single one. This is not a bug — it is honest uncertainty. You still have to narrow the last step manually.

`--first-parent` changes what counts as “reachable”. On a history with many merge commits it gives cleaner attribution (you learn which merge brought the bug in), but it may skip commits that are actually reachable through a branch parent. Use it intentionally.

Exit codes for `bisect run` are strict. Exit 0 means good, 1–124 and 126–127 mean bad. Exit 125 means skip. Any other value aborts the session. In particular, `exit(-1)` in C becomes 255 after truncation — that aborts bisect. Make sure your test script’s exit codes are well-defined.

Dirty working trees block checkout. Bisect checks out a different commit at every step. If you have uncommitted changes that conflict, Git will refuse to proceed. Either stash the changes before starting or use `--no-checkout` if your test does not need a clean tree.

Bisect history is stored in `.git`. The session state lives in `.git/refs/bisect/` and `.git/BISECT_*` files. Deleting the repo or running `git bisect reset` removes it. Do not confuse “bisect reset” with any destructive action — it is safe and idempotent.

Worked examples

Finding a regression in a web app

A feature that worked in the `v4.2.0` release is broken on `main`. You have a test script `scripts/smoke.sh` that exits 0 on success.

```
git bisect start main v4.2.0 --
git bisect run scripts/smoke.sh
```

After roughly twelve steps, Git prints:

```
d7a3c89f is the first bad commit
commit d7a3c89f...
Author: Jane Dev <jane@example.com>
Date:   Thu Mar 14 09:22:00 2026

    Refactor session token generation
```

```
git bisect reset
```

You now know exactly which commit to examine.

Locating the commit that introduced a fix (alternate terms)

You have a security bug that was patched somewhere between `v3.0.0` and `HEAD`, and you want to back-port the fix to an older branch. You need the exact commit that introduced the fix.

```
git bisect start --term-old vulnerable --term-new patched
git bisect patched          # HEAD is already patched
git bisect vulnerable v3.0.0
```

At each step, run your security check and mark accordingly:

```
git bisect patched          # this commit has the fix
git bisect vulnerable       # this commit does not
```

Git converges and prints the first “patched” commit — the exact change you want to cherry-pick.

```
git bisect reset
git cherry-pick <found-commit>
```

Handling a mix of testable and untestable commits

A migration script broke the database schema at some point, but several intermediate commits also have an unrelated build failure. Use `skip` to step over them.

```
git bisect start HEAD v5.0.0 --
# Git checks out a candidate:
make 2>&1 | grep -q "Build failed" && git bisect skip || ./run_tests.sh
# or use a script that returns 125 on build failure:
git bisect run ./scripts/test-with-skip.sh
git bisect reset
```

test-with-skip.sh:

```
#!/bin/sh
make || exit 125          # skip commits that do not build
./run_tests.sh           # exit 0 = good, 1 = bad
```

Reviewing and correcting a mistake mid-session

Halfway through a session you realise you marked step 3 incorrectly.

```
git bisect log > /tmp/bisect-correction.log
# Edit the file and remove the incorrect line, e.g.:
# Delete: git bisect good abc9876
git bisect reset
git bisect replay /tmp/bisect-correction.log
# Continue from the corrected state
```

Recovery

If you are stuck inside a bisect session unexpectedly:

```
git bisect reset          # return to original HEAD, clear all bisect state
```

To return to the identified bad commit rather than the original HEAD:

```
git bisect reset bisect/bad
```

If you marked commits incorrectly and the search converged on the wrong answer, save the log, edit out the bad marks, reset, and replay as shown in the worked example above.

See *Getting out of jams* for broader undo recipes when bisect leaves unrelated changes in the working tree.

See also

- *log* — navigating history to choose good/bad anchor points.
- *commit* — keeping commits small and focused makes bisect faster and its results more actionable.
- *revert* — once bisect identifies the culprit commit, revert it cleanly.
- *cherry-pick* — back-port the identified fix commit to another branch.
- *Getting out of jams* — recovering a detached HEAD or unexpected working tree state left by a bisect session.

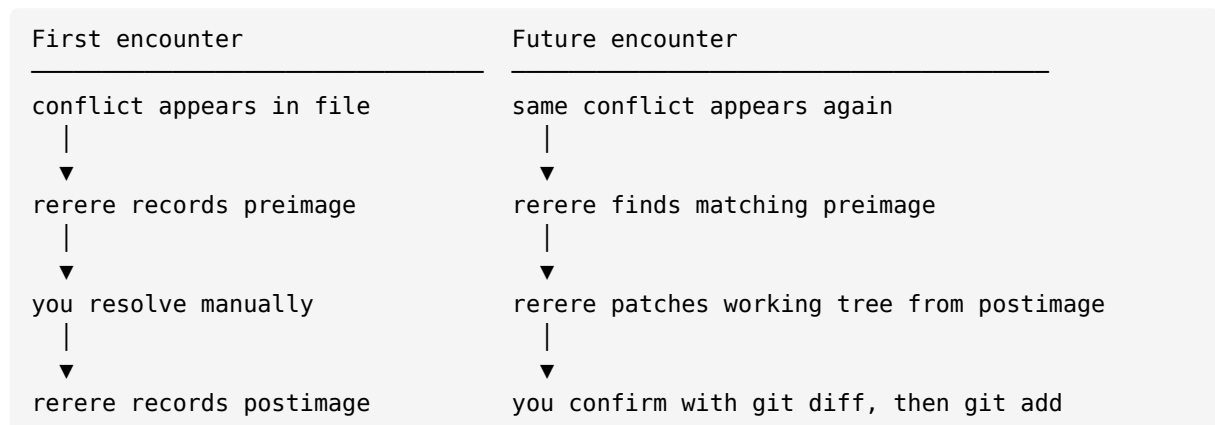
rerere

Record a conflict resolution once and replay it automatically every time the same conflict reappears.

Mental model

The name stands for “reuse recorded resolution.” The idea is simple: the first time you resolve a merge or rebase conflict, git rerere photographs both sides of the conflict and the resolution you produced. The next time Git encounters a conflict whose two sides look identical to a previously photographed pair, rerere silently patches your working tree with the recorded answer — no manual work required.

Rerere stores its data in `.git/rr-cache/`. Each conflict gets a directory named after a hash of the conflict’s two sides. Inside that directory Git keeps the pre-resolution conflict (`preimage`) and, once you resolve and record it, the post-resolution file (`postimage`).



Rerere is most valuable on long-lived topic branches that are repeatedly rebased onto a fast-moving base. Without rerere, every rebase step that hits the same conflict forces a manual re-resolution. With rerere, only the first resolution is manual; every subsequent one is automatic.

Note that rerere never touches the index on its own. After it applies a recorded resolution you still need to inspect the result and `git add` the files. This is intentional: rerere’s replay is a three-way merge of the old preimage, old postimage, and new conflict, and the result deserves a sanity check before it is staged.

Synopsis

```
git rerere [clear | forget <pathspec>... | diff | status | remaining | gc]
```

Running `git rerere` with no subcommand is the normal automated path: Git calls it for you at the right moments (after a failed merge, after a rebase step, after a commit of a merge result). You rarely need to invoke it manually except to use one of the diagnostic or maintenance subcommands.

Everyday usage

Enable rerere for a repository

Rerere is off by default. Turn it on once per repository:

```
git config rerere.enabled true
```

To enable it globally for all repositories on the machine:

```
git config --global rerere.enabled true
```

Let rerere work automatically during a merge

With `rerere.enabled true` you do not need to invoke `git rerere` yourself. Attempt a merge that conflicts:

```
git merge feature/payments
# Auto-merging src/checkout.js
# CONFLICT (content): Merge conflict in src/checkout.js
# Automatic merge failed; fix conflicts and then commit the result.
```

Git has already called `git rerere` internally and recorded the preimage. Open the file, resolve the conflict markers, then commit as normal:

```
# edit src/checkout.js – resolve the conflict markers
git add src/checkout.js
git commit
```

Git calls `git rerere` again on commit, recording the postimage. Next time the same conflict surfaces it will be resolved automatically.

Check what rerere will record

Before staging anything, see which files rerere considers to have in-flight conflicts:

```
git rerere status
```

```
src/checkout.js
```

See what rerere has already applied

After rerere auto-resolves a conflict, verify the result before staging:

```
git rerere diff
```

This shows the diff between the recorded preimage and the current working tree state — useful to confirm rerere applied exactly what you intended the first time.

Find conflicts rerere cannot auto-resolve

During a rebase, some conflicts may be untrackable (conflicting submodules, for example). List them:

```
git rerere remaining
```

Files listed here need manual attention even if rerere resolved others.

Discard a bad recorded resolution

If you resolved a conflict incorrectly the first time, delete the record so rerere does not repeat the mistake:

```
git rerere forget src/checkout.js
```

You can pass multiple paths. After forgetting, resolve the conflict manually again and the corrected resolution will be recorded in its place.

Reset all rerere state for an aborted merge

If you decide to abort a merge or rebase entirely:

```
git merge --abort
# or
git rebase --abort
```

`git rebase --abort` (and `git am --abort`) call `git rerere clear` automatically, discarding in-progress rerere metadata for the abandoned operation. You can also call it directly:

```
git rerere clear
```

Prune stale records

Records accumulate in `.git/rr-cache/` over time. Clean up old ones:

```
git rerere gc
```

By default, unresolved conflicts older than 15 days and resolved conflicts older than 60 days are pruned.

Key options

Subcommand	What it does	When to use it
<i>(none)</i>	Record or replay a conflict resolution	Called automatically; rarely needed by hand
<code>clear</code>	Discard rerere state for the current in-progress operation	Aborting a merge or rebase manually
<code>forget <paths></code>	Delete recorded resolutions for specific files	A previous resolution was wrong
<code>diff</code>	Show diff between recorded preimage and current working tree	Verify what rerere applied
<code>status</code>	Print files with conflicts rerere will record	Audit what is in flight
<code>remaining</code>	Print files rerere cannot auto-resolve	Find conflicts still needing manual work
<code>gc</code>	Prune old records from <code>.git/rr-cache/</code>	Periodic housekeeping

Configuration variables that control rerere behavior:

Variable	Default	Effect
<code>rerere.enabled</code>	<code>false</code>	Must be <code>true</code> to activate rerere
<code>gc.rerereUnresolved</code>	15 days	Age at which unresolved records are pruned by <code>gc</code>
<code>gc.rerereResolved</code>	60 days	Age at which resolved records are pruned by <code>gc</code>

Best practices

Enable rerere globally. There is no downside to having rerere active in every repository. Set `rerere.enabled = true` in your global config and forget about it. The `.git/rr-cache/` overhead is negligible.

Always inspect with `git rerere diff` before staging. Rerere’s replay is a three-way merge. If the surrounding code has changed significantly since the original resolution, the result may be subtly wrong. Use `git rerere diff` and `git diff` together to gain confidence before running `git add`.

Use `git rerere forget` promptly when you spot a bad resolution. If a recorded resolution is incorrect, remove it immediately. Left in place it will silently replicate the same mistake on every future rebase step or re-merge — potentially producing code that compiles and passes tests but behaves incorrectly.

Pair rerere with a rebase-based workflow. Rerere pays dividends when you keep topic branches current by rebasing onto `main` frequently. The first rebase step forces a manual resolution; every subsequent rebase step that replays the same commit through the same changed lines is handled automatically. Without rerere that cost is paid every time. See the *rebase* chapter for the broader workflow.

Run `git rerere gc` as a scheduled maintenance step. In repositories with heavy branching activity the rr-cache can grow to hundreds of entries. Wiring `git rerere gc` into a periodic CI job or `git maintenance` keeps it lean. See the *gc* and *maintenance* chapters for integration points.

Pitfalls & gotchas

Rerere is disabled by default. The most common reason rerere “doesn’t work” is that `rerere.enabled` was never set. Confirm it is active:

```
git config rerere.enabled
# should print: true
```

Rerere never stages files. After rerere resolves a conflict automatically, the working tree file is updated but the index is not. You must run `git add <file>` yourself. Forgetting this leaves you with a resolved working tree that Git still considers conflicted, and the next `git status` will still show the file under “Unmerged paths”.

A changed context can silently produce a wrong result. Rerere performs a three-way merge between the original preimage, the original postimage, and the new conflict. If the lines surrounding the conflict have changed substantially since the first resolution, the three-way merge may succeed while producing semantically broken code. Always verify the output with `git rerere diff`.

Rerere cannot track all conflict types. Conflicting submodule pointers, conflicting file modes, and certain binary conflicts cannot be recorded. `git rerere remaining` lists these so you know they need manual attention.

`git rerere clear` discards in-flight state, not stored resolutions. If you call `git rerere clear` hoping to erase a bad recorded resolution, nothing changes in `.git/rr-cache/`. Use `git rerere forget <pathspec>` to delete a stored resolution. `clear` only discards the metadata for an operation that is currently in progress.

Conflict marker size mismatches prevent recording. If a file uses a non-standard conflict marker size (set via `.gitattributes`) and `conflict-marker-size` is not configured consistently, rerere may fail to detect the conflict markers and will silently skip recording.

Worked examples

Long-lived feature branch rebased repeatedly onto main

Your team lands several commits to `main` each week. You maintain `feature/checkout-v2`, which touches `src/checkout.js` in the same region that `main` also modifies. You need to stay current.

Enable `rerere` first (if not already done globally):

```
git config rerere.enabled true
```

Rebase onto `main` for the first time:

```
git switch feature/checkout-v2
git rebase main
# CONFLICT (content): Merge conflict in src/checkout.js
# error: could not apply alb2c3d... Refactor checkout flow
```

`Rerere` has recorded the preimage. Resolve the conflict manually:

```
# open src/checkout.js, fix the conflict markers
git add src/checkout.js
git rebase --continue
```

`Rerere` records the postimage. A week later, rebase again after more commits have landed on `main`:

```
git rebase main
# CONFLICT (content): Merge conflict in src/checkout.js
# Resolved 'src/checkout.js' using previous resolution.
```

`Rerere` has replayed the resolution. Verify it looks correct before accepting:

```
git rerere diff      # confirm the applied patch is sensible
git diff            # review the full working tree state
git add src/checkout.js
git rebase --continue
```

No manual conflict resolution needed on the second (or third, or fourth) rebase.

Identifying and correcting a bad recorded resolution

During code review your colleague spots that the auto-resolved `src/pricing.js` has a duplicated function body — `rerere` applied an earlier resolution that was itself incorrect.

Forget the bad record for that file:

```
git rerere forget src/pricing.js
```

Abort the current rebase so you can redo the conflicting step cleanly:

```
git rebase --abort
git rebase main
# CONFLICT (content): Merge conflict in src/pricing.js
```

This time `rerere` has no stored answer for this conflict. Resolve it correctly:

```
# edit src/pricing.js carefully
git rerere diff      # double-check the working tree matches your intent
git add src/pricing.js
git rebase --continue
```

Rerere records the corrected postimage. Future rebases will use the right resolution.

Auditing rerere state during an ongoing rebase

A rebase across many commits is in progress and you want to know which files rerere has handled and which still need work.

Files rerere will record (in-flight conflicts):

```
git rerere status
```

```
src/api/auth.js
src/api/payments.js
```

Files rerere cannot auto-resolve:

```
git rerere remaining
```

```
vendor/stripe
```

The submodule `vendor/stripe` needs a manual resolution; the two `.js` files are handled automatically. Fix the submodule pointer, stage everything, then continue:

```
# update vendor/stripe to the correct commit
git add vendor/stripe src/api/auth.js src/api/payments.js
git rebase --continue
```

Recovery

If rerere has applied a resolution that leaves the repository in a bad state, abort the current operation and forget the offending record:

```
git rebase --abort      # or: git merge --abort
git rerere forget src/bad-file.js
```

Then restart the merge or rebase. This time rerere has no stored answer for that file and will stop for manual resolution.

If you want to discard all rr-cache entries and start fresh (rarely needed):

```
rm -rf .git/rr-cache
```

This is safe: rerere records are derived data. Repository history and your working tree are unaffected. You simply lose the accumulated resolution shortcuts and will need to re-resolve conflicts manually until new records build up again.

See *Getting out of jams* for broader undo recipes, including recovering from aborted merges and rebases.

See also

- *merge* — the primary operation where *rerere* saves repeated conflict work.
- *rebase* — *rerere* is especially valuable when rebasing long-lived branches frequently onto a moving base.
- *mergetool* — launch a GUI diff tool to resolve conflict markers by hand instead of editing them in a text editor.
- *gc* — `git gc` triggers `git rerere gc` as part of routine repository housekeeping.
- *maintenance* — schedule periodic *gc* runs to keep *rr-cache* lean over time.

log

Walk the commit graph and display commit history with configurable filtering, formatting, and diff output.

Mental model

Every commit in a Git repository is a node in a directed acyclic graph. Each node points to one or more parent commits. `git log` is a graph-walking engine: it starts at one or more named entry points (branches, tags, HEAD, or any revision expression), follows parent links backwards through history, and prints each commit it visits.

The walk is filtered in two stages:

1. **Commit limiting** — decide which commits to include based on author, date, message content, affected paths, or ancestry constraints.
2. **Commit formatting** — decide how to render each included commit: one line, full metadata, a patch, a diffstat, or a custom template.

Both stages are independent. You can filter to the last ten commits touching `src/auth/` and render each one as a full patch, or you can look at three years of history and render it as a single-line graph. Understanding this separation makes the many options feel organized rather than overwhelming.

```
All commits reachable from <revision>
|
▼ commit limiting (--since, --author, --grep, -- <path> ...)
|
▼ commit ordering (--topo-order, --date-order, --reverse)
|
▼ commit formatting (--oneline, --pretty, --stat, -p ...)
|
▼ output
```

By default, `git log` starts at HEAD and outputs commits in reverse chronological order with the medium format (hash, author, date, and full message).

Synopsis

```
git log [<options>] [<revision-range>] [--] <path>...
```

Key forms:

<code>git log</code>	# history of current branch from HEAD
<code>git log <branch></code>	# history of another branch
<code>git log <since>..<until></code>	# commits reachable from <until> but not <since>
<code>git log <branch1>...<branch2></code>	# symmetric difference (commits on either side)
<code>git log -- <path>...</code>	# only commits that touched these paths
<code>git log -L <start>,<end>:<file></code>	# trace a line range across history
<code>git log -L :<funcname>:<file></code>	# trace a function across history

Everyday usage

Show a compact, one-line-per-commit view of the current branch:

```
git log --oneline
```

```
a3f9c1d Fix null-pointer dereference in parseToken
7b2e8fa Add login validation with rate limiting
c1d4a0b Initial project scaffold
```

Show a decorated graph of all branches and tags — the most common starting point for understanding a repository's topology:

```
git log --oneline --graph --decorate --all
```

Show commits from the last week by a specific author:

```
git log --since="1 week ago" --author="Alice"
```

Show only commits that touched a particular file:

```
git log -- src/auth/login.js
```

Show the changes each commit introduced (full patch):

```
git log -p
```

Show a diffstat summary per commit — useful for understanding scope without reading every diff:

```
git log --stat
```

Show only commits on your feature branch that are not yet on `main`:

```
git log main..HEAD
```

Show commits whose message contains a search term:

```
git log --grep="rate limit"
```

Show the last five commits:

```
git log -5
```

Key options

Commit limiting

Option	What it does	When to use it
<code>-<n></code> / <code>-n <n></code>	Limit output to <code><n></code> commits	Quick look at recent history
<code>--since=<date></code> , <code>--after=<date></code>	Show commits newer than <code><date></code>	Scoping a time window
<code>--until=<date></code> , <code>--before=<date></code>	Show commits older than <code><date></code>	Scoping a time window

<code>--author=<pattern></code>	Filter by author name/email (regex)	Reviewing one person's work
<code>--committer=<pattern></code>	Filter by committer name/email (regex)	Distinguish author from committer
<code>--grep=<pattern></code>	Filter by commit message (regex)	Finding commits about a topic
<code>--all-match</code>	Require all <code>--grep</code> patterns to match (default: any)	Narrowing multiple keywords
<code>--invert-grep</code>	Exclude commits matching <code>--grep</code>	Filtering out noise
<code>-i / --regex-ignore-case</code>	Case-insensitive regex matching	Grep without worrying about case
<code>--no-merges</code>	Exclude merge commits	Cleaner linear history view
<code>--merges</code>	Show only merge commits	Auditing integration points
<code>--first-parent</code>	Follow only first parent of merges	Trunk-focused history on busy repos
<code>--all</code>	Start from all refs (branches, tags, HEAD)	Whole-repository overview

Commit formatting

Option	What it does	When to use it
<code>--oneline</code>	One line per commit: abbreviated hash + subject	Default compact view
<code>--pretty=<fmt> / --format=<fmt></code>	Built-in or custom format string	Scripting or bespoke display
<code>--abbrev-commit</code>	Shorten the hash to a unique prefix	Readability in terminals
<code>--decorate</code>	Show branch/tag names next to commits	Seeing where refs point
<code>--graph</code>	ASCII graph of the branch topology	Visualizing merges
<code>--stat</code>	Diffstat (files changed, insertions, deletions) per commit	Judging scope of each commit
<code>--shortstat</code>	One-line summary of total changes	Even more compact than <code>--stat</code>
<code>--name-only</code>	List changed filenames only	Quick file-level overview
<code>--name-status</code>	Filenames with status letters (A/M/D/R)	Spotting renames and deletions
<code>-p / --patch</code>	Full unified diff per commit	Code review in the terminal
<code>--date=<format></code>	Control date rendering (relative, iso, short, ...)	Readable dates in custom formats

<code>--follow</code>	Continue history past renames (single file)	Tracking a file that was renamed
<code>-L <start>,<end>:<file></code>	Show evolution of a line range	Archaeology on a specific block of code
<code>-L :<funcname>:<file></code>	Show evolution of a function by name	Tracing when a function changed

Revision ranges

Notation	Meaning
<code>A..B</code>	Commits reachable from B but not from A (commits added since A)
<code>A...B</code>	Symmetric difference: commits on either side but not the common ancestor
<code>^A B</code>	Same as <code>A..B</code> (explicit exclusion)
<code>HEAD~3</code>	The commit three steps before HEAD
<code>origin/main..HEAD</code>	Commits in your branch not yet on the remote

Best practices

Use `--oneline --graph --decorate --all` as your default overview. This single invocation shows the entire reference graph in a compact form. Alias it to something short:

```
git config --global alias.lg "log --oneline --graph --decorate --all"
```

Then `git lg` gives you the topology at a glance.

Filter by path to understand a file's history. Before reading a file, look at its history first:

```
git log --oneline -- src/payments/checkout.js
```

The `--` separator is not always required but is unambiguous: everything after `--` is a path, not a branch name.

Use `--follow` when a file has been renamed. Without it, the log stops at the rename commit. With it, Git threads through the rename and continues showing older commits under the original name:

```
git log --follow -- src/payments/checkout.js
```

Prefer `origin/main..HEAD` over guessing a base commit. When reviewing your own branch before pushing, this range shows exactly the commits that will travel:

```
git log --stat origin/main..HEAD
```

Use `-L :<funcname>:<file>` for function-level archaeology. Instead of grepping through the log manually, let Git trace the function for you:

```
git log -L :parseToken:src/auth/token.js
```

Git infers the function boundary from language-specific heuristics (or `.gitattributes`). Every commit that changed the function body appears with its diff, even if other parts of the file changed at the same time.

Use `--no-merges` when reviewing a linear history. On a repository with frequent merges, merge commits dominate the log. `--no-merges` hides them and surfaces the actual work:

```
git log --oneline --no-merges main..feature/payments
```

Combine `--grep` with `-i` and `--all-match` carefully. Multiple `--grep` patterns are ORed by default. Add `--all-match` to require all of them, and `-i` to go case-insensitive:

```
git log --grep="payment" --grep="stripe" --all-match -i
```

Pitfalls & gotchas

`git log <file>` without `--` can be ambiguous. If a branch and a file share a name, Git may interpret the argument as a branch. Always use `--` to separate revision arguments from path arguments:

```
git log -- config # unambiguous: show history of the file named "config"
```

History simplification hides commits by default when filtering by path. When you give `git log -- <path>`, Git uses default history simplification: it follows only the parent that explains the final state of the file, which can make commits disappear. If you suspect a commit is missing, add `--full-history`:

```
git log --full-history -- src/auth/login.js
```

--all does not mean “everything ever”. It includes all refs (branches, tags, HEAD) but not commits that are not reachable from any ref. Commits dangling after a `git reset` or `git branch -D` are not shown. Use `git reflog` to find those.

--graph implies --topo-order. The topological ordering can cause surprising jumps in timestamps. Older commits may appear above newer ones to keep branch lines readable. This is correct behavior, not a bug.

--since and --until use the author date, not the committer date. Rebased commits keep their original author date. A commit written last year and rebased today will still appear in `--since="last year"` output, even though it was rebased recently.

Custom --pretty=format: does not add a trailing newline; tformat: does. If you pipe `--pretty=format:%h` output to another tool and the last line is missing a newline, switch to `--pretty=tformat:%h`. The `tformat:` variant uses terminator semantics (newline after every entry) instead of separator semantics (newline between entries).

-p on merge commits shows no diff by default. Merge commits have multiple parents, and `git log -p` suppresses diffs for them unless you specify a `--diff-merges` option. Use `--diff-merges=first-parent` for a diff against the first parent:

```
git log -p --diff-merges=first-parent
```


Worked examples

Auditing recent changes to a module

Your team just deployed and something in the authentication module looks wrong. Show every commit that touched `src/auth/` in the last two weeks, with a diffstat:

```
git log --since="2 weeks ago" --stat -- src/auth/
```

```
commit a3f9c1d
Author: Alice <alice@example.com>
Date:   Mon Jun 16 14:22:01 2026 +0200

    Throttle login attempts to 5 per minute per IP

src/auth/login.js   | 18 ++++++-----
src/auth/limiter.js |  5 +++++
2 files changed, 21 insertions(+), 2 deletions(-)

commit 7b2e8fa
Author: Bob <bob@example.com>
Date:   Fri Jun 13 09:11:44 2026 +0200

    Add JWT expiry to token validator

src/auth/token.js | 12 ++++++---
1 file changed, 9 insertions(+), 3 deletions(-)
```

Narrow further to commits by a specific person:

```
git log --since="2 weeks ago" --author="Alice" --stat -- src/auth/
```

Understanding how a function evolved

You need to understand why `parseToken` behaves the way it does. Trace its entire history:

```
git log --oneline -L :parseToken:src/auth/token.js
```

Git prints each commit that modified the function body, along with the exact diff of the function at that point. Read from bottom to top to see the function grow from its original form to the present.

Comparing two branches before a merge

Before merging `feature/payments` into `main`, review what will land:

```
git log --oneline --stat main..feature/payments
```

Each commit is shown with the files it touches. If a commit touches unexpected files, you can inspect it further with `git show <hash>`.

Check the same range for any commits that are pure merge noise:

```
git log --oneline --no-merges main..feature/payments
```

Building a custom one-liner for scripting

You need a list of commit hashes and subjects since a given tag, suitable for feeding to another script:

```
git log --pretty=tformat: "%h %s" v2.3.0..HEAD
```

```
a3f9c1d Throttle login attempts to 5 per minute per IP
7b2e8fa Add JWT expiry to token validator
c1d4a0b Migrate payment provider to Stripe
```

The `tformat:` prefix ensures each line ends with a newline, making it safe to pipe to `while read hash subject; do ...`.

Visualizing a branchy repository

You have inherited a repository with many branches and want to understand its topology before starting work:

```
git log --oneline --graph --decorate --all
```

```
* a3f9c1d (HEAD -> main, origin/main) Throttle login attempts
| * 9f1b234 (feature/payments) Add Stripe webhook handler
| * 4d7e890 Scaffold payments module
|/
* 7b2e8fa Add JWT expiry to token validator
* c1d4a0b Initial project scaffold
```

The `*` marks commits, `/` and `|` draw branch lines, and the parenthetical labels show where refs point.

Recovery

`git log` is a read-only command: it cannot modify the repository. There is nothing to undo.

If a range expression returns unexpected results, double-check the notation: `A..B` means “reachable from B, not from A” — not “commits between A and B chronologically”. Test with `--oneline` first to confirm the set before adding `-p` or other heavy output options.

If commits seem missing when filtering by path, see the *history simplification* pitfall above and try `--full-history`.

For recovering commits that are no longer reachable from any ref (after a reset or a deleted branch), see the *reflog* chapter, which walks Git’s internal operation log.

See also

- *show* — display a single commit in full detail (patch + metadata).
- *blame* — annotate a file line-by-line with the commit that last changed each line.
- *reflog* — walk the local operation log to find commits not reachable from refs.
- *shortlog* — summarize `git log` output grouped by author.
- *diff* — compare two trees or commits without the commit metadata.
- *bisect* — binary-search the commit graph to find the commit that introduced a bug.
- *Getting out of jams* — recovering commits that appear to have been lost.

show

Inspect any Git object — commit, tag, tree, or file blob — and display its contents along with the diff it introduces.

Mental model

Everything stored in a Git repository is an *object*: commits, trees (directories), blobs (file contents), and annotated tags. Each object has a 40-character SHA-1 name. `git show` is the universal lens for looking at any one of those objects.

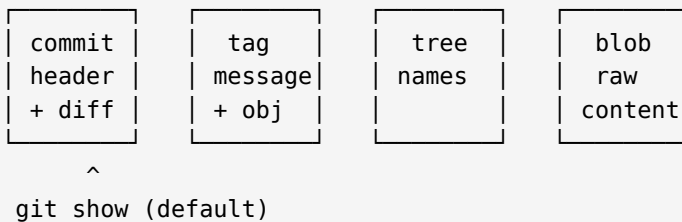
For the most common case — a commit — `git show` prints the commit header (author, date, message) followed by the unified diff of the changes that commit introduced. Think of it as `git log -1` plus `git diff` combined into a single, focused view.

For other object types the output changes accordingly:

- **Annotated tag** — shows the tag message, then recurses into whatever the tag points at (usually a commit).
- **Tree** — lists filenames, like `git ls-tree --name-only`.
- **Blob** — prints the raw file contents with no diff.

The default target is `HEAD`, so a bare `git show` always means “show me the last commit on the current branch”.

Object store



Synopsis

```
git show [<options>] [<object>...]
```

`<object>` can be any revision expression: a commit hash, branch name, tag, `HEAD~3`, `v1.4.0^{commit}`, `main:path/to/file`, and so on. When omitted, `HEAD` is used.

Everyday usage

Show the most recent commit on the current branch:

```
git show
```

Show a specific commit by hash (abbreviated is fine):

```
git show a3f9c1
```

Show a commit at the tip of another branch:

```
git show feature/login
```

Show what a tag points at (tag message first, then the commit):

```
git show v2.0.0
```

Show the contents of a file as it existed in a specific commit:

```
git show HEAD~2:src/config.js
```

Show a commit with a compact file-change summary instead of the full diff:

```
git show --stat a3f9c1
```

Show just the commit message, no diff:

```
git show -s a3f9c1
git show --no-patch a3f9c1    # same effect
```

Show a commit with a custom format (hash and subject only):

```
git show --format="%h %s" --no-patch a3f9c1
```

Key options

Option	What it does	When to use it
<code>-s</code> , <code>--no-patch</code>	Suppress the diff; show only the commit header	Quickly read a commit message without noise
<code>--stat</code>	Replace the full diff with a filename/lines-changed summary	Get a high-level picture of what changed
<code>--name-only</code>	Show only the names of changed files	Feed into scripts or check which files a commit touched
<code>--name-status</code>	Show file names plus a status letter (A/M/D/R/...)	See what happened to each file at a glance
<code>--oneline</code>	Abbreviate commit to one line (short hash + subject)	Compact header when you don't need the full message
<code>--format=<fmt></code> / <code>--pretty=<fmt></code>	Control commit header format (<code>oneline</code> , <code>short</code> , <code>medium</code> , <code>full</code> , <code>fuller</code> , <code>raw</code> , <code>format:<string></code>)	Custom output for scripts or aliases
<code>--abbrev-commit</code>	Show a short hash prefix instead of 40 characters	Readable terminal output
<code>-U<n></code> , <code>--unified=<n></code>	Show <code><n></code> lines of diff context (default 3)	More context when reviewing complex changes
<code>--word-diff</code>	Highlight changed words within lines instead of whole lines	Review prose or config-file changes
<code>--color-moved</code>	Color blocks of moved code differently from additions/deletions	Identify refactors that only moved code

<code>--show-signature</code>	Verify GPG signature and show the verification result	Confirm a signed commit is authentic
<code>--raw</code>	Show diff in raw format (object hashes, mode changes)	Scripting and plumbing use
<code>-p</code> , <code>--patch</code>	Explicitly request a patch diff (on by default for commits)	Force patch output when another option suppressed it

Best practices

Use `HEAD~N` and branch names, not raw hashes, when exploring. `git show HEAD~3` and `git show feature/auth` are self-documenting in shell history. Hashes are essential for precision (a review comment, a script) but awkward for interactive use.

Pair `--stat` with the full diff for large commits. Run `git show --stat <commit>` first to understand the scope of the change, then drop `--stat` to read the diff for files that need scrutiny. This is faster than scrolling past hundreds of diff lines looking for the one file you care about.

Use `<commit>:<path>` to recover a previous file version. The `git show HEAD~1:src/app.js` form prints the file exactly as it was — useful for copying a snippet, or piping to a file for a manual comparison.

Combine `--format` with `-s` for lightweight aliases. A common alias is:

```
git config --global alias.msg 'show -s --format=%B'
```

Running `git msg HEAD~2` then prints just the full commit body, with no diff and no header clutter.

Prefer `--name-only` or `--name-status` in scripts. When a script needs to act on files that a commit touched, these options produce stable, parseable output without parsing unified-diff syntax.

Pitfalls & gotchas

`git show` on a merge commit produces a combined diff by default. The combined diff (`--diff-merges=dense-combined`) only shows lines that differ from *both* parents, which means most of the merge is hidden. If you want to see all changes relative to the first parent, use `--diff-merges=first-parent` (or its shorthand `--dd`). To see diffs against each parent separately, use `--diff-merges=separate`.

Bare `git show` shows `HEAD`, not the working tree. A common confusion: you edited a file, ran `git show`, and the change is not there. `git show` reads committed objects — it knows nothing about uncommitted edits. Use `git diff` for working-tree comparisons.

`-s` and `--no-patch` are synonyms but their names mislead. `-s` stands for “suppress diff”, not “short”. It is documented as `--no-patch` in newer Git versions; both work.

`<commit>:<path>` needs the path relative to the repository root. If you are deep in a subdirectory and run `git show HEAD:widget.js`, Git looks for `widget.js` at the root of the repo, not your current directory. Use the full repo-root-relative path, e.g. `git show HEAD:src/components/widget.js`.

Annotated tags and lightweight tags behave differently. `git show v1.0` on an annotated tag prints the tag message *then* the commit. On a lightweight tag (which is just a pointer to a commit) there is no tag message, and output looks identical to `git show <commit>`. Use `git cat-file -t v1.0` to check which type you have.

--word-diff can be misleading for code. Word-level diffing works well for prose but is noisy for code where a single-character rename spans multiple tokens. Use it deliberately, not as a default.

Worked examples

Reviewing a colleague's commit before merging

Your teammate pushed `feature/payments`. You want to understand what changed before you approve the pull request.

Start with the big picture:

```
git show --stat origin/feature/payments
```

```
commit d4e89f1
Author: Ana Costa <ana@example.com>
Date:   Mon Jun 16 14:32:09 2025 +0200

    Add Stripe webhook handler

src/webhooks/stripe.js | 87 +++++
src/webhooks/index.js  |  3 ++
tests/webhooks/stripe.js | 62 +++++
3 files changed, 152 insertions(+)
```

Now read the diff for just the source file you care about:

```
git show origin/feature/payments -- src/webhooks/stripe.js
```

If the commit is large and you want more context around each change:

```
git show -U10 origin/feature/payments -- src/webhooks/stripe.js
```

Recovering the previous version of a config file

You accidentally overwrote `config/database.yml` and have not committed yet, but you want the content as it was two commits ago.

```
git show HEAD~2:config/database.yml
```

To restore it in place:

```
git show HEAD~2:config/database.yml > config/database.yml
```

Or use `git restore` with a source commit (see the *restore* chapter for that approach).

Checking what a release tag contains

Your deployment pipeline triggers on annotated tags. Before tagging, verify the tag message and the commit it wraps:

```
git show v3.1.0
```

```
tag v3.1.0
Tagger: Thomas Bjork <thomas@example.com>
Date: Tue Jun 17 09:00:00 2025 +0200

Release 3.1.0 – performance improvements and bug fixes

commit 7b2a94c
Author: Thomas Bjork <thomas@example.com>
Date: Mon Jun 16 18:45:00 2025 +0200

    Bump version to 3.1.0

diff --git a/package.json b/package.json
...
```

To show only the tag object’s message without the commit diff:

```
git show -s --format=%B v3.1.0
```

Inspecting a file’s content at a branch tip

You want to compare how `src/auth.js` looks on `main` versus `feature/oauth` without checking either branch out:

```
git show main:src/auth.js > /tmp/auth-main.js
git show feature/oauth:src/auth.js > /tmp/auth-oauth.js
diff /tmp/auth-main.js /tmp/auth-oauth.js
```

Or, if your terminal supports side-by-side diffing, pipe through `diff -y`.

Recovery

`git show` is purely read-only — it cannot modify history or the working tree, so there is nothing to undo. If you ran `git show HEAD~2:path/file > path/file` and overwrote a file accidentally, recover with:

```
git restore path/file # restore from the index (last staged version)
git restore --source HEAD path/file # restore from HEAD
```

See *Getting out of jams* for broader undo recipes.

See also

- `log` — traverse and filter history across many commits; `git show` is `git log -1 -p` in spirit.
- `diff` — compare the working tree or index to a commit without picking a single commit to inspect.
- `restore` — overwrite working-tree files from a commit or the index.
- `tag` — creating and listing annotated and lightweight tags.
- *Getting out of jams* — recovering from accidental file overwrites and lost commits.

blame

Annotate every line of a file with the commit and author that last changed it.

Mental model

Think of `git blame` as a time-stamped margin note printed beside each line of source code. For every line the output answers: *which commit introduced this exact text, who wrote it, and when?*

Underneath, blame walks the commit graph backwards from HEAD (or a specified revision), replaying the diffs in reverse until every line in the file has been attributed to the commit that last touched it. Git follows whole-file renames automatically, so a function that lived in `util.py` before a reorganisation and now lives in `helpers/util.py` is still correctly attributed.

One detail worth internalising: blame shows the commit that *last* changed a line, not the commit that *first* introduced it. If line 42 was written in `a1b2c3` but a later commit `d4e5f6` only re-indented it, blame shows `d4e5f6`. Use `-w` to make whitespace-only changes invisible to blame, and `--ignore-rev` to skip entire bulk-formatting commits.

```
commit history:  alb2c3 → d4e5f6 → HEAD
                  |      |
                  | wrote | reformatted
                  |line 42|   line 42
git blame result: shows d4e5f6 (unless -w or --ignore-rev suppresses it)
```

Synopsis

```
git blame [-c] [-b] [-l] [--root] [-t] [-f] [-n] [-s] [-e] [-p] [-w]
          [--incremental]
          [-L <range>] [-S <revs-file>] [-M] [-C] [-C] [-C]
          [--since=<date>]
          [--ignore-rev <rev>] [--ignore-revs-file <file>]
          [--color-lines] [--color-by-age] [--progress] [--abbrev=<n>]
          [--contents <file>] [<rev> | --reverse <rev>..  
<rev>] [--] <file>
```

Everyday usage

Annotate the current HEAD version of a file:

```
git blame src/auth.py
```

Typical output — each line gets: abbreviated commit hash, author name, date, line number, then the line itself:

IP	Host	Time	Offset	Seq	Len	Window	Flags	Window Size	Checksum	Options	Application
10.0.2.15	10.0.2.15	2025-03-10 09:14:02	+0100	1	10	0	0	0	0		10.0.2.15
10.0.2.15	10.0.2.15	2025-03-10 09:14:02	+0100	2	10	0	0	0	0		10.0.2.15
10.0.2.15	10.0.2.15	2025-03-10 09:14:02	+0100	3	10	0	0	0	0		10.0.2.15
10.0.2.15	10.0.2.15	2025-03-10 09:14:02	+0100	4	10	0	0	0	0		10.0.2.15

A leading `^` marks a boundary commit – a line that has existed since the very first commit in the repository (or since the `--since` boundary).

Narrow blame to a specific line range (lines 30 through 55):

```
git blame -L 30,55 src/auth.py
```


Narrow to a function by name (Git uses the same hunk-header regex as `git diff`):

```
git blame -L ':authenticate' src/auth.py
```

Blame an older revision rather than HEAD:

```
git blame v2.1.0 -- src/auth.py
```

Ignore whitespace-only changes (reformats, re-indent):

```
git blame -w src/auth.py
```

Show email addresses instead of author names:

```
git blame -e src/auth.py
```

Key options

Option	What it does	When to use it
<code>-L <start>,<end></code>	Restrict annotation to a line range; also accepts <code>+<offset></code> , <code>/<regex>/</code> , or <code>:<funcname></code>	Focus on one function or section
<code>-w</code>	Ignore whitespace when comparing parent and child versions	Skip re-indent or reformat commits
<code>--ignore-rev <rev></code>	Attribute lines changed by that commit to the previous touching commit	Skip a single bulk lint/format commit
<code>--ignore-revs-file <file></code>	Read a list of revisions to ignore (same format as <code>fsck.skipList</code>)	Project-wide ignore list committed as <code>.git-blame-ignore-revs</code>
<code>-M[<num>]</code>	Detect lines moved or copied within the same file	Track refactored code blocks that were shuffled around
<code>-C[<num>]</code>	Detect lines moved or copied from other files modified in the same commit; use twice to also check the file-creation commit; use three times to check any commit	Track code moved across files
<code>-e / --show-email</code>	Show author email instead of name	Distinguish contributors who share a display name
<code>-f / --show-name</code>	Always show the filename the line came from	Useful after renames or copy-detection with <code>-C</code>
<code>-n / --show-number</code>	Show the line number in the original commit	Correlate with line numbers from an older version

<code>-s</code>	Suppress author name and timestamp	Compact output for scripting
<code>-l</code>	Show the full 40-character commit hash	Avoid hash collisions in very large repositories
<code>-t</code>	Show the raw Unix timestamp	Scripting or sorting by time
<code>--since=<date></code>	Ignore commits older than the date; attribute those lines to the boundary commit	Show only recent changes in a long-lived file
<code>--reverse <rev>.. <rev></code>	Walk history forward — show the last revision in which a line existed	Track when a line was removed
<code>--first-parent</code>	Follow only the first parent at merge commits	See when a line arrived on an integration branch
<code>-p</code> / <code>--porcelain</code>	Machine-readable output with full commit metadata per block	Scripting and IDE integrations
<code>--line-porcelain</code>	Like <code>--porcelain</code> but repeats commit info for every line	Simpler parsing at the cost of verbosity
<code>--color-lines</code>	Highlight lines from the same commit in cyan	Terminal readability when scanning a file
<code>--color-by-age</code>	Colour lines based on how old they are	Quick visual age map of a file
<code>--abbrev=<n></code>	Use at least <code>n</code> hex digits for abbreviated hashes	Avoid collisions in large monorepos
<code>-b</code>	Show a blank SHA-1 for boundary commits	Cleaner display when <code>--root</code> is not set
<code>--root</code>	Do not treat root commits as boundaries	Annotate all the way back to the initial commit
<code>--show-stats</code>	Append statistics at the end of output	Quick summary of contributor distribution
<code>--contents <file></code>	Annotate using the contents of <code><file></code> instead of the committed version	Blame an unsaved or modified working-tree file

Best practices

Always pass `--` before the filename when there is any ambiguity. If a branch or tag has the same name as the file you want to blame, Git may misinterpret the argument.

```
git blame -- auth.py
```

is unambiguous.

Commit a `.git-blame-ignore-revs` file and configure `blame.ignoreRevsFile` to point at it. Bulk formatting commits (Prettier, Black, gofmt) pollute blame output for months. Put their full hashes in a project-level file:

```
# .git-blame-ignore-revs
# Ran Black across the entire codebase 2026-05-01
e3f1a2b9c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9
```

Then tell every clone to use it automatically:

```
git config blame.ignoreRevsFile .git-blame-ignore-revs
```

GitHub and GitLab both honour this file when rendering their blame views, so committing it benefits everyone — including the web UI.

Use `-w` alongside `--ignore-rev` for reformatting commits. Some reformatters change only indentation, not content. `-w` tells blame to ignore whitespace differences when computing attribution, so even if a hash is not in the ignore list, a purely cosmetic re-indent will not hijack the blame.

Use `-C -C` when investigating copied code. A single `-C` only examines files modified in the same commit as the file being blamed. Two `-C` flags extend the search to the commit that created the file, catching copy-paste from an older module.

Combine `-L` with a function-name pattern for focused investigation. Rather than scrolling through hundreds of lines, jump straight to the function in question:

```
git blame -L ':parseToken' -- src/jwt.js
```

Git resolves the function boundary using the same hunk-header rules as `git diff`, so this works out of the box for most languages without any configuration.

Use porcelain output for tooling, not the default format. When writing a script that counts lines per author or feeds blame data into another tool, use `--line-porcelain` rather than parsing the human-readable default format, which can vary with terminal width and locale settings.

Pitfalls & gotchas

Blame shows who last *touched* a line, not who *wrote* it. A bulk search-and-replace, a copyright header update, or a trailing-whitespace fix can make every line in a file attribute to the same housekeeping commit. Always check with `-w` and `--ignore-rev` before concluding that an author owns a block of code.

Line ranges go stale. If you bookmark “lines 40-60 of auth.py” and someone inserts ten lines above that range in a later commit, your range now points at different code. Prefer the function-name form (`-L ':funcname'`) whenever possible — it re-anchors on every invocation.

`-M` and `-C` are expensive on large files or deep histories. Each additional `-C` flag adds another full pass over the commit graph. On a file with thousands of commits, triple `-C` can be slow. Use it deliberately rather than adding it to an alias you run constantly.

`--reverse` requires a range, not a single revision. A common mistake is writing `git blame --reverse v1.0 -- file`. Git accepts this and expands it to `v1.0..HEAD` automatically, but being explicit avoids surprises when HEAD is detached.

Blame cannot show deleted lines. If a line has been removed, blame cannot tell you who deleted it — because the line no longer exists in the file. Use `git log -S'<search string>'` (the pickaxe interface) or `git log -G'<regex>'` to find the commit that removed a particular string. See the *log* chapter for pickaxe details.

The `^` prefix on a hash is a display marker, not ref syntax. When you see `^a1b2c3d` in blame output, it means that line has existed in the file since a boundary commit — either the repository’s root commit or the boundary set by `--since`. It is not the same `^` used in `HEAD^` revision syntax.

--ignore-rev can leave some lines unattributable. When a commit is ignored, lines it touched that cannot be attributed to any nearby commit are marked with `*` in the output (if `blame.markUnblamableLines` is set) or `?` (if `blame.markIgnoredLines` is set). Check your config if attribution looks wrong after adding a revision to the ignore list.

Worked examples

Finding who introduced a failing line

A test started failing overnight. The assertion on line 88 of `src/parser.py` looks wrong. Find out when and why it changed:

```
git blame -L 88,88 -- src/parser.py
```

```
f7c3a1d2 (Carol 2026-01-14 23:41:05 +0100 88)      assert result["status"] == "ok"
```

The commit is `f7c3a1d2`. Inspect the full change:

```
git show f7c3a1d2
```

The diff reveals that Carol changed `"success"` to `"ok"` to match a new API contract, but the downstream test was not updated at the same time. You now have the full context — author, date, commit message, and diff — to write the fix and reference the original commit.

Skipping a bulk-format commit

Your project ran Black over all Python files in commit `b0rked01`. Every Python file now blames that commit for most of its lines. Add it to a project-level ignore list and re-run blame:

```
# Record the full hash in the ignore file
printf '\n# Black reformat 2026-05-01\nb0rked01deadbeef1234567890abcdef12345678\n' \
  >> .git-blame-ignore-revs

# Confirm blame now attributes lines to real authors
git blame --ignore-revs-file .git-blame-ignore-revs -- src/models.py
```

Make the ignore list automatic for every contributor and for the GitHub blame UI:

```
git config blame.ignoreRevsFile .git-blame-ignore-revs
git add .git-blame-ignore-revs
git commit -m "Add blame ignore-revs for Black reformat"
```

Counting lines per author with porcelain output

Count which authors contributed the most lines to a file:

```
git blame --line-porcelain -- src/core.py \
| grep '^author ' \
| sort \
| uniq -c \
| sort -rn
```

```
142 author Alice
61 author Bob
23 author Carol
```

This works reliably because `--line-porcelain` emits an `author` header for every single line, making the `grep-sort-uniq` pipeline straightforward.

Tracking code that moved across files

A function in `lib/validate.py` looks like it was copied from `lib/util.py`. Use double `-C` to follow the copy, and `-f` to show the originating filename:

```
git blame -C -C -f -- lib/validate.py
```

Lines that originated in `lib/util.py` will show that filename in the output, together with the commit that first introduced them there. This confirms the copy origin and reveals whether the two copies have since diverged.

Recovery

`git blame` is a read-only inspection command — it never modifies the working tree, the index, or the commit graph. There is nothing to undo.

If blame is returning confusing results, the most common causes are:

- A bulk-format commit is claiming attribution. Add it to `--ignore-revs-file` (or the project's `.git-blame-ignore-revs`) and re-run.
- Whitespace changes are masking the real author. Add `-w` and re-run.
- You are blaming the wrong revision. Confirm with `git log --oneline -5 -- <file>` that the expected revision is in the file's history.

See *Getting out of jams* for commit-level undo recipes if you need to reverse a change that blame helped you identify.

See also

- *log* — `git log -S` (pickaxe) and `git log -G` to find commits that added or removed a specific string; essential for tracking deleted lines that blame cannot show.
- *show* — inspect the full diff of the commit blame identifies.
- *diff* — compare file versions directly when you need to see what changed between two revisions.
- *bisect* — binary-search the commit graph when blame points to the wrong region and you need to narrow down a regression automatically.
- *Getting out of jams* — undoing changes once blame has identified the offending commit.

shortlog

Summarize commit history by author (or any grouping) in a format ready for release notes and contributor reports.

Mental model

`git shortlog` is a post-processor for commit history. Internally it does what `git log` does — walks the ancestry graph — but instead of printing each commit individually it collects them into buckets and prints one bucket per author (or per whatever grouping you ask for). The output looks like:

```
Alice Johnson (14):
  Add OAuth2 login flow
  Fix token expiry off-by-one
  ...

Bob Nakamura (7):
  Refactor middleware stack
  ...
```

Two design details are worth knowing up front.

First, `git shortlog` respects the mailmap. A contributor who has committed under several email addresses or name variations is folded into a single entry automatically. See the *Mapping authors* note in `gitmailmap(5)`.

Second, `git shortlog` can read from standard input instead of the repository. Pipe `git log --pretty=short` into it and it works identically — useful in scripts that already have log output in hand, or when running outside any repository.

Synopsis

```
git shortlog [<options>] [<revision-range>] [--] <path>...
git log --pretty=short | git shortlog [<options>]
```

Everyday usage

Show every contributor and their commit subjects for the whole history, sorted alphabetically by name:

```
git shortlog HEAD
```

Show just the count per contributor, sorted from most commits to fewest:

```
git shortlog -sn HEAD
```

Typical output:

```
143 Alice Johnson
 89 Bob Nakamura
 52 Carol Wei
```

Include email addresses next to each name:

```
git shortlog -sne HEAD
```

Summarise only what is new since the last release tag:

```
git shortlog -sn v1.3.0..HEAD
```

Show contributors to a specific directory only:

```
git shortlog -sn HEAD -- src/auth/
```

Group by the Reviewed-by trailer to see who has reviewed the most work:

```
git shortlog -sn --group=trailer:reviewed-by HEAD
```

Key options

Option	What it does	When to use it
<code>-n</code> , <code>--numbered</code>	Sort by commit count descending instead of alphabetically	Leaderboard-style contributor lists
<code>-s</code> , <code>--summary</code>	Suppress commit subjects; show only the count and name	Quick contributor counts
<code>-e</code> , <code>--email</code>	Show each author's email address	Identifying duplicate identities before updating <code>.mailmap</code>
<code>--group=<type></code>	Group by author (default), committer, trailer:<field>, or format:<format>	Grouping by reviewer, co-author, or any commit metadata
<code>-c</code> , <code>--committer</code>	Alias for <code>--group=committer</code>	When you want to see who landed commits, not who wrote them
<code>--format=<format></code>	Replace commit subjects with a <code>git log --format</code> string	Showing short hashes or dates instead of subjects
<code>--date=<format></code>	Format date fields used in <code>--group=format:<format></code>	Grouping by month or year
<code>-w[<width>[,<indent1>[,<indent2>]]]</code>	Wrap output at <width> columns (default 76)	Fitting output into a terminal or email
<code>--since=<date></code> , <code>--after=<date></code>	Limit to commits newer than a date	Activity reports for a sprint or release cycle
<code>--until=<date></code> , <code>--before=<date></code>	Limit to commits older than a date	Historical snapshots
<code>--author=<pattern></code>	Limit to commits whose author matches the pattern	Auditing a single contributor's work
<code>--no-merges</code>	Exclude merge commits	Counts that reflect only substantive work

<code>--first-parent</code>	Follow only the first parent of each merge	Summary of a trunk branch without feature-branch noise
<code>--all</code>	Include all refs, not just HEAD	Repository-wide contributor totals

Best practices

Always pair `-s` with `-n` for contributor lists. The combination `-sn` produces output sorted by impact (most commits first) with no commit subject noise. This is the idiomatic form used in almost every project’s release announcement.

Exclude merge commits with `--no-merges`. On repositories that merge feature branches rather than rebasing, each merge commit is credited to the person who pressed the merge button. That inflates their count and deflates the counts of feature authors. `--no-merges` makes the numbers represent actual authorship.

```
git shortlog -sn --no-merges v2.0.0..HEAD
```

Use a revision range for release notes. Running `git shortlog HEAD` sums the entire project history, which is rarely what you want for a changelog. Anchor both ends explicitly:

```
git shortlog -sn v1.4.0..v1.5.0
```

Keep a `.mailmap` file. Contributors change employers, use different machines, or configure Git inconsistently. Without a `.mailmap`, the same person appears as multiple authors. See the *log* chapter for how mailmap interacts with history display, and `gitmailmap(5)` for the file format.

Use `--group=trailer:co-authored-by` to credit pair-programming partners. Many teams follow the GitHub convention of adding `Co-authored-by` trailers to commits. Shortlog can aggregate those:

```
git shortlog -sn --group=author --group=trailer:co-authored-by HEAD
```

The `--group` flag may be specified more than once; a commit is counted under each matching group value.

Pitfalls & gotchas

`-n` alone does not suppress commit subjects. The flag changes the sort order but not the output format. Use `-sn` together, not just `-n`, when you want the count-only summary.

Merge commits double-count work on trunk-based repos. On a repo where all feature branches are merged (not rebased), every commit on a feature branch appears once in the feature author’s bucket and the merge commit appears in the merger’s bucket. Add `--no-merges` to eliminate the merge entries; or use `--first-parent` to see only the trunk commits.

The revision range is required when piping. When reading from standard input, `git shortlog` does not attach to the repository, so revision-range expansion such as `HEAD` or tag names cannot be resolved. Pass the pre-expanded commit list through `git log` and pipe it in instead:

```
git log --pretty=short v1.3.0..HEAD | git shortlog -sn
```


Commit counts do not measure contribution quality. A contributor who writes ten small, focused commits will rank higher than one who writes a single large architectural change. Use the numbers as a starting point for recognition, not as a definitive ranking.

--group=trailer:<field> silently ignores commits missing the trailer. Commits without the specified trailer are excluded from the count entirely. If only some commits have Reviewed-by trailers, the resulting totals reflect only those commits.

-w line-wrapping applies to commit subjects, not to names. The width option wraps the individual commit-subject lines inside each author block. It does not truncate long author names.

Worked examples

Generating a release announcement contributor list

Your project tags releases. You want the contributor section of the `CHANGELOG` entry for `v2.1.0`:

```
git shortlog -sn --no-merges v2.0.0..v2.1.0
```

```
31 Alice Johnson
18 Bob Nakamura
12 Carol Wei
5 David Osei
3 Eve Martinez
```

To produce the formatted paragraph often seen in announcements (“Thanks to Alice Johnson, Bob Nakamura, ...”), combine with `awk` or a simple shell pipeline:

```
git shortlog -sn --no-merges v2.0.0..v2.1.0 | awk '{ $1=""; print substr($0,2) }' |
paste -sd ', '
```

Reviewing activity on a single directory over the last quarter

You want to know who touched the authentication subsystem in the past 90 days, excluding merge commits:

```
git shortlog -sne --no-merges --since="90 days ago" HEAD -- src/auth/
```

```
17 Alice Johnson <alice@example.com>
9 Bob Nakamura <bob@example.com>
2 Carol Wei <carol@example.com>
```

The `-e` flag reveals email addresses. If you see the same person listed twice under different emails, that is a signal to add an entry to `.mailmap`.

Grouping by reviewer trailer

Your team adds Reviewed-by trailers. You want a quarterly review leaderboard:

```
git shortlog -sn --group=trailer:reviewed-by --since="2026-01-01" HEAD
```

```
42 Alice Johnson
27 Bob Nakamura
15 Carol Wei
```

Only commits that carry the trailer are counted, so the total will be lower than the total commit count if not all commits have been reviewed.

Piping from `git log` for a custom format

You want to list each contributor alongside the short hash and subject of their most recent commit — useful for a digest email. Use `--format` to replace the default commit subject:

```
git shortlog --format="%h %s" -n HEAD
```

Each author block then shows `<hash> <subject>` instead of just the subject.

Recovery

`git shortlog` is a read-only command. It never modifies refs, the index, or the working tree, so there is nothing to undo.

If your contributor counts look wrong (duplicates, missing authors), the fix is usually a `.mailmap` correction rather than anything in `git shortlog` itself. See the *log* chapter for the mailmap format and how to verify it with `git log --use-mailmap`.

See also

- *log* — the underlying history traversal that `git shortlog` builds on; also covers `.mailmap` and `--format` strings.
- *blame* — attribute individual lines of a file to the commit that last changed them.
- *show* — inspect a single commit in full detail.
- *Getting out of jams* — if history looks wrong, this is the place to start.

describe

Produce a human-readable name for any commit by anchoring it to the nearest reachable tag.

Mental model

Every commit has a SHA-1 hash, but hashes are opaque. `git describe` gives you something you can read, say aloud, and embed in a build artifact:

```
v1.4.2-7-g3a8f91c
|      |      |
|      |      └─ abbreviated SHA of the commit being described
|      └───────── number of commits on top of the tag
└──────────────── most recent reachable annotated tag
```

Git walks backwards through the commit graph from the target commit (HEAD by default), looking for the nearest tag. If the commit is exactly tagged, the tag name alone is returned. If not, Git counts how many commits separate the target from that tag and appends that count plus a `g`-prefixed hash.

The `g` prefix stands for “git” and exists so that version strings stay unambiguous in environments where other SCMs might produce similar-looking output.

By default, only annotated tags are considered. Lightweight tags (created with `git tag` without `-a` or `-s`) are ignored unless you pass `--tags`.

Synopsis

```
git describe [--all] [--tags] [--contains] [--abbrev=<n>]
             [--candidates=<n>] [--exact-match] [--long]
             [--match <pattern>] [--exclude <pattern>]
             [--always] [--first-parent] [--debug]
             [<commit-ish>...]
git describe [--all] [--tags] [--contains] [--abbrev=<n>] --dirty[=<mark>]
git describe <blob>
```

Everyday usage

Describe the current HEAD:

```
git describe
# v2.1.0-14-g9f3c20a
```

Describe a specific commit or branch:

```
git describe main
git describe a3f9c1b
```

Check whether the working tree is clean relative to the nearest tag (useful in release scripts):

```
git describe --dirty
# v2.1.0          (clean)
# v2.1.0-dirty    (uncommitted changes present)
```

Get just the nearest tag name, with no suffix:

```
git describe --abbrev=0
# v2.1.0
```

Always produce output even when no tag exists yet:

```
git describe --always
# 9f3c20a (falls back to abbreviated commit hash)
```

Key options

Option	What it does	When to use it
<code>--tags</code>	Match lightweight tags in addition to annotated tags	Repos that use lightweight tags for releases
<code>--all</code>	Match any ref under <code>refs/</code> (branches, remote-tracking refs, tags)	Exploratory use; rarely useful in scripts
<code>--contains</code>	Find the nearest tag that comes after the commit and contains it	Determine which release first shipped a given commit
<code>--abbrev=<n></code>	Use <code><n></code> hex digits for the hash suffix; <code>0</code> suppresses it entirely	Normalize output length; <code>--abbrev=0</code> to get just the tag name
<code>--long</code>	Always emit the long <code>tag-N-gHASH</code> format, even on exact tag matches	Scripts that always need to parse all three components
<code>--dirty[=<mark>]</code>	Append <code>-dirty</code> (or a custom mark) if the working tree has local modifications	Build scripts and version-string generators
<code>--broken[=<mark>]</code>	Like <code>--dirty</code> but appends <code>-broken</code> if the repo state cannot be determined	CI environments where a corrupt repo is possible
<code>--exact-match</code>	Only print output if the commit is directly tagged; exit non-zero otherwise	Release gates that must confirm an exact tagged commit
<code>--candidates=<n></code>	Consider up to <code><n></code> recent tags as candidates (default 10)	Deep histories where the nearest tag is more than 10 candidates back
<code>--match <pattern></code>	Only consider tags matching the given glob pattern	Repos with multiple tag namespaces (e.g., <code>v*</code> vs. <code>rc*</code>)
<code>--exclude <pattern></code>	Skip tags matching the given glob pattern	Exclude pre-release tags from version strings
<code>--always</code>	Fall back to an abbreviated commit hash when no tag is reachable	Guarantee output in shallow clones or untagged repos
<code>--first-parent</code>	Follow only first parents when walking the graph	Avoid picking up tags from merged feature branches
<code>--debug</code>	Print the search strategy to stderr	Diagnosing why describe chose a particular tag

Best practices

Use annotated tags for releases. `git describe` prefers annotated tags over lightweight ones, and annotated tags carry authorship and a message. Create them with `git tag -a v1.0.0 -m "Release 1.0.0"` rather than plain `git tag v1.0.0`. Your release tooling will then work with the default `git describe` invocation and no extra flags.

Embed the describe output in build artifacts. A common pattern is to bake the version string into a compiled binary or a deployed package at build time:

```
VERSION=$(git describe --tags --always --dirty)
echo "Building version: $VERSION"
```

The `--tags` flag catches lightweight tags, `--always` prevents failure in shallow clones, and `--dirty` makes it obvious when someone shipped uncommitted work.

Use `--abbrev=0` to extract the base tag only. If you only need the tag name for changelog generation or milestone comparison, `--abbrev=0` strips the commit-count and hash suffix cleanly:

```
LAST_TAG=$(git describe --abbrev=0)
git log "${LAST_TAG}..HEAD" --oneline
```

Use `--match` to enforce a tag namespace in a monorepo. In projects with multiple release tracks, scope describe to the relevant tag prefix:

```
git describe --match "api/v*"
git describe --match "frontend/v"
```

Pair `--contains` with `--tags` when tracing when a fix shipped. Given a commit hash from a bug report, find the earliest release that contains it:

```
git describe --contains --tags abc1234
# v3.2.1~5 (the commit is 5 hops before v3.2.1 on the path to HEAD)
```

Pitfalls & gotchas

No annotated tags means no output. On a fresh repo, or in a shallow clone that did not fetch tags, `git describe` exits with a non-zero status and prints nothing. Add `--always` to fall back to the commit hash, or `--tags` if lightweight tags are available.

Shallow clones truncate the graph. CI environments often fetch with `--depth 1` or `--depth 50`. If the nearest annotated tag is outside that window, `git describe` cannot find it. Either deepen the clone (`git fetch --unshallow`) or use `--tags --always` to get at least something useful.

`--abbrev=0` is not the same as `--exact-match`. `--abbrev=0` silently drops the suffix even when the commit is not directly tagged — the tag name it returns may be many commits behind HEAD. `--exact-match` fails explicitly when the commit is not directly tagged, which is what release gate scripts usually want.

Annotated tags are preferred, but `--tags` does not remove that preference. When both an annotated and a lightweight tag point to the same commit, the annotated one wins. This is usually the right behavior but can surprise you if you created a lightweight tag intending to override an earlier annotated one.

--contains walks forward, not backward. The semantics reverse: it finds a tag that the commit eventually leads to, not a tag it descended from. The output format changes too — you may see `v1.2.0~3` rather than `v1.1.0-4-gabcdef0`.

Tag patterns use glob(7), not regex. The `--match` and `--exclude` patterns are shell-style globs (`v*`, `v[0-9]*`), not regular expressions. A pattern like `v\d+\.\d+` will not work as expected.

Worked examples

Generating a version string for a release build

Your CI pipeline needs a reproducible version string for every build, whether it is an exact release or a pre-release snapshot.

```
# In your build script:
VERSION=$(git describe --tags --long --always --dirty=-dev)
echo "$VERSION"
```

```
v2.3.0-0-g7a4f1c2      # exact tag, clean tree (long format requested)
v2.3.0-3-g9b1e88a      # 3 commits past the tag
v2.3.0-3-g9b1e88a-dev  # same, with uncommitted changes
7a4f1c2                # no tags at all (--always fallback)
```

Using `--long` ensures the format is always `tag-N-gHASH`, so a parser can reliably split on `-` without special-casing exact matches.

Finding which release first shipped a commit

A customer reports a regression introduced after `v4.0.0`. You have the commit hash from `git bisect` (see the *bisect* chapter) and want to know which release first contained it.

```
git describe --contains --tags 8f3c901
```

```
v4.1.2~12
```

The commit is an ancestor of `v4.1.2`, twelve hops before it on the path to that tag. The regression first appeared in `v4.1.2`.

If the commit is too recent to have a tag above it, the command exits non-zero. Check `git log --oneline 8f3c901..HEAD` to see how far it is from the tip.

Scoping describe to a tag namespace in a monorepo

A monorepo contains two independently versioned components. Tags look like `api/v1.2.3` and `ui/v2.0.1`. Describe each component separately:

```
# Component versions
API_VERSION=$(git describe --match "api/v*" --tags --abbrev=0)
echo "${API_VERSION#api/}" # strip the prefix → v1.2.3

UI_VERSION=$(git describe --match "ui/v*" --tags --abbrev=0)
echo "${UI_VERSION#ui/}"  # strip the prefix → v2.0.1
```

The `--match` flag prevents the API describe from accidentally anchoring to a UI tag that happens to be closer in the graph.

Guarding a release pipeline with `--exact-match`

A release pipeline should only proceed when HEAD is an exact tagged commit.

```
if ! git describe --exact-match --tags HEAD 2>/dev/null; then
    echo "HEAD is not tagged. Tag the release commit before running this
    pipeline."
    exit 1
fi
```

`--exact-match` exits non-zero and prints nothing if HEAD is not directly tagged, making it ideal as a guard condition. The `2>/dev/null` redirect suppresses the error message Git would otherwise emit to stderr.

Recovery

`git describe` is a read-only command — it never modifies the repository. There is nothing to undo.

If the command produces unexpected output, use `--debug` to inspect the search strategy:

```
git describe --debug
```

Git prints to stderr which tags it evaluated and how many commits apart each one was, letting you trace exactly why it chose a particular anchor.

If describe fails because no tags exist in the repo, see the *tag* chapter for how to create an annotated tag, then re-run describe.

See also

- *tag* — creating the annotated tags that `git describe` anchors to.
- *log* — `git log <tag>..HEAD` lists commits since the last described tag.
- *bisect* — finding the commit that introduced a bug; pair with `git describe --contains` to name the release that shipped it.
- *shortlog* — summarising commits between two described versions.
- *Getting out of jams* — general recovery when the repository state is unexpected.

reflog

Show, manage, and recover from Git's local record of every position HEAD and branch tips have ever occupied.

Mental model

Every time a reference moves — you commit, switch branches, rebase, reset, merge, cherry-pick, or pull — Git appends an entry to a log file stored under `.git/logs/`. That log is the reflog. It is purely local: it never leaves your machine during a push or fetch, and it is never shared with collaborators.

Think of the reflog as a personal undo stack for references. Each entry records the old SHA-1, the new SHA-1, a timestamp, the identity of the person who made the change, and a short description of what happened.

old SHA-1	-> new SHA-1	identity	timestamp	description
a3f9c1	8b4d72	you	2 hours ago	commit: Fix null pointer
8b4d72	HEAD	you	3 hours ago	commit: Add login form

The most important reflog is `HEAD`, which records every position HEAD has occupied. Each branch also has its own reflog under `.git/logs/refs/heads/<branch>`.

You refer to reflog entries with the `@{n}` syntax: `HEAD@{0}` is the current position, `HEAD@{1}` is where HEAD was before the last operation, `HEAD@{2}` two operations ago, and so on. You can also use time-based syntax: `HEAD@{1.hour.ago}` or `main@{yesterday}`.

The reflog is the safety net that makes destructive operations — hard resets, rebases, amends — survivable. As long as a commit was ever reachable from a local ref, its SHA-1 appears in the reflog and can be retrieved.

Synopsis

```
git reflog [show] [<log-options>] [<ref>]
git reflog list
git reflog expire [--expire=<time>] [--expire-unreachable=<time>]
    [--rewrite] [--updateref] [--stale-fix]
    [--dry-run | -n] [--verbose] [--all] [--single-worktree] | <refs>...
git reflog delete [--rewrite] [--updateref]
    [--dry-run | -n] [--verbose] <ref>@{<specifier>}...
git reflog drop [--all] [--single-worktree] | <refs>...
git reflog exists <ref>
```

Everyday usage

Show the reflog for HEAD (the default):

```
git reflog
```

```
8b4d72a (HEAD -> main) HEAD@{0}: commit: Add rate limiting
a3f9c1b HEAD@{1}: reset: moving to HEAD~1
f7e2c80 HEAD@{2}: commit: Fix null pointer in parseToken
```

Show the reflog for a specific branch:

```
git reflog show feature/login
```


Show the last ten entries (the reflog accepts any `git log` option):

```
git reflog -10
```

Show entries with full dates instead of relative times:

```
git reflog --date=iso
```

Find the SHA-1 of where HEAD was three moves ago:

```
git show HEAD@{3}
```

Find where the branch `main` was yesterday:

```
git show main@{yesterday}
```

List all refs that have a reflog:

```
git reflog list
```

Check whether a reflog exists for a ref:

```
git reflog exists refs/heads/main
```

Key options

`git reflog show` accepts all options that `git log` accepts. The options below cover the most useful ones for each subcommand.

Option	Subcommand	What it does	When to use it
<code><ref></code>	<code>show</code>	Show the reflog for the named ref (default: <code>HEAD</code>)	Inspect a branch's movement history
<code>-n <number></code>	<code>show</code>	Limit output to the last <code>n</code> entries	Keep the output manageable
<code>--date=<format></code>	<code>show</code>	Format timestamps (<code>relative</code> , <code>iso</code> , <code>short</code> , <code>local</code> , <code>raw</code>)	Correlate with wall-clock time
<code>--oneline</code>	<code>show</code>	Condense each entry to one line	Quick scanning
<code>--all</code>	<code>expire</code> , <code>drop</code>	Apply to every ref's reflog	Repository-wide view or bulk pruning
<code>--single-worktree</code>	<code>expire</code> , <code>drop</code>	Limit <code>--all</code> to the current worktree	Multi-worktree repositories
<code>--expire=<time></code>	<code>expire</code>	Prune entries older than <code><time></code> (default: <code>gc.reflogExpire</code> , 90 days)	Reclaim space in old repositories

<code>--expire-unreachable=<time></code>	<code>expire</code>	Prune unreachable entries older than <code><time></code> (default: 30 days)	Faster pruning of abandoned work
<code>--rewrite</code>	<code>expire</code> , <code>delete</code>	Adjust predecessor SHA-1s after pruning to keep entries internally consistent	Repair scenarios
<code>--updateref</code>	<code>expire</code> , <code>delete</code>	Move the ref to <code><ref>@{0}</code> if the previous top entry was pruned	Rare repair scenarios
<code>--stale-fix</code>	<code>expire</code>	Also prune entries pointing at broken (unreachable, object-missing) commits	Repairing corruption from old Git versions
<code>-n</code> / <code>--dry-run</code>	<code>expire</code> , <code>delete</code>	Show what would be pruned without doing it	Safe preview before destructive runs
<code>--verbose</code>	<code>expire</code> , <code>delete</code>	Print each pruned entry	Audit trails

Best practices

Check the reflog before declaring data lost. Before reaching for a backup or asking a colleague for their copy, run `git reflog`. If the commit was ever on your machine, the SHA-1 is there. Only a `git gc` run after the default 30-day unreachable-entry window will have removed it.

Use `HEAD@{n}` as a safe target for `git reset`. When recovering from a bad rebase or reset, find the pre-operation entry in the reflog and reset directly to its SHA-1 rather than guessing with `HEAD~n`. The entry description (`rebase: start`, `reset: moving to ...`) tells you exactly which operation to undo.

```
git reflog
# Find the entry just before the bad operation, e.g. HEAD@{4}
git reset --hard HEAD@{4}
```

Combine `--date=iso` with `grep` when the list is long. Time-based selectors like `HEAD@{2.days.ago}` can be imprecise on machines with irregular git activity. Displaying ISO timestamps and grepping for a known time window is more reliable.

```
git reflog --date=iso | grep "2026-06-15"
```

Do not rely on `git reflog` in CI or shared environments. The reflog is local-only. Clones start with no reflog at all. Do not write scripts that assume the reflog is present or complete outside a developer workstation.

Let `git gc` manage expiry automatically. You rarely need to run `git reflog expire` by hand. The defaults — 90 days for reachable entries, 30 days for unreachable ones — strike a sensible balance between safety and disk usage. Override them with `gc.reflogExpire` and `gc.reflogExpireUnreachable` in your git config if the defaults do not suit your workflow.

```
git config --global gc.reflogExpire 180.days
git config --global gc.reflogExpireUnreachable 90.days
```

Pitfalls & gotchas

The reflog does not exist in a fresh clone. Cloning downloads objects and references but not `.git/logs/`. If you clone a repository and immediately check the reflog, it will be empty or contain only the initial checkout entry. This catches people who expect to see a remote team member's history in the reflog.

@{n} counts operations, not commits. A rebase of ten commits is one reflog entry for the final HEAD position (plus internal entries during the rebase). Do not assume `HEAD@{10}` is ten commits back — it is ten HEAD movements back, which may span a single rebase or a single merge.

Time-based selectors depend on the local clock. `HEAD@{1.week.ago}` is evaluated against your system clock at the moment you run the command. If your clock was wrong or you changed time zones, the entry you get may not be what you expect. Verify with `--date=iso` when precision matters.

Reflog entries are not immutable. Running `git reflog expire --expire=all --all` or `git reflog drop --all` permanently removes entries. After `git gc` runs (automatically or on demand), entries past the expiry window are gone and the associated loose objects may be garbage-collected.

Rebasing creates many entries quickly. An interactive rebase of a twenty-commit branch can add dozens of entries to the HEAD reflog. If something went wrong mid-rebase, the entry you need is buried. Use `--oneline` and `grep` the description column:

```
git reflog --oneline | grep "rebase: start"
```

`git reflog show` is `git log -g --abbrev-commit --pretty=oneline` under the hood. Most `git log` options work, but options that imply a graph traversal (like `--graph`) produce confusing output because the reflog is a linear sequence, not a DAG.

Worked examples

Recovering commits after a hard reset

You ran `git reset --hard` to a commit several steps back, discarding work you needed. The branch pointer no longer points at the commits you lost.

```
git reflog
```

```
d9a1c03 (HEAD -> feature/auth) HEAD@{0}: reset: moving to d9a1c03
7f3e812 HEAD@{1}: commit: Add JWT refresh logic
4b2a990 HEAD@{2}: commit: Implement token expiry
d9a1c03 HEAD@{3}: commit: Scaffold auth module
```

The two commits you lost are at `HEAD@{1}` (`7f3e812`) and `HEAD@{2}` (`4b2a990`). Reset back to the most recent of the lost commits:

```
git reset --hard HEAD@{1}
```

Your branch now points at `7f3e812` again — both lost commits restored.

Recovering a dropped stash

You ran `git stash drop` on the wrong stash entry. The stash reference is gone but the commit object still exists in the object database.

Find the stash object via the stash reflog:

```
git reflog show refs/stash
```

```
b5c7f21 refs/stash@{0}: WIP on main: 8b4d72a Add rate limiting
a91e034 refs/stash@{1}: WIP on main: f7e2c80 Fix null pointer
```

`refs/stash@{1}` is the dropped entry. Apply its SHA-1 directly:

```
git stash apply a91e034
```

Or create a branch from it so you can inspect it first:

```
git checkout -b recovered-stash a91e034
```

Finding when a file was in a known-good state

You know `src/config.js` was correct sometime last week but cannot identify the commit. Use `--date=iso` to find reflog entries from that period:

```
git reflog --date=iso | grep "2026-06-10"
```

```
c3d44f1 HEAD@{2026-06-10 14:23:11 +0200}: commit: Update DB pool settings
```

Inspect the file at that commit:

```
git show c3d44f1:src/config.js
```

If this is the right version, restore it to the working tree:

```
git checkout c3d44f1 -- src/config.js
```

Diagnosing an interrupted rebase

Your rebase was interrupted by conflicts and you are not sure what state the repository is in. Read the reflog to reconstruct the sequence:

```
git reflog -20 --oneline
```

```
e1a3c55 HEAD@{0}: rebase (pick): Add input validation
9b2f440 HEAD@{1}: rebase (pick): Extract helper functions
3d7c118 HEAD@{2}: rebase: checkout main
8b4d72a HEAD@{3}: rebase: start
```

The entry at `HEAD@{3}` (`8b4d72a`) is where HEAD was before the rebase began. To abort the rebase cleanly:

```
git rebase --abort
```

If rebase state was already cleaned up and `--abort` is not available, reset directly to the pre-rebase SHA-1:

```
git reset --hard 8b4d72a
```

Recovery

The reflog is itself the primary recovery mechanism for most local catastrophes. If entries have already been pruned by `git gc`, the objects may still exist as loose objects in the database for a short window after expiry — use `git fsck --unreachable` to locate dangling commits before they are collected.

To reduce the risk of losing reflog coverage before you need it, increase the expiry windows:

```
git config --global gc.reflogExpire 180.days
git config --global gc.reflogExpireUnreachable 90.days
```

There is no way to reverse a completed `git reflog expire` or `git reflog drop` run. The entries — and any objects they exclusively referenced — are gone once garbage collection has run.

See *Getting out of jams* for step-by-step undo recipes that use reflog entries as recovery targets.

See also

- *reset* — the most common reason to need the reflog; `git reset --hard HEAD@{n}` is the canonical recovery pattern.
- *rebase* — interactive rebases generate many reflog entries; know how to read them before rebasing large branches.
- *stash* — stash entries have their own reflog at `refs/stash` and can be recovered the same way as lost commits.
- *log* — `git reflog show` is built on `git log -g` all log options apply.
- *gc* — controls when reflog entries expire via `gc.reflogExpire` and `gc.reflogExpireUnreachable`.
- *Getting out of jams* — the troubleshooting chapter with concrete undo recipes anchored on reflog entries.

grep

Search for patterns across every file Git tracks — or across blobs in any tree object — without leaving your repository.

Mental model

`git grep` is a repository-aware search engine. Where a plain `grep -r` walks the filesystem and stumbles over build artefacts, `node_modules`, and binary blobs, `git grep` works from what Git knows: tracked files in the working tree, the index, or any commit (or tag, or branch tip) you name.

Three search targets are available:

Working tree	—	the default; what is on disk right now
Index	—	<code>--cached</code> ; what is staged
Tree object	—	<code>git grep <pattern> <tree></code> ; any commit/branch/tag

Because Git controls the file list, results are always relevant to the project. Untracked files and anything listed in `.gitignore` are invisible by default. When you need to go back in time — “was this constant defined in the release-2.0 tag?” — you hand `git grep` a tree reference instead of hunting through `git log` output.

Synopsis

```
git grep [-a | --text] [-I] [-i | --ignore-case] [-w | --word-regexp]
        [-v | --invert-match] [--full-name]
        [-E | --extended-regexp] [-G | --basic-regexp] [-P | --perl-regexp]
        [-F | --fixed-strings] [-n | --line-number] [--column]
        [-l | --files-with-matches] [-L | --files-without-match]
        [-z | --null] [-o | --only-matching] [-c | --count]
        [-q | --quiet]
        [--break] [--heading] [-p | --show-function]
        [-A <num>] [-B <num>] [-C <num>] [-W | --function-context]
        [(-m | --max-count) <num>] [--threads <num>]
        [-f <file>] [-e <pattern>]
        [--and | --or | --not | -e <pattern>...] [--all-match]
        [--cached | --untracked | --no-index]
        [--recurse-submodules]
        [<tree>...] [--] [<paths-spec>...]
```

Everyday usage

Search all tracked files for a string:

```
git grep 'TODO'
```

Case-insensitive search:

```
git grep -i 'deprecated'
```

Show line numbers alongside matches:

```
git grep -n 'MAX_RETRIES'
```

List only the filenames that contain a match — useful for piping:

```
git grep -l 'console\.log'
```

Search a specific subdirectory or file pattern:

```
git grep 'error_code' -- 'src/*.c'
```

Search across a past commit or branch:

```
git grep 'old_function_name' main
git grep 'old_function_name' v2.3.0
```

Show a few lines of context around each match:

```
git grep -n -C 3 'initDatabase'
```

Count matches per file rather than printing each line:

```
git grep -c 'import React'
```

Key options

Option	What it does	When to use it
<code>-n</code> , <code>--line-number</code>	Prefix each match with its line number	Navigate to matches in an editor
<code>-i</code> , <code>--ignore-case</code>	Match regardless of letter case	Searching identifiers with inconsistent casing
<code>-w</code> , <code>--word-regexp</code>	Match only at word boundaries	Avoid <code>count</code> matching <code>discount</code>
<code>-l</code> , <code>--files-with-matches</code>	Print only filenames, not matching lines	Feed to <code>xargs</code> or another command
<code>-L</code> , <code>--files-without-match</code>	Print only filenames that have no match	Audit which files lack a required header
<code>-v</code> , <code>--invert-match</code>	Print lines that do not match	Find files missing something expected
<code>-c</code> , <code>--count</code>	Print match count per file	Quick frequency survey
<code>-E</code> , <code>--extended-regexp</code>	POSIX extended regex (<code>+</code> , <code>?</code> , <code> </code> , <code>()</code>)	Most regex needs; cleaner than basic regex
<code>-P</code> , <code>--perl-regexp</code>	Perl-compatible regex (PCRE)	Lookaheads, lookbehinds, <code>\b</code> , <code>\K</code>
<code>-F</code> , <code>--fixed-strings</code>	Treat pattern as a literal string	Searching for <code>.</code> or <code>*</code> without escaping
<code>-A <num></code>	Show <code><num></code> lines after each match	See what follows a match
<code>-B <num></code>	Show <code><num></code> lines before each match	See what precedes a match

<code>-C <num></code>	Show <code><num></code> lines before and after	General context around a match
<code>-W</code> , <code>--function-context</code>	Show the whole enclosing function	Understand a match without opening the file
<code>-p</code> , <code>--show-function</code>	Print the preceding function-name line above each match	Locate which function contains the match
<code>--cached</code>	Search the index instead of the working tree	Check what is staged
<code>--untracked</code>	Also search untracked files	Cast a wider net; includes new files
<code>--no-index</code>	Search outside a Git repo (like <code>grep -r</code>)	Scripts that may run in non-Git directories
<code>-e <pattern></code>	Explicitly mark the next argument as the pattern	Required when the pattern starts with <code>-</code> also used for Boolean combinations
<code>--and</code> , <code>--or</code> , <code>--not</code>	Combine multiple <code>-e</code> patterns with Boolean logic	Multi-condition searches
<code>--all-match</code>	Restrict to files where every <code>--or</code> pattern appears	Audit files that must contain two independent things
<code>-q</code> , <code>--quiet</code>	Exit 0/non-zero without printing	Shell conditionals and scripts
<code>-o</code> , <code>--only-matching</code>	Print only the matched portion of each line	Extract values rather than whole lines
<code>--color[=<when>]</code>	Highlight matches in colour (<code>always</code> , <code>never</code> , <code>auto</code>)	Terminal readability
<code>--break</code>	Print a blank line between results from different files	Visual separation in long output
<code>--heading</code>	Print the filename once above its matches instead of on every line	Cleaner output when many lines match per file
<code>--recurse-submodules</code>	Descend into active submodules	Monorepos with submodules
<code>-f <file></code>	Read patterns from a file, one per line	Large or machine-generated pattern lists
<code>-m <num></code> , <code>--max-count <num></code>	Stop after <code><num></code> matches per file	Speed up exploratory searches
<code>--threads <num></code>	Set the number of worker threads	Tune performance on large repositories
<code>-z</code> , <code>--null</code>	Delimit filenames with <code>\0</code> in output	Safe piping to <code>xargs -0</code>

Best practices

Anchor searches to a path when you know where to look. A project-wide search for `id` returns thousands of hits. Narrowing to a subdirectory or glob finds what you actually want in milliseconds:


```
git grep -n 'user_id' -- 'src/models/*.py'
```

Use **-l** with **-z** when piping into **xargs**. Filenames can contain spaces. Combining **--null** with **xargs -0** handles them correctly:

```
git grep -lz 'console\.log' | xargs -0 sed -i '' 's/console\.log/logger.debug/g'
```

Prefer **-E** or **-P** over the default basic regex. Basic regex requires backslash-escaping **+** and **|**. Extended (**-E**) or Perl (**-P**) regex are far more readable for anything beyond a simple literal:

```
# Basic regex – confusing escapes required
git grep 'function\|method'

# Extended regex – clear alternation
git grep -E 'function|method'
```

Search historical trees to answer “when did this change?” Use known tags or branch tips as the search target. Once you find the boundary, use **git bisect** (see the *bisect* chapter) to narrow to a specific commit:

```
git grep 'legacyAuth' v1.0.0 # present in v1.0.0?
git grep 'legacyAuth' v2.0.0 # present in v2.0.0?
```

Use **-q** in scripts instead of parsing output. The exit code is reliable: 0 means at least one match, 1 means no matches, 128 means an error. Parsing text output is fragile by comparison:

```
if git grep -q 'set -e' scripts/deploy.sh; then
    echo "deploy.sh enables strict mode"
fi
```

Configure defaults in your global config. Always having line numbers and extended regex avoids typing the same flags every day:

```
git config --global grep.lineNumber true
git config --global grep.patternType extended
```

Pitfalls & gotchas

Untracked and ignored files are invisible by default. If you just created a file that is not yet staged, **git grep** will not find it unless you add **--untracked**. Vendor directories in **.gitignore** are always excluded unless you use **--no-index**, which bypasses Git’s file list entirely.

The default regex engine is basic, not extended. Characters like **+**, **?**, **|**, **(**, and **)** must be escaped or you must pass **-E**. A common surprise: **git grep 'foo+bar'** matches the literal string **foo+bar**, not “one or more **o**s followed by **bar**”.

-w (--word-regexp) uses C-locale word boundaries. It treats any non-alphanumeric, non-underscore character as a word separator. This matches POSIX **grep** behaviour but can produce unexpected results with Unicode identifiers.

Searching a tree does not search the working tree. `git grep 'foo' HEAD` searches the committed HEAD snapshot, not your unsaved edits. To check staged changes use `--cached` to check the latest commit use `HEAD` as the tree argument.

Boolean expressions require `-e` for every pattern. The `--and`, `--or`, and `--not` operators only work when each pattern is introduced with `-e`. Omitting `-e` makes Git treat the bare word as a pathspec:

```
# Wrong: 'FIXME' is interpreted as a pathspec limiter, not a pattern
git grep -e 'TODO' --and 'FIXME'

# Correct
git grep -e 'TODO' --and -e 'FIXME'
```

Multi-threaded searches over the object store can be slower with `--textconv`. When grepping with `--cached` or a tree argument and text conversion is involved, try `--threads=1` if performance is unexpectedly poor.

Worked examples

Finding all call sites before renaming a function

You are renaming `fetchUser` to `getUser` across a Go codebase. Before touching anything, map every call site:

```
git grep -n 'fetchUser' -- '*.go'
```

```
api/handler.go:14:    u, err := fetchUser(ctx, id)
api/handler.go:87:    return fetchUser(ctx, req.UserID)
db/queries.go:32:func fetchUser(ctx context.Context, id int) (*User, error) {
internal/cache.go:11:    cached := fetchUser(ctx, key)
```

Get only the filenames for a bulk replacement, using `\0` as delimiter so spaces in paths cause no problems:

```
git grep -lz 'fetchUser' -- '*.go' | xargs -0 sed -i '' 's/fetchUser/getUser/g'
```

Confirm no occurrences remain — the exit code should be 1:

```
git grep 'fetchUser' -- '*.go'
echo "exit: $?"
```

```
exit: 1
```

Auditing for a required copyright header

Your team requires every Python file to open with a copyright comment. Find which files are missing it:

```
git grep -L 'Copyright (c) Acme Corp' -- '*.py'
```

```
scripts/legacy_import.py
tests/fixtures/sample_data.py
```

Only those two files need the header; the rest already have it.

Comparing a pattern across branches

You want to confirm whether a feature flag named `enable_new_checkout` was present at the last release tag but has since been removed in `main`:

```
git grep -l 'enable_new_checkout' v3.0.0
git grep -l 'enable_new_checkout' main
```

If the first command lists files and the second prints nothing, the flag was cleaned up after the release — or was never backported to the release branch, depending on your workflow.

Boolean search: lines that both define a symbol and mention a unit

Find every line in C headers that contains `#define` and also contains the string `_MS` (a millisecond suffix convention):

```
git grep -e '#define' --and -e '_MS' -- '*.h'
```

Both patterns must match the same line. Without `--and`, `--or` is the default and either pattern alone would qualify.

Recovery

`git grep` is read-only — it never modifies the working tree, the index, or history. There is nothing to undo from the `grep` itself.

If you piped `git grep -l` output into a command that modified files and the result was wrong, restore the affected files with `git restore`:

```
# Restore a single file to its last committed state
git restore path/to/file.py

# Discard all working-tree changes
git restore .
```

See *Getting out of jams* for broader recovery strategies, including recovering from unintended bulk edits triggered by `xargs` pipelines.

See also

- `log` — search commit messages with `git log --grep` combine with `git grep` to find both the code and the commit that introduced it.
- `bisect` — after `git grep` shows a pattern exists in one tag but not another, use `git bisect` to pinpoint the exact commit.
- `blame` — once `git grep -n` tells you the line, use `git blame` to see who last changed it and which commit introduced it.
- `add` — use `git grep -l` to build a targeted file list before staging with `git add`.
- *Getting out of jams* — recovering from bulk working-tree modifications driven by `xargs` pipelines built on `git grep -l` output.

range-diff

Compare two versions of a patch series to see exactly what changed between iterations.

Mental model

When you rebase a branch, revise commits after a review, or port a series of patches from one base to another, the individual commits change — their hashes are new, their diffs may be slightly different, and some commits may appear or disappear. A plain `git diff` between the two branch tips tells you only what the net result is; it says nothing about which commits were modified or added.

`git range-diff` answers a different question: *how did the patch series itself evolve?*

It works by treating each commit range as a set of patches and computing a “diff of diffs”. For each pair of commits from the two ranges, it generates both patches and then diffs those patches against each other. A cost matrix tells it which old commit corresponds to which new commit (a minimum-cost bipartite matching problem solved with the Jonker-Volgenant algorithm). The result is a side-by-side listing: old commit on the left, matched new commit on the right, with the inner diff showing exactly what changed in the patch itself.

Before rebase / old iteration	After rebase / new iteration
1: f00dbal Fix the crash	1: decafe1 Fix the crash
2: c0debee Add a helpful message	2: cab005e Add a helpful message
3: bedead TO-UNDO	(removed)
(none)	3: 0ddba11 Prepare for the inevitable

The output uses a single-character symbol in each line to summarise the relationship:

- `=` — the patch is identical (only the hash changed, e.g. due to a rebase)
- `!` — the patch changed; the inner diff follows
- `<` — the commit exists only in the old range (was dropped)
- `>` — the commit exists only in the new range (was added)

Synopsis

```
git range-diff [--color=<when>] [--no-color] [<diff-options>]
               [--no-dual-color] [--creation-factor=<factor>]
               [--left-only | --right-only] [--diff-merges=<format>]
               [--remerge-diff]
               ( <range1> <range2> | <rev1>...<rev2> | <base> <rev1> <rev2> )
               [--] <path>...
```

There are three ways to name the two commit ranges:

```
# Two explicit ranges
git range-diff origin/main..v1 origin/main..v2

# Three-dot notation (symmetric difference shorthand)
git range-diff v1...v2

# Two tips sharing a common base
git range-diff origin/main my-topic@{1} my-topic
```

Everyday usage

After an interactive rebase, compare what changed from the previous version of your branch:

```
git range-diff @{u} @{l} @
```

`@{u}` is the upstream (the merge base), `@{l}` is the branch as it was before the rebase (saved in the reflog), and `@` is the current HEAD.

Compare two explicit range references when reviewing a re-rolled patch series:

```
git range-diff origin/main..topic-v1 origin/main..topic-v2
```

Use the three-argument form to compare two branches from the same base without spelling out the ranges twice:

```
git range-diff origin/main topic-v1 topic-v2
```

Limit the comparison to a particular file to cut noise when only some commits touched a specific path:

```
git range-diff origin/main topic-v1 topic-v2 -- src/auth.c
```

Suppress the color output for piping into a pager or file:

```
git range-diff --no-color origin/main topic-v1 topic-v2
```

Key options

Option	What it does	When to use it
<code>--no-dual-color</code>	Color lines purely by the outer +/- markers, ignoring the inner diff's original colors	Simpler terminal output when dual-color is visually confusing
<code>--creation-factor=<percent></code>	Tune the cost threshold (default 60) for deciding whether a changed commit is a rewrite or a new commit	Increase when large refactors are incorrectly shown as delete+add; decrease when unrelated commits are incorrectly matched
<code>--left-only</code>	Show only commits present in the first (old) range	Audit what was dropped from a series
<code>--right-only</code>	Show only commits present in the second (new) range	Audit what was added in the new iteration
<code>--diff-merges=<format></code>	Include merge commits using the specified diff format (merge commits are ignored by default)	When the range includes merges that you care about
<code>--remerge-diff</code>	Shorthand for <code>--diff-merges=remerge</code>	Reviewing how a merge conflict was resolved across rebase iterations

	shows only conflicts resolved on top of Git's auto-merge	
<code>--[no-]notes[=<ref>]</code>	Pass through to <code>git log</code> to include or exclude commit notes	When patches carry notes that are part of the review record
<code>--color=[<when>]</code>	Control color output (<code>always</code> , <code>never</code> , <code>auto</code>)	Scripting or forcing color in a CI log
<code>-- <path>...</code>	Limit both ranges to commits that touch the given paths	Reducing noise when only a subset of files is relevant

Best practices

Run it immediately after every rebase. The reflog entry `@{1}` points to the branch tip before the rebase for as long as the reflog retains it. The three-argument invocation `git range-diff @{u} @{1} @` becomes a muscle-memory post-rebase check — it confirms you did not accidentally drop a commit or introduce an unintended change while resolving conflicts.

Paste the output into code review comments. When you re-roll a patch series after a review, attaching the `range-diff` to the cover letter (or pull-request description) lets reviewers focus only on what changed rather than re-reading the entire series. Tools like `git format-patch` support `--range-diff` for this reason.

Interpret `=` commits carefully. A `=` means the patch content is identical, but it does not mean the commit is meaningless — its position in the series may have changed, or it may now apply to a different base. Read the summary line to confirm the commit landed in the right place.

Use `--creation-factor` to fix mismatched pairings. If the output shows a large commit as deleted and an unrelated commit as added, `range-diff` has decided they are too different to pair. Raise `--creation-factor` above the default of 60 to force it to look harder for a match. Conversely, lower it if superficially similar commits (e.g. one-line changes to a big file) are being matched when they should be treated as independent.

Limit paths in large repositories. In a monorepo where a series touches tens of thousands of lines across many subsystems, passing `-- <path>` narrows the diff computation to commits relevant to a particular directory or file, which cuts both runtime and output length dramatically.

Pitfalls & gotchas

The output is human-readable only. The `range-diff` manual explicitly states that the output format is not stable across Git versions and is not intended for machine parsing. Do not script against it; use `git log --format` or `git patch-id` instead if you need programmatic access.

Passing `--stat` or other summary options may produce useless output. Some diff options affect the inner patch generation in ways `range-diff` does not yet interpret meaningfully. Stick to color and path-limiting options unless the output looks sensible.

`@{1}` becomes stale. The reflog entry `@{1}` is overwritten the next time any operation moves the branch (another rebase, a reset, a new commit). If you do not run `range-diff` immediately after the rebase, capture the pre-rebase SHA explicitly before doing more work:

```
git rev-parse HEAD # record this before rebasing
```

Then use the saved SHA in place of `@{1}`.

Merge commits are silently skipped by default. If your range includes merge commits and you expect them to show up, add `--diff-merges=remerge` (or the `--remerge-diff` shorthand). Without it, merges are omitted without any warning.

The three-dot form means something specific. `git range-diff v1...v2` is *not* the same as `git diff v1...v2`. It expands to `v1..v2 v2..v1` — the symmetric difference of both ranges. Use the explicit two-range form if you need to compare ranges that do not share the same base.

Color output requires a capable terminal. The dual-color mode (on by default) uses background-color escape codes. In terminals that do not support them, or when piping to a file, pass `--no-color` to avoid cluttered output.

Worked examples

Verifying a clean rebase

You have just rebased `feature` onto the latest `main` and want to confirm that no commit changed unexpectedly during conflict resolution.

```
git range-diff @{u} @{1} @
```

Typical output when the rebase was clean:

```
1: alb2c3d = 1: e4f5a6b Add rate limiting to login endpoint
2: 7890abc = 2: 1234def Extend session expiry to 30 days
3: fedcba9 = 3: abcdef0 Update changelog
```

Every line shows `=` — the patches are identical. The new hashes are simply because the parent commit changed during the rebase.

If one line shows `!`, expand the output (color is on by default in a terminal) to read the inner diff and understand what was altered.

Reviewing a re-rolled patch series

A colleague has sent v2 of a three-patch series. The first version is already on `origin/topic-v1` the new one is on `origin/topic-v2`. You want to see only what changed without re-reading the whole series.

```
git range-diff origin/main origin/topic-v1 origin/topic-v2
```

Sample output:

```
-: ----- > 1: 0a1b2c3 Add feature flag infrastructure
1: f1e2d3c ! 2: 4d5e6f7 Implement dark-mode toggle
@@ -3,7 +3,7 @@
   Author: Ada Lovelace <ada@example.com>

   -This commit adds a CSS class toggle on the body element.
   +This commit adds a CSS class toggle on the body element and
   +updates the test to cover the edge case in Safari.

@@ -15,6 +15,8 @@
   -body.toggleClass('dark-mode');
```

```
++expect(document.body.classList).toContain('dark-mode');
2: c0ffee1 = 3: badf00d Docs: document the dark-mode toggle
```

Reading the output:

- Commit 1 of v2 (0a1b2c3) is new — it was added to the series.
- Commit 1 of v1 maps to commit 2 of v2 and is marked **!**: the commit message was expanded and a new test line was added to the diff.
- The docs commit transferred unchanged (=).

You can now comment specifically on the changed hunk rather than reviewing the entire series from scratch.

Comparing branches that diverged from the same base

Two developers each started from `origin/main` and implemented the same feature independently. You want to see how their approaches differ at the commit level.

```
git range-diff origin/main alice/feature bob/feature
```

Commits that both developers wrote equivalently appear as `=`. Commits that one wrote but the other did not appear as `>` or `<`. This gives you a high-level picture of where the implementations diverge before you read the individual diffs.

Recovery

`range-diff` is a read-only inspection command — it does not modify the repository. There is nothing to undo.

If a rebase that you have already run produced unexpected changes, and you have lost the pre-rebase SHA from `@{1}`, recover it from the reflog:

```
git reflog show HEAD
```

Identify the commit entry immediately before the rebase line, then use that SHA in the three-argument form:

```
git range-diff origin/main <pre-rebase-sha> HEAD
```

See *Getting out of jams* for recovering a branch from the reflog after a destructive rebase.

See also

- `log` — the underlying patch generation that `range-diff` uses internally.
- `rebase` — the primary operation that creates the “before and after” ranges that `range-diff` compares.
- `reflog` — how to find the pre-rebase SHA when `@{1}` is no longer current.
- `format-patch` — patch-series workflow; supports `--range-diff` to embed the comparison in a cover letter.
- *Getting out of jams* — recovering a branch tip from the reflog after a rebase goes wrong.

remote

Manage the set of named remote repositories whose branches your local repository tracks.

Mental model

A remote is nothing more than a short alias — usually `origin` — that maps to a URL and a set of rules about which branches to track. Git stores this in `.git/config`:

```
[remote "origin"]
  url      = git@github.com:acme/api.git
  fetch    = +refs/heads/*:refs/remotes/origin/*
```

The fetch refspec `+refs/heads/*:refs/remotes/origin/*` tells Git: when you fetch from this remote, copy every branch from the server into the local namespace `origin/<branch>`. Those copies are called *remote-tracking branches* — read-only snapshots of where the remote’s branches were the last time you fetched.

`git remote` itself never talks to the network. It is purely a bookkeeping command: it reads and writes the `[remote]` sections in `.git/config`. The network calls happen in *fetch*, *pull*, and *push*.

```
.git/config
└─ [remote "origin"]      ← git remote manages this
    url = ...
    fetch = ...

refs/remotes/origin/main ← git fetch populates this
```

A repository can have any number of remotes. The name `origin` is a convention set by `git clone` nothing requires it.

Synopsis

```
git remote [-v | --verbose]
git remote add      [-t <branch>] [-m <master>] [-f] [--[no-]tags]
                    [--mirror=(fetch|push)] <name> <URL>
git remote rename  [--[no-]progress] <old> <new>
git remote remove  <name>
git remote set-head <name> (-a | --auto | -d | --delete | <branch>)
git remote set-branches [--add] <name> <branch>...
git remote get-url [--push] [--all] <name>
git remote set-url [--push] <name> <newurl> [<oldurl>]
git remote set-url --add      [--push] <name> <newurl>
git remote set-url --delete  [--push] <name> <URL>
git remote [-v | --verbose] show    [-n] <name>...
git remote prune    [-n | --dry-run] <name>...
git remote [-v | --verbose] update  [-p | --prune] [(<group> | <remote>)...]
```

Everyday usage

List all remotes (names only):

```
git remote
```

```
origin
upstream
```

List remotes with their URLs:

```
git remote -v
```

```
origin    git@github.com:you/api.git (fetch)
origin    git@github.com:you/api.git (push)
upstream  git@github.com:acme/api.git (fetch)
upstream  git@github.com:acme/api.git (push)
```

Add a new remote:

```
git remote add upstream git@github.com:acme/api.git
```

Remove a remote you no longer need:

```
git remote remove old-fork
```

Rename a remote:

```
git remote rename origin github
```

See full details about a remote — its URL, tracked branches, and whether your local branches are ahead or behind:

```
git remote show origin
```

```
* remote origin
Fetch URL: git@github.com:you/api.git
Push  URL: git@github.com:you/api.git
HEAD branch: main
Remote branches:
  main          tracked
  feature/auth   tracked
Local branch configured for 'git pull':
  main merges with remote main
Local ref configured for 'git push':
  main pushes to main (up to date)
```

Delete stale remote-tracking branches that no longer exist on the server:

```
git remote prune origin
```

Preview what prune would remove without actually removing it:

```
git remote prune --dry-run origin
```

Key options

git remote add

Option	What it does	When to use it
<code>-f</code>	Run <code>git fetch <name></code> immediately after adding	Get remote-tracking branches right away
<code>-t <branch></code>	Track only <code><branch></code> instead of all branches	Shallow-track a single branch of a large repo
<code>-m <master></code>	Set <code>refs/remotes/<name>/HEAD</code> to point at <code><master></code>	Define a default branch for the remote
<code>--tags</code>	Import all tags on every fetch	Explicit tag mirroring
<code>--no-tags</code>	Never import tags from this remote	Keep tag namespace clean
<code>--mirror=fetch</code>	Mirror all refs into local <code>refs/</code> (bare repos only)	Backup mirrors
<code>--mirror=push</code>	Always push as a mirror	Publishing mirrors

git remote show

Option	What it does	When to use it
<code>-n</code>	Use cached info; do not query the remote	Inspect config offline

git remote prune

Option	What it does	When to use it
<code>-n</code> / <code>--dry-run</code>	Report what would be pruned without doing it	Verify before destructive cleanup

git remote set-url

Option	What it does	When to use it
<code>--push</code>	Operate on push URLs instead of fetch URLs	Set a different push destination
<code>--add</code>	Add a new URL rather than replacing the existing one	Push to multiple remotes at once
<code>--delete</code>	Delete all URLs matching a regex	Remove one URL from a multi-push set

git remote update

Option	What it does	When to use it
<code>-p</code> / <code>--prune</code>	Prune stale tracking branches for each updated remote	Keep refs tidy after a bulk fetch

Best practices

Use `upstream` for the canonical source, `origin` for your fork. The convention that `origin` points to *your* fork and `upstream` points to the project you forked is widely understood. Inverting them confuses collaborators and tools.

```
git remote add upstream git@github.com:acme/api.git
git remote set-url origin git@github.com:you/api.git
```

Change URLs with `set-url`, not by editing `.git/config` by hand. The subcommand validates the new URL and handles the push/fetch URL distinction cleanly. Hand-editing is error-prone and bypasses that validation.

Prune regularly — or pass `--prune` to fetch. Branches deleted on the server linger as stale remote-tracking refs until you prune. Run `git remote prune origin` after a sprint ends, or configure automatic pruning once:

```
git config fetch.prune true
```

With that setting, every `git fetch` behaves as if `--prune` was passed, so `git remote prune` becomes unnecessary day-to-day.

Use `-t` when adding a large monorepo remote you only partially need. Tracking all branches of a repo with thousands of refs wastes bandwidth and clutters your ref namespace. Track only the branches you care about:

```
git remote add -t main -t release/v3 upstream git@github.com:acme/mono.git
```

Verify the URL before pushing to a new remote. A mistyped URL can create a new repository on some hosts instead of failing cleanly. Check it first:

```
git remote show new-remote
```

Prefer SSH URLs for remotes you push to. HTTPS requires credential entry on every push (or a credential helper). SSH keys authenticate silently and do not expire the same way personal access tokens do.

Pitfalls & gotchas

`git remote remove` deletes remote-tracking branches silently. Running `git remote remove upstream` also removes all `refs/remotes/upstream/*` refs. Those branches disappear from your local view. The data still exists on the actual server; re-add the remote and fetch to restore the remote-tracking refs.

Renaming `origin` breaks default push/pull for existing local branches. Local branches are configured with `branch.<name>.remote = origin`. After a rename, that value is stale. Update each affected branch:

```
git remote rename origin github
git branch --set-upstream-to=github/main main
```

For repositories with many local branches, re-cloning is often simpler.

`git remote show` makes a live network call. It queries the remote using `git ls-remote`. On a slow or offline connection this hangs. Pass `-n` to use cached information instead.

A remote with multiple push URLs pushes to all of them. This is useful for maintaining mirrors, but surprises people who added a second URL experimentally. Verify with `git remote get-url --push --all <name>` before pushing to avoid unintended writes.

--mirror=fetch is only safe in bare repositories. In a normal working-tree repo, a fetch mirror overwrites local commits because it copies every ref unconditionally. Use it only in repos created with `git init --bare` or `git clone --bare`.

set-url --delete matches a regex, not a literal string. Characters like `.` match any character in a URL. Verify the match first with `git remote get-url --all <name>` before deleting.

Worked examples

Setting up a fork workflow

You have cloned your fork and want to keep it in sync with the upstream project.

```
# Your clone already has origin pointing to your fork.
git remote -v
```

```
origin  git@github.com:you/api.git (fetch)
origin  git@github.com:you/api.git (push)
```

Add the upstream remote:

```
git remote add upstream git@github.com:acme/api.git
git fetch upstream
```

```
From git@github.com:acme/api.git
* [new branch]  main      -> upstream/main
* [new branch]  v2        -> upstream/v2
```

Rebase your feature branch on the latest upstream main:

```
git switch feature/auth
git rebase upstream/main
```

Push the updated branch to your fork:

```
git push origin feature/auth --force-with-lease
```

See the *rebase* chapter for details on interactive rebase and `--autosquash`, and the *push* chapter for `--force-with-lease`.

Correcting a remote URL after a repository migration

Your team migrated from Bitbucket to GitHub. The old URL stops working.

Check the current URL:

```
git remote get-url origin
```

```
git@bitbucket.org:acme/api.git
```

Update it:

```
git remote set-url origin git@github.com:acme/api.git
```

Verify:

```
git remote -v
```

```
origin  git@github.com:acme/api.git (fetch)
origin  git@github.com:acme/api.git (push)
```

Test that the new URL resolves before pushing:

```
git remote show origin
```

Pushing to two remotes simultaneously

You want every push to `origin` to also update a backup mirror on a self-hosted Gitea instance.

```
git remote set-url --add --push origin git@github.com:acme/api.git
git remote set-url --add --push origin git@gitea.internal:acme/api.git
```

Confirm both push URLs are registered:

```
git remote get-url --push --all origin
```

```
git@github.com:acme/api.git
git@gitea.internal:acme/api.git
```

From now on, `git push origin main` sends to both destinations in sequence. A failure on either URL causes that destination to report an error while the successful one proceeds normally.

Tracking a subset of branches from a large remote

You only need the `main` branch of a large upstream repository. Adding it without `-t` would download hundreds of branches.

```
git remote add -t main -f upstream git@github.com:acme/mono.git
```

The `-f` flag fetches immediately. Confirm only the requested branch was tracked:

```
git branch -r | grep upstream
```

```
upstream/main
```

Later, if you also need `release/v4` :

```
git remote set-branches --add upstream release/v4
git fetch upstream
```

Recovery

If you accidentally remove a remote, re-add it and fetch:

```
git remote add origin git@github.com:acme/api.git
git fetch origin
```

Your local commits are unaffected. Remote-tracking branches are cached pointers; the underlying commit objects remain in your local object database until a garbage-collection run removes unreachable objects.

If a rename or URL change breaks branch tracking, restore it with:

```
git branch --set-upstream-to=origin/main main
```

See *Getting out of jams* for broader undo strategies, including recovering from a mistaken force push.

See also

- *fetch* — download objects and update remote-tracking branches.
- *pull* — fetch followed by merge or rebase.
- *push* — upload local commits to a remote.
- *branch* — list and manage remote-tracking branches with `-r`.
- *clone* — the command that creates the initial `origin` remote.

fetch

Download objects and refs from a remote repository without modifying your working tree or current branch.

Mental model

Your local repository holds two kinds of branch pointers. First, your own branches (`main` , `feature/login` , etc.) — these move when you commit. Second, remote-tracking branches (`origin/main` , `origin/feature/login` , etc.) — these are read-only snapshots of what the remote looked like the last time you contacted it.

`git fetch` reaches out to the remote, downloads any new commits and objects, and advances those remote-tracking pointers. It never touches your branches. After a fetch, you have the remote's work locally, but none of it has been merged into anything you own. You are free to inspect it, compare it, or integrate it on your own terms.

```
Remote:  A -- B -- C -- D   (origin/main after fetch)

Local:   A -- B -- C       (your main, unchanged)
          \
           E               (your unshared work)
```

Think of `git fetch` as refreshing your map of the world: the terrain may have changed, but you have not moved yet.

Synopsis

```
git fetch [<options>] [<repository> [<refspec>...]]
git fetch [<options>] <group>
git fetch --multiple [<options>] [(<repository> | <group>)...]
git fetch --all [<options>]
```

Everyday usage

Update all remote-tracking branches from the default remote (`origin`):

```
git fetch
```

Fetch from a specific remote:

```
git fetch origin
```

Fetch and prune stale remote-tracking branches in one step:

```
git fetch --prune
```

See what the remote has that you do not, after fetching:

```
git fetch origin
git log HEAD..origin/main --oneline
```

Fetch a single branch only:


```
git fetch origin feature/payments
```

Fetch from every configured remote at once:

```
git fetch --all
```

Inspect a remote branch before integrating it:

```
git fetch origin  
git diff main origin/main
```

Key options

Option	What it does	When to use it
<code>--all</code>	Fetch from every configured remote	Monorepos or repos with multiple remotes
<code>-p</code> , <code>--prune</code>	Remove remote-tracking refs that no longer exist on the remote	Keeping the ref namespace tidy after upstream branches are deleted
<code>-P</code> , <code>--prune-tags</code>	Remove local tags that no longer exist on the remote (requires <code>--prune</code>)	Syncing tag lifecycle with the remote
<code>-t</code> , <code>--tags</code>	Fetch all remote tags in addition to branches	Ensuring all release tags are present locally
<code>-n</code> , <code>--no-tags</code>	Disable automatic tag following	CI environments or shallow clones where tags are not needed
<code>--depth=<depth></code>	Limit fetched history to <code><depth></code> commits from each branch tip	Speeding up fetches in large repos; also deepens existing shallow clones
<code>--unshallow</code>	Convert a shallow clone into a complete one	When you need full history after cloning with <code>--depth</code>
<code>--dry-run</code>	Show what would be fetched without making any changes	Verifying the effect of a refspec before running for real
<code>-f</code> , <code>--force</code>	Allow updating a local branch even if it is not a fast-forward	Overriding safety checks on explicitly mapped refspecs
<code>--atomic</code>	Update all local refs in one atomic transaction	Ensuring refs are never in a partially-updated state on error
<code>--multiple</code>	Allow fetching from several remotes or groups in one command	Batch fetching when you want to name remotes explicitly
<code>-j <n></code> , <code>--jobs=<n></code>	Fetch remotes or submodules in parallel	Speeding up <code>--all</code> or submodule-heavy repos

<code>--recurse-submodules[=yes on-demand no]</code>	Also fetch new commits in populated submodules	Projects that use submodules and need them up to date
<code>--set-upstream</code>	Record the fetched remote as the upstream for the current branch	Setting up tracking after a manual fetch
<code>-q, --quiet</code>	Suppress progress output	Scripts and CI pipelines
<code>-v, --verbose</code>	Print more information, including up-to-date refs	Debugging what was and was not fetched
<code>--porcelain</code>	Machine-parseable output on stdout	Scripts that parse fetch results
<code>--show-forced-updates</code>	Always check whether any branch was force-updated	Auditing whether remote history was rewritten

Best practices

Fetch before you branch or rebase. Before creating a feature branch or rebasing onto `main`, run `git fetch origin` so your remote-tracking branches reflect the current upstream state. Branching from a stale `origin/main` means you may miss recent merges and encounter more conflicts later.

Prefer `git fetch` + explicit integration over `git pull`. `git pull` runs fetch and immediately integrates. Fetching first gives you a moment to look at what arrived — `git log HEAD..origin/main --oneline` — before deciding whether to merge or rebase. See the *pull* chapter for when that convenience is justified.

Enable pruning by default. Upstream branches that have been deleted linger as stale remote-tracking refs unless you prune them. Add this to your global config once:

```
git config --global fetch.prune true
```

After that, every `git fetch` automatically removes stale refs, and `git branch -r` stays clean.

Fetch all remotes on build servers. On a developer laptop with a single `origin`, plain `git fetch` is enough. On CI machines or multi-remote workflows, use `git fetch --all` or set `fetch.all = true` in config to keep every remote current without thinking about it.

Use `--dry-run` when experimenting with refspecs. Custom refspec mappings (for example `+refs/pull/*/head:refs/remotes/origin/pr/*`) can have surprising side effects. Verify with `--dry-run` before running for real.

Pitfalls & gotchas

`git fetch` does not update your working tree. New commits on `origin/main` do not appear in your `main` until you integrate them. A common surprise is running `git fetch` and then wondering why `git log` shows nothing new — your branch pointer has not moved. Follow up with `git merge origin/main` or `git rebase origin/main` as appropriate.

Stale remote-tracking branches accumulate silently. When a teammate deletes a remote branch, your local `refs/remotes/origin/<branch>` persists until you prune. Without `--prune` (or `fetch.prune = true`), `git branch -r` fills with dead refs over time. On repos with heavy branch churn this can degrade performance of ref-scanning operations.

--prune-tags removes local tags not on the remote — including ones you created yourself. If you use local tags as personal bookmarks and never push them, they will vanish. Use **--prune-tags** only when your local tags are intended to mirror the remote exactly.

Shallow clones give incomplete results for history-dependent commands. A clone made with `git clone --depth=1` fetches only a slice of history. Commands like `git bisect`, `git blame` on old commits, and `git log --follow` across renames silently return incomplete results. Run `git fetch --unshallow` when you need full history.

FETCH_HEAD is overwritten on every fetch. The file `.git/FETCH_HEAD` records what the most recent fetch retrieved. A subsequent `git fetch` replaces it entirely. Rely on named remote-tracking branches (`origin/main`) rather than `FETCH_HEAD` in any script that outlives a single command.

Force-fetching with an explicit <src>:<dst> refspec overwrites the local branch. Using `git fetch origin main:main --force` overwrites your local `main` unconditionally, discarding any unpushed commits. Reserve this pattern for read-only mirror setups.

Worked examples

Reviewing a colleague's branch before merging

A pull request for `feature/checkout` has been opened upstream. You want to inspect it locally without switching branches.

```
# Bring remote-tracking branches up to date
git fetch origin

# See what commits the branch adds over main
git log origin/main...origin/feature/checkout --oneline
```

```
e4a91bc Add address validation to checkout form
8f30c17 Extract PaymentService into its own module
```

```
# Diff the branch against main
git diff origin/main...origin/feature/checkout
```

Nothing in your working tree or branches changed. When you are ready to integrate, merge or check out the remote-tracking branch:

```
git checkout -b feature/checkout origin/feature/checkout
```

Cleaning up after a team sprint

Your team deleted several merged branches on the remote. Your local remote-tracking refs still list them.

```
git branch -r
```

```
origin/HEAD -> origin/main
origin/feature/auth
origin/feature/search
origin/main
```

```
origin/release/v2.1
origin/release/v2.2
```

`feature/auth` and `feature/search` were merged and deleted upstream. Fetch with `--prune`:

```
git fetch --prune
```

```
From github.com:acme/storefront
- [deleted]          (none)    -> origin/feature/auth
- [deleted]          (none)    -> origin/feature/search
abc1234..def5678  main       -> origin/main
```

```
git branch -r
```

```
origin/HEAD -> origin/main
origin/main
origin/release/v2.1
origin/release/v2.2
```

The stale refs are gone and `origin/main` reflects the latest upstream commit.

Fetching GitHub pull-request refs for local testing

GitHub exposes pull requests under `refs/pull/<number>/head`. These are not fetched by the default refspec. Fetch PR #42 into a local tracking branch:

```
git fetch origin refs/pull/42/head:refs/remotes/origin/pr/42
git checkout origin/pr/42
```

To keep all open PRs current automatically, add a permanent refspec to your remote configuration:

```
git config --add remote.origin.fetch '+refs/pull/*/head:refs/remotes/origin/pr/*'
git fetch origin
```

Now every `git fetch origin` updates all PR branches under `refs/remotes/origin/pr/`.

Recovery

`git fetch` is non-destructive with respect to your branches — it never modifies commits you own. There is nothing to undo in the traditional sense.

If `--prune` removed a remote-tracking ref you still needed (because the upstream branch was deleted), the ref pointer is gone but the underlying commits remain in the object database until garbage collection. Locate the commit hash with `git reflog` or `git fsck --unreachable`, then recreate a branch from it:

```
git branch recover-work <commit-hash>
```

If you fetched with an explicit `<src>:<dst>` refspec and accidentally overwrote a local branch, the overwritten commit is still reachable via the reflog:

```
git reflog show main
# find the entry just before the overwrite
git branch recover-main <sha-before-overwrite>
```

See *Getting out of jams* for broader undo recipes.

See also

- *remote* — managing the remote entries that `git fetch` reads from.
- *pull* — fetch followed by an immediate merge or rebase.
- *merge* — integrating fetched remote-tracking branches into your own.
- *rebase* — replaying your commits on top of freshly fetched upstream work.
- *log* — inspecting what arrived after a fetch with range syntax (`HEAD..origin/main`).
- *Getting out of jams* — recovering from unexpected ref changes.

pull

Fetch changes from a remote repository and integrate them into the current branch in one step.

Mental model

`git pull` is a convenience command that runs two operations back to back:

1. `git fetch` — downloads new commits and updates the remote-tracking branches (e.g. `origin/main`) without touching your working tree.
2. Either `git merge` or `git rebase` — integrates those downloaded commits into your current branch.

```
Remote: A---B---C          (origin/main)

Local:  A---B---D          (main, one local commit)

After git pull (merge mode):
      A---B---C
        \
      A---B---D---E      (E is the merge commit)

After git pull --rebase:
      A---B---C---D'     (D replayed on top of C)
```

The remote-tracking branch (`origin/main`) is always updated first to reflect what the server has. The integration step — merge or rebase — then happens locally. Understanding that distinction matters when things go wrong: a network failure aborts at step one; a conflict surfaces at step two.

By default Git uses merge. If you prefer rebase, set `pull.rebase = true` in your config, or pass `--rebase` per-invocation.

Synopsis

```
git pull [<options>] [<repository> [<refspec>...]]
```

Common forms:

```
git pull
git pull origin main
git pull --rebase
git pull --rebase origin main
git pull --ff-only
```

Everyday usage

Update the current branch from its configured upstream (the common case):

```
git pull
```

Pull from a specific remote and branch when no upstream is configured:

```
git pull origin main
```

Pull and rebase instead of merge — keeps history linear:

```
git pull --rebase
```

Pull only if the result would be a fast-forward (no divergence allowed):

```
git pull --ff-only
```

Stash local changes automatically before pulling, then re-apply them:

```
git pull --autostash
```

Key options

Option	What it does	When to use it
<code>--rebase[=(false true merges interactive)]</code>	Rebase the current branch on top of the upstream instead of merging	Keeping a linear history; avoiding noisy merge commits
<code>--no-rebase</code>	Force merge even when <code>pull.rebase</code> is configured to <code>true</code>	Explicitly merging when your default is rebase
<code>--ff-only</code>	Abort if the update cannot be a fast-forward	CI pipelines, automation, or when you never want a local merge commit
<code>--no-ff</code>	Always create a merge commit even when fast-forward is possible	Preserving a visible record of every integration
<code>--autostash</code>	Stash dirty working tree before pulling, re-apply after	Pulling quickly without committing or stashing manually
<code>--squash</code>	Collapse all incoming commits into a single staged diff without committing	Squash-merging a remote branch into your working tree
<code>--no-commit</code>	Perform the merge but stop before creating the commit	Inspecting or editing the merge result before it becomes permanent
<code>-s <strategy></code>	Use a specific merge strategy (<code>ort</code> , <code>octopus</code> , <code>ours</code> , etc.)	Unusual merge topologies or intentionally discarding one side
<code>-X <option></code>	Pass a strategy-specific option (e.g. <code>-X ours</code> , <code>-X theirs</code>)	Resolving conflicts by always preferring one side
<code>--allow-unrelated-histories</code>	Merge two repos that share no common ancestor	Combining two independent Git projects for the first time
<code>--depth=<n></code>	Limit the fetch to <code><n></code> commits from the tip	Shallow clones; reducing download size
<code>--recurse-submodules[=(yes on-demand no)]</code>	Also fetch and update submodules	Projects that use Git submodules

<code>--stat</code>	Show a diffstat of what was merged	Reviewing what changed at a glance after pulling
<code>-q, --quiet</code>	Suppress progress and merge output	Scripts and CI
<code>-v, --verbose</code>	Pass <code>--verbose</code> to both fetch and merge	Debugging network or merge issues

Best practices

Prefer `--rebase` for feature branches. A merge pull creates a merge commit with a generic message like “Merge branch ‘main’ into feature/x”. Over the life of a long-lived branch, those commits accumulate and obscure the real work. Rebasing replays your commits on top of the updated upstream, resulting in a clean linear history that is easier to review and bisect.

Set `pull.rebase` in your global config rather than typing `--rebase` every time. This makes the behavior consistent across all your repos:

```
git config --global pull.rebase true
```

If a specific repo needs merge pulls, override locally:

```
git config pull.rebase false
```

Use `--ff-only` in automation. On CI, deployment scripts, or repository mirrors you never want an unexpected merge commit. `--ff-only` makes the command fail loudly if the branches have diverged, forcing a human to decide how to reconcile.

```
git pull --ff-only origin main
```

Pull before you push. Pushing to a remote that has commits you do not have locally fails with a rejection. Running `git pull --rebase` first, resolving any conflicts, and then pushing is the standard loop for shared branches.

Keep your working tree clean before pulling. Git will refuse to pull (into a merge) if uncommitted local changes would be overwritten. Either commit, stash manually, or pass `--autostash` to let Git handle it. Surprises from `--autostash` are rare but possible when the re-applied stash conflicts with the newly merged changes.

Pitfalls & gotchas

The default behavior changed — and it varies by version. Older Git versions (before 2.27) defaulted to merge without warning. Starting with 2.27, Git warns on the first `git pull` in a new clone when `pull.rebase` is not set. Configure it explicitly rather than relying on any particular default.

`--rebase` rewrites history. Rebasing replays your local commits as new objects with new SHA-1 hashes. If you have already pushed those commits to a shared remote, rewriting them forces everyone else to reconcile the divergence. Only use `--rebase` on commits that exist solely on your machine, or on a private branch.

Fast-forward is not always possible. If your local branch and the remote branch have both advanced since they last shared a commit, no fast-forward exists. Running `git pull --ff-only` in that situation aborts with:


```
fatal: Not possible to fast-forward, aborting.
```

This is a signal to investigate: have you accidentally committed to a branch you share with others, or has the remote been force-pushed?

--autostash can surface conflicts on re-apply. Git stashes your dirty working tree, performs the pull, then runs `git stash pop`. If the pulled changes touch the same lines as your stashed work, the pop can conflict. The stash is not lost — Git leaves it in the stash list — but you need to resolve the conflict manually.

Submodule updates require an extra step. A plain `git pull` updates the superproject's pointer to the submodule's commit but does not check out that commit inside the submodule directory. Use:

```
git pull --recurse-submodules
```

Or, after the pull, run `git submodule update --init --recursive`.

Pulling from a remote and specifying a refspec merges that ref immediately. If you run `git pull origin next`, Git fetches `next` and merges it into your current branch. If you run `git pull origin` with no refspec, it fetches all configured refs from `origin` but only merges the tracked one. The distinction trips people up when they want to fetch a branch without merging — use `git fetch` alone for that.

Worked examples

Daily sync on a shared feature branch

You and a colleague both push to `feature/checkout-flow`. You start the morning by syncing:

```
git pull --rebase origin feature/checkout-flow
```

```
From https://github.com/example/shop
* branch                feature/checkout-flow -> FETCH_HEAD
Successfully rebased and updated refs/heads/feature/checkout-flow.
```

Your local commits are replayed on top of your colleague's commits. History stays linear and `git log --oneline` remains readable.

If a conflict arises during the rebase:

```
# Git pauses and shows which file conflicts
git status
# Edit the file to resolve, then stage it:
git add src/checkout.js
git rebase --continue
```

Pulling into a dirty working tree

You are mid-edit on `README.md` and need to pull an urgent fix your teammate just pushed:

```
git pull --autostash --rebase
```

```
Created autostash: stash@{0}
HEAD is now at f3a9c11 Start draft of README section
From https://github.com/example/docs
 * branch                main -> FETCH_HEAD
   f3a9c11..a2d4f90      main -> origin/main
Successfully rebased and updated refs/heads/main.
Applied autostash.
```

Your uncommitted changes are back in the working tree, layered on top of the newly fetched commits.

Pulling an unrelated history when combining two projects

You start a new repo and want to pull in a skeleton from a separate upstream project that was initialized independently:

```
git remote add upstream https://github.com/example/skeleton
git pull upstream main --allow-unrelated-histories
```

Without `--allow-unrelated-histories`, Git refuses because the two repos share no ancestor commit.

Using `--ff-only` in a deploy script

A deployment script that runs on every merge to `main` needs to be certain the branch is always in a known, linear state:

```
#!/bin/sh
git pull --ff-only origin main || {
    echo "ERROR: main has diverged from origin/main. Manual intervention required."
    exit 1
}
```

If the branch has diverged — someone force-pushed, or a commit landed locally by mistake — the script exits non-zero rather than silently creating a merge commit.

Recovery

If a pull triggered a merge or rebase that you want to undo, use `ORIG_HEAD`. Git sets `ORIG_HEAD` before any operation that moves the branch pointer in a potentially disruptive way:

```
# Undo a merge pull — return to the state before git pull ran
git reset --merge ORIG_HEAD
```

If the pull used rebase and you want to abort while it is still in progress:

```
git rebase --abort
```

If you have already completed the pull but realize the result is wrong, the reflog records every position your branch has been at:

```
git reflog
# Find the SHA before the pull, then:
git reset --hard <sha-before-pull>
```

See *Getting out of jams* for a broader treatment of undoing merges and recovering from rebase disasters.

See also

- *fetch* — the first half of what `pull` does; use it when you want to inspect remote changes before integrating.
- *merge* — how Git integrates divergent histories; the second half of a merge-mode pull.
- *rebase* — replaying commits onto a new base; the second half of `git pull --rebase`.
- *stash* — manually saving and restoring a dirty working tree before pulling.
- *reflog* — recovering commits after a rebase or hard reset goes wrong.
- *Getting out of jams* — undoing pulls, merges, and rebases safely.

push

Upload local branch commits to a remote repository, advancing its branch pointers to match yours.

Mental model

Every repository is a self-contained database of objects and refs. When you commit locally, only your copy changes. `git push` is the act of transmitting those objects across the network and updating the remote's branch pointer to match your local one.

Git enforces one constraint by default: the update must be a *fast-forward*. A fast-forward means the remote's current tip is an ancestor of the commit you are pushing — in other words, you are only adding new commits on top of what is already there, never discarding history.

Before push:

```
local:   A---B---C  (feature)
remote:  A---B      (origin/feature)
```

After git push:

```
remote:  A---B---C  (origin/feature)
```

If someone else has pushed in the meantime, the remote's tip is no longer an ancestor of your local tip, and Git refuses the push to protect their work. The solution is to integrate their changes first (see the *fetch* and *pull* chapters) and then push again.

Pushing also sets a *tracking relationship*: once a branch has an upstream, plain `git push` (with no arguments) knows where to send commits.

Synopsis

```
git push [--all | --branches | --mirror | --tags] [--follow-tags]
        [--atomic] [-n | --dry-run]
        [-f | --force]
        [--force-with-lease[=<refname>[:<expect>]]] [--force-if-includes]]
        [-d | --delete] [--prune]
        [-u | --set-upstream]
        [-o <string> | --push-option=<string>]
        [--[no-]signed | --signed=(true|false|if-asked)]
        [--no-verify]
        [-q | --quiet] [-v | --verbose]
        [--repo=<repository>]
        [<repository> [<refspec>...]]
```

Everyday usage

Push the current branch to its configured upstream (the most common invocation after the upstream is set up):

```
git push
```

Push a branch for the first time and set the upstream tracking reference so future pushes and pulls need no arguments:

```
git push -u origin feature/login
```

Push to a specific remote and branch by name:

```
git push origin main
```

Push the current branch to the identically-named branch on origin without having to type the branch name:

```
git push origin HEAD
```

Delete a remote branch you no longer need:

```
git push origin --delete feature/old-experiment
# shorthand with the empty-src refspec:
git push origin :feature/old-experiment
```

Push all local tags to the remote:

```
git push --tags
```

Push annotated tags that point at commits you are pushing (usually the right choice over `--tags`):

```
git push --follow-tags
```

Preview what would be pushed without actually sending anything:

```
git push --dry-run
```

Key options

Option	What it does	When to use it
<code>-u</code> , <code>--set-upstream</code>	Set the upstream tracking reference for the pushed branch	First push of a new branch
<code>-f</code> , <code>--force</code>	Overwrite the remote ref even if not a fast-forward	After a rebase or amend on a private branch
<code>--force-with-lease</code>	Force only if the remote ref hasn't changed since your last fetch	Safer alternative to <code>--force</code> on shared branches
<code>--force-if-includes</code>	Verify remote-tracking ref changes are locally integrated; use alongside <code>--force-with-lease</code>	Extra safety when background fetches may silently refresh remote-tracking refs
<code>-n</code> , <code>--dry-run</code>	Do everything except actually send objects	Verify what would be pushed before committing
<code>--all</code> , <code>--branches</code>	Push all branches under <code>refs/heads/</code>	Sync a full local mirror to a remote

<code>--tags</code>	Push all tags under <code>refs/tags/</code>	Publish all tags alongside commits
<code>--follow-tags</code>	Push only annotated tags reachable from the pushed commits	Standard release tagging workflow
<code>-d</code> , <code>--delete</code>	Delete the listed refs from the remote	Clean up merged remote branches
<code>--prune</code>	Remove remote branches that have no local counterpart	Keep the remote in sync after local branch deletions
<code>--atomic</code>	Succeed or fail all ref updates as a unit	Multiple refs that must land together or not at all
<code>-o <str></code> , <code>--push-option=<str></code>	Pass an arbitrary string to remote pre/post-receive hooks	Custom CI signals (e.g. <code>ci.skip</code>)
<code>--no-verify</code>	Skip the pre-push hook	Emergency bypass — fix the hook issue afterward
<code>--mirror</code>	Push all refs including remotes; force-update or delete on the remote to match locally	Migrating a repository to a new hosting location
<code>--signed=(true false if-asked)</code>	GPG-sign the push request	Servers that validate signed pushes via hooks
<code>-q</code> , <code>--quiet</code>	Suppress output except errors	Scripts and CI pipelines
<code>-v</code> , <code>--verbose</code>	Show updated refs and additional detail	Diagnosing unexpected push behaviour

Best practices

Set the upstream once, then push without arguments. Running

`git push -u origin feature/auth` the first time records `origin/feature/auth` as the upstream. Every subsequent `git push` on that branch sends to the right place without any arguments. Configure `push.autoSetupRemote = true` globally to have Git do this automatically on every first push.

Prefer `--force-with-lease` over `--force`. When you rebase a branch and need to overwrite the remote, `--force` is a blunt instrument — it pushes regardless of what anyone else may have added. `--force-with-lease` checks that the remote ref still matches your remote-tracking ref and refuses if someone else has pushed in the meantime. This catches the most common force-push foot-gun with almost no extra effort.

Use `--follow-tags` for releases instead of `--tags`. `--tags` pushes every tag in your repository, including old, incomplete, or local-only tags. `--follow-tags` pushes only annotated tags reachable from the commits being pushed — exactly what a release workflow needs.

Delete merged remote branches promptly. Stale remote branches accumulate and confuse teammates. After a branch is merged via a pull request, delete it:

```
git push origin --delete feature/done
```

Protect shared branches at the server level. Never rely solely on client behaviour. Configure branch protection rules on GitHub or GitLab so that force-pushes to `main` and `release/*` are server-rejected regardless of what any client sends.

Validate with `--dry-run` before unusual pushes. Before a push involving multiple refsspecs, `--delete`, or `--force`, add `-n` to see exactly which refs would change without touching anything.

Pitfalls & gotchas

“Rejected — non-fast-forward” means someone else pushed first. Fetch and integrate their work, then push again. Never reach for `--force` as the first response to a rejection on a shared branch.

`--force` applies to all refsspecs in the command. When `push.default` is `matching`, or when you list multiple refsspecs, `--force` applies to all of them — not just the one you had in mind. Use the `+` prefix on a specific refspect to force only that one ref:

```
git push origin +feature/rebased
```

Force-pushing rewrites public history. Any collaborator who has already fetched your branch will have a diverged copy after you force-push. They must run `git fetch` and then reset or rebase on top of your new history. Agree with your team before force-pushing anything someone else might have checked out.

`--force-with-lease` can be defeated by background fetches. If your editor or another tool runs `git fetch` automatically, the remote-tracking ref gets updated even though you have not explicitly reviewed the new commits. `--force-with-lease` then sees the remote as matching your tracking ref and permits the push — which may silently overwrite a collaborator’s commit. Add `--force-if-includes` to require that any newly fetched commits appear in your local reflog before the force push proceeds.

Pushing tags is not automatic. `git push` never sends tags unless you add `--tags` or `--follow-tags`. A newly created tag is invisible to collaborators until you push it explicitly.

The pre-push hook can abort a push. The hook receives the remote name, URL, and list of refs being updated. If it exits non-zero the push is aborted — this is intentional. Use `--no-verify` only as a last resort and fix the underlying hook issue as soon as possible.

Deleting a remote branch does not delete the local branch. After `git push origin --delete feature/done`, your local `feature/done` still exists. Delete it separately with `git branch -d feature/done`.

Worked examples

Pushing a new feature branch

You have finished work on a local branch and want to share it for review.

```
# See what you are about to push
git log --oneline origin/main..HEAD
```

```
3f2a1b8 Add rate limiting to login endpoint
c9d04e1 Extract auth middleware into its own file
```

```
# Push and set the upstream in one step
git push -u origin feature/rate-limit
```

```
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 10 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (7/7), 1.23 KiB | 1.23 MiB/s, done.
Total 7 (delta 4), reused 0 (delta 0), pack-reused 0
To github.com:acme/api.git
 * [new branch]      feature/rate-limit -> feature/rate-limit
Branch 'feature/rate-limit' set up to track remote branch 'feature/rate-limit'
from 'origin'.
```

Future pushes on this branch require only `git push`.

Safely pushing a rebased branch

A reviewer asked for changes. You rewrote commits with an interactive rebase (see the *rebase* chapter). The remote branch now diverges from your local one, so a plain push is rejected.

```
git rebase -i origin/main      # rewrite commits
git push --force-with-lease    # push only if nobody else has pushed
```

If a collaborator pushed to the same branch in the meantime, Git refuses:

```
error: failed to push some refs to 'github.com:acme/api.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally.
```

Fetch their work, reconcile, and try again:

```
git fetch origin
git rebase origin/feature/rate-limit
git push --force-with-lease
```

Publishing a release with a tag

```
# Create a signed, annotated tag
git tag -a v2.1.0 -m "Release 2.1.0 – add rate limiting"

# Push commits and the new tag together
git push --follow-tags
```

`--follow-tags` sends the commits on the current branch plus any annotated tags pointing at reachable commits. Lightweight bookmarking tags stay behind.

Cleaning up a merged branch

```
# Confirm the branch is fully merged
git branch -r --merged origin/main | grep feature/done
```



```
origin/feature/done
```

```
# Delete on the remote and locally
git push origin --delete feature/done
git branch -d feature/done
```

Recovery

If you pushed commits you did not intend to share, you have two options.

Safe option — add a revert commit. Never rewrites history, always safe on a shared branch. Use the *revert* chapter for the mechanics:

```
git revert <bad-commit>
git push
```

Destructive option — force-push to remove the commit. Only appropriate if you are certain nobody has fetched the bad commit yet:

```
git reset --hard HEAD~1      # remove the commit locally
git push --force-with-lease   # overwrite the remote
```

If collaborators already have the bad commit, they must run `git fetch` and realign their local branches to the new remote state — coordinate with them before doing this.

See *Getting out of jams* for recipes covering diverged branches and recovering from an accidental force-push.

See also

- *fetch* — download remote changes without merging; run before pushing when the remote may have advanced.
- *pull* — fetch and integrate in one step.
- *remote* — manage remote names and URLs.
- *branch* — list, create, and delete branches before pushing.
- *rebase* — rewrite local history cleanly before pushing to a shared branch.
- *tag* — create the annotated tags that `--follow-tags` will push.
- *Getting out of jams* — recovering from rejected or accidentally overwritten pushes.

submodule

Embed one Git repository inside another as a tracked dependency pinned to a specific commit.

Mental model

A submodule is a pointer, not a copy. When you add a submodule, Git records two things:

1. **.gitmodules** — a plain-text file at the superproject root that maps each submodule path to its remote URL (and optionally a tracking branch).
2. **A gitlink** — a special index entry for the submodule path that stores a single SHA-1: the exact commit the superproject expects that submodule to be at.

The submodule directory itself is a fully independent Git repository. It has its own `.git` directory (or a `.git` file pointing into the superproject's `.git/modules/` store), its own history, and its own remotes. The superproject does not track the submodule's files — only that one commit SHA-1.

```
superproject/
.gitmodules      ← URL + path registry
.git/modules/ui/ ← submodule's embedded git dir
ui/              ← submodule working tree (detached HEAD)
  .git           ← file: pointer to ../.git/modules/ui
  src/
  ...
```

When a teammate clones your superproject they get the `.gitmodules` file and the gitlink, but not the submodule's files — they must run `git submodule update --init` to populate the working tree.

The most common confusion: **updating a submodule to a newer commit does not happen automatically**. You must explicitly pull inside the submodule (or use `update --remote`), then stage and commit the updated gitlink in the superproject. Think of it as bumping a version pin.

Synopsis

```
git submodule [--quiet] [--cached]
git submodule [--quiet] add [-b <branch>] [-f] [--name <name>]
                        [--depth <depth>] [--] <repository> [<path>]
git submodule [--quiet] status [--cached] [--recursive] [--] [<path>...]
git submodule [--quiet] init [--] [<path>...]
git submodule [--quiet] deinit [-f|--force] [--all|[--] <path>...]
git submodule [--quiet] update [--init] [--remote] [-N|--no-fetch]
                        [-f|--force] [--checkout|--rebase|--merge]
                        [--depth <depth>] [--recursive] [-j <n>] [--] [<path>...]
git submodule [--quiet] set-branch (-b <branch>|-d) [--] <path>
git submodule [--quiet] set-url [--] <path> <newurl>
git submodule [--quiet] summary [--cached|--files] [-n <n>] [--] [<path>...]
git submodule [--quiet] foreach [--recursive] <command>
git submodule [--quiet] sync [--recursive] [--] [<path>...]
git submodule [--quiet] absorbgitdirs [--] [<path>...]
```

Everyday usage

Add a submodule to the superproject:

```
git submodule add https://github.com/example/ui-lib.git vendor/ui
git commit -m "Add ui-lib as a submodule at vendor/ui"
```

Clone a repository that already has submodules — populate everything in one step:

```
git clone --recurse-submodules https://github.com/example/app.git
```

If you forgot `--recurse-submodules` at clone time, fix it afterwards:

```
git submodule update --init --recursive
```

Check the state of all submodules. The prefix characters tell you the story: `-` means not initialized, `+` means the checked-out commit differs from the superproject's recorded SHA-1, `U` means merge conflict.

```
git submodule status
```

```
a3f9cld vendor/ui (v2.1.0)
+7b20e44 vendor/proto (heads/main-3-g7b20e44)
-0000000 vendor/icons (not initialized)
```

Update submodules to the commits the superproject records:

```
git submodule update --init --recursive
```

Pull the latest upstream commit of a submodule and record it in the superproject:

```
cd vendor/ui
git fetch
git checkout main
git pull
cd ../../
git add vendor/ui
git commit -m "Bump ui-lib to latest main"
```

Or use `update --remote` without entering the directory:

```
git submodule update --remote vendor/ui
git add vendor/ui
git commit -m "Bump ui-lib to latest upstream HEAD"
```

Run a command in every submodule (the `$sm_path` variable holds each submodule's path relative to the superproject):

```
git submodule foreach 'git fetch && echo "fetched $sm_path"'
```

Key options

Option	Applies to	What it does	When to use it
--------	------------	--------------	----------------

<code>add <repo> [<path>]</code>	—	Register a remote repo as a submodule at <path>	Adding a new dependency
<code>update --init</code>	<code>update</code>	Initialize uninitialized submodules before updating	After a fresh clone or pull
<code>update --recursive</code>	<code>update</code> , <code>status</code> , <code>foreach</code> , <code>sync</code>	Descend into nested submodules	Repos with multi-level nesting
<code>update --remote</code>	<code>update</code>	Fetch and check out the submodule's upstream tracking branch instead of the pinned SHA-1	Pulling latest upstream into a submodule
<code>update --merge</code>	<code>update</code>	Merge the superproject's recorded commit into the submodule's current branch	Keep submodule on a real branch
<code>update --rebase</code>	<code>update</code>	Rebase the submodule's current branch onto the recorded commit	Keep submodule on a real branch
<code>-j <n> / --jobs <n></code>	<code>update</code>	Clone or fetch submodules in parallel	Many submodules or slow network
<code>--depth <n></code>	<code>add</code> , <code>update</code>	Shallow-clone with history truncated to n commits	Large submodule repos, CI
<code>deinit <path></code>	—	Remove a submodule's working tree and local config entry (does not remove the gitlink)	Stopping work on a submodule temporarily
<code>deinit --all</code>	<code>deinit</code>	Deinit every registered submodule	Cleaning up the entire workspace
<code>-f / --force</code>	<code>add</code> , <code>deinit</code> , <code>update</code>	<code>add</code> : allow adding an otherwise-ignored path; <code>deinit</code> / <code>update</code> : proceed even when local modifications exist	Recovery; use with care
<code>foreach <cmd></code>	—	Run a shell command in each checked-out submodule	Bulk operations: fetch, status, branch
<code>foreach --recursive</code>	<code>foreach</code>	Also descend into nested submodules	Deep nesting

<code>sync</code>	—	Update <code>.git/config</code> remote URLs from <code>.gitmodules</code>	After an upstream URL change
<code>set-url <path> <url></code>	—	Change the URL of a submodule and sync local config	Migrating a submodule to a new host
<code>set-branch -b <branch></code>	—	Set the remote tracking branch stored in <code>.gitmodules</code>	Control which branch <code>update --remote</code> follows
<code>set-branch -d</code>	—	Remove the tracking-branch setting, reverting to remote HEAD	Reset to default upstream branch
<code>status --cached</code>	<code>status</code>	Show the SHA-1 stored in the index rather than in HEAD	See what is staged for commit
<code>summary</code>	—	Show commits in submodules between superproject HEAD and the index	Quick diff of what changed
<code>absorbgitdirs</code>	—	Move submodule <code>.git</code> dirs into the superproject's <code>.git/modules/</code> store	Housekeeping after a manual clone-and-add
<code>--quiet</code>	all	Only print error messages	Scripts and CI pipelines

Best practices

Always commit `.gitmodules` and the gitlink together. When you add or bump a submodule, stage both the `.gitmodules` file and the submodule path in the same commit. Leaving one staged without the other produces an inconsistent state for every collaborator who pulls.

Pin submodules to stable refs, not moving branches. The superproject stores a commit SHA-1, not a branch name. Pointing a submodule at a branch with `--branch` only affects `update --remote` collaborators who run a plain `update` still get the recorded SHA-1. Make deliberate bump commits rather than silently drifting with `--remote` in CI.

Use `--recurse-submodules` everywhere. Add it to your `clone` invocations, and set it globally so `pull`, `checkout`, and `switch` automatically keep submodules in sync:

```
git config --global submodule.recurse true
```

Without this you constantly run `git submodule update` after every pull, and occasionally forget.

Parallelize on large trees. When a project has many submodules, use `-j` to clone or fetch them in parallel. Set the default once:

```
git config --global submodule.fetchJobs 8
```

Then any `git submodule update` will use eight parallel jobs automatically.

Shallow-clone large submodule histories in CI. If a submodule has a long history you do not need, `--depth 1` cuts clone time dramatically:

```
git submodule update --init --depth 1
```

Keep submodule development on a branch. By default `update --checkout` leaves the submodule in detached HEAD state. Any commits you make there are easily lost. Before hacking on a submodule, check out a branch:

```
cd vendor/ui
git checkout -b feature/my-change origin/main
```

Now your commits belong to a branch and cannot be silently overwritten by a future `submodule update`.

Pitfalls & gotchas

Empty directory after clone. Pulling a commit that adds a new submodule leaves an empty directory in your working tree. The fix is always `git submodule update --init --recursive`. Without it the directory exists but contains nothing, and any build referencing it will fail silently.

Detached HEAD inside the submodule. A plain `git submodule update` checks out the pinned commit on a detached HEAD. Any commits you make in that state are dangling — no branch points at them. After switching the superproject away and back, `update` will silently replace your work. Always `git checkout -b <branch>` inside the submodule before making changes.

deinit does not remove the gitlink. Running `git submodule deinit vendor/ui` removes the working tree and the `.git/config` entry, but the path still exists as a gitlink in the index and in `.gitmodules`. To fully remove a submodule from the project, you must also run `git rm vendor/ui`. See the Worked examples section for the complete sequence.

URL changes are not automatic. If an upstream repository moves to a new URL, collaborators must run `git submodule sync` after you commit the updated `.gitmodules` otherwise their local `.git/config` still points at the old address and fetches fail.

update --remote without staging creates drift. Running `git submodule update --remote` updates the submodule’s checked-out commit but does not stage the gitlink in the superproject. If you stop there, the superproject still records the old SHA-1. Always follow with `git add <path>` and a commit.

Shallow history breaks older commit checkouts. If a submodule was cloned with `--depth 1`, attempting to check out an older commit recorded in the superproject fails with “object not found”. Deepen the clone first:

```
cd vendor/ui
git fetch --unshallow
```

Nested submodules multiply complexity. Every submodule can have its own submodules. Without `--recursive` you only process one level. Use `--recursive` consistently or nested submodules silently fall out of sync.

Worked examples

Setting up a project that uses libraries as submodules

You are building an application that depends on two libraries in separate repositories.

```
git init app && cd app

# Add the first library
git submodule add https://github.com/example/core.git vendor/core

# Add a second library, tracking the stable-2 branch for update --remote
git submodule add -b stable-2 https://github.com/example/utils.git vendor/utils

git status
```

On branch main

```
Changes to be committed:
  new file:   .gitmodules
  new file:   vendor/core
  new file:   vendor/utils
```

```
git commit -m "Add core and utils as submodules"
```

A collaborator who clones the project populates the submodules with:

```
git clone --recurse-submodules https://github.com/example/app.git
```

Or, after a plain clone:

```
git submodule update --init --recursive
```

Bumping a submodule to a newer release

The `vendor/core` library has published `v3.0.0` and you want the superproject to track it.

```
cd vendor/core
git fetch --tags
git checkout v3.0.0
cd ../..

# The superproject sees vendor/core as modified (+ prefix means the
# checked-out commit no longer matches the recorded SHA-1)
git submodule status
```

```
+e8a712f vendor/core (v3.0.0)
c901234 vendor/utils (stable-2)
```

```
git add vendor/core
git commit -m "Bump core to v3.0.0"
```

Teammates get the updated pin by running:

```
git pull
git submodule update --recursive
```

Fully removing a submodule

You have decided to vendor the `vendor/Utils` source directly and no longer need it as a submodule.

```
# 1. Remove the working tree and the .git/config entry
git submodule deinit vendor/Utils

# 2. Remove the gitlink from the index and delete the directory;
#    also removes the [submodule "vendor/Utils"] block from .gitmodules
git rm vendor/Utils

# 3. Remove the leftover modules store entry
rm -rf .git/modules/vendor/Utils

git commit -m "Remove Utils submodule; vendor source directly"
```

Skip any of those three steps and you leave behind orphaned data that confuses future `submodule update` runs.

Running bulk operations with foreach

Print the current branch of every submodule:

```
git submodule foreach 'echo "$sm_path: $(git branch --show-current)'"
```

Pull the latest changes in every submodule, continuing even if one fails:

```
git submodule foreach 'git pull --ff-only || :'
```

Run the same command recursively through all nested submodules:

```
git submodule foreach --recursive 'git fetch origin'
```

Fixing a URL change after a repository migration

The `vendor/core` library moved from GitHub to an internal GitLab instance.

```
git submodule set-url vendor/core https://gitlab.internal/core.git

# Propagate the new URL to .git/config (local clone config)
git submodule sync vendor/core

git add .gitmodules
git commit -m "Migrate core submodule URL to internal GitLab"
```

Every teammate pulls and runs:

```
git submodule sync          # update their .git/config from .gitmodules
git submodule update        # re-fetch if needed
```


Recovery

Submodule directory is empty after clone or pull:

```
git submodule update --init --recursive
```

Accidentally made commits on detached HEAD inside a submodule. You ran `git submodule update` and your commits disappeared:

```
cd vendor/ui
git reflog                      # find the last commit you made
git branch rescue-work abc1234  # create a branch at that commit
git checkout rescue-work
```

Then cherry-pick or merge `rescue-work` into your intended branch, and record the updated gitlink in the superproject.

update fails with “reference is not a tree”. The superproject records a commit the submodule does not have locally, which is common with shallow clones or after the submodule’s remote changed:

```
cd vendor/ui
git fetch --unshallow  # if shallow; otherwise a plain fetch is enough
git fetch origin
cd ../..
git submodule update
```

Wrong commit after a merge conflict on the gitlink. A merge brought in a gitlink pointing at a different commit. See what each side wanted:

```
git diff HEAD MERGE_HEAD -- vendor/ui
```

Check out the commit you want, stage the path, and complete the merge commit.

See *Getting out of jams* for broader undo recipes.

See also

- *clone* — `--recurse-submodules` for initial population; `--depth` and `--shallow-submodules` for partial histories.
- *add* — staging the gitlink after updating a submodule’s checked-out commit.
- *commit* — recording the bumped submodule pointer in the superproject.
- *diff* — `--submodule=log` shows which commits changed inside a submodule between two superproject revisions.
- *Getting out of jams* — recovering detached-HEAD commits and other submodule mishaps.

worktree

Check out multiple branches simultaneously into separate directories, all backed by a single repository clone.

Mental model

A normal clone gives you one working tree: one directory, one `HEAD`, one index. `git worktree` lets you attach additional working trees to that same repository. Each linked worktree has its own `HEAD`, its own index, and its own in-progress changes — but they all share the same object database and the same branch namespace.

```
~/.git/           ← single object database, all branches
  worktrees/
    hotfix/        ← per-worktree HEAD + index
    experiment/    ← per-worktree HEAD + index

~/project/        ← main worktree (branch: main)
~/project-hotfix/  ← linked worktree (branch: hotfix)
~/project-experiment/ ← linked worktree (detached HEAD)
```

Think of linked worktrees as cheap, instant checkouts that do not require a second clone. Switching between them is as fast as `cd`, and each one keeps its own uncommitted work without any stashing.

The repository enforces one rule: a branch can only be checked out in one worktree at a time. If `hotfix` is already open in another worktree, a second `add` of the same branch is rejected — unless you use `--force`.

Synopsis

```
git worktree add [-f] [--detach] [--checkout] [--lock [--reason <string>]]
                  [--orphan] [(--b | -B) <new-branch>] <path> [<commit-ish>]
git worktree list [-v | --porcelain [-z]]
git worktree lock [--reason <string>] <worktree>
git worktree move <worktree> <new-path>
git worktree prune [-n] [-v] [--expire <expire>]
git worktree remove [-f] <worktree>
git worktree repair [<path>...]
git worktree unlock <worktree>
```

Everyday usage

Create a linked worktree for a new branch based on `HEAD`:

```
git worktree add ../hotfix
# Creates branch "hotfix" from HEAD and checks it out at ../hotfix
```

Create a worktree for an existing branch:

```
git worktree add ../review origin/feature/payments
```

Create a worktree with an explicit new branch name:

```
git worktree add -b fix/login-timeout ../login-fix main
```

List all worktrees, including branch and commit:

```
git worktree list
```

```
/home/alice/project      abc1234 [main]
/home/alice/hotfix       def5678 [hotfix]
/home/alice/login-fix    ghi9012 [fix/login-timeout]
```

Remove a worktree once the work is done (the branch is not deleted):

```
git worktree remove ../hotfix
```

Clean up stale administrative records for worktrees deleted without `remove` :

```
git worktree prune
```

Key options

Option	What it does	When to use it
<code>-b <branch></code>	Create <code><branch></code> from <code><commit-ish></code> and check it out	Starting fresh topic work in its own directory
<code>-B <branch></code>	Like <code>-b</code> but resets the branch if it already exists	Reuse a branch name after it was already created
<code>-d</code> / <code>--detach</code>	Check out in detached HEAD mode	Throwaway investigation; no branch needed
<code>-f</code> / <code>--force</code>	Override the “branch already checked out” safety check	Rare: open the same branch in two places simultaneously
<code>--lock</code>	Lock the worktree immediately after <code>add</code>	Worktrees on removable drives or network shares
<code>--reason <string></code>	Record why the worktree is locked (with <code>lock</code> or <code>add --lock</code>)	Document context for teammates or future self
<code>--no-checkout</code>	Create the worktree without populating the working tree	Preparing a sparse checkout before files land
<code>--orphan</code>	Create the worktree with an empty index and an unborn branch	Starting a new branch with no history
<code>--guess-remote</code>	Base the new branch on a matching remote-tracking branch	Quickly mirror remote branches locally
<code>-n</code> / <code>--dry-run</code>	With <code>prune</code> : show what would be removed without removing it	Safely preview stale-record cleanup
<code>-v</code> / <code>--verbose</code>	With <code>list</code> : show more detail; with <code>prune</code> : report removals	Diagnosing which worktrees are known to git
<code>--porcelain</code>	With <code>list</code> : machine-readable output stable across git versions	Scripting and tooling
<code>--expire <time></code>	With <code>prune</code> : only remove records older than <code><time></code>	Conservative cleanup

Best practices

Keep linked worktrees beside, not inside, the main worktree. Place them as siblings in the parent directory (e.g. `../hotfix`) rather than as subdirectories. Git will not add sibling directories to the main worktree's index, whereas nested paths can confuse tools that scan the working tree recursively.

Name the path after the branch. When `git worktree add ../hotfix` is run without an explicit branch name, git creates a branch called `hotfix`. Keeping the directory name and branch name in sync makes `git worktree list` output immediately readable and avoids confusion months later.

Use worktrees instead of stash for context-switching. If you are mid-way through feature work and an urgent bug comes in, open a new worktree on `main` rather than stashing. Your feature work stays exactly as you left it; there is no stash to pop and no risk of conflicts when you return. See the *stash* chapter for the cases where stash is still the right tool.

Lock worktrees on removable or network locations. If a worktree lives on a USB drive or a network mount that is not always present, run

`git worktree lock --reason "on external drive" ../worktree-name` to prevent git from pruning its administrative records when the path is temporarily unavailable.

Remove worktrees explicitly. `git worktree remove` cleans up both the directory and the administrative records in `.git/worktrees/`. Deleting the directory with `rm -rf` leaves stale records behind. `git worktree prune` will eventually clean them up but only after the configured expiry period (`gc.worktreePruneExpire`, default 3 months).

Prefer bare clones as a worktree host for server-side or multi-branch workflows. A bare repo has no main working tree of its own, making every working tree a linked one. Clone with `git clone --bare <url> project.git` and then add worktrees at `project-main/`, `project-staging/`, and so on. This avoids the awkwardness of a branch occupying the “main” slot permanently.

Pitfalls & gotchas

A branch can only be checked out in one worktree at a time. Attempting to add a worktree for a branch already open elsewhere fails with a message like

`fatal: 'hotfix' is already checked out at '/home/alice/hotfix'`. Use `--force` to override, but two worktrees on the same branch will see each other's committed changes immediately — committing in both simultaneously produces a diverged branch.

Deleting the directory does not remove the administrative records. If you

`rm -rf ../hotfix` instead of running `git worktree remove ../hotfix`, git still thinks that worktree exists. `git worktree list` shows it as missing or prunable. Run `git worktree prune` to clean up, or `git worktree repair` if the directory was moved rather than deleted.

The main worktree cannot be removed. `git worktree remove` only applies to linked worktrees. The main worktree (created by `git init` or `git clone`) is permanent for the lifetime of that clone. If you need to work without a fixed main working tree, start from a bare clone.

Per-worktree state does not follow you when you `cd`. Each worktree has its own `HEAD`, index, and mid-operation state (`MERGE_HEAD`, `CHERRY_PICK_HEAD`, and so on). Running `git status` in the wrong directory will show a completely different picture than expected. Always verify your working directory with `pwd` or a shell prompt that shows the current path before running git commands.

Refs inside `refs/bisect` and `refs/worktree` are per-worktree, not shared. A bisect session started in one worktree is invisible in another. This is usually desirable, but it can surprise you if you expect `git bisect reset` in the main worktree to clean up a bisect started in a linked one. See the *bisect* chapter for details.

Moving the main worktree manually breaks linked worktrees. Linked worktrees record the path back to `.git` as an absolute path. Moving the main repository directory with `mv` causes every linked worktree to lose its back-reference. Fix this by running `git worktree repair` from inside the moved main worktree's new location.

Worked examples

Handling a hotfix without leaving feature work

You are deep in a feature branch when a P1 bug is reported against `main`.

```
# From inside your feature worktree
git worktree add -b hotfix/payment-crash ../payment-hotfix main
cd ../payment-hotfix
```

Fix the bug, commit, and push:

```
# (edit the file)
git add src/payments.js
git commit -m "Fix null dereference on expired card"
git push origin hotfix/payment-crash
```

Return to your feature work instantly — nothing has changed:

```
cd ../myproject
```

When the hotfix branch is merged and no longer needed, clean up:

```
git worktree remove ../payment-hotfix
git branch -d hotfix/payment-crash
```

Running two versions of the app side by side

You want to compare the behaviour of `main` against an experimental renderer without switching branches back and forth.

```
# main is already checked out in ~/myproject
git worktree add -b experiment/new-renderer ../myproject-experiment main
```

Run both servers simultaneously in separate terminals:

```
# Terminal 1 – current main
cd ~/myproject && npm start -- --port 3000

# Terminal 2 – experimental branch
cd ~/myproject-experiment && npm start -- --port 3001
```

Compare them in the browser at the same time. When the experiment is settled, remove the worktree:

```
git worktree remove ~/myproject-experiment
```

Recovering from a manually moved repository

You moved your main repository from `~/old-name` to `~/new-name` using `mv`, and now `git status` in a linked worktree at `~/hotfix` fails:

```
fatal: not a git repository: /Users/alice/old-name/.git
```

Fix the broken back-reference from inside the new main worktree location:

```
cd ~/new-name
git worktree repair
```

Git rewrites the `gitdir` file in each linked worktree to point at the new location. Verify everything is connected:

```
git worktree list
```

```
/Users/alice/new-name    abc1234 [main]
/Users/alice/hotfix      def5678 [hotfix]
```

Recovery

Remove a worktree and discard any uncommitted changes inside it:

```
git worktree remove --force ../linked-worktree
```

If you deleted a worktree directory manually and want to clean up the stale administrative records:

```
git worktree prune --dry-run  # preview what will be removed
git worktree prune           # remove stale records
```

If a worktree directory was moved manually, re-link it to the repository without losing your work:

```
cd /new/path/to/linked-worktree
git worktree repair
```

If a worktree is locked and you cannot remove or move it, unlock it first:

```
git worktree unlock <worktree>
git worktree remove <worktree>
```

See *Getting out of jams* for strategies when a worktree is left in a mid-merge or mid-rebase state with no straightforward way to abort.

See also

- *branch* — creating and deleting the branches that worktrees check out.
- *checkout* — the single-worktree equivalent for switching branches; also explains detached HEAD mode.

- *stash* — a lighter alternative for context-switching when a full worktree is more than the task requires.
- *rebase* — running an interactive rebase inside a dedicated worktree keeps the main worktree clean during the operation.
- *bisect* — bisect sessions are per-worktree; run a bisect in a linked worktree to avoid interrupting ongoing work.
- *Getting out of jams* — recovering from mid-operation states left in a linked worktree.

apply

Read a diff file and apply it to the working tree and/or the index, without creating a commit.

Mental model

A patch file is a text description of changes: which lines to remove and which to add, in which file, at which location. `git apply` reads that description and executes it — surgically editing your files as instructed.

The key distinction from `git am` is that `apply` is deliberately low-level: it modifies files (and optionally the index), but it never creates a commit. That makes it useful for importing a single diff snippet, testing whether a patch would apply cleanly, or selectively applying parts of a larger patch series.

```
patch file  —git apply—> working tree   (default)
patch file  —git apply --index—> working tree + index
patch file  —git apply --cached—> index only (working tree untouched)
patch file  —git apply --check—> dry run (nothing written)
```

The patch does not have to come from Git. `git apply` accepts standard unified diff output from GNU diff, `diff -u`, or any tool that produces the same format. Outside a Git repository it behaves like a better version of `patch(1)`.

When a hunk cannot apply — because the surrounding context lines no longer match — `git apply` aborts the entire patch by default, leaving the working tree untouched. Use `--reject` to apply only the hunks that succeed and write the failures to `*.rej` files for manual resolution.

Synopsis

```
git apply [--stat] [--numstat] [--summary] [--check]
          [--index | --cached] [-N | --intent-to-add]
          [-3 | --3way] [--ours | --theirs | --union]
          [-R | --reverse] [--reject]
          [-p<n>] [-C<n>] [--directory=<root>]
          [--exclude=<path-pattern>] [--include=<path-pattern>]
          [--ignore-space-change | --ignore-whitespace]
          [--whitespace=<action>]
          [--recount] [--inaccurate-eof] [--unidiff-zero]
          [--apply] [--no-add] [--allow-empty]
          [--verbose | --quiet] [--unsafe-paths]
          [<patch>...]
```

Pass `-` as `<patch>` to read from standard input.

Everyday usage

Inspect what a patch would change before touching any file:

```
git apply --stat feature.patch      # diffstat: files and line counts
git apply --check feature.patch     # dry run: exits non-zero if it would fail
```

Apply a patch to the working tree only (you still need to `git add` afterwards):

```
git apply feature.patch
```


Apply a patch and immediately stage the result (equivalent to apply + add):

```
git apply --index feature.patch
```

Apply a patch received on stdin, for example piped from curl :

```
curl -s https://example.com/fix.patch | git apply --index -
```

Reverse a patch that was already applied (undo it):

```
git apply -R feature.patch
```

Apply as much as possible, writing rejected hunks to *.rej files:

```
git apply --reject partial.patch
```

Key options

Option	What it does	When to use it
--stat	Print a diffstat instead of applying	Preview the scope of a patch
--numstat	Machine-readable diffstat (decimal counts, full paths)	Scripting and automation
--summary	Show creations, renames, and mode changes	Understand structural changes at a glance
--check	Dry run — verify applicability without modifying anything	Validate a patch before applying it
--index	Apply to both working tree and index	Apply and stage in one step
--cached	Apply to the index only, leave working tree untouched	Construct staged content directly
-N, --intent-to-add	Mark new files added by the patch as intent-to-add in the index	When you want git diff to show new-file additions
-3, --3way	Fall back to a three-way merge when context does not match	Applying an old patch against an evolved codebase
--ours, --theirs, --union	Resolve three-way conflicts automatically (requires --3way)	Scripted, unattended patch application
-R, --reverse	Apply the patch backwards	Undo a previously applied patch
--reject	Apply matching hunks; write failures to *.rej files	Partial application with manual follow-up
-p<n>	Strip <n> leading path components from patch paths (default: 1)	Patches from foreign repositories with different path prefixes

<code>-C<n></code>	Require at least <code><n></code> lines of context to match	Tighten or relax context matching
<code>--directory=<root></code>	Prepend <code><root></code> to all paths in the patch	Apply a patch into a subdirectory
<code>--exclude=<pattern></code>	Skip files matching a path pattern	Import a patch series while ignoring vendored files
<code>--include=<pattern></code>	Only apply changes to files matching a pattern	Cherry-pick a subset of files from a large patch
<code>--ignore-space-change</code> , <code>--ignore-whitespace</code>	Ignore whitespace differences in context lines	Patch produced with different line endings or indentation
<code>--whitespace=<action></code>	Control how trailing-whitespace errors are handled (<code>nowarn</code> , <code>warn</code> , <code>fix</code> , <code>error</code> , <code>error-all</code>)	Enforce or relax whitespace hygiene
<code>--recount</code>	Infer hunk line counts from the patch body	Apply hand-edited patches with incorrect hunk headers
<code>--inaccurate-eof</code>	Work around diffs that miss a missing-newline marker	Patches generated by buggy diff tools
<code>--apply</code>	Force application even when a “turns off apply” flag is present	Get diffstat output <i>and</i> apply the patch together
<code>--no-add</code>	Ignore additions; apply deletions only	Extract the common base between two files
<code>--allow-empty</code>	Succeed on patches that contain no diff	Pipelines where empty patches are normal
<code>--unsafe-paths</code>	Allow patches that reference paths outside the working area	Using <code>git apply</code> as a GNU <code>patch</code> replacement
<code>-v</code> , <code>--verbose</code>	Print more progress information to stderr	Diagnosing why a patch fails
<code>-q</code> , <code>--quiet</code>	Suppress all progress output	Silent scripted use

Best practices

Always run `--check` first on untrusted or unfamiliar patches. It costs nothing and catches conflicts before anything is written to disk. Make it a habit before `--index` or `--cached` .

```
git apply --check incoming.patch && git apply --index incoming.patch
```

Use `--3way` when applying against an evolved codebase. Context lines drift as history accumulates. Rather than failing outright, `--3way` attempts a merge and leaves conflict markers in place — the same workflow you already know from `git merge` . Without it, even a one-line context mismatch aborts the entire patch.

Prefer `--index` over a bare `apply`. A bare `git apply` modifies files but does not touch the index. You must then run `git add` on every changed file before committing. Passing `--index` does both in a single step and is less error-prone.

Strip path prefixes deliberately with `-p`. Standard `git diff` output uses an `a/` and `b/` prefix on every path. The default `-p1` strips one component, turning `a/src/main.c` into `src/main.c` .

Patches from foreign projects (Subversion exports, GNU diff output, mailing-list patches) often have different prefix depths. Check with `--stat` first and adjust `-p` accordingly.

Use `--directory` to relocate a patch into a subdirectory. When integrating a standalone project's patch into a monorepo, `--directory` saves hand-editing every path in the patch file.

```
git apply --directory=libs/mylib -p1 mylib-fix.patch
```

Combine `--exclude` and `--include` to import partial patch series. Large patch sets from mailing lists often contain generated files (translations, minified assets) you want to skip. Use `--exclude` to filter them out without editing the patch.

Pitfalls & gotchas

`git apply` does not create a commit. This surprises people who expect it to behave like `git am`. If you want a full commit (with author, date, and message from the patch), use `git am` instead. `git apply` is the lower-level primitive.

`--reject` leaves your working tree in a partially applied state. After `--reject`, some hunks are applied, others are not. Inspect every `*.rej` file, apply its changes manually, delete the `.rej` files, then stage and commit. Running `git apply` again on the same patch will fail because some hunks are already in place.

Context lines must match exactly by default. Even a one-line difference in surrounding context causes the hunk to fail. If you receive the error `patch does not apply`, the most common causes are: the file has diverged from the base the patch was created against, or whitespace differences (line endings, indentation). Try `--ignore-whitespace` or `--3way` before concluding the patch is broken.

`--index` requires working tree and index to be in sync. If you have staged changes in the index that differ from the working tree for the same file, `--index` raises an error even if the patch would apply cleanly to both in isolation. Commit or stash your staged changes first.

`-p0` retains the full path as written in the patch. This is almost never what you want when applying a Git-generated patch (which uses `a/ / b/` prefixes). Using `-p0` on such a patch causes Git to look for a directory literally named `a/` at the repo root.

`--unsafe-paths` opens a security risk. Never use it on patches from untrusted sources. A malicious patch could overwrite files outside the repository — including dotfiles, cron jobs, or SSH keys.

Binary patches need no special flag anymore. Older documentation mentions `--binary` as required for binary content. Modern Git always allows binary patch application; the flag is a no-op and can be ignored.

Worked examples

Inspecting and applying a mailing-list patch

A contributor sends `0001-fix-null-deref.patch` by email (or via a download link). Before touching the tree, inspect it:

```
git apply --stat 0001-fix-null-deref.patch
```

```
src/parser.c | 4 ++--
1 file changed, 2 insertions(+), 2 deletions(-)
```

Verify it applies cleanly to the current HEAD:

```
git apply --check 0001-fix-null-deref.patch
```

No output and exit code 0 means success. Now apply and stage in one step:

```
git apply --index 0001-fix-null-deref.patch
git commit -m "Fix null dereference in parser (from patch)"
```

If you want the original author and commit message preserved, use `git am` instead:

```
git am 0001-fix-null-deref.patch
```

Applying a patch from a foreign project with a different path layout

An upstream GNU project provides a patch with paths like `mylib-1.4/src/util.c`. Your copy of the library lives under `libs/mylib/`. The patch has no `a/ / b/` prefix — it is a plain `diff -u` output, so the prefix depth is 0 for the project root.

First, see how many path components separate the patch paths from your layout:

```
git apply --stat -p1 upstream.patch # try stripping one component
```

```
error: mylib-1.4/src/util.c: does not exist in index
```

```
git apply --stat -p2 upstream.patch # strip two components
```

```
src/util.c | 8 ++++---
1 file changed, 4 insertions(+), 4 deletions(-)
```

The files live under `libs/mylib/`, so prepend that with `--directory`:

```
git apply --check -p2 --directory=libs/mylib upstream.patch
git apply --index -p2 --directory=libs/mylib upstream.patch
```

Salvaging a patch that no longer applies cleanly

A patch was prepared against a version of the file that has since been refactored. The plain apply fails:

```
git apply feature.patch
```

```
error: patch failed: src/auth.c:42
error: src/auth.c: patch does not apply
```

Try the three-way fallback:

```
git apply --3way feature.patch
```

If the content Git needs is in the object database (because the patch records blob SHAs), this succeeds or leaves familiar conflict markers. Resolve them with your editor or `git mergetool` (see the *mergetool* chapter), then stage and commit normally.

If `--3way` also fails, use `--reject` to apply what is possible and fix the rest manually:

```
git apply --reject feature.patch
# Edit src/auth.c, incorporating the hunks from src/auth.c.rej
rm src/auth.c.rej
git add src/auth.c
git commit -m "Apply feature patch (manually resolved conflicts)"
```

Undoing a patch that was applied by mistake

You ran `git apply --index wrong.patch` and staged the result, but have not yet committed. Reverse the patch:

```
git apply -R --index wrong.patch
```

The working tree and index are restored to their previous state. If you had already committed, see *revert* or the *Getting out of jams* chapter for commit-level undo strategies.

Recovery

If `git apply` left your working tree in a broken partial state (for example after `--reject`), discard all uncommitted changes and return to HEAD:

```
git restore .
```

If you applied with `--index` and want to unstage without touching the working tree:

```
git restore --staged .
```

To undo a commit that was created immediately after a `git apply --index`, keep the staged changes but remove the commit:

```
git reset --soft HEAD~1
```

See *Getting out of jams* for broader undo strategies including recovering from an accidental `--cached apply`.

See also

- *am* — apply a patch series as commits, preserving author and message.
- *format-patch* — generate patch files from commits, suitable for `git am` or `git apply`.
- *diff* — produce the unified diff output that `git apply` consumes.
- *add* — stage files after a bare `git apply` (without `--index`).
- *mergetool* — resolve conflicts left by `--3way`.
- *revert* — create a commit that undoes the effect of a previous commit.
- *Getting out of jams* — undo strategies when `apply` leaves the tree in a bad state.

am

Apply a series of patches from a mailbox file, reconstructing commits with their original author, date, and message.

Mental model

`git am` is the receiving end of an email-based patch workflow. The sending end — `git format-patch` — turns commits into RFC 2822 email messages where the subject line becomes the commit title, the body becomes the commit message, and the diff is appended inline. `git am` reverses that process: it reads one or more of those messages, extracts the author identity and date from the email headers, applies the diff to the working tree, and records a real commit.

Contributor machine	Maintainer machine
<code>git format-patch origin/main</code> ↳ 0001-fix-login.patch —email→	<code>git am 0001-fix-login.patch</code> ↳ commit in history

Think of it as `git apply` plus `git commit` in one step, with the commit metadata coming from the email message rather than from you interactively. Because the original author and date are preserved, the contributor's identity appears in `git log` even though you are the one running the command on your machine.

`git am` is stateful: when a patch fails to apply, it pauses and leaves a session in `.git/rebase-apply/`. You resolve the conflict, then tell it to `--continue`, `--skip`, or `--abort`, just like a rebase. The command refuses to start a new session until the current one is resolved or aborted.

Synopsis

```
git am [--signoff] [--keep] [--[no-]keep-cr] [--[no-]utf8]
      [--[no-]3way] [--interactive] [--no-verify]
      [--committer-date-is-author-date] [--ignore-date]
      [--ignore-space-change | --ignore-whitespace]
      [--whitespace=<action>] [-C<n>] [-p<n>]
      [--directory=<dir>] [--exclude=<path>] [--include=<path>]
      [--reject] [-q | --quiet] [--[no-]scissors]
      [-S[<keyid>]] [--patch-format=<format>]
      [--quoted-cr=<action>] [--empty=(stop|drop|keep)]
      [<mailbox> | <Maildir>]...

git am (--continue | --skip | --abort | --quit | --retry |
      --show-current-patch[=(diff|raw)] | --allow-empty)
```

Everyday usage

Apply a single patch file produced by `git format-patch`:

```
git am 0001-fix-null-pointer-in-login.patch
```

Apply all patches in the current directory in order:

```
git am *.patch
```

Apply patches piped from standard input (common when saving email directly):

```
git am < incoming.mbox
```

Apply a patch with 3-way merge fallback so conflicts land in the working tree rather than causing a hard failure:

```
git am --3way 0001-fix-null-pointer-in-login.patch
```

Add your own Signed-off-by trailer to each applied commit (required by some projects such as the Linux kernel):

```
git am --signoff patches/*.patch
```

Key options

Option	What it does	When to use it
-3 , --3way	Fall back to 3-way merge when the patch does not apply cleanly	Almost always — makes conflicts resolvable instead of fatal
-s , --signoff	Append a Signed-off-by trailer with your identity	Projects that require DCO sign-off from the person applying the patch
--keep	Pass -k to git mailinfo preserve the original subject prefix	Prevent [PATCH] or [RFC] from being stripped out of the title
--no-utf8	Do not re-encode the commit message to UTF-8	Rare; when the source encoding must be preserved verbatim
--[no-]keep-cr	Preserve (or strip) carriage returns at line endings	Patches originating from Windows mail clients
--scissors	Drop everything before the scissors line (-- >8 --) in the email body	Patches forwarded with extra quoting above the actual diff
-p<n>	Strip <n> leading path components from filenames in the diff	Applying a patch with a different directory depth
-C<n>	Require <n> lines of surrounding context to match	Tighten the match when the patch was made against a different base
--directory=<dir>	Prepend <dir> to all filenames in the patch	Applying a patch into a subdirectory
--reject	Write unmatched hunks as .rej files instead of aborting	Partial application when you want to salvage what applies cleanly
--whitespace=<action>	Control how whitespace errors are handled (nowarn , warn , fix , error , error-all)	fix automatically corrects trailing whitespace issues

<code>--patch-format=<fmt></code>	Force a specific patch format (<code>mbox</code> , <code>mboxrd</code> , <code>stgit</code> , <code>stgit-series</code> , <code>hg</code>)	When auto-detection picks the wrong format
<code>--empty=(stop drop keep)</code>	What to do with email messages that contain no patch	<code>drop</code> skips non-patch messages silently; default is <code>stop</code>
<code>-m</code> , <code>--message-id</code>	Include the Message-ID header in the commit message	Traceability back to the mailing list archive
<code>--committer-date-is-author-date</code>	Set the committer date to match the author date from the email	Reproducible builds or bisect scenarios requiring exact timestamps
<code>--ignore-date</code>	Ignore the email date; use the current time as author date	When the patch is old but you want today's timestamp
<code>-q</code> , <code>--quiet</code>	Print only error messages	Scripted bulk application
<code>-S[<keyid>]</code> , <code>--gpg-sign</code>	GPG-sign each applied commit	Verified commit workflows
<code>--no-verify</code>	Skip the <code>applypatch-msg</code> and <code>pre-applypatch</code> hooks	Emergency; investigate the failing hook afterward
<code>--interactive</code>	Confirm each patch before applying	Reviewing a batch of patches one at a time
<code>--continue</code> , <code>-r</code> , <code>--resolved</code>	Resume after manually resolving a conflict	Conflict recovery (see Recovery section)
<code>--skip</code>	Discard the current failing patch and move to the next	When a patch is genuinely inapplicable and you want to move on
<code>--abort</code>	Restore the branch to its state before <code>git am</code> started	Start over from scratch
<code>--quit</code>	Stop the session without restoring HEAD or the index	Rare; leaves the tree in mid-session state
<code>--retry</code>	Retry the last failing patch, useful for passing extra options	Re-attempt with <code>--3way</code> after an initial failure without it
<code>--show-current-patch[=(diff raw)]</code>	Print the patch that caused the current failure	Inspect what you are trying to resolve

Best practices

Always pass `--3way` for patches against a moving target. A patch produced against an old commit rarely applies perfectly by line number. With `--3way` , `git` falls back to a three-way merge when the context does not match exactly, giving you standard conflict markers in the working tree instead of a hard failure. Make it your default by setting `am.threeWay` in your config:

```
git config --global am.threeWay true
```

Verify the patch before applying it. `git am` creates real commits in your history. Inspect the patch file first to understand what it changes:


```
git apply --stat 0001-fix-login.patch # summary of changed files
git apply --check 0001-fix-login.patch # dry-run: does it apply cleanly?
```

If `--check` passes, `git am` will too (absent 3-way considerations).

Keep patches numbered and ordered. `git format-patch` names files `0001-...`, `0002-...`. Apply them in that order. Applying `0003` before `0002` almost always fails because later patches depend on earlier ones. Use a glob that expands in numeric order:

```
git am *.patch
```

Confirm the applied commits before pushing. After a successful `git am`, run `git log --oneline` to confirm author names, dates, and messages look correct. A misformatted patch can corrupt the author line silently:

```
git log --format="%h %aN <%aE> %s" -5
```

Use `--signoff` when the project requires it. The Linux kernel, U-Boot, and many other projects treat the Signed-off-by trailer as a legal declaration under the Developer Certificate of Origin. If you are the person integrating a patch on behalf of a contributor, add your own sign-off:

```
git am --signoff patches/*.patch
```

This appends Signed-off-by: Your Name <you@example.com> to every commit while preserving the contributor's original authorship.

Pitfalls & gotchas

Patch context lines must match the working tree. `git am` computes patch offsets using surrounding context. If the base commit differs too much from what the patch was created against, even `--3way` may not help. Check whether the patch records base commit information (produced by `git format-patch --base=...`) and, if possible, apply onto the same base first.

Email clients mangle patches. Quoted-printable encoding, line-wrapping at 76 characters, converting tabs to spaces, and HTML multipart messages all corrupt patches in transit. The safest delivery method is attaching the raw `.patch` file; the second safest is saving the raw message from a command-line mail client with no re-encoding. If the patch arrives mangled, `--whitespace=fix` and `--keep-cr` may help at the margins, but there is no substitute for a clean raw file. The `git send-email` / `git am` pair avoids this problem entirely by sending patches as plain-text attachments.

`git am` is not the same as `git apply`. `git apply` modifies the working tree and optionally the index but creates no commit. `git am` always creates a commit. If you want to inspect or partially apply a change without committing, use the *apply* command instead.

`ORIG_HEAD` is set before any patches are applied. Git records the tip of the branch in `ORIG_HEAD` at the start of the session. If something goes wrong mid-series you can return to the pre-am state:

```
git reset --hard ORIG_HEAD
```

This is faster than `--abort` after a failed session and works even if the session state has been corrupted.

An in-progress session blocks new work. If you run `git am` while a previous session is unfinished, git refuses:

```
error: previous rebase directory .git/rebase-apply still exists but mbox given.
```

Either finish the session (`--continue` , `--skip`) or abandon it (`--abort`) before starting a new one.

Empty messages stop the session by default. If an mbox contains forwarded emails or cover letters without a diff, `git am` halts with `--empty=stop` (the default). Pass `--empty=drop` to silently skip non-patch messages:

```
git am --empty=drop patches.mbox
```

Worked examples

Applying a patch series from a mailing list

A contributor sends a three-patch bug-fix series. You save the thread as an mbox file from your mail client (`patches.mbox`).

```
# Preview what will change
git apply --stat patches.mbox

# Dry-run to catch obvious failures early
git apply --check patches.mbox

# Apply for real, with 3-way fallback and your sign-off
git am --3way --signoff patches.mbox
```

Successful output:

```
Applying: Fix null pointer in login handler
Applying: Add regression test for login edge case
Applying: Update CHANGELOG for login fix
```

Verify the result:

```
git log --oneline -3
```

```
c4a1e8f Update CHANGELOG for login fix
9d73b2a Add regression test for login edge case
3f51a04 Fix null pointer in login handler
```

Resolving a conflict mid-series

You are applying a five-patch series. Patch 3 fails:

```
Applying: Refactor authentication module
error: patch failed: src/auth.c:142
```

```
error: src/auth.c: patch does not apply
hint: Use --reject to leave rejected hunks in files with .rej extension.
hint: Alternatively, use git am --skip to skip this patch.
hint: To restore the original branch and stop patching run "git am --abort".
```

Inspect the failing patch:

```
git am --show-current-patch=diff
```

If you did not start with `--3way`, abort and retry with it:

```
git am --abort
git am --3way patches/*.patch
```

If conflict markers appear in the working tree, resolve them manually, stage the result, and continue:

```
# Edit src/auth.c to resolve the conflict markers
git add src/auth.c
git am --continue
```

`git am` applies the remaining patches automatically. If patch 4 or 5 also conflicts, repeat the process for each one.

Applying a patch from a GitHub commit URL

GitHub's patch endpoint (`/commit/<sha>.patch`) returns a valid mbox message. Fetch and apply it directly:

```
curl -L https://github.com/example/repo/commit/abc1234.patch | git am --3way
```

Successful output:

```
Applying: Fix race condition in worker pool
```

If it fails, inspect the situation with `--show-current-patch`, resolve, and run `git am --continue`.

Recovering from applying onto the wrong branch

You realize mid-session that you started on the wrong branch. Abort completely:

```
git am --abort
git switch correct-branch
git am --3way patches/*.patch
```

`--abort` restores the branch to the state captured in `ORIG_HEAD` before any patches were applied.

Recovery

If patches have been applied and you want to undo all of them at once:

```
git reset --hard ORIG_HEAD
```

`ORIG_HEAD` is set automatically when `git am` starts. This is the fastest path when the session completed but the result is wrong.

To abort a session that is currently paused on a conflict:

```
git am --abort
```

This restores the branch and clears the `.git/rebase-apply/` state directory.

To undo only the most recently applied commit after a clean `git am`:

```
git reset --soft HEAD~1    # keep changes staged
# or
git reset HEAD~1           # keep changes in the working tree
```

To see all commits applied in the session (useful before deciding how far to reset):

```
git reflog
```

See *Getting out of jams* for additional undo recipes and how to recover commits using the *reflog* command.

See also

- *format-patch* — produces the patch files that `git am` consumes.
- *apply* — applies a patch to the working tree and index without creating a commit; useful for inspection or partial application.
- *cherry-pick* — an alternative way to bring commits across when you have direct repository access rather than a patch file.
- *rebase* — shares the same `--continue` / `--skip` / `--abort` session model; understanding one helps with the other.
- *rerere* — record conflict resolutions once and replay them automatically; valuable when applying recurring patch series against the same code.
- *commit* — the commit objects that `git am` creates follow the same structure described in the *commit* chapter.
- *Getting out of jams* — recipes for undoing applied patches and recovering from a failed `git am` session.

format-patch

Turn commits into email-ready patch files that can be reviewed, applied, and archived outside of a shared repository.

Mental model

`git format-patch` serialises commits as UNIX mailbox messages. Each commit becomes one `.patch` file (or one mbox message on stdout) containing three parts:

1. An email header — `From:`, `Date:`, and `Subject:` derived from the commit author, date, and first line of the commit message.
2. The remainder of the commit message as the email body.
3. The diff itself, separated from the body by a line containing only `---`.

```
Commit object  —format-patch→ 0001-fix-login.patch
                                |— email headers (From, Date, Subject)
                                |— commit message body
                                |— ---
                                |— diff -p --stat output
```

The fixed timestamp `Mon Sep 17 00:00:00 2001` in the `From` line is a magic marker that tools like `file(1)` use to recognise format-patch output rather than a real mailbox.

The round-trip counterpart is `git am`, which reads these files and replays the commits. Because the patch carries author identity and the full log message, the resulting commit is faithful to the original — not a squash or a cherry-pick with a generic message.

This mechanism predates GitHub pull requests. It remains the contribution workflow for projects like the Linux kernel and Git itself, which conduct code review on mailing lists. Even when email is not the delivery channel, the format is useful: the files are self-documenting, can be archived, and apply cleanly with `git am`.

Synopsis

```
git format-patch [-<n>] [--stdout] [-o <dir>]
                  [--cover-letter] [--subject-prefix=<prefix>]
                  [--to=<email>] [--cc=<email>]
                  [-v <n> | --reroll-count=<n>]
                  [--in-reply-to=<message-id>] [--thread[=<style>]]
                  [--rfc[=<rfc>]] [--base[=<commit>]]
                  [--notes[=<ref>]] [--interdiff=<previous>]
                  [--range-diff=<previous>]
                  [<common-diff-options>]
                  [ <since> | <revision-range> ]
```

Everyday usage

Export every commit on the current branch that is not in `origin/main`:

```
git format-patch origin/main
```

Git writes one numbered `.patch` file per commit in the current directory:

```
0001-Add-rate-limiting-to-login-endpoint.patch
0002-Extract-token-validation-into-a-helper.patch
```

Export only the last three commits:

```
git format-patch -3
```

Export a specific range and write the files to a `patches/` directory:

```
git format-patch -o patches/ origin/main..HEAD
```

Export exactly one commit by its hash:

```
git format-patch -1 a3f9c1b
```

Preview all patches concatenated to stdout — useful for piping to `git am`:

```
git format-patch --stdout origin/main
```

Apply those patches on another branch in one step:

```
git format-patch --stdout origin/main | git am -3
```

Key options

Option	What it does	When to use it
<code>-<n></code>	Export the topmost <code><n></code> commits	Quick single- or few-commit export
<code>-o <dir></code>	Write files into <code><dir></code> (created if absent)	Keep patches out of the repo root
<code>--stdout</code>	Print all patches to stdout as one mbox stream	Piping directly to <code>git am</code>
<code>--cover-letter</code>	Generate a <code>0000-cover-letter.patch</code> with a shortlog and diffstat	Multi-patch series; fill in summary before sending
<code>--subject-prefix=<prefix></code>	Replace the default <code>[PATCH]</code> tag in the subject line	Per-project conventions, e.g. <code>[PATCH subsystem]</code>
<code>--to=<email></code>	Add a <code>To:</code> header to every generated message	Embed recipient so <code>git send-email</code> picks it up
<code>--cc=<email></code>	Add a <code>Cc:</code> header to every generated message	Copy maintainers or reviewers automatically
<code>-v <n> /</code> <code>--reroll-count=<n></code>	Mark the series as the <code><n></code> -th revision; prepends <code>v<n></code> to filenames and appends it to the subject prefix (e.g. <code>[PATCH v2 1/3]</code>)	Resending a revised patch series after review

<code>--rfc[=<rfc>]</code>	Prepend RFC (or custom string) before PATCH in subject	Experimental patches sent for discussion, not application
<code>--in-reply-to=<message-id></code>	Set In-Reply-To: so the patch threads under an existing message	Responding to a review thread on a mailing list
<code>--thread[=<style>]</code>	Add In-Reply-To and References headers to thread the series (shallow or deep)	Keep a series readable as a thread in mail clients
<code>--ignore-if-in-upstream</code>	Skip any commit whose patch content is already reachable from the upstream	Avoid resubmitting already-merged work
<code>--always</code>	Include commits that introduce no change (normally omitted)	Carry documentation-only or merge-point commits
<code>--base[=<commit>]</code>	Record the base tree information block in the first message	Help reviewers identify which tree the series applies to
<code>--notes[=<ref>]</code>	Append git notes after the --- line	Carry review commentary that is not part of the commit message
<code>--interdiff=<previous></code>	Insert an interdiff into the cover letter	Show reviewers what changed between patch series versions
<code>--range-diff=<previous></code>	Insert a range-diff into the cover letter	Alternative to --interdiff uses git range-diff format
<code>-n / --numbered</code>	Always include n/m numbering in the subject, even for a single patch	Consistency when series size varies across revisions
<code>-N / --no-numbered</code>	Never include n/m numbering	Single patches that should not look like part of a series
<code>--start-number <n></code>	Start the sequence at <n> instead of 1	Appending to an existing partially-sent series
<code>--numbered-files</code>	Use plain numbers as filenames, without the commit subject appended	Scripting workflows that do not need human-readable names
<code>--signature=<sig> / --no-signature</code>	Override or suppress the Git version signature appended to each patch	Cleaner output; suppress the default -- \n2.x.y trailer
<code>--suffix=<sfx></code>	Change the filename extension from .patch	Some review tools or mail filters expect .txt
<code>-s / --signoff</code>	Add a Signed-off-by: trailer using your committer identity	Projects requiring DCO sign-off
<code>-k / --keep-subject</code>	Do not strip or add [PATCH] from the first line of the commit message	Preserving custom subject prefixes already in the message

<code>--binary</code>	Include binary diffs; implies <code>--full-index</code> automatically	Patches that add images or other binary assets
<code>--root</code>	Treat the revision argument as a range even when it is a single commit	Export everything from the very first commit
<code>-q / --quiet</code>	Suppress the list of generated filenames on stdout	Scripting

Best practices

Export into a dedicated directory. Running `git format-patch -o patches/ origin/main` keeps the generated files out of the repository root and away from `git status` noise. Add `patches/` to `.gitignore` if you work this way regularly.

Add a cover letter for any series longer than one patch. A cover letter gives reviewers the big picture: what the series does, how it is structured, and any context that does not fit in individual commit messages. Generate it with `--cover-letter` and fill in the `*** SUBJECT HERE ***` and `*** BLURB HERE ***` placeholders before sending.

Number revisions with `--reroll-count`. When you revise a series after review feedback, use `-v 2`, `-v 3`, and so on. The subject becomes `[PATCH v2 1/3]` and filenames gain a `v2-` prefix so reviewers can tell iterations apart and mail threads stay separate.

```
git format-patch --cover-letter -v 2 -o /tmp/myfeature-v2/ origin/main
```

Use `--in-reply-to` to continue a thread. When a reviewer posted comments, grab the `Message-ID` of that message and thread your reply onto it:

```
git format-patch \
  --in-reply-to="<20260615123456.12345-1-reviewer@example.com>" \
  origin/main
```

Your revised series lands in the same mail thread rather than starting a new one.

Verify patches apply cleanly before sending. Test the generated files on a clean branch using `git am`. Discovering whitespace corruption or a wrong base after sending wastes reviewers' time.

```
git checkout -b test-apply origin/main
git am /tmp/patches/*.patch
```

Set per-project defaults in git config. If a project requires a fixed prefix or a default recipient, add a `[format]` section to the local config rather than typing the same flags every time:

```
git config format.subjectPrefix "PATCH net"
git config format.to "netdev@vger.kernel.org"
git config format.coverLetter auto
git config format.outputDirectory /tmp/patches
```

Pitfalls & gotchas

Merge commits are silently dropped. `format-patch` skips merge commits entirely — a plain patch file cannot represent a merge. If your branch contains merges, the series will not match the

topology of the original history. Rebasing onto the target branch first (see the *rebase* chapter) to produce a linear series before exporting.

Email clients corrupt whitespace. Sending patches through webmail (Gmail, Outlook web) or through Thunderbird with word-wrap enabled mangles diff context lines. Corrupted context lines cause `git am` to fail with “patch does not apply”. Use `git send-email` which sends raw text over SMTP, or test by sending a patch to yourself and applying it with `git am` before submitting.

--thread and git send-email threading conflict. `git send-email` adds its own threading headers by default. If you also run `git format-patch --thread`, the two interact and the `In-Reply-To` chain can be wrong. Either let `send-email` handle threading (the default) or pass `--no-thread` to `send-email` when you want `format-patch` to control threading instead.

The default output is the current directory. Running `git format-patch origin/main` drops `.patch` files right next to your source files unless you use `-o`. This clutters `git status` output.

--stdout produces an mbox stream, not individual files. Writing `git format-patch --stdout origin/main > all.mbox` gives you a single mbox file. Apply it with `git am < all.mbox`, not by listing individual patch files.

--ignore-if-in-upstream compares patch content, not commit hashes. A commit that was cherry-picked with a different message still matches if the diff is identical. Conversely, a commit with a trivial rebase conflict resolution that changes one context line will not match and will be included unexpectedly. Always review the output before sending.

File names are truncated at 64 bytes by default. Very long commit subjects produce shortened filenames. Adjust with `--filename-max-length=<n>` if your project’s tooling expects the full subject in the filename.

Worked examples

Submitting a small feature series to a mailing list

You have three commits on `feature/rate-limit` that are not yet in `origin/main`.

```
# Generate patches with a cover letter into a dedicated directory
git format-patch \
  --cover-letter \
  --subject-prefix="PATCH net" \
  --to="netdev@vger.kernel.org" \
  --cc="alice@example.com" \
  -o /tmp/rate-limit-v1/ \
  origin/main
```

This produces:

```
/tmp/rate-limit-v1/0000-cover-letter.patch
/tmp/rate-limit-v1/0001-Add-rate-limit-infrastructure.patch
/tmp/rate-limit-v1/0002-Wire-rate-limiter-into-login-handler.patch
/tmp/rate-limit-v1/0003-Add-rate-limit-tests.patch
```

Open `0000-cover-letter.patch`, replace the placeholder subject and body, then send with `git send-email`:

```
git send-email /tmp/rate-limit-v1/*.patch
```

Revising a series after review feedback

A reviewer asked you to split the second commit and tighten the error messages. After making the changes the branch now has four commits. Mark the new export as v2 and include a range-diff so reviewers see exactly what changed:

```
git format-patch \
  --cover-letter \
  --reroll-count=2 \
  --subject-prefix="PATCH net" \
  --to="netdev@vger.kernel.org" \
  --in-reply-to="<20260615123456.12345-1-reviewer@example.com>" \
  --range-diff=feature/rate-limit-v1 \
  -o /tmp/rate-limit-v2/ \
  origin/main
```

Filenames now begin with `v2-` and each subject reads `[PATCH net v2 n/m]`. The `--range-diff` flag inserts a diff-of-diffs in the cover letter so reviewers immediately see what changed between v1 and v2 without reading every patch from scratch.

Transferring commits between machines without a shared remote

You have local commits on a laptop that the target machine cannot reach via a remote.

On the laptop:

```
git format-patch --stdout origin/main > ~/Desktop/feature.mbox
```

Copy `feature.mbox` to the target machine (via USB, `scp`, etc.), then:

```
# On the target machine, on a clean branch tracking origin/main
git am -3 ~/feature.mbox
```

The `-3` flag asks `git am` to attempt a three-way merge if a patch does not apply cleanly, which is more forgiving when the base has diverged slightly.

Exporting a single historic commit by hash

You want to export commit `a3f9c1b` regardless of where it sits in history:

```
git format-patch -1 a3f9c1b -o /tmp/
```

This writes exactly one file, for example `0001-Fix-off-by-one-in-parser.patch`, without any context about the surrounding branch.

Recovery

If `git am` chokes on a patch file you generated, abort and inspect:

```
git am --abort
```

Common causes and fixes:

- **Whitespace corruption** — regenerate with `git format-patch` and apply with `git am --whitespace=fix` to auto-correct trailing-whitespace issues.

- **Base mismatch** — check out the commit the patch was made against (or use `git am -3` to fall back to a three-way merge).
- **Merge commits in the range** — rebase the branch to linearise history before re-running `format-patch`.

If you accidentally generated patches with the wrong range or options, simply delete the `.patch` files and re-run `git format-patch`. No repository state is modified by generating patches.

See *Getting out of jams* for broader undo recipes, including recovering from a partially applied `git am` session.

See also

- *am* — apply patch files produced by `format-patch`.
- *send-email* — send the generated `.patch` files via SMTP without a GUI mail client.
- *rebase* — linearise a branch before exporting, and collapse fixup commits with `--autosquash` before generating the series.
- *commit* — writing good commit messages that become good patch subjects.
- *cherry-pick* — an alternative way to transplant individual commits when email delivery is not needed.
- *range-diff* — compare two versions of a patch series directly, outside of the cover-letter context.

send-email

Send a collection of commits as formatted patch emails directly to a mailing list or maintainer.

Mental model

Many open-source projects — the Linux kernel, Git itself, U-Boot, and hundreds more — accept contributions through email rather than pull requests. A contributor emails patches to a mailing list; maintainers review them inline, reply with comments, and eventually apply the accepted patches with `git am`.

`git send-email` is the bridge between your local commits and that email workflow. It reads one or more patch files (typically produced by `git format-patch`), opens an SMTP connection, and delivers each patch as a proper RFC 2822 email. Threading, reply-to chains, cover letters, and Cc lists are all handled automatically.

```
Local commits
└─ git format-patch → .patch files
    └─ git send-email → SMTP → mailing list
```

Think of `git format-patch` as your word processor (it shapes the content) and `git send-email` as your mail client (it delivers it). The two are designed to be used together, but `send-email` also accepts raw mbox files or revision ranges, so you can pass format-patch arguments directly on the command line and skip the intermediate files entirely.

Synopsis

```
git send-email [<options>] (<file>|<directory>)...
git send-email [<options>] <format-patch-options>
git send-email --dump-aliases
git send-email --translate-aliases
```

Everyday usage

Configure SMTP once, then send patches

Add your mail server settings to `~/.gitconfig` (or the repo's `.git/config` for project-specific accounts):

```
git config --global sendemail.smtpServer smtp.gmail.com
git config --global sendemail.smtpServerPort 587
git config --global sendemail.smtpEncryption tls
git config --global sendemail.smtpUser you@gmail.com
```

Send the last commit

```
git send-email HEAD~1
```

`send-email` runs `format-patch` internally, generates the patch file, and prompts for `To:`, `Cc:`, and subject if they are not configured.

Send a range of commits

```
git send-email origin/main..my-feature
```

Each commit becomes one email. They are threaded as replies to the first email in the series by default.

Generate patch files first, review them, then send

```
git format-patch --cover-letter -o outgoing/ origin/main
# edit outgoing/0000-cover-letter.patch to fill in the summary
git send-email outgoing/
```

Separating the two steps gives you a chance to inspect the exact bytes that will be sent before committing to delivery.

Do a dry run before sending

```
git send-email --dry-run origin/main
```

`--dry-run` performs every step — generates patches, resolves recipients, prints what it would do — without opening an SMTP connection. Use this to catch addressing mistakes before they reach the list.

Annotate patches interactively before sending

```
git send-email --annotate outgoing/
```

Your editor opens for each patch so you can add notes or adjust headers before delivery.

Key options

Composing

Option	What it does	When to use it
<code>--to=<address></code>	Set the primary To: recipient(s)	Required; the mailing list or maintainer address
<code>--cc=<address></code>	Add a Cc: address	Cc a co-author or subsystem reviewer
<code>--bcc=<address></code>	Add a Bcc: address	Blind-copy yourself or an archive
<code>--from=<address></code>	Override the sender address	When your committer identity differs from your mail account
<code>--subject=<string></code>	Set the email subject (first patch or cover letter)	Rarely needed; <code>format-patch</code> sets this from the commit title
<code>--in-reply-to=<msg-id></code>	Thread the series as a reply to an existing Message-ID	Re-rolling a patch series so it nests under the original thread
<code>--compose</code>	Open an editor to write a cover-letter-style introduction	Send a short explanation before the first patch
<code>--annotate</code>	Open each patch in an editor before sending	Review and add notes to individual patches
<code>--reply-to=<address></code>	Set a Reply-To: header	Redirect replies to a different address than <code>--from</code>

Sending

Option	What it does	When to use it
<code>--smtp-server=<host></code>	SMTP server hostname or IP	Specify your mail provider's server; defaults to localhost if unset and no sendmail binary is found
<code>--smtp-server-port=<port></code>	Override the default port	Use 587 for STARTTLS submission
<code>--smtp-encryption=<tls ssl></code>	Encryption mode (<code>tls</code> = STARTTLS, <code>ssl</code> = implicit TLS on port 465)	Always use one of these; never send on plain port 25
<code>--smtp-user=<user></code>	SMTP authentication username	Required by most hosted mail providers
<code>--smtp-pass[=<pw>]</code>	SMTP password (omitting the whole option lets <code>git credential</code> prompt)	Avoid putting passwords on the command line
<code>--smtp-auth=<mechanisms></code>	Restrict SMTP-AUTH mechanisms (e.g. <code>PLAIN LOGIN</code>)	When your server requires a specific mechanism
<code>--no-smtp-auth</code>	Disable SMTP authentication	Relay-only servers on localhost
<code>--sendmail-cmd=<cmd></code>	Use a local sendmail-compatible binary instead of SMTP	When routing through <code>msmtp</code> , <code>sendmail</code> , etc.
<code>--batch-size=<n></code>	Reconnect after every <i>n</i> messages	Some servers (e.g. <code>smtp.163.com</code>) cap messages per session
<code>--relogin-delay=<sec></code>	Wait this many seconds before reconnecting	Use with <code>--batch-size</code>

Threading and recipients

Option	What it does	When to use it
<code>--[no-]thread</code>	Add <code>In-Reply-To</code> / <code>References</code> headers (on by default)	Disable only for lists that reject threaded patches
<code>--[no-]chain-reply-to</code>	Each patch replies to the previous one (chained) vs. all reply to the first (off by default)	Enable for deeply threaded review workflows
<code>--[no-]signed-off-by-cc</code>	Auto-Cc addresses from <code>Signed-off-by</code> trailers (on by default)	Disable when you do not want all signers notified
<code>--suppress-cc=<category></code>	Stop auto-Cc for <code>self</code> , <code>author</code> , <code>sob</code> , <code>body</code> , <code>all</code> , etc.	Prevent self-Cc, or tighten who gets notified
<code>--cc-cmd=<command></code>	Run a command once per patch to generate extra Cc addresses	Automated per-patch routing based on changed files

Administering

Option	What it does	When to use it
<code>--dry-run</code>	Do everything except send	Always run this first on an unfamiliar list
<code>--confirm=<mode></code>	When to pause for confirmation: <code>always</code> , <code>never</code> , <code>cc</code> , <code>compose</code> , <code>auto</code>	<code>always</code> for peace of mind; <code>never</code> for scripted sends
<code>--[no-]validate</code>	Check patches for lines over 998 chars and run <code>sendemail-validate</code> hook	On by default; only disable if a hook is incorrectly blocking valid patches
<code>--quiet</code>	Print one line per email instead of verbose output	CI or scripted sends
<code>--force</code>	Send even if safety checks fail	Last resort
<code>--dump-aliases</code>	Print configured alias names, one per line	Inspect which aliases are available

Best practices

Configure SMTP credentials in `~/.gitconfig`, not on the command line. Passwords typed on the command line appear in shell history. Set `sendemail.smtpUser` and rely on `git credential` for the password:

```
git config --global sendemail.smtpUser you@example.com
# git-credential prompts on first use, then caches the password
```

Always do a `--dry-run` before sending to a real list. A mis-addressed patch reaching a high-traffic kernel mailing list is embarrassing and cannot be recalled. `--dry-run` costs nothing.

Use `format-patch --cover-letter` for series of more than one patch. A cover letter gives reviewers context before they read individual patches. Edit the generated `0000-cover-letter.patch` to explain the motivation, any design choices, and what has changed since the previous version.

Thread re-rolls under the original series. When you revise patches after review feedback, use `--in-reply-to` with the Message-ID of your original cover letter (or first patch). This keeps the entire discussion in one thread, making it easy for maintainers to track the history.

```
git send-email --in-reply-to="<20260101123456.1234-1-you@example.com>" \
-v2 origin/main
```

Let `--signed-off-by-cc` stay enabled. The default behaviour of auto-Cc'ing anyone in a `Signed-off-by` trailer is intentional: those people already agreed to be associated with the work and should see the review discussion.

Store per-project addresses in a `sendemail.identity` subsection. If you contribute to multiple lists with different To/Cc defaults, use identities rather than typing addresses each time:

```
[sendemail "kernel"]
to = linux-kernel@vger.kernel.org
```

```
cc = subsystem-maintainer@example.com
smtpUser = you@example.com
```

Then send with:

```
git send-email --identity=kernel origin/main
```

Use `--suppress-cc=self` when you don't want to Cc yourself. By default your own address can end up in the Cc list when it appears in trailer lines. Suppress it with:

```
git config --global sendemail.suppressCc self
```

Pitfalls & gotchas

Gmail requires an app password or OAuth2, not your account password. Google disabled regular password authentication for SMTP. Generate an app-specific password at security.google.com/settings/security/apppasswords, or configure `smtpAuth = OAUTHBEARER` with a credential helper. Trying to authenticate with your normal password will produce an authentication error.

Microsoft Outlook no longer supports app-specific passwords. For `smtp.office365.com` you must use OAuth2 with `smtpAuth = XOAUTH2`. `git send-email` automatically applies `--outlook-id-fix` for that server because Outlook rewrites Message-IDs and breaks threading otherwise. Use `--no-outlook-id-fix` only if you are sure the server does not rewrite Message-IDs.

Patches can arrive out of order if your MTA reorders messages. Threading via `In-Reply-To` depends on Message-IDs being preserved end-to-end. Some corporate mail gateways strip or rewrite these headers. Use `--smtp-debug=1` to watch the SMTP conversation and confirm Message-IDs are being transmitted correctly.

--chain-reply-to is off by default for a reason. Chained threading (each patch replying to the previous) makes following a series harder in most MUAs. Stick with the default (all patches reply to the first message) unless the target list specifically requires chained mode.

Line-length validation catches real problems. The `--validate` flag (on by default) warns when a patch contains lines over 998 characters, the RFC 5322 limit. This usually means a generated file ended up in the diff. Split the commit or use `--transfer-encoding=quoted-printable` only if you understand why the long lines are necessary.

The `sendemail.*` config namespace conflicts with `sendmail.*`. If you have any `sendmail.*` variables in your git config (a common mistake), `git send-email` will abort with a warning. Remove or rename those keys, or set `sendemail.forbidSendmailVariables = false` to suppress the check.

Passwords in config files are stored in plain text. Set only `sendemail.smtpUser` in `.gitconfig` and let `git credential` handle the password. Never commit a config file that contains `smtpPass`.

Worked examples

Sending a single bug-fix patch to a mailing list

Your fix is in one commit on top of `origin/main`. The target list is `patches@example-project.org`.


```
# Generate and inspect the patch first
git format-patch -1 HEAD -o /tmp/fix/
cat /tmp/fix/*.patch      # verify Subject, diff, and trailers

# Dry run
git send-email --to=patches@example-project.org --dry-run /tmp/fix/

# Send for real
git send-email --to=patches@example-project.org /tmp/fix/
```

On first run you will be prompted for your SMTP password. After that, `git credential` caches it.

Sending a multi-patch series with a cover letter

You have three commits that together implement a new feature. You want to send them to the list as a numbered series with an introductory cover letter.

```
# Format patches: cover letter plus three numbered patches
git format-patch --cover-letter -n -o outgoing/ origin/main

# Fill in the cover letter subject and body
# (the generated file has placeholder text)
${EDITOR:-vi} outgoing/0000-cover-letter.patch

# Dry run to confirm addressing
git send-email --to=patches@example-project.org --dry-run outgoing/

# Send
git send-email --to=patches@example-project.org outgoing/
```

The resulting thread looks like:

```
[PATCH 0/3] Implement feature X
 [PATCH 1/3] Add data model for X
 [PATCH 2/3] Wire up the API handler
 [PATCH 3/3] Add tests for X
```

Re-rolling a series after review feedback

The maintainer asked for changes. You amend the commits, then re-send as version 2, threaded under the original cover letter so the full discussion history is in one place.

Find the Message-ID of your original cover letter from the list archive or your sent-mail folder — it looks like `<20260601090000.1234-1-you@example.com>`.

```
# Format v2 patches with the version tag
git format-patch --cover-letter -v2 -o outgoing-v2/ origin/main

# Edit the cover letter to summarize what changed since v1
${EDITOR:-vi} outgoing-v2/v2-0000-cover-letter.patch

# Send v2 as a reply to the v1 cover letter
git send-email \
    --to=patches@example-project.org \
```

```
--in-reply-to="<20260601090000.1234-1-you@example.com>" \  
outgoing-v2/
```

The list now sees:

```
[PATCH 0/3] Implement feature X          ← v1 original  
[PATCH 1/3] Add data model for X  
...  
[PATCH v2 0/3] Implement feature X      ← v2 nested under v1  
[PATCH v2 1/3] Add data model for X  
...
```

Using msmtplib instead of direct SMTP

If your environment blocks outbound SMTP or you prefer a local MTA, route through `msmtplib`:

```
git config --global sendemail.sendmailCmd "/usr/bin/msmtplib"  
# msmtplib reads its own ~/.msmtplibrc for server and credentials  
git send-email --to=patches@example-project.org outgoing/
```

`--sendmail-cmd` overrides `--smtp-server` the two cannot be used together.

Recovery

Wrong recipient — email already sent. Email cannot be recalled. Send a follow-up to the same list/address acknowledging the error and, if needed, resend the corrected patch with a note explaining the mistake.

Sent to the right list but forgot `--in-reply-to` for a re-roll. The new series will appear as a separate thread. Reply manually to both threads pointing to the other, so readers can find the full context.

Authentication failure mid-series (some patches sent, some not). Note which patches were sent (check your outbox or the list archive), then resend only the remaining patches by passing those specific `.patch` files instead of the whole directory. Use `--in-reply-to` with the Message-ID of the last successfully delivered patch so threading is preserved.

Accidentally sent a draft with placeholder text in the cover letter. Resend the full series with a corrected cover letter and add a note at the top explaining the previous send was in error. Use `--in-reply-to` to thread it under the draft so the context is clear.

See *Getting out of jams* for general advice on undoing local history mistakes before they reach a mailing list.

See also

- *format-patch* — generates the `.patch` files that `send-email` delivers.
- *am* — the receiving end: applies mailed patches to a repository.
- *commit* — `--signoff` adds the Signed-off-by trailer that mailing-list projects require.
- *Getting out of jams* — recovering from history mistakes before sending.

request-pull

Generate a formatted summary asking an upstream project to pull your changes.

Mental model

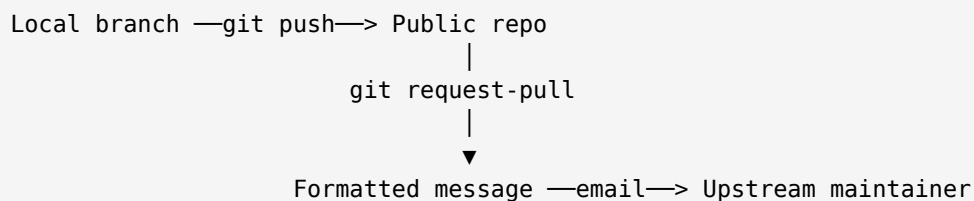
`git request-pull` is an email-first workflow tool that predates GitHub pull requests by years. The Linux kernel, Git itself, and many other projects that operate over mailing lists still use it today.

The mental model is simple: you have pushed your work to a public repository that the upstream maintainer can reach. You now need to send them a message that says “starting from commit X, please pull everything up to commit Y from this URL”. `git request-pull` writes that message for you. It prints to standard output — you then paste or pipe it into your email client.

The output contains three parts:

1. The branch description (from `git branch --edit-description`, if set).
2. A `git log`-style summary of the commits between `<start>` and `<end>`.
3. The URL and ref the upstream should pull from.

Git verifies that the `<end>` commit is actually reachable at the given URL and emits a warning if the two do not match. This catches the common mistake of generating a pull request before the push has completed.



Synopsis

```
git request-pull [-p] <start> <URL> [<end>]
```

Everyday usage

The typical workflow: you have branched from a tag or commit that the upstream already has, done some work, pushed it to your public fork, and now want to ask the upstream to pull.

Generate a pull request message from tag `v2.3` to your current HEAD:

```
git request-pull v2.3 https://github.com/you/project
```

Send the output directly to your email client (here, `mutt`):

```
git request-pull v2.3 https://github.com/you/project | mutt -s "Pull request: add TLS 1.3 support" maintainer@example.org
```

Include the full patch text in the message body (useful for small changes where you want the maintainer to review inline without fetching):

```
git request-pull -p v2.3 https://github.com/you/project
```

When your local branch name differs from the remote branch name — for example you pushed `feature/tls` as `for-upstream` — use the `<local>:<remote>` syntax for `<end>`:

```
git push origin feature/tls:for-upstream
git request-pull v2.3 https://github.com/you/project feature/tls:for-upstream
```

Key options

Option	What it does	When to use it
<code>-p</code>	Include patch text in the output	Small patches you want reviewable inline, or projects that prefer patch-in-email
<code><start></code>	The commit already in upstream history to begin the summary from	Always required; typically a tag (<code>v1.0</code>) or a known shared commit
<code><URL></code>	The repository URL the upstream should fetch from	Always required; must be publicly reachable by the maintainer
<code><end></code>	The commit to end the summary at (defaults to <code>HEAD</code>); accepts <code><local>:<remote></code>	When your local ref name differs from the pushed ref name

Best practices

Push before you run `git request-pull`. The command checks whether the tip commit is reachable at the given URL. If it is not, Git warns you. Always verify the push succeeded before generating the message.

Use a tag or a well-known shared commit as `<start>`. The `<start>` argument names a commit that the upstream already has. Using a release tag (`v1.0` , `v2.3.1`) is unambiguous. Using a branch name from the upstream (such as `origin/main`) works too, but the resolved commit may advance between your push and the maintainer’s pull, making the log summary incomplete.

Set a branch description to give context. Maintainers appreciate a short paragraph explaining what a series does before they read the commit list. Set one with:

```
git branch --edit-description
```

`git request-pull` includes that description at the top of the output automatically.

Use the `<local>:<remote>` form when you rename on push. Kernel contributors routinely push a topic branch under a different remote name (`master:fixes-for-v6.9`). Without the colon form, Git generates the wrong fetch refspec in the output and the maintainer’s pull will fail.

Redirect output to a file for review before sending. Long patch series benefit from a quick sanity check:

```
git request-pull v2.3 https://github.com/you/project > /tmp/pull-request.txt
less /tmp/pull-request.txt
```

Pitfalls & gotchas

Warning: “No match for commit ... found at ...” Git could not find your `<end>` commit at the given URL. The most common cause is forgetting to push, or pushing to a different remote than the one in the URL. Re-push and regenerate.

`<start>` must be in upstream history, not just your local history. If you pass a commit that the upstream does not have, the log summary covers more than you intend and the maintainer

cannot identify where to start the merge. Always anchor to a shared tag or the upstream's branch tip.

The generated URL and ref must be fetchable by the maintainer. A URL on `localhost` or an SSH remote that only you can reach is useless. Use an HTTPS or public SSH URL for a hosting service (GitHub, GitLab, kernel.org) that the maintainer can actually clone from.

-p can produce very large output. For a series of dozens of commits, the patch output can run to thousands of lines. Many mailing lists reject messages above a certain size. For large series, omit `-p` and let the maintainer fetch; reserve `-p` for single-patch submissions.

git request-pull does not send email. It only prints to stdout. Piping directly into `git send-email` does not work the way `format-patch` does — use your regular mail client or a script to deliver the output.

Worked examples

Submitting a patch series to a mailing list

You maintain a fork of an open-source library. Upstream tags releases. You have three commits built on top of `v4.1.0` that add WebSocket support, and you have pushed them to your public fork.

```
# Push the work
git push origin websocket

# Verify the branch is live at the URL
git ls-remote https://github.com/you/library websocket
# 3a8f9c1 refs/heads/websocket

# Generate the pull-request message
git request-pull v4.1.0 https://github.com/you/library websocket
```

The output looks like:

```
The following changes since commit d4e7a12... (v4.1.0):

    Release 4.1.0 (2026-03-10 14:22:01 +0000)

are available in the git repository at:

    https://github.com/you/library websocket

for you to fetch changes up to 3a8f9c1...:

    Add WebSocket handshake negotiation (2026-06-01 09:15:00 +0100)

-----
Alice Smith (3):
    Add WebSocket frame parser
    Add WebSocket handshake negotiation
    Add integration tests for WebSocket echo server

lib/websocket.c      | 182 +++++
lib/websocket.h      |  38 +++++
tests/websocket_test.c |  94 +++++
3 files changed, 314 insertions(+)
```

Copy this output, add a brief cover note at the top, and send to the project mailing list.

Pushing under a different remote branch name

The upstream project asks contributors to push to a branch named `for-<maintainer>`. Your local branch is `fix/null-deref`.

```
# Push local branch under the name the project expects
git push origin fix/null-deref:for-torvalds

# Generate the message with the local:remote form for <end>
git request-pull v6.9 https://git.kernel.org/pub/scm/linux/kernel/git/you/linux
fix/null-deref:for-torvalds
```

Git uses `fix/null-deref` to identify the local tip commit for the log summary, and emits `for-torvalds` as the ref name the maintainer should fetch. Without the colon form, Git would look for a ref named `fix/null-deref` on the remote and fail to match.

Including the patch for a single-commit fix

For a one-liner documentation fix, some maintainers prefer to receive the patch inline rather than fetching a whole branch.

```
git push origin fix-typo
git request-pull -p v3.0 https://github.com/you/docs fix-typo
```

The `-p` flag appends the full patch text after the summary, giving the maintainer everything they need to review and apply without cloning.

Recovery

`git request-pull` is read-only: it never modifies the repository. If the generated output is wrong, simply re-run with corrected arguments.

If you sent a pull request message with the wrong URL or the wrong `<start>` commit, send a follow-up email to the mailing list noting the correction and include the corrected `git request-pull` output.

If the upstream has already attempted a pull and gotten nothing (because you had not pushed yet), push your branch and ask them to retry with `git fetch` followed by `git merge FETCH_HEAD`, or resend the corrected request.

See *Getting out of jams* for help undoing any local commits before the push stage.

See also

- *format-patch* — create patch files suitable for `git am` or mailing list submission when the upstream does not want a fetch URL.
- *send-email* — send `format-patch` output directly from the command line.
- *am* — how the upstream applies a patch series received by email.
- *push* — the required step before `git request-pull` to make your commits publicly reachable.
- *Getting out of jams* — undoing local commits before they are pushed.

gc

Clean up unnecessary files and optimize the local repository by packing loose objects, pruning unreachable history, and updating ancillary indexes.

Mental model

Every time you commit, amend, rebase, or stash, Git writes new objects to the repository's object database — blobs for file contents, trees for directories, and commit objects tying them together. Some of those objects eventually become unreachable: the dangling commit left behind by an amend, the blob that was staged but never committed, the stash entry you dropped. They take up disk space and slow object lookups.

Git also stores objects in two ways. Small, recently created objects live as individual files under `.git/objects/` — one file per object, named by its SHA-1 hash (loose objects). Over time, `git gc` gathers loose objects into a single binary file called a packfile, which applies delta compression between similar objects and is far more space-efficient and faster to read.

Think of `git gc` as a janitor that does four things in one pass:

1. **Repacks** loose objects into a packfile, delta-compressing the contents.
2. **Prunes** unreachable loose objects that are older than the grace period (default: two weeks).
3. **Packs refs** from individual files under `.git/refs/` into `.git/packed-refs`.
4. **Updates ancillary indexes** — the commit-graph, the reflog expiry, and rerere metadata.

You rarely need to run `git gc` by hand. Porcelain commands (`git commit`, `git fetch`, `git pull`) call `git gc --auto` automatically once the number of loose objects or packs crosses a configurable threshold.

```
.git/objects/xx/yyyy...  (loose objects, one per SHA)
      |
      └─ git gc -> .git/objects/pack/pack-<sha>.{idx,pack}
                           (delta-compressed packfile)
```

Unreachable objects older than `gc.pruneExpire` (default 2 weeks) → deleted

Synopsis

```
git gc [--aggressive] [--auto] [--[no-]detach] [--quiet]
      [--prune=<date> | --no-prune] [--force] [--keep-largest-pack]
```

Everyday usage

Run a standard housekeeping pass:

```
git gc
```

Output looks like:

```
Enumerating objects: 4021, done.
Counting objects: 100% (4021/4021), done.
Delta compression using up to 10 threads
Compressing objects: 100% (1387/1387), done.
Writing objects: 100% (4021/4021), done.
Total 4021 (delta 2614), reused 4021 (delta 2614), pack-reused 0
```

Check whether housekeeping is needed before doing any work (the same check automatic gc performs):

```
git gc --auto
```

Prune all unreachable objects immediately rather than waiting for the default two-week grace period:

```
git gc --prune=now
```

Suppress progress output in scripts or cron jobs:

```
git gc --quiet
```

Run in the foreground and wait for completion, even if background detach is configured:

```
git gc --no-detach
```

Key options

Option	What it does	When to use it
<code>--aggressive</code>	Repack with <code>--no-reuse-delta</code> , larger window/depth; takes much longer	One-off optimization of a repo with a history of many force-pushes or filter-branch runs
<code>--auto</code>	Exit immediately if housekeeping thresholds are not exceeded	Called by porcelain commands; rarely useful interactively
<code>--[no-]detach</code>	Override the <code>gc.autoDetach</code> config to run in foreground or background	Use <code>--no-detach</code> in scripts that must wait for gc to finish
<code>--prune=<date></code>	Delete unreachable objects older than <code><date></code> <code>--prune=now</code> deletes all	Safe on a single-user repo; risky when other processes may be writing
<code>--no-prune</code>	Skip deletion of unreachable loose objects entirely	Skip pruning when you want to inspect unreachable objects first with <code>git fsck --unreachable</code>
<code>--force</code>	Run even if another <code>git gc</code> may already be running	Use with care; two concurrent gc runs can corrupt packs
<code>--keep-largest-pack</code>	Consolidate all packs except the largest non-cruft pack, any <code>.keep</code> -marked packs, and cruft packs	Reduce pack count while preserving the pack with the most data
<code>--quiet</code>	Suppress progress output	CI, cron, and scripts where noise is unwanted

Best practices

Let automatic gc do its job. The default `gc.auto` threshold (6700 loose objects) and `gc.autoPackLimit` threshold (50 packs) are well-tuned for most workflows. Manual runs are needed only after bulk imports, after heavy use of `git filter-branch`, or when you want to reclaim space on a constrained machine.

Do not use `--prune=now` on shared repositories. The two-week grace period exists because `git gc` can run concurrently with another process that has written objects but has not yet created references to them. Pruning those objects immediately can corrupt the repository for the other process.

Treat `--aggressive` as a one-off, not a habit. It throws away all existing deltas and recomputes them from scratch using a larger window (250 by default vs. 10 normally). The improvement is mostly persistent, so running it once after a major history rewrite is worthwhile; running it repeatedly on a healthy repo wastes time without meaningful gain.

Schedule `git gc --quiet` in maintenance windows for server-side repos. Hosting platforms run their own maintenance, but self-hosted bare repositories benefit from a weekly cron job. Pair `--quiet` with output redirection so failures are noticed without noise on success. Consider the newer `git maintenance` command (see the *maintenance* chapter) for finer-grained scheduling.

Use `gc.bigPackThreshold` to protect large packs from being repacked. Set it to the size of your baseline pack to avoid unpacking and repacking hundreds of megabytes every time small changesets arrive:

```
git config gc.bigPackThreshold 200m
```

Pitfalls & gotchas

`git gc` will not delete objects still referenced anywhere. References include not just branches and tags but also the index, remote-tracking branches, reflogs, and everything under `refs/*`. If you expect an object to be deleted but it persists, check whether a reflog, stash, or remote-tracking branch still points to it.

The two-week grace period can mask expected pruning. After a rebase or amend, the old commits remain as loose objects until `gc.pruneExpire` elapses. Running `git gc` the day after does not remove them. Run `git gc --prune=now` if you explicitly want them gone immediately (single-user repos only).

Background detach by default can hide failures. With `gc.autoDetach=true` (the default), `git gc --auto` returns immediately. Errors are written to `.git/gc.log` and suppressed for `gc.logExpiry` (default: one day) to avoid spamming every subsequent command. Check `.git/gc.log` if you suspect `gc` is failing silently.

`--force` is not safe to use routinely. If a `gc` run is truly in progress in another process (not just a stale lock file), forcing a second concurrent run can leave packs in an inconsistent state. Verify the other process is dead before using `--force`.

`--aggressive` may produce larger packfiles in some cases. The repack discards existing deltas and recomputes them. Occasionally the new computation finds a worse set of deltas for certain object graphs, resulting in a slightly larger pack. This is acknowledged in the documentation and is an accepted trade-off.

Worked examples

Reclaiming space after rewriting history

You have just run `git filter-branch` (or `git filter-repo`) to remove a large binary that was accidentally committed. The rewrite created thousands of new objects and left the old ones dangling.

```
# Confirm there are many loose objects before gc
git count-objects -v
```

```
count: 8342
size: 94832
in-pack: 0
packs: 1
size-pack: 12451
prune-packable: 0
garbage: 0
size-garbage: 0
```

```
# Remove the old refs that filter-branch left in refs/original/
git for-each-ref --format="%(refname)" refs/original/ \
  | xargs -r git update-ref -d

# Expire all reflog entries that still reference the old objects
git reflog expire --expire=now --all

# Prune immediately and repack aggressively
git gc --aggressive --prune=now
```

```
# Verify the reduction
git count-objects -v
```

```
count: 0
size: 0
in-pack: 3105
packs: 1
size-pack: 4218
prune-packable: 0
garbage: 0
size-garbage: 0
```

The repository has shrunk from 95 MB of loose objects plus a 12 MB pack down to a single 4 MB pack.

Running gc in a CI maintenance job

A self-hosted bare repository is used as the central remote for a team. You want a weekly cron job that compacts the repository without interrupting pushes or fetches.

```
# /etc/cron.weekly/git-gc-repos
#!/bin/sh
set -e
REPOS_ROOT=/srv/git
LOG=/var/log/git-gc.log

find "$REPOS_ROOT" -name HEAD -execdir \
  git gc --quiet --no-detach \; >> "$LOG" 2>&1
```

`--no-detach` ensures the script waits for gc to finish before the cron job exits, so the log captures the full output and errors surface immediately. `--quiet` suppresses normal progress output so only errors appear in the log.

Inspecting what would be pruned

Before committing to a destructive `--prune=now`, audit the unreachable objects:

```
# List all unreachable objects
git fsck --unreachable 2>/dev/null | grep "^unreachable commit" | awk '{print $3}'
```

Pipe one of those hashes to `git show` to confirm it is safe to remove:

```
git show <hash>
```

If the output shows a commit or file you no longer need, proceed with:

```
git gc --prune=now
```

See the *fsck* chapter for a deeper look at inspecting repository integrity.

Recovery

`git gc` deletes objects permanently once they have been pruned. If you ran `--prune=now` and removed commits you still needed, recovery depends on whether those commits are recorded in the reflog.

Check the reflog first:

```
git reflog
```

If the commit hash appears there, restore the branch:

```
git branch recover-branch <hash>
```

If the reflog does not contain it (for example, the reflog was also expired), and you did not push the branch anywhere, the objects are unrecoverable. This is why the two-week default grace period exists and why `--prune=now` should be used only when you are certain.

See *Getting out of jams* for reflog-based recovery recipes.

See also

- *fsck* — verify repository integrity and list unreachable objects before pruning.
- *maintenance* — finer-grained, scheduled alternative to `git gc` for busy repositories.
- *reflog* — the reflog is the last safety net before pruned objects are unrecoverable.
- *rebase* — creates unreachable commits as a side effect; gc cleans them up after the grace period.
- *Getting out of jams* — recovering commits that have been pruned or amended away.

fsck

Verify the integrity and connectivity of every object in the repository's object database.

Mental model

Git stores everything — commits, trees, blobs, tags — as content-addressed objects: the file name is the SHA-1 hash of the content. That design means a single bit-flip produces a detectable mismatch: the stored hash no longer matches the actual content.

`git fsck` exploits this property in two passes:

1. **Validity pass.** For each object it reads, `fsck` recomputes the hash and checks the internal structure (valid author line, parseable tree entries, well-formed tag, etc.). A mismatch or a malformed structure is reported as corruption.
2. **Connectivity pass.** Starting from every known root — branch tips, tags, the index, and reflogs — `fsck` walks the object graph and flags any object that is referenced but absent (*missing*), or any object that is present but reachable from nowhere (*dangling* or *unreachable*).

```
refs/heads/*, tags/*, index, reflogs
|
└─ walk object graph ──> flag missing objects
                        flag dangling/unreachable objects
                        verify hash of every object visited
```

Dangling objects are not corruption. They are the normal residue of deleted branches, amended commits, reset heads, and rebases. Git's garbage collector (`git gc`) eventually prunes them, but only after the configured grace period. `fsck` surfaces them so you can inspect or recover them before they disappear.

Synopsis

```
git fsck [--tags] [--root] [--unreachable] [--cache] [--no-reflogs]
        [--[no-]full] [--strict] [--verbose] [--lost-found]
        [--[no-]dangling] [--[no-]progress] [--connectivity-only]
        [--[no-]name-objects] [--[no-]references] [<object>...]
```

Everyday usage

Run a full integrity check (the default — checks all refs, the index, and reflogs):

```
git fsck
```

Show every unreachable object along with dangling ones — useful when hunting for a dropped commit:

```
git fsck --unreachable
```

List dangling objects with human-readable names that describe how each object was referenced:

```
git fsck --dangling --name-objects
```

Write dangling blobs and commits to `.git/lost-found/` so you can inspect their content:

```
git fsck --lost-found
ls .git/lost-found/commit/    # each filename is a dangling commit SHA-1
ls .git/lost-found/other/    # dangling blobs and trees
```

Suppress dangling-object noise when you only care about hard errors:

```
git fsck --no-dangling
```

Skip the reflog scan (finds commits that used to be in a branch but were later removed and are no longer referenced by any current ref):

```
git fsck --no-reflogs
```

Key options

Option	What it does	When to use it
<code>--unreachable</code>	Print objects that exist but are not reachable from any ref	Audit leftover objects after branch deletions
<code>--[no-]dangling</code>	Print or suppress objects present but never directly used (default: on)	Suppress noise when hunting only hard errors
<code>--lost-found</code>	Write dangling objects to <code>.git/lost-found/</code>	Recover content from dropped commits or stashes
<code>--name-objects</code>	Include a rev-parse-compatible name alongside each SHA-1	Make unreachable output human-readable
<code>--connectivity-only</code>	Check only that referenced objects exist; skip deep content validation	Faster check on large repos; will miss blob corruption
<code>--no-full</code>	Restrict check to the main object directory only	Exclude alternate object pools (and their pack subdirectories) from the walk
<code>--strict</code>	Catch file modes with the group-write bit set, created by old Git versions	New projects; avoid on established repos with legacy objects
<code>--no-reflogs</code>	Exclude reflogs from the set of reachability roots	Find commits that have fallen out of all current refs even if still referenced by a reflog
<code>--root</code>	Report root commit nodes	Auditing repos that should have exactly one root
<code>--tags</code>	Report tag objects	Auditing tag integrity
<code>--cache</code>	Treat index entries as additional reachability roots	Include staged-but-uncommitted objects in the walk
<code>--verbose</code>	Print every object examined	Diagnosing where in the object graph a walk stalls
<code>--[no-]progress</code>	Force or suppress the progress indicator on stderr	Use <code>--no-progress</code> in scripts; <code>--progress</code> in TTY-less CI

<code>--[no-]references</code>	Control whether to verify the refs database via <code>git refs verify</code> (default: on)	Disable when only checking object integrity, not ref consistency
--------------------------------	--	--

Best practices

Run `git fsck` before and after risky operations on critical repositories. Before a destructive rebase or a `filter-repo` run, a clean `fsck` output confirms the starting state is sound. After the operation, a second run confirms nothing was corrupted in transit.

Pipe output through `grep` to separate signal from noise. A healthy repo will report dozens of dangling objects — normal residue from everyday work. Restrict attention to hard problems:

```
git fsck 2>&1 | grep -v "^dangling"
```

Use `--lost-found` as the first recovery step, not a last resort. When you realize a stash drop, a reset, or a force-push discarded work, run `--lost-found` immediately — before `git gc` runs and permanently removes the objects. The default grace period is two weeks, but shared server configurations may be shorter.

Automate a periodic `fsck` on bare server repositories. Disk errors and filesystem bugs are silent until they compound. A weekly cron job running `git fsck --no-dangling` on every bare clone provides an early-warning system that catches corruption before it spreads to every developer's clone.

Prefer `fsck.skipList` over silencing entire message classes. If a legacy repository has a handful of known-bad objects that pre-date your ownership, list their SHA-1s in `fsck.skipList` rather than setting `fsck.<msg-id> = ignore`. The skip list is surgical; a message-class ignore hides all future instances of that problem class silently.

Pitfalls & gotchas

Dangling objects are normal — do not panic. Every `git commit --amend`, `git rebase`, `git reset`, and dropped stash leaves dangling objects. A long list of `dangling commit` lines does not indicate corruption; it means Git's safety net is doing its job. Focus on `missing` and `hash mismatch` lines.

`--connectivity-only` misses blob corruption. The fast path verifies that referenced blobs exist but does not read and re-hash their contents. A silently bit-flipped blob will pass

`--connectivity-only` and fail a full scan. Use the full scan for integrity audits; reserve

`--connectivity-only` for quick reachability checks on very large repositories.

`fsck` cannot repair what it finds. When `fsck` reports a missing or corrupt object, the only remedy is to restore the object from a backup, a remote clone, or another replica. The object database has no built-in redundancy: one corrupt loose file or one flipped bit in a pack file is unrecoverable from the local repo alone.

Reflogs extend reachability — `--no-reflogs` changes what “dangling” means. With reflogs active (the default), a commit that was `HEAD` two weeks ago is still reachable and will not be reported as dangling. Pass `--no-reflogs` when you specifically want to find commits that are no longer in any reflog — useful for auditing what `git gc --prune=now` would remove.

`--strict` produces false positives on established repositories. The group-write file-mode check targets modes created by very old Git versions. Running `--strict` on repositories such as

the Linux kernel or Git itself produces a flood of warnings about objects that are not actually harmful. Use `--strict` only on repositories created with a recent Git version.

A **hash mismatch** is a hardware or filesystem problem, not a Git bug. SHA-1 hash mismatches almost always trace to disk corruption, a failing drive, a bad RAM stick, or a misbehaving network filesystem. Fix the underlying hardware issue before attempting recovery; otherwise replacement objects may also corrupt.

Worked examples

Recovering a dropped stash

You ran `git stash drop` on the wrong entry. The commit is gone from `git stash list` but is still a dangling object inside the object database.

Write all dangling objects to `.git/lost-found/`:

```
git fsck --lost-found
```

```
Checking object directories: 100% (256/256), done.
dangling commit 7f3a1b2c...
dangling commit d4e9f012...
```

Inspect each dangling commit to find the stash:

```
for sha in $(ls .git/lost-found/commit/); do
  echo "=== $sha ==="
  git log -1 --oneline "$sha"
done
```

Once you identify the correct SHA-1, apply the stash content or branch from it:

```
# Apply the changes directly
git stash apply d4e9f012

# Or create a branch to review the diff first
git checkout -b recovered-stash d4e9f012
```

Diagnosing a suspected corrupt repository

A colleague reports that `git log` occasionally exits with a “bad object” error on the shared bare repository. Run a full check, suppressing dangling-object noise:

```
git fsck --no-dangling 2>&1
```

```
Checking object directories: 100% (256/256), done.
Checking connectivity: done.
missing blob alb2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2
```

A **missing blob** means a file’s content is referenced by a tree but the object is absent. Identify which path points to the missing object:

```
git ls-tree -r --full-tree HEAD | grep alb2c3d4
```

```
100644 blob a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2 src/payment/retry.js
```

Restore the object from a healthy clone:

```
# On the healthy clone – extract the blob
git cat-file blob a1b2c3d4e5f6a1b2c3d4e5f6a1b2c3d4e5f6a1b2 > /tmp/retry.js

# Back on the corrupt repo – re-inject the object
git hash-object -w /tmp/retry.js
```

Confirm the fix with another fsck run:

```
git fsck --no-dangling
# Checking object directories: 100% (256/256), done.
# Checking connectivity: done.
```

Finding orphaned commits after a force-push

A collaborator force-pushed and overwrote three commits on `main`. The commits are gone from the branch but may still exist as dangling objects.

Run fsck without reflogs to list objects that are truly unreachable (not just reflog-only reachable):

```
git fsck --unreachable --no-reflogs 2>&1 | grep "^unreachable commit"
```

```
unreachable commit 9c3f1a2b...
unreachable commit 5d7e8f90...
unreachable commit 1a2b3c4d...
```

Inspect the chain to confirm these are the lost commits:

```
git log --oneline 9c3f1a2b
```

```
9c3f1a2b Add payment gateway retry logic
5d7e8f90 Wire up retry config to environment
1a2b3c4d Add integration test for payment retries
```

Restore the work as a branch:

```
git branch recovered-payment-retries 9c3f1a2b
git push origin recovered-payment-retries
```

Recovery

`git fsck` is strictly read-only — it never modifies the repository. There is nothing to undo after running it.

If fsck reports **dangling objects** you want to keep, act before `git gc` removes them. Use `--lost-found` to extract them, then create a branch or re-apply them as a stash:

```
git fsck --lost-found
git checkout -b rescue-branch <dangling-commit-sha>
```


If fsck reports **missing or corrupt objects**, recovery requires an external source — a remote clone, a backup, or a bundle. See *Getting out of jams* for step-by-step recipes covering how to rehydrate a corrupt repository from a remote and how to recover individual objects from a bundle.

See also

- *Getting out of jams* — step-by-step recovery for corrupt repositories and lost commits.
- *stash* — `--lost-found` is the primary recovery tool when a stash entry is accidentally dropped.
- *reflog* — the first place to look for lost commits before reaching for fsck; reflogs cover the common cases faster and without filesystem side-effects.
- *gc* — the garbage collector that eventually prunes the dangling objects fsck surfaces; understand the grace-period settings before running `gc --prune=now`.
- *rebase* — heavy rebases leave many dangling objects; knowing that helps interpret fsck output after a rebase session.
- *bundle* — a portable object archive that can supply missing objects to a corrupt repository.

maintenance

Run background housekeeping tasks to keep a repository fast and lean over time.

Mental model

Every time you run `git add`, `git fetch`, or `git commit`, Git prioritizes responsiveness over tidiness. Loose objects accumulate. Pack files fragment. The commit-graph — a precomputed index that makes `git log` and reachability checks fast — goes stale. Remote-tracking branches fall behind without a full fetch.

`git maintenance` is the scheduled janitor. It runs a configurable set of *tasks*, each targeting one aspect of repository health, without interrupting your normal workflow. Because each run takes a lock on the object database, tasks cannot race against each other or corrupt in-flight operations.

Think of it in two modes:

- **On-demand:** `git maintenance run` — you trigger it explicitly, choosing which tasks to run.
- **Background schedule:** `git maintenance start` — Git registers the repository with your OS scheduler (launchctl on macOS, systemd timers on Linux, cron as a fallback, schtasks on Windows) and runs `git maintenance run --schedule` hourly, daily, and weekly without any further action on your part.

```
git maintenance start
  |
  ▼
OS scheduler (launchctl / systemd / crontab / schtasks)
  | hourly → commit-graph, prefetch
  | daily  → loose-objects, incremental-repack
  └ weekly → pack-refs
```

The `register` and `unregister` subcommands manage which repositories appear in the global `maintenance.repo` list without touching the OS scheduler.

Synopsis

```
git maintenance run [--auto] [--schedule] [--quiet] [--task=<task>...]
git maintenance start [--scheduler=auto|crontab|systemd-timer|launchctl|schtasks]
git maintenance stop
git maintenance register [--config-file=<file>]
git maintenance unregister [--force] [--config-file=<file>]
```

Everyday usage

Register the current repository and start the background schedule:

```
git maintenance start
```

Run all currently-enabled tasks immediately, in the foreground:

```
git maintenance run
```

Run a specific task without changing what is persistently enabled:

```
git maintenance run --task=commit-graph
git maintenance run --task=loose-objects
```

Run multiple tasks in one pass (executed in the order listed):

```
git maintenance run --task=loose-objects --task=incremental-repack
```

Stop background maintenance (leaves the repository registered):

```
git maintenance stop
```

Remove the current repository from the global maintenance list entirely:

```
git maintenance unregister
```

Check which tasks the incremental strategy will run and when — look at the global config after `git maintenance start`:

```
git config --global --list | grep maintenance
```

Key options

Option	What it does	When to use it
<code>--task=<task></code>	Run only the named task(s), in that order	Target one area without touching the rest
<code>--auto</code>	Run tasks only when their threshold is exceeded (e.g. loose-object count, pack-file count)	Light-touch check after commands that add data
<code>--schedule[=<frequency>]</code>	Run tasks only if their scheduled interval has elapsed since last run	Used internally by the OS scheduler (e.g. <code>--schedule=hourly</code>); also accepts bare <code>--schedule</code>
<code>--quiet</code>	Suppress progress and informational output on stderr	Cron jobs and scripts where noise pollutes logs
<code>--scheduler=<value></code>	Choose the OS scheduler backend: <code>auto</code> , <code>crontab</code> , <code>systemd-timer</code> , <code>launchctl</code> , <code>schtasks</code>	Force a specific scheduler when <code>auto</code> picks the wrong one

Best practices

Use `git maintenance start` on every repository you work in regularly. One command enables the full incremental strategy: `commit-graph` and `prefetch` run hourly, `loose-objects` and `incremental-repack` run daily, `pack-refs` runs weekly. You pay nothing at commit or fetch time; the cost moves to the background.

Prefer the incremental strategy over `git gc` for active repositories. The `gc` task repacks everything into a single pack file. On a large repository this can take minutes and holds the object-

database lock for the entire duration, blocking concurrent operations. The `incremental-repack` task does the same work in small batches using the multi-pack-index, which finishes quickly and releases the lock sooner. See the `gc` chapter for cases where a full `git gc` is still appropriate.

Do not mix `git gc` with `git maintenance run`. The `git gc` command does not take the same lock as `git maintenance run`. Running both concurrently can leave the object database in an inconsistent state. If you need garbage collection in a maintenance context, use `git maintenance run --task=gc` instead.

Set `maintenance.strategy = incremental` explicitly on shared developer machines. When `git maintenance register` runs, it sets this value automatically. On machines where you manage git config centrally, set it in the global config to ensure every registered repository inherits a sensible schedule without further per-repo configuration.

```
git config --global maintenance.strategy incremental
```

Tune batch sizes for very large repositories. The `loose-objects` task defaults to processing fifty thousand objects per run. If your repository grows loose objects faster than that, increase the batch size — or set it to zero to remove the limit and process all of them in one pass:

```
git config maintenance.loose-objects.batchSize 0
```

Verify the scheduler is running after `git maintenance start`. On macOS, check that the launch agents are registered:

```
ls ~/Library/LaunchAgents/org.git-scm.git*
```

On Linux with systemd:

```
systemctl --user list-timers | grep git-maintenance
```

Pitfalls & gotchas

unregister does not stop the background scheduler. It removes the repository from the `maintenance.repo` list, so future scheduled runs skip it. But if a run is already in progress it will complete. Call `git maintenance stop` first if you want an immediate halt.

stop does not unregister the repository. After `git maintenance stop`, the current repository is still in `maintenance.repo`. A future `git maintenance start` will pick it up again. This is intentional — stop means “pause”, not “forget”.

The object-database lock causes skipped windows on busy machines. Each `git maintenance run` takes the lock for its entire duration. If a previous run has not finished when the scheduler fires again, the new run exits without doing any work. On machines with many large registered repositories, consider moving expensive tasks (especially `gc`) to weekly or disabling them in favour of `incremental-repack`.

`maintenance.auto = false` persists after `unregister`. When you run `git maintenance register`, it sets `maintenance.auto = false` in the repository config to prevent double-maintenance (automatic + scheduled). That setting stays after `git maintenance unregister`. If you want foreground auto-maintenance back, re-enable it manually:

```
git config maintenance.auto true
```

Prefetch stores refs under `refs/prefetch/`, not under `refs/remotes/`. The `prefetch` task deliberately avoids updating your remote-tracking branches so that a `git fetch` you run interactively appears to complete instantly. But your remote-tracking branches (`origin/main`, etc.) do not reflect the prefetched state. Run a regular `git fetch` when you need those refs updated.

Credentials must be available for prefetch to work. The `prefetch` task calls `git fetch` in a non-interactive context (the OS scheduler). If your remote requires interactive authentication (e.g. a browser OAuth flow), the fetch will silently fail. Use SSH keys or a credential helper that stores tokens persistently.

On macOS, do not use `crontab` for maintenance. The cron environment on macOS does not have a full user context, so credential helpers cannot access the system keychain.

`git maintenance start` uses `launchctl` by default on macOS for exactly this reason. If you override with `--scheduler=crontab`, `prefetch` will fail silently for remotes that require stored credentials.

Worked examples

Enabling background maintenance on a new clone

You have just cloned a large monorepo and want ongoing performance optimisation without any manual intervention.

```
cd ~/projects/monorepo
git maintenance start
```

```
$ git config --global --list | grep maintenance
maintenance.repo=/Users/alice/projects/monorepo
maintenance.strategy=incremental
```

From this point on, the OS scheduler runs `git maintenance run --schedule=hourly` every hour. At midnight it also runs daily tasks; on Sunday midnight it runs weekly tasks. Your `git log`, `git fetch`, and `git status` commands will gradually get faster as the commit-graph and pack structure are kept up to date.

Running specific tasks manually before a release

You are about to tag a release and want the repository in peak shape. You decide to pack loose objects and regenerate the commit-graph, but skip a full `gc` to avoid the long lock time.

```
git maintenance run --task=loose-objects --task=incremental-repack --task=commit-graph
```

Tasks run in the order specified. When the command returns, loose objects are consolidated into a new pack file, the multi-pack-index is updated, and the commit-graph reflects all commits on every reachable branch.

Verify the commit-graph was written:

```
ls .git/objects/info/commit-graphs/
```

```
commit-graph-chain graph-abc123.graph graph-def456.graph
```

Migrating from a cron-based `git gc` to managed maintenance

Your team runs `git gc` nightly via a custom cron job. You want to replace that with `git maintenance` to gain the lock-safety and incremental approach.

1. Remove the old cron entry:

```
crontab -e  
# delete the line that calls git gc
```

1. Register each repository and enable the incremental strategy:

```
cd ~/projects/service-a && git maintenance start  
cd ~/projects/service-b && git maintenance start
```

1. Confirm all repositories are registered:

```
git config --global --get-all maintenance.repo
```

```
/Users/ops/projects/service-a  
/Users/ops/projects/service-b
```

1. Disable the `gc` task explicitly (it is off by default in the incremental strategy, but make it explicit for documentation purposes):

```
git config --global maintenance.gc.enabled false
```

The incremental-repack and loose-objects tasks now handle what `git gc` used to do, spread across daily runs with no long lock windows.

Inspecting and adjusting the schedule

You want the commit-graph rebuilt daily instead of hourly on a repository that receives only a few commits per day.

```
git config maintenance.commit-graph.schedule daily
```

On the next hourly scheduler invocation, `git maintenance run --schedule=hourly` checks whether `commit-graph` is configured for `hourly` — it is not — and skips it. The nightly daily run picks it up instead.

To see the full current schedule for the repository:

```
git config --list | grep 'maintenance\.'
```

Recovery

If `git maintenance run` hangs or leaves a stale lock, check for a lock file in the object directory:

```
ls .git/objects/pack/*.lock
```

If no `git` process is running against this repository, remove the stale lock manually:

```
rm .git/objects/pack/*.lock
```

If the commit-graph becomes corrupt, delete it and let the next maintenance run rebuild it:

```
rm -rf .git/objects/info/commit-graphs/  
git maintenance run --task=commit-graph
```

If background maintenance stops firing on macOS, re-register the launch agents:

```
git maintenance stop  
git maintenance start
```

If you accidentally unregistered a repository, add it back without restarting the full scheduler:

```
git maintenance register
```

See *Getting out of jams* for help with object-database corruption and pack-file recovery.

See also

- `gc` — full garbage collection; the heavier alternative to incremental maintenance.
- `fsck` — verify object integrity; use after suspecting corruption that maintenance alone cannot fix.
- `reflog` — understanding what `reflog-expire` removes during maintenance.
- `rerere` — what the `rerere-gc` task cleans up.
- `worktree` — what the `worktree-prune` task removes.
- `fetch` — the underlying command the `prefetch` task drives.
- *Getting out of jams* — recovering from stale locks and object corruption.

archive

Package any tree, commit, or tag from your repository into a tar or zip archive — without checking anything out.

Mental model

Every commit in git points to a tree object: a complete snapshot of every tracked file at that point in time. `git archive` walks that tree and writes the files directly into a tar or zip container. No working tree is created, no index is modified, and no temporary files are scattered around your disk.

The key distinction that trips people up is the difference between giving `git archive` a **tree ID** versus a **commit or tag ID**:

- A tree ID carries no timestamp, so each file in the archive gets the current wall-clock time as its modification time.
- A commit or tag ID carries the commit timestamp, so every file gets that time — which means archives of the same commit are bit-for-bit reproducible regardless of when you run the command.

For release tarballs, always use a tag or commit, not a bare tree hash.

tag / commit	→	tree	→	git archive	→	.tar.gz / .zip
(timestamp		(file				(reproducible
embedded)		snapshots)				by default)

Git embeds the commit ID in a global extended pax header when producing tar output, and as a file comment in zip output. You can recover it later with `git get-tar-commit-id`.

Synopsis

```
git archive [--format=<fmt>] [--list] [--prefix=<prefix>/]
           [-o <file> | --output=<file>]
           [--add-file=<file>] [--add-virtual-file=<path>:<content>]
           [--mtime=<time>] [--worktree-attributes]
           [--remote=<repo>] [--exec=<git-upload-archive>]]
           <tree-ish> [<path>...]
```

Everyday usage

Create a release tarball from a tag and write it to a file:

```
git archive --prefix=myapp-1.2.0/ -o myapp-1.2.0.tar.gz v1.2.0
```

The format is inferred from the `.tar.gz` extension. The `--prefix` option adds a top-level directory inside the archive — without it, the files land in the root of the archive, which surprises users who extract it.

Create a zip archive of the current HEAD:

```
git archive -o latest.zip HEAD
```

Archive only a subdirectory (e.g., the `docs/` tree):

```
git archive -o docs-snapshot.zip HEAD docs/
```


Pipe directly to tar for an immediate extract — useful for deploying to a staging server without storing an intermediate file:

```
git archive --prefix=deploy/ HEAD | tar -x -C /var/www/staging/
```

List all formats supported on the current installation:

```
git archive --list
```

Key options

Option	What it does	When to use it
<code>--format=<fmt></code>	Choose the output format (<code>tar</code> , <code>zip</code> , <code>tar.gz</code> , <code>tgz</code> , or any custom format)	When the format cannot be inferred from the output filename
<code>-o <file></code> , <code>--output=<file></code>	Write to a file instead of stdout	Almost always; piping to stdout is for chaining with tar
<code>--prefix=<prefix>/</code>	Prepend a directory prefix to every path in the archive	Release tarballs — consumers expect a top-level folder
<code>-l</code> , <code>--list</code>	Print all available archive formats and exit	Before scripting, to check what formats are registered
<code>--add-file=<file></code>	Bundle an untracked file into the archive	Adding a generated <code>configure</code> script or build artifact
<code>--add-virtual-file=<path>:<content></code>	Inject an in-memory file without it existing on disk	Embedding a version string or build metadata
<code>--mtime=<time></code>	Override the modification time stamped on every entry	Reproducible builds that must match a specific timestamp
<code>--worktree-attributes</code>	Read <code>.gitattributes</code> from the working tree in addition to the archived tree	Tweaking <code>export-ignore</code> rules after committing
<code>--remote=<repo></code>	Fetch the archive from a remote repository instead of the local one	Packaging a version from a server without a full clone
<code>--exec=<cmd></code>	Override the path to <code>git-upload-archive</code> on the remote side	Non-standard server installations
<code>-v</code> , <code>--verbose</code>	Report progress to stderr	Large repositories where you want confirmation it is running
<code>-<digit></code> (zip)	Set zip compression level, <code>0</code> (store only) through <code>9</code> (best ratio)	Balancing archive size against CPU time

Best practices

Always include a `--prefix`. Without a prefix, all files land directly in whatever directory the user extracts into. Anyone who extracts a prefix-less archive in their home directory ends up with

`README.md` next to their `Music` folder. The convention `<project>-<version>/` matches what users expect from release tarballs and what packaging tools (`rpm`, `dpkg`, `Homebrew`) require.

Use a tag as `<tree-ish>` for releases. Tags are stable and carry the commit timestamp, producing bit-for-bit reproducible archives. A branch name like `HEAD` or `main` is a moving target — fine for dev snapshots but not for releases.

Leverage `export-ignore` in `.gitattributes` to keep archives clean. Files and directories marked `export-ignore` are silently omitted from the archive. This is the right place to exclude test suites, CI configuration, and development tooling that consumers of your release do not need.

```
# .gitattributes
tests/          export-ignore
.github/        export-ignore
.editorconfig   export-ignore
```

Use `export-subst` to embed version metadata. Mark a file (e.g. `VERSION`) with the `export-subst` attribute and git will expand `$Format:%H$`, `$Format:%D$`, and similar placeholders when archiving. Consumers of your tarball get the exact commit hash without needing git itself.

```
# .gitattributes
VERSION        export-subst
```

```
# VERSION
commit: $Format:%H$
ref:    $Format:%D$
```

Prefer `-o` over shell redirection for format inference. When you write `git archive ... > myapp.tar.gz`, git sees only stdout and cannot infer the format; you must add `--format=tar.gz`. When you write `-o myapp.tar.gz`, git reads the extension and picks the format automatically.

Pitfalls & gotchas

Forgetting `--prefix` produces a “tarbomb”. Extracting a prefix-less archive drops all files directly into the current directory. If you are testing a tarball before a release, always extract into a fresh temporary directory first.

Submodules are not included. `git archive` only knows about files tracked in the top-level repository. If your project uses submodules, none of the submodule contents appear in the archive. You must either archive each submodule separately and merge the results, or switch to a different packaging strategy.

Tree IDs produce non-reproducible archives. If you pass a raw tree hash (`HEAD^{tree}`) the modification time of every entry becomes the current time. Two runs of the same command one second apart produce different archives. This is usually undesirable; pass a commit or tag instead. The only reason to use a bare tree ID is when you deliberately want to strip the pax commit-ID header from tar output.

`--worktree-attributes` reads your local checkout, not the archived tree. If `.gitattributes` in your working tree differs from the one in the commit being archived, the working-tree version wins. This can cause unexpected inclusions or exclusions. Keep your working tree `.gitattributes` in sync, or rely on `$GIT_DIR/info/attributes` for overrides that should never be committed.

Remote archives are subject to server-side restrictions. When using `--remote`, the remote server runs `git-upload-archive` and may refuse arbitrary `<tree-ish>` expressions — often only tags and branch tips are permitted. Check the remote's `uploadArchive` configuration if you receive an unexpected rejection.

Custom tar formats must be configured before use. Formats like `tar.xz` or `tar.bz2` do not exist by default. You must configure them first with `git config tar.<format>.command`. Running `git archive --list` after configuring confirms they are registered.

Worked examples

Packaging a release

The project is tagged `v2.5.0`. Create a tarball and a zip ready to upload to GitHub Releases.

```
# Tarball
git archive --prefix=myapp-2.5.0/ -o myapp-2.5.0.tar.gz v2.5.0

# Zip (for Windows users who prefer it)
git archive --prefix=myapp-2.5.0/ -o myapp-2.5.0.zip v2.5.0
```

Verify the top-level directory is present before publishing:

```
tar tzf myapp-2.5.0.tar.gz | head -5
```

```
myapp-2.5.0/
myapp-2.5.0/README.md
myapp-2.5.0/LICENSE
myapp-2.5.0/src/
myapp-2.5.0/src/main.c
```

Confirm the commit ID is embedded in the tarball:

```
git get-tar-commit-id < myapp-2.5.0.tar.gz
```

```
a3f9c1e8b2d47f601c8453d09e4f2a18b7c3e561
```

Deploying to a server without a full clone

You want to push the current HEAD to a staging server over SSH. The server has git installed but you do not want to clone the full history there.

```
git archive --prefix=app/ HEAD \
| ssh deploy@staging.example.com 'tar -x -C /var/www/ --strip-components=1'
```

The `--strip-components=1` on the receiving end removes the `app/` prefix after extraction, landing files directly in `/var/www/`.

Adding a generated file not tracked by git

Your build process creates a `configure` script that should ship in the release tarball but is intentionally absent from the repository. Bundle it alongside the archived tree:

```
autoconf # generates ./configure
git archive \
  --prefix=myapp-3.0/ \
  --add-file=configure \
  -o myapp-3.0.tar.gz HEAD
```

Because `--add-file` uses the last active `--prefix`, `configure` lands at `myapp-3.0/configure` in the archive.

Configuring a custom xz format

Register the format once (stored in your global or project git config):

```
git config tar.tar.xz.command "xz -c"
```

Then use it like any built-in format:

```
git archive --format=tar.xz --prefix=myapp-4.0/ -o myapp-4.0.tar.xz v4.0.0
# or, with the format inferred from the extension:
git archive --prefix=myapp-4.0/ -o myapp-4.0.tar.xz v4.0.0
```

Recovery

`git archive` is entirely read-only — it never writes to the repository, modifies the index, or changes any branch pointer. There is nothing to undo. If the output file is wrong, delete it and re-run with corrected options.

If you discover after publishing that a release archive contains files that should have been excluded, add `export-ignore` to `.gitattributes`, commit that change, re-tag if necessary (see the *tag* chapter), and regenerate the archive.

See *Getting out of jams* for help recovering from problems with the commits or tags you are archiving, rather than the archive operation itself.

See also

- *tag* — creating and managing the version tags used as `<tree-ish>`.
- *commit* — understanding commit objects and how timestamps are stored.
- *Getting out of jams* — recovering from bad commits or tags before archiving.

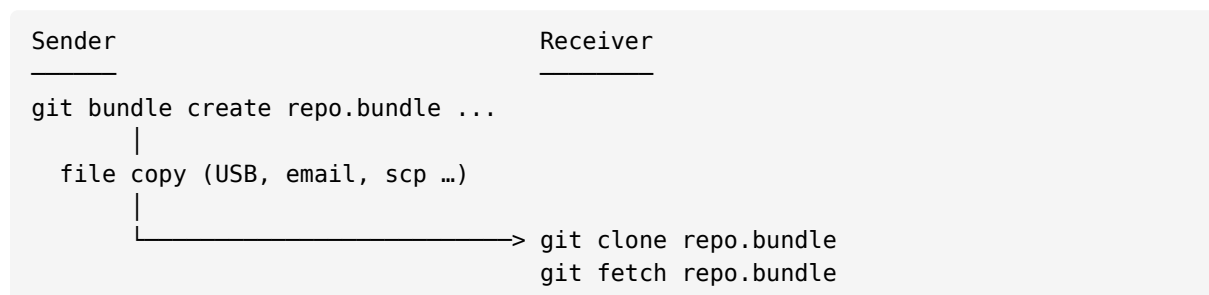
bundle

Package repository objects and refs into a single portable file for offline transfer or archival.

Mental model

A bundle is a self-contained snapshot of part of a repository — commits, trees, blobs, and the refs that point to them — packed into one file you can copy like any other file. Think of it as a transportable fetch operation: the receiving end runs a fetch or clone against the bundle file exactly as it would against a live remote, but no server process is needed on the other side.

Internally a bundle is a pack file (the same format git uses on the wire) with a small header listing the refs it contains and any prerequisite commits the recipient must already have. When the prerequisites are met, `git fetch` or `git clone` against the bundle file is indistinguishable from fetching from a real remote.



Because a bundle only carries refs and the objects reachable from them, it does not capture the working tree, the index, stash entries, per-repository configuration, or hooks.

Synopsis

```
git bundle create [-q | --quiet | --progress] [--version=<version>]
                  <file> <git-rev-list-args>
git bundle verify [-q | --quiet] <file>
git bundle list-heads <file> [<refname>...]
git bundle unbundle [--progress] <file> [<refname>...]
```

Pass `-` as `<file>` to write the bundle to stdout (`create`) or to read from stdin (all other subcommands).

Everyday usage

Create a full bundle of every ref in the repository:

```
git bundle create backup.bundle --all
```

Verify that a bundle is valid and that your repository satisfies its prerequisites:

```
git bundle verify backup.bundle
```

See which refs a bundle exposes:

```
git bundle list-heads backup.bundle
```

Clone a new repository from a bundle:

```
git clone -b main /path/to/backup.bundle ~/new-repo
```

Fetch updates from a bundle into an existing repository:

```
git fetch /path/to/incremental.bundle main:main
```

Key options

Option	What it does	When to use it
<code>create <file> <rev-list-args></code>	Pack objects and refs into a bundle file	Sending or archiving history
<code>verify <file></code>	Check format validity and confirm prerequisites exist locally	Before importing any bundle received from outside
<code>list-heads <file></code>	Print refs contained in the bundle	Discovering what a bundle offers
<code>unbundle <file></code>	Import objects into the object database (plumbing; prefer <code>git fetch</code>)	Low-level scripting
<code>--all</code>	Include every ref (passed as a rev-list arg to <code>create</code>)	Full backups
<code>--progress</code>	Show progress on stderr even when not connected to a terminal	Scripted transfers where progress should be logged
<code>-q / --quiet</code>	Suppress progress output	Automated or cron jobs
<code>--version=<version></code>	Force bundle format version (2 = SHA-1 only, 3 = supports extensions)	Compatibility with older Git installations

Best practices

Tag the watermark after each incremental bundle. Incremental bundles depend on both sides agreeing where the last transfer ended. Keep a dedicated tag (`last-usb-sync` , `lastR2bundle` , etc.) and advance it immediately after creating each bundle. That tag becomes the prerequisite for the next run.

```
git bundle create inc.bundle last-usb-sync..main
git tag -f last-usb-sync main
```

Always verify before importing. Run `git bundle verify` on every bundle you receive from outside your control. It confirms the format is sound and that your local repository has every prerequisite commit. Discovering a missing prerequisite after a long copy operation wastes time; verifying first costs seconds.

Use `--all` for archival snapshots, rev-ranges for incremental transfers. A full `--all` bundle can be cloned from scratch on any machine. A range bundle (`old..new`) is smaller but requires the recipient to already have `old` . Choose based on whether the destination is a blank machine or an existing mirror.

Treat the bundle file as a remote, not a one-shot import. After cloning from a bundle, the `origin` remote URL points to that file path. When you drop a newer bundle at the same path, `git pull` in the clone just works. This makes bundles a clean drop-in for sneakernet update workflows.

Prefer `git fetch` over `git bundle unbundle` for consuming bundles. `unbundle` is plumbing intended for `git fetch` to call internally. Using `git fetch <bundle-path> <refspec>` gives you proper remote-tracking refs, ref-mapping, and familiar output.

Pitfalls & gotchas

The tip of a range must resolve to a named ref. Git refuses to create a bundle whose tip is a bare commit hash rather than a ref:

```
git bundle create head.bundle $(git rev-parse HEAD)
# fatal: Refusing to create empty bundle.
```

Use a branch name, tag, or `HEAD` explicitly instead of a raw hash.

Bundles do not include everything you might expect. The index, working tree, stash, reflogs, per-repository config, and hooks are all absent. A bundle restores commit history, not a complete developer environment.

Prerequisites must be present on the receiving side. An incremental bundle created with `old..new` can only be fetched into a repository that already contains `old`. If the recipient has been re-initialized or diverged since the last sync, the verify step will report the missing prerequisites and the fetch will fail. The remedy is a fresh full bundle.

Cloning from a bundle sets the bundle file path as `origin`. Moving or deleting the bundle file breaks `git fetch` in the clone. After cloning for a long-lived repository, update the remote URL to a real remote immediately:

```
git remote set-url origin git@github.com:org/repo.git
```

The watermark tag is only on the sender. Nothing automatically records on the receiving side where the last bundle left off. If the sending repository is ever cloned fresh or the watermark tag is lost, you must start again from a full bundle.

Worked examples

Full backup and restore

Create a complete archive of the current repository, named with today's date:

```
git bundle create ~/backups/myrepo-$(date +%Y%m%d).bundle --all
```

To restore on another machine, clone directly from the file:

```
git clone ~/backups/myrepo-20260617.bundle ~/restored-repo
cd ~/restored-repo
git log --oneline -5
git branch -a
```

Because `--all` was used, every branch and tag is present. The result is a fully functional clone.

Sneakernet incremental sync across an air-gapped network

Machine A holds the authoritative repository. Machine B sits on an isolated network. A USB drive carries bundles between them.

Initial transfer — on machine A:

```
cd /srv/repos/project
git bundle create /mnt/usb/project-full.bundle --all
git tag -f last-usb-sync main
```

Initial import — on machine B:

```
git clone -b main /mnt/usb/project-full.bundle ~/project
cd ~/project
```

Subsequent transfers — after further work on machine A:

```
cd /srv/repos/project
git bundle create /mnt/usb/project-inc.bundle last-usb-sync..main
git tag -f last-usb-sync main
```

Verify and pull on machine B after the drive arrives:

```
cd ~/project
git bundle verify /mnt/usb/project-inc.bundle
git pull /mnt/usb/project-inc.bundle main:main
```

Sharing a topic branch without a shared remote

Your colleague has a clone of the same repository but no network access to your fork. Bundle just the commits on your feature branch that they do not have yet:

```
git bundle create feature-login.bundle origin/main..feature/login
```

Email or copy `feature-login.bundle` to your colleague. They apply it:

```
git bundle verify feature-login.bundle
git fetch ./feature-login.bundle feature/login:feature/login
git log --oneline feature/login
```

They can now review, test, or merge `feature/login` without you needing a shared remote at all.

Recovery

If a fetch from a bundle fails because prerequisites are missing, request or re-create a full bundle rather than an incremental one:

```
# On the sender side — fall back to a complete bundle
git bundle create full-resync.bundle --all
```

If you cloned from a bundle and the file has since moved, update the remote URL:

```
git remote set-url origin <new-file-path-or-real-remote>
```


See *Getting out of jams* for general recovery from broken or diverged remote states.

See also

- *clone* — `git clone` accepts a bundle file path as the source URL.
- *fetch* — the primary way to import refs from a bundle into an existing repository.
- *remote* — managing remote URLs, including fixing a bundle-backed origin after the file moves.
- *tag* — using tags as watermarks to track incremental bundle boundaries.
- *Getting out of jams* — recovering from broken or diverged repository states.

notes

Attach, read, and manage free-form annotations on any Git object without altering the object itself.

Mental model

A commit is immutable once written — its hash is derived from every byte of its content, so changing the message after the fact changes the hash. `git notes` sidesteps that constraint by storing annotations in a separate reference, `refs/notes/commits` by default, rather than inside the commit object.

Think of notes as sticky tabs on the pages of a book. The text on each page (the commit) never changes, but you can peel off and replace a sticky tab at any time without reprinting anything.

Under the hood, notes live in their own commit history. Each time you add or change a note, Git creates a new commit on the notes ref. The annotated object's SHA-1 becomes a path in the notes tree; the note itself is a blob stored at that path:

Commit history	Notes ref (refs/notes/commits)
a1b2c3 ←	blob: "Deployed to staging 2026-06-10"
d4e5f6 ←	blob: "CVE-2026-1234 patched here"
...	

When `git log` shows a commit, it looks up the notes ref and appends any matching note below the commit message. Notes are visible without modifying the commit object or its hash.

Synopsis

```
git notes [list [<object>]]
git notes add    [-f] [--allow-empty] [-m <msg> | -F <file> | (-c|-C) <object>]
                [-e] [--separator=<str>] [--[no-]strip-space] [<object>]
git notes append [--allow-empty] [-m <msg> | -F <file> | (-c|-C) <object>]
                [-e] [--separator=<str>] [--[no-]strip-space] [<object>]
git notes copy   [-f] (--stdin | <from-object> [<to-object>])
git notes edit   [--allow-empty] [--[no-]strip-space] [<object>]
git notes show   [<object>]
git notes remove [--ignore-missing] [--stdin] [<object>...]
git notes prune  [-n] [-v]
git notes merge  [-v | -q] [-s <strategy>] <notes-ref>
git notes merge  --commit [-v | -q]
git notes merge  --abort  [-v | -q]
git notes get-ref
```

All subcommands default to `HEAD` when no `<object>` is given.

Everyday usage

Add a note to the most recent commit:

```
git notes add -m "Deployed to staging 2026-06-10 by alice"
```

Show the note for a specific commit:

```
git notes show a1b2c3
```

`git log` includes notes automatically:

```
git log --oneline -3
```

```
a1b2c3 Fix connection pool exhaustion
Notes:
  Deployed to staging 2026-06-10 by alice
```

Append more information to an existing note instead of overwriting it:

```
git notes append -m "Also deployed to production 2026-06-14" a1b2c3
```

Edit a note interactively in your configured editor:

```
git notes edit a1b2c3
```

Remove a note entirely:

```
git notes remove a1b2c3
```

List every note in the default notes ref:

```
git notes list
```

```
7f3a9e... a1b2c3...
9c815d... d4e5f6...
```

Key options

Option	What it does	When to use it
<code>-m <msg></code>	Set the note message inline	Short, one-off annotations
<code>-F <file></code>	Read the note message from a file (<code>-F -</code> reads stdin)	Long notes or notes generated by a script
<code>-f</code> , <code>--force</code>	Overwrite an existing note instead of aborting	Replace a note that is already set
<code>--allow-empty</code>	Store an empty note (default behaviour removes empty notes)	Explicitly mark a commit as reviewed with no comments
<code>-e</code>	Open the editor even when <code>-m</code> or <code>-F</code> is provided	Review and adjust the message before saving
<code>-C <object></code>	Reuse an existing blob verbatim as the note	Copy a note from another object
<code>-c <object></code>	Like <code>-C</code> but open the editor to modify it	Adapt an existing note for a new commit
<code>--separator=<str></code>	Custom separator inserted between multiple <code>-m</code> / <code>-F</code> paragraphs	Use <code>---</code> instead of a blank line
<code>--no-separator</code>	No separator between paragraphs	Concatenate messages directly

<code>--strip-space</code>	Normalise whitespace (default for <code>-m / -F</code>)	Keep notes tidy
<code>--no-strip-space</code>	Preserve whitespace exactly	Notes with intentional indentation
<code>--ref=<ref></code>	Operate on a notes ref other than <code>refs/notes/commits</code>	Multiple independent annotation namespaces
<code>--ignore-missing</code>	Do not error when removing a note that does not exist	Safe <code>remove</code> calls in scripts
<code>-n, --dry-run</code>	Show what <code>prune</code> would remove, without doing it	Audit before cleanup
<code>-s <strategy></code>	Merge strategy: <code>manual</code> , <code>ours</code> , <code>theirs</code> , <code>union</code> , <code>cat_sort_uniq</code>	Control conflict resolution during <code>merge</code>
<code>--commit</code>	Finalise an in-progress <code>notes merge</code> after manually resolving conflicts	After editing files in <code>.git/NOTES_MERGE_WORKTREE</code>
<code>--abort</code>	Cancel an in-progress <code>notes merge</code>	Bail out of a conflicted merge
<code>-v</code>	Verbose output for <code>merge</code> (reports conflict details) and <code>prune</code> (lists removed objects)	Scripting and CI logs
<code>-q</code>	Quiet output for <code>merge</code> only	Suppress merge output in scripts

Best practices

Use notes for information that only exists after the commit is written. Notes are ideal for things like deployment timestamps, downstream test results, ticket status updates, or automated code-review scores. If the information belongs in the commit message, write it there at commit time; notes are not a substitute for a good message.

Push and fetch notes explicitly – they do not travel with a normal push. A plain `git push` transfers your branch refs but not `refs/notes/*`. Add the notes refspec to your remote configuration so notes travel automatically:

```
# One-time setup: push notes alongside branches
git config --add remote.origin.push '+refs/notes/*:refs/notes/*'

# One-time setup: fetch notes from the remote
git config --add remote.origin.fetch '+refs/notes/*:refs/notes/*'
```

Or push and fetch by hand when needed:

```
git push origin refs/notes/commits
git fetch origin '+refs/notes/commits:refs/notes/commits'
```

Use separate refs for independent annotation streams. The default `refs/notes/commits` ref is a single shared namespace. If a CI system records test results and a deploy script records environment data, keeping them in the same ref clutters `git log`. Use `--ref` to separate them, and configure `notes.displayRef` to control which namespaces each developer sees:

```
git notes --ref=refs/notes/ci      add -m "Build #1042: PASS" HEAD
git notes --ref=refs/notes/deploy add -m "Deployed prod 2026-06-14" HEAD

git config notes.displayRef 'refs/notes/commits'
git config --add notes.displayRef 'refs/notes/ci'
```

Enable note rewriting so notes survive rebase and amend. When you rebase or amend a commit, Git creates a new commit object with a new hash. Notes attached to the original hash become orphaned unless rewriting is configured:

```
git config notes.rewrite.rebase true
git config notes.rewrite.amend true
git config notes.rewriteRef refs/notes/commits
```

Prefer append over add -f when notes come from multiple sources. Overwriting with `add -f` silently discards whatever was already there. Use `append` to accumulate contributions from multiple people or systems without losing history.

Pitfalls & gotchas

Notes are invisible to collaborators until you push them. This is the most common surprise. After adding notes locally and pushing the branch, teammates see nothing. The fix is always an explicit push of `refs/notes/commits` (or configuring the remote refspec shown above).

Rebasing or amending silently drops notes when rewriting is not configured. `git rebase` and `git commit --amend` (see the *commit* chapter) produce new objects with new hashes. Any note pointing to the old hash becomes orphaned — it still exists in the notes ref but is never shown because no commit carries that SHA-1 any more. Set `notes.rewrite.rebase` and `notes.rewriteRef` before this matters.

add aborts if a note already exists. Invoking `git notes add` on an object that already has a note exits with an error unless you pass `-f`. Use `append` to accumulate notes, or `-f` to intentionally replace the existing one.

Notes history grows without bound on active repositories. Each `add`, `append`, `remove`, or `edit` creates a commit on the notes ref. After a history rewrite that produces many new objects, run `git notes prune -n` first to preview, then `git notes prune` to remove orphaned note objects.

Merge conflicts on notes require manual intervention by default. When two people annotate the same commit on different clones and then merge, Git cannot automatically reconcile the two blobs. The merge pauses, checks out the conflicting notes in `.git/NOTES_MERGE_WORKTREE`, and waits. Edit those files, then run `git notes merge --commit` to finalise, or `git notes merge --abort` to bail out. Alternatively, choose an automated strategy such as `union` or `cat_sort_uniq` upfront.

The --ref value is normalised, not passed verbatim. A value starting with `refs/notes/` is used as-is; one starting with `notes/` gets `refs/` prepended; anything else gets `refs/notes/` prepended. So `--ref=ci` targets `refs/notes/ci`, not `refs/ci`. Verify with `git notes get-ref` after setting `GIT_NOTES_REF` or `core.notesRef`.

Worked examples

Recording deployment events without modifying commits

A deploy script annotates each commit it ships so that the release history is visible in `git log` without any secondary database.

```
# In the deploy pipeline
COMMIT=$(git rev-parse HEAD)
TIMESTAMP=$(date -u +%Y-%m-%dT%H:%M:%SZ)
git notes --ref=refs/notes/deploy append \
    -m "Deployed to production at ${TIMESTAMP} by ${DEPLOY_USER}" \
    "$COMMIT"
git push origin refs/notes/deploy
```

A developer fetches and enables the namespace:

```
git fetch origin '+refs/notes/deploy:refs/notes/deploy'
git config --add notes.displayRef refs/notes/deploy
git log --oneline -3
```

```
alb2c3 Fix connection pool exhaustion
Notes (refs/notes/deploy):
    Deployed to production at 2026-06-14T09:41:00Z by ci-bot
d4e5f6 Increase pool size to 20
Notes (refs/notes/deploy):
    Deployed to production at 2026-06-10T14:22:00Z by ci-bot
```

No commits were altered; the deployment record lives entirely in the notes ref.

Adding a post-hoc code-review note to a shared branch

You have reviewed a colleague's branch and want to record the outcome against the specific commit you reviewed, not as a comment in the pull-request interface.

```
# Identify the commit
git log --oneline feature/payment-refactor
# b7d91a Refactor Stripe checkout path
# 3cc4f0 Extract PaymentGateway interface

# Write the review note
git notes add -m "Code review by alice 2026-06-17: approved.
Latency concern tracked in #892." b7d91a

# Share it
git push origin refs/notes/commits
```

A teammate retrieves and reads it:

```
git fetch origin '+refs/notes/commits:refs/notes/commits'
git notes show b7d91a
```

```
Code review by alice 2026-06-17: approved.
Latency concern tracked in #892.
```

Resolving a notes merge conflict

Two CI systems both write notes to the same commit on the same notes ref. When you merge the two notes branches a conflict arises.

```
git notes merge refs/notes/other-ci
# CONFLICT (content): Merge conflict in bf/fe30.../680d5a...
# Automatic merge failed; fix conflicts and then commit the result.
```

Inspect and edit the conflicting file in the worktree:

```
ls .git/NOTES_MERGE_WORKTREE/
vim .git/NOTES_MERGE_WORKTREE/bf/fe30.../680d5a...
# Remove conflict markers, keep both entries
git notes merge --commit
```

Or restart the merge using the `union` strategy to concatenate automatically:

```
git notes merge --abort
git notes merge -s union refs/notes/other-ci
```

Recovery

Remove a note you added by mistake:

```
git notes remove HEAD
```

If you need to recover a note that was deleted, the notes ref has its own commit history. Inspect it to find the state just before the deletion:

```
git log --oneline refs/notes/commits
# Find the commit just before the delete
git show <old-notes-commit>:<first-2-chars-of-sha>/<rest-of-sha>
```

If a rebase or amend silently dropped notes because `notes.rewrite` was not configured, retrieve the old note using the original commit SHA from the reflog:

```
# Find the original commit SHA
git reflog

# Read the note from the historical notes tree
git log --all --oneline refs/notes/commits
git show <historical-notes-commit>:<path-in-notes-tree>
```

See *Getting out of jams* for broader strategies around recovering lost objects.

See also

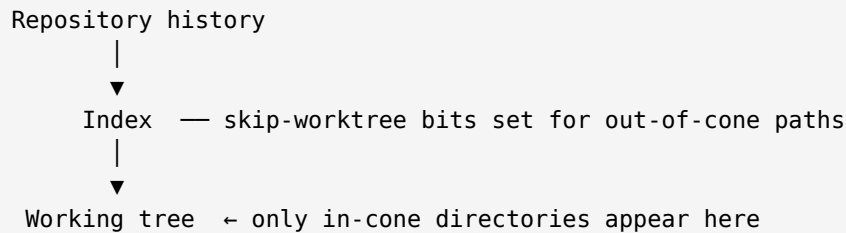
- `commit` — writing the commits that notes annotate; `--amend` and rebase both orphan notes when rewriting is not configured.
- `log` — where notes surface in everyday use; `notes.displayRef` controls which refs it reads.
- `rebase` — configure `notes.rewrite.rebase` so notes survive history rewrites.
- *Getting out of jams* — recovering deleted notes and orphaned note objects.

sparse-checkout

Reduce your working tree to a subset of tracked files so you only check out the directories you actually need.

Mental model

Every file in a repository lives in the index. Normally the working tree is a full mirror of that index — every tracked file is present on disk. Sparse checkout breaks that mirror deliberately: it marks files whose directories you did not ask for with the **skip-worktree** bit, and Git stops materialising them on disk. The files are still in the repository history and in the index; they are simply absent from your working tree.



The default mode is **cone mode**: you name directories, and Git includes everything beneath each directory, plus the files at the toplevel and in each ancestor directory along the path. This constraint lets Git use fast hash-based lookups rather than $O(N \times M)$ pattern matching, and it is the only mode that supports `--sparse-index`.

A **sparse index** takes the optimisation further: instead of storing one index entry per file for out-of-cone directories, Git stores a single entry for the directory itself. This can dramatically shrink the index and speed up `git status`, `git add`, and related commands in very large repositories.

When Git operations such as merge or rebase need to materialise files outside the cone to resolve conflicts, they do so temporarily. Use `git sparse-checkout reapply` afterwards to re-hide those files.

Synopsis

```
git sparse-checkout list
git sparse-checkout set [--cone | --no-cone] [--[no-]sparse-index] [--stdin]
<dir>...
git sparse-checkout add [--stdin] <dir>...
git sparse-checkout reapply [--[no-]cone] [--[no-]sparse-index]
git sparse-checkout disable
git sparse-checkout check-rules [--rules-file <file>] [-z]
```

`init` is listed in the help as a deprecated alias for `set` with no paths and should not be used in new workflows.

Everyday usage

Set up a sparse checkout that checks out only the `src/` and `docs/` directories (plus toplevel files):

```
# Start from a full clone, or a partial clone for maximum effect
git clone --filter=blob:none --no-checkout https://github.com/example/monorepo
cd monorepo
git sparse-checkout set src docs
git checkout main
```


From inside an existing full clone, narrow the working tree at any time:

```
git sparse-checkout set services/api services/auth
```

See which directories are currently included:

```
git sparse-checkout list
# services/api
# services/auth
```

Add another directory without replacing what is already there:

```
git sparse-checkout add services/payments
```

Restore the full working tree and disable sparse checkout:

```
git sparse-checkout disable
```

Key options

Option	Applies to	What it does	When to use it
<code>--cone</code>	set , reapply	Interpret arguments as directory names (default)	Always — cone mode is faster and supports sparse index
<code>--no-cone</code>	set , reapply	Interpret arguments as gitignore-style patterns	Avoid; deprecated and incompatible with <code>--sparse-index</code>
<code>--sparse-index</code>	set , reapply	Collapse out-of-cone directories to single index entries	Large monorepos where <code>git status</code> feels slow
<code>--no-sparse-index</code>	set , reapply	Expand the index back to per-file entries	Recovering compatibility with older tools
<code>--stdin</code>	set , add	Read the directory list from stdin (newline-delimited)	Scripting; piping the output of another command
<code>--rules-file <file></code>	check-rules	Test paths against rules in <code><file></code> instead of the live rules	Previewing a rule change before applying it
<code>-z</code>	check-rules	Use NUL-delimited input/output instead of newlines	Paths that contain spaces or special characters

Best practices

Combine sparse checkout with a partial clone. `git clone --filter=blob:none` skips downloading file contents at clone time; Git fetches blobs on demand. Paired with sparse checkout you avoid both downloading and writing to disk the files you never need. For very large repositories, use `--filter=tree:0` to also defer tree objects.

```
git clone --filter=blob:none --no-checkout https://github.com/example/monorepo
cd monorepo
git sparse-checkout set my/team/dir
git checkout main
```

Enable the sparse index for speed. If your sparse cone is a small fraction of the total repository, add `--sparse-index` when you first call `set`. You can also add it later via `reapply`:

```
git sparse-checkout reapply --sparse-index
```

Check whether it is active with `git config --local index.sparse`.

Use `set` rather than `init` + `set`. The old two-step workflow (`init` then `set`) first stripped the working tree of almost all files and then re-added them, which was slow and confusing. `set` handles all required config settings in a single operation.

Let `set` upgrade worktree config automatically. When you call `set`, Git writes the sparsity rules into worktree-specific config so that different worktrees (see the *worktree* chapter) can have different cone definitions. This happens automatically; you do not need to configure it.

Stick to cone mode. Non-cone mode (gitignore-style patterns) has $O(N \times M)$ performance, prevents use of `--sparse-index`, is incompatible with some merge strategies, and is deprecated. If you think you need a complex pattern, reconsider whether restructuring the directories would be cleaner.

Pitfalls & gotchas

Toplevel files are always included. Cone mode unconditionally includes every file at the root of the repository — `README.md`, `Makefile`, `.gitignore`, and friends. You cannot exclude them without switching to the deprecated non-cone mode.

Merge and rebase can leak files into the working tree. When Git needs to show a conflict it materialises the conflicting file even if it is outside your cone. After resolving the conflict and committing or aborting, run `git sparse-checkout reapply` to hide those files again. Forgetting this step leaves out-of-cone files present on disk, which can confuse editors and build tools.

`git commit -a` ignores files outside the cone. This is intentional — Git will not mark out-of-cone paths as deleted just because they are absent from the working tree. But it means `-a` can give a misleading “nothing to commit” when you expect it to pick up changes in directories you have not checked out.

Switching branches does not update out-of-cone paths. If a branch adds or changes files outside your current cone, those changes will not appear in the working tree after `git switch`. The index tracks them correctly, but your disk does not reflect them until you widen the cone.

The sparse index can confuse external tools. Older IDEs, language servers, and Git GUIs that inspect the index directly may not understand sparse directory entries. If you encounter strange behaviour, disable the sparse index:

```
git sparse-checkout reapply --no-sparse-index
```

Untracked files block directory removal. When you narrow the cone with a new `set` call, Git tries to delete directories that have fallen out of scope. If any such directory contains untracked files that are not gitignore-d, Git warns and leaves the directory in place. Stage and commit (or explicitly remove) those files before narrowing the cone.

--sparse-index is still experimental. The help text warns that some commands may be slower until they are fully integrated, and that external tools may not handle the format correctly. Enable it in workspaces where you control all tooling.

Worked examples

Checking out one service from a monorepo

A monorepo contains hundreds of services under `services/`. You only maintain `services/billing`.

```
# Partial clone – no file contents downloaded yet
git clone --filter=blob:none --no-checkout git@github.com:corp/platform.git
cd platform

# Set the cone and enable sparse index for speed
git sparse-checkout set --sparse-index services/billing

# Now materialise the branch
git checkout main
```

```
$ git sparse-checkout list
services/billing
```

The working tree contains only `services/billing/` (all depths), every file immediately under `services/`, and every file at the repository root. The other 190-odd service directories are absent from disk.

To pull in a shared library you discover is needed:

```
git sparse-checkout add lib/common
```

Temporarily widening the cone to resolve a conflict

You are rebasing a branch that touches files in both `services/billing` and `services/auth`. Git materialises the conflicting `services/auth` files to present the conflict.

```
git rebase origin/main
# ... CONFLICT (content): Merge conflict in services/auth/config.go
```

Resolve the conflict in your editor, then continue:

```
git add services/auth/config.go
git rebase --continue
```

After the rebase completes, re-hide the out-of-cone files:

```
git sparse-checkout reapply
```

`services/auth/` disappears from the working tree again.

Previewing rules before applying them

You are writing a script that will configure sparse checkout for a new team member and want to verify the rule set catches the right paths before touching anyone's working tree.

```
# Write candidate rules to a file
printf 'services/billing\nlib/common\n' > /tmp/billing-rules.txt

# Ask which paths from the index would be included
git ls-tree -r --name-only HEAD | \
git sparse-checkout check-rules --rules-file /tmp/billing-rules.txt
```

The output is the subset of tracked paths that match those rules, without modifying the current sparse-checkout configuration.

Recovery

Re-enable a full working tree after accidentally over-narrowing the cone:

```
git sparse-checkout disable
```

This sets `core.sparseCheckout` to `false` and repopulates all tracked files.

Restore a specific out-of-cone file without changing the cone definition:

```
git checkout HEAD -- path/to/file
```

The file reappears on disk; the skip-worktree bit remains cleared for that path until the next cone-changing operation.

Recover a sparse index that broke a tool:

```
git sparse-checkout reapply --no-sparse-index
```

This rewrites the index to use per-file entries without changing which directories are included.

See *Getting out of jams* for broader undo recipes, including recovering from bad merges and rebases that may have happened while outside your sparse cone.

See also

- *clone* — `--filter=blob:none` and `--filter=tree:0` for partial clones that pair naturally with sparse checkout.
- *worktree* — each worktree maintains its own independent sparse-checkout definition.
- *checkout* and *switch* — how branch switches interact with out-of-cone paths.
- *rebase* and *merge* — operations that can temporarily materialise out-of-cone files, requiring `reapply` afterwards.
- *Getting out of jams* — restoring a working tree that has ended up in an unexpected state.

replace

Transparently substitute any Git object with a different one, without rewriting history.

Mental model

Every commit, tree, blob, and tag in a Git repository is identified by its SHA-1 hash. `git replace` writes a reference into the `refs/replace/` namespace that says: “whenever any command would resolve SHA-1 X, hand back SHA-1 Y instead.” The original object stays in the object database — nothing is deleted or rewritten. The swap is purely a lookup table.

Object database	refs/replace/
deadbeef (original)	deadbeef → cafef00d
cafef00d (replacement)	

`git log:` sees cafef00d content for the commit named deadbeef
`git --no-replace-objects log:` sees the real deadbeef

Because replacements are stored as refs, they can be pushed and fetched like branches or tags. A team can share a set of “amended” commits without forcing every contributor to rewrite their local clone.

The mechanism is transparent to almost all porcelain commands. The only carve-outs are reachability-traversal operations (`prune` , `pack transfer`, and `fsck`) which intentionally ignore replacements so the object database stays self-consistent.

Synopsis

```
git replace [-f] <object> <replacement>
git replace [-f] --edit <object>
git replace [-f] --graft <commit> [<parent>...]
git replace [-f] --convert-graft-file
git replace -d <object>...
git replace [--format=<format>] [-l [<pattern>]]
```

Everyday usage

List all active replacements:

```
git replace -l
git replace          # no arguments also lists
```

Show replacements with both SHA-1s and object types:

```
git replace --format=long -l
```

Replace a commit with a hand-crafted one (the replacement object already exists):

```
git replace deadbeef cafef00d
```

Delete a replacement so the original object is visible again:

```
git replace -d deadbeef
```

Edit a commit's metadata (author, message, tree) interactively — Git opens `$EDITOR`, you save the changes, and a replacement ref is written automatically:

```
git replace --edit deadbeef
```

Graft a commit onto a different set of parents (useful for joining two separate history lines):

```
git replace --graft <tip-of-new-history> <desired-parent>
```

Disable all replacements for one command:

```
git --no-replace-objects log
```

Key options

Option	What it does	When to use it
<code>-f</code> , <code>--force</code>	Overwrite an existing replace ref for the same object	Updating a replacement without deleting it first
<code>-d</code> , <code>--delete</code>	Remove replace refs for the given objects	Undoing a replacement
<code>--edit <object></code>	Open the object in <code>\$EDITOR</code> , write a replacement from the result	Fixing a commit message or metadata without a full rebase
<code>--raw</code>	With <code>--edit</code> , present raw binary content instead of pretty-printed	Repairing a corrupted tree object
<code>--graft <commit> [<parent>...]</code>	Create a replacement commit with a different parent list	Joining histories or cutting a commit free from its parents
<code>--convert-graft-file</code>	Migrate <code>\$GIT_DIR/info/grafts</code> to replace refs and delete the file	Upgrading repos that still use the deprecated grafts mechanism
<code>-l [<pattern>]</code> , <code>--list [<pattern>]</code>	List replace refs, optionally filtered by pattern	Auditing what replacements are active
<code>--format=<format></code>	Control listing output: <code>short</code> , <code>medium</code> , or <code>long</code>	Getting a quick overview (<code>short</code>) or full type info (<code>long</code>)

Best practices

Treat replace refs like branches: push and document them. A replacement that lives only in your local clone is a silent trap for teammates — they will see a different history than you do. Push replace refs explicitly and note them in the project's contributing guide:

```
git push origin 'refs/replace/*'
```

On the receiving end, fetch them with a matching refspec:

```
git fetch origin 'refs/replace/*:refs/replace/*'
```

Prefer `--edit` for one-off metadata fixes over a full rebase. When a commit message contains a typo or a broken `Closes:` trailer but the diff is correct, `git replace --edit` corrects the metadata without touching any other commit's hash. Use the *rebase* chapter's techniques when you need to change file content or restructure multiple commits.

Use `--graft` to join histories, not to rewrite them. Grafting is the right tool when you are stitching an import (a repo created from a tarball) back onto the original upstream history, or when a repository was split and you want to reconnect the two halves for `git log` purposes. It leaves both the original root and the synthetic parent intact.

Convert legacy graft files before they cause confusion. The `$GIT_DIR/info/grafts` file has been deprecated since Git 2.18. Run `git replace --convert-graft-file` once, push the resulting replace refs to the shared remote, and remove any documentation that references the old mechanism.

Scope `--no-replace-objects` in scripts that care about canonical hashes. Automation that validates commit hashes, generates release notes from `git log`, or produces SBOMs should pass `--no-replace-objects` (or set `GIT_NO_REPLACE_OBJECTS=1`) so it always sees the real object graph.

Pitfalls & gotchas

Replacements are invisible unless everyone fetches `refs/replace/`. The default `git fetch` refspec does not include `refs/replace/*`. A developer who has not fetched the replace refs sees the original, unreplaced history. This asymmetry is the most common source of confusion around the feature.

The replaced and replacement objects must be the same type — unless you use `-f`.

Replacing a commit with a blob fails without `--force`. Using `--force` to bypass the type check is almost always a mistake; Git commands that dispatch on object type will behave unpredictably.

`git reset --hard` to a replaced commit moves to the replacement. This is a documented bug. If you run `git reset --hard deadbeef` and `deadbeef` has a replacement, your working tree will reflect the replacement commit's tree, not the original. Use `--no-replace-objects` when you specifically need the original object:

```
git --no-replace-objects reset --hard deadbeef
```

Reachability commands ignore replacements. `git fsck`, `git gc --prune`, and pack-transfer operations see the raw object graph. A replacement object that is not otherwise reachable (not pointed to by any branch, tag, or other ref) can be pruned away, silently breaking the replacement. Make sure the replacement commit is reachable by pushing it to the remote.

`--edit` on a tree requires `--raw` for binary content. Tree objects are stored in a compact binary format. Without `--raw` they are shown in a human-readable listing; with `--raw` you are editing bytes. Most editors cannot round-trip binary data cleanly. Avoid `--raw` on trees unless you have a hex editor configured and a specific reason to work at that level.

Comparing a replaced object with its replacement does not work properly.

`git diff deadbeef cafef00d` (where one is a replacement for the other) can produce misleading output. Use `--no-replace-objects` when you need to see what actually changed between the two:

```
git --no-replace-objects diff deadbeef cafef00d
```

Worked examples

Fixing a commit message without rebasing

A release commit landed on `main` with the wrong issue number in the message. The commit is `a1b2c3d4`. You cannot rebase `main` because other branches have already diverged from it.

Open the commit in your editor:

```
git replace --edit a1b2c3d4
```

Git pretty-prints the commit to a temporary file. The file looks like:

```
tree 9f8e7d6c5b4a3f2e1d0c9b8a7f6e5d4c3b2a1f0e
parent 7654321f
author Jane Dev <jane@example.com> 1700000000 +0100
committer Jane Dev <jane@example.com> 1700000000 +0100

Release v2.4.0 - closes #421
```

Change `#421` to `#412`, save, and quit. Git writes a new commit object and creates the replace ref automatically.

Verify the replacement is active:

```
git replace --format=long -l
```

```
a1b2c3d4 (commit) → e5f6a7b8 (commit)
```

```
git log --oneline -3
```

```
e5f6a7b8 Release v2.4.0 - closes #412
...
```

Share the fix with the team:

```
git push origin 'refs/replace/*'
```

Teammates fetch it:

```
git fetch origin 'refs/replace/*:refs/replace/*'
```

Joining an imported repository to its upstream history

Your organization's copy of a library was created from a tarball in 2019 and has accumulated two years of local patches. You now have the real upstream available and want `git log` to show the full lineage, but you cannot rewrite the patch commits because they are already referenced by release tags.

The goal: make the root commit of the local-patch history (`loc0001`) appear to descend from the tip of the upstream history (`ups9999`).


```
# Add upstream as a remote and fetch it
git remote add upstream https://example.com/library.git
git fetch upstream

# Create the graft: loc0001 gets ups9999 as its parent
git replace --graft loc0001 ups9999
```

Check the result:

```
git log --oneline upstream/main..HEAD
```

The log now walks through your local patches and continues back into upstream history. The original `loc0001` object is untouched; the graft exists only as a replace ref.

Push so CI and teammates see the same combined history:

```
git push origin 'refs/replace/*'
```

Migrating from the deprecated grafts file

A legacy repository has `$GIT_DIR/info/grafts`. Migrate it to replace refs:

```
git replace --convert-graft-file
```

Git reads each line from the grafts file, creates a replacement commit for it, deletes the file, and prints a summary. Push the new replace refs to a shared remote and update the team's fetch configuration so everyone gets them on the next `git fetch`.

Recovery

Remove a specific replacement and restore the original object's visibility:

```
git replace -d <original-sha1>
```

Remove all replace refs in one step:

```
git for-each-ref --format='%(refname)' refs/replace/ | \
  xargs -r git update-ref -d
```

Inspect the original version of any object regardless of active replacements:

```
git --no-replace-objects cat-file -p <original-sha1>
```

See *Getting out of jams* for broader object-recovery techniques.

See also

- *rebase* — rewriting history by actually changing commit hashes; reach for this when you need to change file content or restructure multiple commits.
- *notes* — attaching metadata to commits without touching the commit object or creating replace refs.
- *log* — reading history; pass `--no-replace-objects` to see the unmodified object graph.
- *fsck* — inspecting the raw object graph; always bypasses replacements.

filter-branch

Rewrite every commit in a branch by applying shell-based filters to the tree, environment, message, parents, or commit object itself.

Mental model

Every commit in a Git repository is immutable — its SHA-1 hash is derived from its content, its parent hashes, and its metadata. When you need to change something that appears in many commits (a leaked password, a wrong email address, a file that was never supposed to be tracked), you cannot edit commits in place. You must create a parallel chain of new commits with the undesired content removed or corrected.

`git filter-branch` does this by walking a set of commits in topological order and, for each one, running one or more shell commands — the *filters* — that transform the commit. The output of each filter becomes the input for the next. When all filters have run, `git filter-branch` calls `git commit-tree` to assemble a new commit object from the transformed pieces. The branch pointer is then advanced to the last rewritten commit.

```
Original history:  A -- B -- C -- D  (branch HEAD)
                  |
                  filters run on every commit
                  |
Rewritten history: A'-- B'-- C'-- D' (branch HEAD, new SHAs)

Original saved at: refs/original/refs/heads/<branch>
```

The original commits are not deleted immediately — they are backed up under `refs/original/`. They remain in the object database until garbage collection removes them, giving you a safety net.

A critical caveat before you start: Git's own documentation marks `filter-branch` as deprecated and strongly recommends `git filter-repo` for new work. `filter-branch` is architecturally slow (it shells out for every commit), prone to silent data corruption with non-ASCII filenames or BSD/GNU userland differences, and leaves many traps for the unwary. This chapter documents `filter-branch` accurately because it remains widely available and because existing scripts depend on it — but treat it as a last resort. For any non-trivial rewrite, evaluate `git filter-repo` first.

Synopsis

```
git filter-branch [--setup <command>]
                  [--subdirectory-filter <directory>]
                  [--env-filter <command>]
                  [--tree-filter <command>]
                  [--index-filter <command>]
                  [--parent-filter <command>]
                  [--msg-filter <command>]
                  [--commit-filter <command>]
                  [--tag-name-filter <command>]
                  [--prune-empty]
                  [--original <namespace>]
                  [-d <directory>]
                  [-f | --force]
                  [--state-branch <branch>]
                  [--] [<rev-list-options>...]
```

Filters are applied in the order they appear in the synopsis — `--env-filter` before `--tree-filter` before `--index-filter`, and so on. Multiple filters can be combined in one invocation.

Everyday usage

Remove a file from every commit on the current branch (fast, index-only):

```
git filter-branch --index-filter \  
  'git rm --cached --ignore-unmatch secrets.env' \  
  --prune-empty -- --all
```

Fix a wrong email address across all commits:

```
git filter-branch --env-filter '  
  if [ "$GIT_AUTHOR_EMAIL" = "old@example.com" ]; then  
    GIT_AUTHOR_EMAIL="correct@example.com"  
    GIT_AUTHOR_NAME="Correct Name"  
  fi  
  if [ "$GIT_COMMITTER_EMAIL" = "old@example.com" ]; then  
    GIT_COMMITTER_EMAIL="correct@example.com"  
    GIT_COMMITTER_NAME="Correct Name"  
  fi  
' -- --all
```

Extract a subdirectory and make it the new repository root:

```
git filter-branch --subdirectory-filter lib/mypackage -- --all
```

Update tags to point to the rewritten commits (append to any invocation that rewrites commits referenced by tags):

```
git filter-branch --tag-name-filter cat -- --all
```

Key options

Option	What it does	When to use it
<code>--env-filter <cmd></code>	Runs <code><cmd></code> before creating each commit; set or modify <code>GIT_AUTHOR_*</code> / <code>GIT_COMMITTER_*</code> variables	Fix wrong author name, email, or timestamps
<code>--tree-filter <cmd></code>	Checks out each commit's tree into a temporary directory, runs <code><cmd></code> with that directory as cwd, then re-indexes everything	Modify or delete files when you need full working-tree access; very slow
<code>--index-filter <cmd></code>	Like <code>--tree-filter</code> but operates on the index directly without checking out files	Remove or rename tracked files; significantly faster than <code>--tree-filter</code>

<code>--msg-filter <cmd></code>	Reads the original commit message on stdin; writes the new message on stdout	Strip SVN IDs, add trailers, reformat messages
<code>--commit-filter <cmd></code>	Replaces the <code>git commit-tree</code> call; receives the tree SHA and parent flags as arguments, must print the new commit SHA	Skip commits entirely; apply per-commit logic; use <code>skip_commit "\$@"</code> to drop a commit
<code>--parent-filter <cmd></code>	Receives the parent string on stdin, writes the new parent string on stdout	Graft histories; add or remove parents
<code>--tag-name-filter <cmd></code>	Called for every tag pointing to a rewritten object; original tag name on stdin, new name on stdout	Keep tags in sync after a rewrite; <code>--tag-name-filter cat</code> passes names through unchanged
<code>--subdirectory-filter <dir></code>	Rewrites history to use <code><dir></code> as the project root, discarding all other paths	Split a subdirectory into its own repository
<code>--prune-empty</code>	Drops commits that become empty (identical tree to parent) after filtering	Avoid cluttering history with no-op commits; cannot be combined with <code>--commit-filter</code>
<code>--setup <cmd></code>	Runs <code><cmd></code> once before the loop; defines shell functions or variables shared by other filters	Factor out common logic used by multiple filters
<code>--original <namespace></code>	Override the backup location for original refs (default: <code>refs/original</code>)	Avoid collisions when running <code>filter-branch</code> more than once
<code>-d <directory></code>	Use a custom temporary directory for tree checkout (e.g. a tmpfs path)	Speed up <code>--tree-filter</code> on large repositories
<code>-f, --force</code>	Allow running even if <code>refs/original/</code> already exists or the temporary directory is present	Re-run after a failed or aborted rewrite
<code>--state-branch <branch></code>	Load and save the old-to-new SHA mapping to a named branch	Incremental rewrites of very large repositories

Best practices

Always work on a fresh clone. Before running any `filter-branch` command, clone the repository to a throwaway directory:

```
git clone --no-local /path/to/repo /tmp/repo-rewrite
cd /tmp/repo-rewrite
```

`--no-local` forces Git to copy objects over the transport layer rather than hard-linking them, so the rewrite directory is fully independent. (A `file://` URL achieves the same effect without the flag.) If anything goes wrong you still have the original.

Rewrite all refs together. Passing `-- --all` rewrites every branch and tag. Rewriting only one branch while leaving others untouched is the leading cause of old-and-new history becoming intertwined. Pass `--tag-name-filter cat` at the same time to keep annotated tags attached to the correct commits.

Use `--index-filter` instead of `--tree-filter` whenever possible. `--tree-filter` checks out the entire working tree for every commit. On a repository with 50 000 files and 10 000 commits, that is five hundred million file operations. `--index-filter` manipulates the index directly and is orders of magnitude faster.

Add `--ignore-unmatch` when removing files. The file you are removing will not be present in every commit. Without `--ignore-unmatch`, `git rm --cached` returns a non-zero exit status for commits where the file does not exist, and `filter-branch` aborts.

```
git rm --cached --ignore-unmatch path/to/secret.key
```

Verify before distributing. After the rewrite, confirm the sensitive content is gone:

```
git log --all --full-history -- path/to/secret.key
git grep 'password123' $(git rev-list --all)
```

Shrink the repository after removing large files. Filtering does not immediately reclaim disk space — the old objects remain until garbage collected. After confirming the rewrite is correct:

```
# Remove the backup refs left by filter-branch
git for-each-ref --format="%(%refname)" refs/original/ \
  | xargs -n 1 git update-ref -d

# Expire reflogs
git reflog expire --expire=now --all

# Collect garbage
git gc --prune=now
```

Or clone the result to a clean repository:

```
git clone file:///tmp/repo-rewrite /path/to/clean-repo
```

Pitfalls & gotchas

Only the filtered branches are rewritten. If you pass `HEAD` instead of `-- --all`, only the current branch gets new SHAs. Other branches still point to the old commits. When team members fetch and merge, old and new history will be merged back together, restoring whatever you tried to remove.

--subdirectory-filter does not automatically rewrite all branches. Pass `-- --all` explicitly to rewrite every branch, not just the current one.

Tags are not automatically updated. Running `filter-branch` without `--tag-name-filter` leaves existing tags pointing to original (unfiltered) commit objects. The sensitive content remains reachable. Always include `--tag-name-filter cat` when the goal is to truly remove something.

Annotated tag signatures are stripped unconditionally. Any GPG-signed tag that points to a rewritten commit will have its signature removed. There is no way to preserve signatures through a rewrite. Notify key maintainers so they can re-sign after the rewrite.

--commit-filter and **--prune-empty** are mutually exclusive. If you need to drop empty commits while using a commit filter, use the built-in helper `git_commit_non_empty_tree "$@"` inside the filter instead:

```
git filter-branch --commit-filter '
    git_commit_non_empty_tree "$@"
' HEAD
```

Shell portability bites silently on macOS. The `sed`, `xargs`, and `date` commands differ between BSD (macOS) and GNU (Linux). A filter that works on Linux may silently produce wrong output on macOS, or vice versa. Test on the same OS where the rewrite will run, and prefer POSIX-portable constructs.

Filenames with spaces or non-ASCII characters break naive filters. Shell pipelines like `git ls-files | grep ... | xargs git rm` will mishandle filenames containing spaces or characters that the shell quotes. Use `-z` / `-0` flags throughout:

```
git ls-files -z | grep -z pattern | xargs -0 git rm --cached
```

refs/original/ is not a true backup. The backup dereferences annotated tags to their underlying commits. If you restore from `refs/original/` and re-run `filter-branch`, you will lose the annotated tag objects. Keep an out-of-band backup (a bundle or a separate clone) before any rewrite.

Commit messages are not updated. If commit messages reference other commit SHAs, those references will point to non-existent (old) objects after the rewrite. Use `--msg-filter` with a `sed` replacement if you need to update cross-references.

Performance is genuinely terrible. A repository with tens of thousands of commits and files can take hours. If the run is interrupted, use `--force` to restart (after verifying the partial state is safe). For large repositories, point `-d` at a RAM-backed filesystem to reduce I/O:

```
# Linux
git filter-branch -d /dev/shm/git-rewrite --index-filter '...' -- --all

# macOS: create a RAM disk first
diskutil erasevolume HFS+ RAMDisk $(hdiutil attach -nomount ram://2097152)
git filter-branch -d /Volumes/RAMDisk/rewrite --index-filter '...' -- --all
```

Worked examples

Remove a committed secret from all history

A credential file `config/database.yml` was committed six months ago and has propagated through dozens of commits. It must be completely expunged.

```
# Work in a fresh clone
git clone file:///path/to/original /tmp/scrub && cd /tmp/scrub

# Remove the file from every commit on every branch and tag
git filter-branch --force --index-filter \
    'git rm --cached --ignore-unmatch config/database.yml' \
    --prune-empty --tag-name-filter cat -- --all
```

Verify the file is gone from all reachable history:

```
git log --all --full-history -- config/database.yml
# should produce no output
```

Clean up backup refs and reclaim disk space:

```
git for-each-ref --format="%(%refname)" refs/original/ \
    | xargs -n 1 git update-ref -d
git reflog expire --expire=now --all
git gc --prune=now --aggressive
```

Force-push all branches and tags to the remote (coordinate with your team first — everyone must re-clone or rebase):

```
git push origin --force --all
git push origin --force --tags
```

Rotate the leaked credential immediately. Removing it from Git history does not invalidate any key or password that was already exposed.

Extract a subdirectory into a standalone repository

A monorepo contains `packages/logging/` which has grown large enough to deserve its own repository.

```
git clone file:///path/to/monorepo /tmp/logging-extract && cd /tmp/logging-extract

# Rewrite all branches so that packages/logging/ becomes the root
git filter-branch --subdirectory-filter packages/logging -- --all
```

The result has all the commit history that touched `packages/logging/`, with paths relative to that directory. Commits that never touched `packages/logging/` are pruned automatically by `--subdirectory-filter`.

Push to a new remote:

```
git remote set-url origin git@github.com:org/logging.git
git push --all
git push --tags
```

Fix a wrong author email across the entire repository

An early contributor committed under `dev@localhost` instead of their real address. All commits, branches, and tags must be corrected.

```
git clone file:///path/to/repo /tmp/email-fix && cd /tmp/email-fix

git filter-branch --env-filter '
    OLD_EMAIL="dev@localhost"
    CORRECT_NAME="Alice Smith"
    CORRECT_EMAIL="alice@example.com"

    if [ "$GIT_AUTHOR_EMAIL" = "$OLD_EMAIL" ]; then
        GIT_AUTHOR_NAME="$CORRECT_NAME"
        GIT_AUTHOR_EMAIL="$CORRECT_EMAIL"
        export GIT_AUTHOR_NAME GIT_AUTHOR_EMAIL
    fi
    if [ "$GIT_COMMITTER_EMAIL" = "$OLD_EMAIL" ]; then
        GIT_COMMITTER_NAME="$CORRECT_NAME"
        GIT_COMMITTER_EMAIL="$CORRECT_EMAIL"
        export GIT_COMMITTER_NAME GIT_COMMITTER_EMAIL
    fi
' --tag-name-filter cat -- --all
```

Confirm the change:

```
git log --all --format="%ae %ce" | sort -u
# dev@localhost should no longer appear
```

Recovery

If the rewrite produced wrong results, the originals are saved under `refs/original/`:

```
# List what was saved
git for-each-ref refs/original/

# Restore a branch (e.g. main)
git checkout -B main refs/original/refs/heads/main

# Once confident the originals are no longer needed, delete the backups
git for-each-ref --format="%(refname)" refs/original/ \
    | xargs -n 1 git update-ref -d
```

If you already deleted `refs/original/` but have not yet garbage-collected, the original commit objects may still be in the object database. Check `git reflog` and the `ORIG_HEAD` ref:

```
git reflog show HEAD
```

For a complete guide to recovering from a botched history rewrite, see *Getting out of jams*.

See also

- *replace* — lightweight history grafting without rewriting all SHAs; a non-destructive alternative for many use cases.
- *rebase* — the right tool for dropping or editing a small number of recent commits interactively; see the *rebase* chapter for `--interactive` and `--autosquash` workflows.
- *reflog* — finding original commit SHAs after a rewrite goes wrong.
- *gc* — reclaiming disk space after backup refs are removed.
- *Getting out of jams* — recovering from a rewrite that corrupted history.

hooks

Scripts that Git executes automatically at defined points in its workflow, letting you enforce policy, automate checks, and integrate with external systems without changing how contributors invoke Git commands.

Mental model

A hook is an executable file sitting in `.git/hooks/`. Git calls it by name at a specific moment — before a commit is written, after a push completes, when a merge message is prepared — and inspects its exit code. Exit zero means “proceed”; exit non-zero means “abort” (for hooks that can abort). The hook receives context through command-line arguments or standard input depending on which hook it is.

```
git commit
|
├─> pre-commit          exit ≠ 0 → abort
├─> prepare-commit-msg  (fills the editor buffer)
├─> commit-msg          exit ≠ 0 → abort
└─> post-commit         (notification only, cannot abort)
```

Three things determine whether a hook fires:

1. A file with the exact hook name exists in the hooks directory.
2. The file has the executable bit set (`chmod +x`).
3. The file is not bypassed by a flag like `--no-verify` .

The hooks directory defaults to `$GIT_DIR/hooks` (`.git/hooks` in a normal clone), but you can redirect it for the whole repository with:

```
git config core.hooksPath /path/to/shared/hooks
```

This is how teams share hooks without committing them into `.git/`, which is not version-controlled.

Before invoking any hook, Git changes its working directory to the repository root (non-bare) or to `$GIT_DIR` (bare). An exception: hooks triggered during a push (`pre-receive` , `update` , `post-receive` , `post-update` , `push-to-checkout`) always run in `$GIT_DIR`, even in non-bare repositories. Git also exports environment variables such as `GIT_DIR` and `GIT_WORK_TREE` so that commands inside the hook find the right repository.

Synopsis

Hooks are not a subcommand — there is no `git hooks run` in day-to-day use. The interface is the filesystem:

```
# Location (default)
$GIT_DIR/hooks/<hook-name>

# Override location for all hooks in this repo
git config core.hooksPath <dir>

# Bypass hooks that support --no-verify
git commit --no-verify
git push    --no-verify
```

```
git am      --no-verify

# Bypass hooks that support --no-verify for merge
git merge  --no-verify
```

A minimal hook skeleton:

```
#!/bin/sh
# Hook name: pre-commit
# Exit 0 to allow, non-zero to abort.
exit 0
```

Everyday usage

Install a pre-commit hook

```
cat > .git/hooks/pre-commit << 'EOF'
#!/bin/sh
# Run the project's lint script before every commit.
npm run lint --silent
EOF
chmod +x .git/hooks/pre-commit
```

From now on, `git commit` runs the linter. A non-zero exit from `npm run lint` aborts the commit and prints the linter output.

Enforce a commit-message format

```
cat > .git/hooks/commit-msg << 'EOF'
#!/bin/sh
# Require the subject line to reference a ticket: PROJ-1234 or HOTFIX.
msg=$(head -1 "$1")
if ! echo "$msg" | grep -qE '^(PROJ-[0-9]+|HOTFIX)'; then
    echo "commit-msg: subject must start with PROJ-NNNN or HOTFIX" >&2
    exit 1
fi
EOF
chmod +x .git/hooks/commit-msg
```

The hook receives the path to the commit-message file as `$1` read or edit that file in place.

Auto-populate the commit message with a ticket number

```
cat > .git/hooks/prepare-commit-msg << 'EOF'
#!/bin/sh
# Prepend the branch ticket number to the commit message.
branch=$(git symbolic-ref --short HEAD 2>/dev/null)
ticket=$(echo "$branch" | grep -oE 'PROJ-[0-9]+')
if [ -n "$ticket" ] && [ "$2" != "merge" ] && [ "$2" != "squash" ]; then
    sed -i.bak "s/^/$ticket: /" "$1"
fi
EOF
chmod +x .git/hooks/prepare-commit-msg
```

`prepare-commit-msg` receives one to three arguments: the message file path; an optional source keyword (`message` , `template` , `merge` , `squash` , or `commit`); and, when the source is `commit` , the commit object name as a third argument. Guard against the merge and squash sources, which already have content.

Run tests before a push

```
cat > .git/hooks/pre-push << 'EOF'
#!/bin/sh
# Run the full test suite; abort push on failure.
npm test
EOF
chmod +x .git/hooks/pre-push
```

`pre-push` receives the remote name and URL as arguments and reads `<local-ref> <local-object-name> <remote-ref> <remote-object-name>` lines from stdin, one per ref being pushed. A non-zero exit prevents the push.

Share hooks across the team via a tracked directory

```
mkdir -p scripts/hooks
cp .git/hooks/pre-commit scripts/hooks/pre-commit
git add scripts/hooks/pre-commit
git commit -m "Add pre-commit lint hook"
# Configure all contributors' clones to use it:
git config core.hooksPath scripts/hooks
```

Each contributor who clones and runs this config command gets the same hooks. Automate the config step in a setup script or `Makefile` .

Key options

Hooks themselves have no flags; the relevant options belong to the commands that invoke them.

Context	Option / variable	What it does
<code>git commit</code>	<code>--no-verify</code>	Skips <code>pre-commit</code> and <code>commit-msg</code> hooks
<code>git merge</code>	<code>--no-verify</code>	Skips <code>pre-merge-commit</code> hook
<code>git push</code>	<code>--no-verify</code>	Skips <code>pre-push</code> hook
<code>git am</code>	<code>--no-verify</code>	Skips <code>applypatch-msg</code> and <code>pre-applypatch</code> hooks
<code>git config</code>	<code>core.hooksPath</code>	Redirects all hook lookups to the given directory
Hook file	executable bit	Must be set (<code>chmod +x</code>); hooks without it are silently ignored
Hook file	shebang line	Determines the interpreter; use <code>#!/bin/sh</code> for portability
<code>pre-receive</code>	<code>GIT_PUSH_OPTION_COUNT</code> / <code>GIT_PUSH_OPTION_N</code>	Environment variables carrying push options passed with <code>git push --push-option=...</code>

Best practices

Keep hooks fast. Every `git commit` goes through `pre-commit`. A hook that takes five seconds trains contributors to reach for `--no-verify`. Run only the subset of checks that is fast locally; save expensive steps (full test suite, integration tests) for `pre-push` or CI.

Use `core.hooksPath` to version-control your hooks. The `.git/` directory is not committed, so a hook installed directly in `.git/hooks/` vanishes after a fresh clone. Put hooks in a tracked directory (e.g. `scripts/hooks/`) and document the one-time setup command. Consider automating it in a `Makefile` target or a `postinstall` script.

Write hooks in portable shell. Use `#!/bin/sh`, not `#!/bin/bash`, unless you need Bash-specific features. Many CI environments and developer machines use different shells. POSIX `sh` is available everywhere Git runs.

Exit with a meaningful message to `stderr`. When a hook rejects an operation, the developer needs to know why and how to fix it:

```
echo "pre-commit: trailing whitespace found in src/api.js:42" >&2
exit 1
```

Write to `stderr` (`>&2`), not `stdout`. Git forwards `stderr` to the terminal.

Distinguish notification hooks from gating hooks. Hooks like `post-commit` and `post-receive` run after the fact and their exit code is ignored (for outcome purposes). Use them for notifications, cache invalidation, or deployments. Do not write critical logic in them expecting they can block anything.

On the server side, prefer `pre-receive` over `update`. The `update` hook fires once per ref, which can produce a flood of emails or Slack messages. `pre-receive` fires once per push operation and sees all refs being updated in a single `stdin` read.

Do not call `git commit` or `git push` from inside a hook that git itself invoked. Re-entrant Git operations inside a hook inherit the lock on the repository and will deadlock or corrupt state. Use object database plumbing, or stage changes to a file and let the caller commit them.

Pitfalls & gotchas

Missing executable bit silently skips the hook. Git does not warn you when a hook file exists but is not executable. If your hook appears to do nothing, the first thing to check is

```
ls -l .git/hooks/pre-commit — the permissions must include x.
```

```
chmod +x .git/hooks/pre-commit # fix it
```

`prepare-commit-msg` is not bypassed by `--no-verify`. Developers who reach for `--no-verify` to skip `pre-commit` are sometimes surprised that the message template hook still runs. The Git manual is explicit: `--no-verify` skips `pre-commit` and `commit-msg`, but not `prepare-commit-msg`.

Windows line endings break shell hooks. A hook file with `\r\n` line endings will fail on Linux/macOS with a cryptic `bad interpreter` error because the shebang line becomes `#!/bin/sh\r`. Ensure your editor and `.gitattributes` do not convert the hook files to CRLF:

```
# .gitattributes
scripts/hooks/* text eol=lf
```

Hooks in `core.hooksPath` must still have the executable bit. The directory override does not change this requirement. After copying hooks to a tracked directory, remember `chmod +x scripts/hooks/*`.

`post-rewrite` is not called by `git filter-repo` or `git fast-import`. The documentation is explicit: full history-rewriting tools typically skip this hook. If you rely on `post-rewrite` to update notes or external references after a rebase, test whether your specific rewriting tool calls it.

Environment variables inherited from Git can interfere with subshells. If a hook needs to run Git commands against a different repository, unset the Git environment variables first, otherwise they point the subprocess at the wrong repo:

```
foreign_desc=$(unset $(git rev-parse --local-env-vars); git -C ../other-repo
describe)
```

Client-side hooks are not enforced. Anyone can delete `.git/hooks/` or pass `--no-verify`. Use server-side hooks (`pre-receive` , `update`) for hard policy enforcement. Client-side hooks are developer-experience tools, not security controls.

Worked examples

Example 1: Linting and formatting on every commit

The goal: run ESLint and Prettier on staged JavaScript files only (not the whole repo), and abort if either tool reports an error.

```
cat > .git/hooks/pre-commit << 'EOF'
#!/bin/sh
# Collect staged .js and .ts files.
staged=$(git diff --cached --name-only --diff-filter=ACM | grep -E '\.(js|ts)$')
[ -z "$staged" ] && exit 0 # nothing to check

# Run Prettier check.
echo "$staged" | xargs npx prettier --check
prettier_exit=$?

# Run ESLint.
echo "$staged" | xargs npx eslint
eslint_exit=$?

# Fail if either tool found issues.
[ $prettier_exit -ne 0 ] || [ $eslint_exit -ne 0 ] && exit 1
exit 0
EOF
chmod +x .git/hooks/pre-commit
```

Running `git commit` now produces output like:

```
Checking formatting...
[warn] src/api.js
[warn] Code style issues found in the above file. Run Prettier with
      --write to fix.
```

Fix the formatting, re-stage, and commit again.

Example 2: Server-side branch protection with pre-receive

The goal: reject any push that attempts to force-update `main` or `release/*` on the central bare repository.

```
cat > /srv/repos/myproject.git/hooks/pre-receive << 'EOF'
#!/bin/sh
# Read each ref being updated from stdin.
while IFS=' ' read -r old new ref; do
    case "$ref" in
        refs/heads/main|refs/heads/release/*)
            # Detect a force push: new commit is not a descendant of old.
            if [ "$old" != "0000000000000000000000000000000000000000" ]; then
                if ! git merge-base --is-ancestor "$old" "$new" 2>/dev/null; then
                    echo "pre-receive: force push to $ref is not allowed." >&2
                    exit 1
                fi
            fi
        ;;
    esac
done
exit 0
EOF
chmod +x /srv/repos/myproject.git/hooks/pre-receive
```

The hook reads `<old-oid> <new-oid> <ref-name>` triplets from stdin. When `old` is the all-zeroes object name, this is a new branch being created, not a force push — the check is skipped. For existing branches, `git merge-base --is-ancestor` returns zero only if the old commit is an ancestor of the new one (a fast-forward); otherwise it exits non-zero, triggering the rejection.

Example 3: Triggering a deployment from post-receive

The goal: after a successful push to the `production` branch on the server, update the working tree of a checked-out deployment directory.

```
cat > /srv/repos/myproject.git/hooks/post-receive << 'EOF'
#!/bin/sh
# Deploy when the production branch is updated.
while IFS=' ' read -r old new ref; do
    if [ "$ref" = "refs/heads/production" ]; then
        echo "post-receive: deploying production..."
        GIT_WORK_TREE=/var/www/myapp git checkout -f production
        echo "post-receive: done."
    fi
done
EOF
chmod +x /srv/repos/myproject.git/hooks/post-receive
```

`post-receive` cannot abort the push — the data is already stored. It is safe to use for side effects such as deployments, cache purges, or webhook calls. Both stdout and stderr are forwarded to the pushing client, so `echo` messages appear in the developer's terminal.

Recovery

Hooks occasionally go wrong — a broken script blocks all commits, or a half-finished hook was left behind. Quick escapes:

Skip a hook for one invocation using `--no-verify` on supported commands:

```
git commit --no-verify -m "Emergency fix – re-enable hook after"
git push --no-verify
```

Use this sparingly. Fix the hook as soon as possible; leaving `--no-verify` in a team's muscle memory defeats the purpose of the hook.

Disable a hook permanently by removing the executable bit or renaming it:

```
chmod -x .git/hooks/pre-commit # disable without deleting
mv .git/hooks/pre-commit .git/hooks/pre-commit.disabled
```

Restore a deleted or corrupted hook from your tracked hooks directory:

```
cp scripts/hooks/pre-commit .git/hooks/pre-commit
chmod +x .git/hooks/pre-commit
```

Debug a misbehaving hook by running it directly:

```
sh -x .git/hooks/pre-commit # print each command as it executes
```

See *Getting out of jams* for broader undo recipes when a hook-related abort has left the repository in a partial state (e.g. a failed `am` or interrupted rebase).

See also

- `commit` — the `--no-verify` flag that bypasses `pre-commit` and `commit-msg`.
- `push` — the `--no-verify` flag that bypasses `pre-push`.
- `am` — `applypatch-msg` and `pre-applypatch` hooks called during patch application.
- `rebase` — triggers `post-rewrite` after squash and fixup operations.
- `gc` — `pre-auto-gc` hook called before automatic garbage collection.
- *Getting out of jams* — recovering from interrupted operations caused by hook failures.

Getting out of jams: git

Every git user hits a wall at some point — a detached HEAD, a rejected push, a secret that slipped into a commit. This chapter is an indexed field guide to those moments. Each entry names the symptom, explains what is really happening, and gives numbered steps you can copy-paste straight into a terminal.

Detached HEAD

What it means. HEAD normally points to a branch name, which in turn points to a commit. “Detached HEAD” means HEAD points directly to a commit SHA rather than to any branch. Any commits you make here belong to no branch and will be garbage-collected once you move away.

Why it happens. You checked out a specific commit hash, a tag, or a remote-tracking reference directly:

```
git checkout v1.4.0      # tag
git checkout abc1234     # raw SHA
git checkout origin/main # remote ref, not a local branch
```

How to get out.

1. If you haven’t made any commits, just return to a branch:

```
git switch main
# or: git checkout main
```

2. If you made commits you want to keep, give them a branch first:

```
git switch -c my-experiment
# Now HEAD is attached to the new branch and your commits are safe.
```

3. If you made commits you don’t want, simply switch away (they will eventually be collected by `git gc`):

```
git switch main
```

How to avoid it. When you want to explore an old commit, create a branch at the same time:

`git switch -c explore-v1.4 v1.4.0`. When checking out a remote branch, use `git switch feature` (without the `origin/` prefix) — git will automatically create a local tracking branch named `feature` that follows `origin/feature`, as long as no local branch of that name already exists.

Merge conflicts and rebase conflicts

What it means. Two commits changed the same lines and git cannot decide which version to keep. It pauses and asks you to resolve the conflict manually before continuing.

Why it happens. During `git merge`, two branches both modified the same region of a file. During `git rebase`, each replayed commit is applied in sequence; if a commit conflicts with the current state of the branch, the rebase pauses at that commit.

How to get out — merge conflict.

1. Identify the conflicting files:

```
git status
# "both modified: src/config.js" means there's a conflict
```

2. Open each conflicting file. Find the conflict markers and decide what the final content should be:

```
<<<<<<< HEAD
const timeout = 5000;
=====
const timeout = 3000;
>>>>>>> feature/fast-requests
```

Edit the file to the correct result, removing all `<<<<<<<`, `=====`, and `>>>>>>>` lines.

1. Stage the resolved file and finish the merge:

```
git add src/config.js
git merge --continue
# git will open the editor for the merge commit message
```

Or, if you want to abort entirely and restore the pre-merge state:

```
git merge --abort
```

How to get out — rebase conflict.

1. Resolve the conflicting file the same way as above.
2. Stage the resolved file, then continue the rebase:

```
git add src/config.js
git rebase --continue
```

3. If the conflicting commit is empty after resolution (it was entirely superseded by a prior commit), skip it:

```
git rebase --skip
```

4. To abandon the rebase and restore the original branch:

```
git rebase --abort
```

How to avoid it. Rebase onto the target branch frequently rather than letting branches diverge for weeks. See *rerere* — git can remember how you resolved a particular conflict and replay that resolution automatically next time.

“Updates were rejected (non-fast-forward)” on push

What it means. The remote branch has commits that your local branch does not have. Pushing would discard those commits on the remote, so git refuses.

Why it happens. Someone else pushed to the same branch while you were working, or you amended / rebased commits that were already on the remote.

How to get out.

Case 1: Someone else pushed new commits — you just need to integrate them.

1. Pull the remote changes and rebase your work on top:

```
git pull --rebase origin main
```

2. Resolve any conflicts that arise (see the previous entry), then push:

```
git push origin main
```

Case 2: You rewrote history (amend, rebase) on a branch only you own.

1. Use `--force-with-lease` instead of `--force`. It pushes only if the remote tip matches what you last fetched, protecting against overwriting someone else's concurrent push:

```
git push --force-with-lease origin my-feature
```

How to avoid it. Never rebase or amend commits that have been pushed to a shared branch (e.g. `main`, `develop`). On personal feature branches it is acceptable, but communicate with your team. Enable branch protection on `main` to prevent accidental force-pushes.

Committed to the wrong branch

What it means. You made one or more commits on `main` (or another branch) when they should have gone on a feature branch, or vice versa.

Why it happens. You forgot to create or switch to a feature branch before starting work.

How to get out.

Move the last N commits to a new branch (commits not yet pushed).

1. Create the correct branch at the current HEAD:

```
git switch -c feature/my-work
```

2. Go back to the original branch and remove the commits with a hard reset:

```
git switch main
git reset --hard origin/main
# main is now back to the remote tip; feature/my-work has the commits
```

If something goes wrong, the discarded commits are still reachable via `git reflog` for 90 days — see *Recover deleted commits and branches via the reflog* later in this chapter.

Cherry-pick one commit onto an existing branch.

1. Note the SHA of the commit to move:

```
git log --oneline -5
```

2. Switch to the target branch and cherry-pick:

```
git switch correct-branch
git cherry-pick <sha>
```

3. Remove the commit from where it shouldn't be:

```
git switch wrong-branch
git reset --hard HEAD~1 # if it's the latest commit and not yet pushed
```

As above, if you reset too far, `git reflog` will show you the previous HEAD so you can recover.

How to avoid it. Run `git branch` or check your shell prompt before starting work. Many teams configure their prompt to show the current branch.

Undo the last commit — soft vs mixed vs hard reset vs revert

What it means. You want to un-do a commit, but the right tool depends on whether you want to keep the changes and where you want them to land.

Why it happens. Wrong message, wrong files staged, premature commit.

How to get out. Pick the mode that matches your goal:

Goal	Command
Keep changes staged, re-commit with a different message	<code>git reset --soft HEAD~1</code>
Keep changes in working tree (unstaged), re-stage selectively	<code>git reset HEAD~1</code> (mixed, the default)
Discard the commit and all its changes entirely	<code>git reset --hard HEAD~1</code>
Undo a commit that is already pushed (safe for shared branches)	<code>git revert HEAD</code>

Steps for each:

1. **Soft reset** — changes remain in the index, ready to recommit:

```
git reset --soft HEAD~1
git commit -m "Better commit message"
```

2. **Mixed reset** (default) — changes return to the working tree, unstaged:

```
git reset HEAD~1
# edit / re-stage as needed
git add -p
git commit -m "Corrected commit"
```

3. **Hard reset** — changes are gone (recoverable via reflog for 90 days):

```
git reset --hard HEAD~1
```

4. **Revert** — creates a new commit that undoes the previous one; history is preserved and the branch can still be pushed normally:

```
git revert HEAD
# an editor opens for the revert commit message; save and close
```

How to avoid it. Use `git commit --amend` to fix the most recent commit message or add a forgotten file, as long as you haven't pushed yet. For anything beyond the last commit, lean on `git rebase -i`.

Recover deleted commits and branches via the reflog

What it means. A branch was deleted, a `git reset --hard` discarded commits, or a rebase went wrong. The commits appear to be gone.

Why it happens. Commits that are no longer reachable from any branch or tag are “dangling” — git's garbage collector will remove them, but they remain in the reflog for 90 days by default.

How to get out.

1. Inspect the reflog to find the SHA of the lost commit:

```
git reflog
```

Output looks like:

```
8f44520 HEAD@{0}: reset: moving to HEAD~1
6530db9 HEAD@{1}: commit: Add login validation
8f44520 HEAD@{2}: commit: Fix null pointer
```

The commit you want is `6530db9`.

1. Restore it to a new branch:

```
git switch -c recovered-work 6530db9
```

2. Or, if you want to reset the current branch back to that point:

```
git reset --hard 6530db9
```

For a deleted branch, look at the branch-specific reflog:

```
git reflog show deleted-branch-name
```

If the branch is already gone from the reflog listing, check `HEAD` — the commits will appear there as long as you haven't run `git gc` since the deletion.

See the *reflog* chapter for the full `@{n}` and time-based syntax.

How to avoid it. Before a destructive operation (hard reset, branch delete, force-push), note the current SHA with `git log --oneline -1`. If something goes wrong you have the hash ready.

Safely undo a commit that was already pushed

What it means. A commit is on a shared branch (e.g. `main`) and needs to be undone without rewriting history for collaborators.

Why it happens. A bug slipped through, a migration script ran, or a file was accidentally committed.

How to get out.

1. Use `git revert` to create a new commit that undoes the bad one. This is always safe on shared branches because it only adds a commit:

```
git revert <bad-sha>
git push origin main
```

To revert a merge commit, specify which parent to return to (usually `1`, the main-line parent):

```
git revert -m 1 <merge-sha>
```

1. If you must rewrite history on a branch you own (e.g. a feature branch in review), use `--force-with-lease` rather than `--force`:

```
git reset --hard HEAD~1
git push --force-with-lease origin my-feature
```

`--force-with-lease` will refuse if someone else pushed to the branch since your last fetch, preventing accidental data loss. The discarded commit remains reachable via `git reflog` for 90 days if you need to recover it.

How to avoid it. Require pull-request reviews before merging to `main`. Enable branch protection rules so direct pushes to `main` are blocked.

A secret got committed (and pushed)

What it means. A password, API key, private key, or other credential ended up in a commit. Because the secret is in history — not just the working tree — deleting the file is not enough.

Why it happens. A `.env` file was not listed in `.gitignore`, credentials were hardcoded for a quick test, or an IDE auto-staged a config file.

How to get out.

Step 0 — rotate the secret immediately. Assume it is compromised. Revoke and regenerate the token, key, or password before anything else.

1. Remove the secret from history using `git filter-repo` (preferred over the older `git filter-branch` install it via your package manager):

```
git filter-repo --path secrets.env --invert-paths
# removes secrets.env from every commit in history
```

Or replace the secret value itself everywhere it appears:

```
git filter-repo --replace-text <(echo 'MY_SECRET_VALUE==>REDACTED')
```

1. Force-push the rewritten history (coordinate with all collaborators first — they will need to re-clone or rebase):

```
git push --force-with-lease --all
git push --force-with-lease --tags
```

2. Ask GitHub/GitLab/Bitbucket to purge cached views of the old commits (each platform has a “cached data” removal tool or support process).
3. Add the file to `.gitignore` so it cannot be accidentally staged again:

```
echo "secrets.env" >> .gitignore
git add .gitignore
git commit -m "Ignore secrets.env"
```

How to avoid it. Add secrets files to `.gitignore` before creating them. Use a secrets manager or environment variables injected at runtime. Install a pre-commit hook (e.g. `detect-secrets` or `gitleaks`) to scan for credential patterns before every commit.

“fatal: refusing to merge unrelated histories”

What it means. You tried to merge or pull two branches that have no common ancestor — their histories are completely separate trees.

Why it happens. You initialized a repository locally (`git init`) and then tried to pull from a remote that was initialized separately, or you are merging two repositories that were never related.

How to get out.

1. If the merge is intentional (e.g. importing a separate project), allow it explicitly:

```
git merge origin/main --allow-unrelated-histories
# resolve any conflicts, then commit
```

2. If you hit this during a `git pull`, pass the flag the same way:

```
git pull origin main --allow-unrelated-histories
```

How to avoid it. When creating a new repository, do not check the “add a README” box on the hosting service if you already have local commits you plan to push. Either initialize the remote empty or clone before adding files.

Line-ending / `.gitattributes` surprises (CRLF/LF)

What it means. Files are shown as modified, diffs are full of `^M` characters, or the same file is checked out differently on Windows versus macOS/Linux even though nothing meaningful changed.

Why it happens. Windows uses CRLF (`\r\n`) and Unix uses LF (`\n`). If `core.autocrlf` is set inconsistently across contributors, git silently converts line endings on checkout and commit, creating spurious diffs.

How to get out.

1. Add a `.gitattributes` file to the repository root to declare the policy once for everyone, regardless of their `core.autocrlf` setting:

```
# .gitattributes
# normalize all text files to LF in the repository
* text=auto

# force LF for shell scripts
*.sh text eol=lf

# force CRLF for Windows batch files
*.bat text eol=crlf

# treat binary files as binary – never touch line endings
*.png binary
*.jpg binary
*.pdf binary
```

2. After adding `.gitattributes`, normalize existing files in the working tree to match the new policy:

```
git add --renormalize .
git commit -m "Normalize line endings via .gitattributes"
```

3. Tell collaborators to re-checkout their working trees after pulling the commit, or the stale line endings in their local files will cause spurious diffs. **This sequence runs `git reset --hard`, which permanently discards uncommitted changes — stash or commit first, every time:**

```
git stash          # ALWAYS run first if you have any uncommitted work
git rm --cached -r .
git reset --hard
git stash pop      # restore stashed work, if you stashed any
```

How to avoid it. Commit a `.gitattributes` file as the first commit in every new repository. The `text=auto` rule is the safe default: git detects text files and normalizes them to LF in the repository, converting to the platform line ending on checkout.

Accidentally committed a huge file

What it means. A large binary (compiled artifact, dataset, video, log file) ended up in the repository. Even after deletion, it bloats every clone because the blob is still in history.

Why it happens. The file was not listed in `.gitignore`, or someone ran `git add .` without thinking.

How to get out.

If the commit is not yet pushed — rewrite locally.

1. Remove the file from the last commit without touching it on disk:

```
git rm --cached path/to/bigfile.bin
git commit --amend --no-edit
```


If the commit is already pushed — rewrite history.

1. Use `git filter-repo` to strip the file from every commit:

```
git filter-repo --path path/to/bigfile.bin --invert-paths
```

2. Force-push all refs:

```
git push --force-with-lease --all
git push --force-with-lease --tags
```

3. Collaborators must re-clone or run `git fetch --prune` followed by a rebase, because their remotes still reference the old objects.
4. Add the file to `.gitignore` immediately:

```
echo "path/to/bigfile.bin" >> .gitignore
git add .gitignore
git commit -m "Ignore bigfile.bin"
```

For ongoing large-file storage needs, consider Git LFS (`git lfs track "*.bin"`), which stores large blobs outside the main repository object store.

How to avoid it. Maintain a thorough `.gitignore` and never use `git add .` on directories that may contain build artifacts or data files. A pre-commit hook that rejects files over a size threshold (e.g. 5 MB) catches accidents before they reach the repository.

“Unable to create ‘.git/index.lock’” / stale lock

What it means. Git refuses to run because another git process appears to be using the index.

Why it happens. A previous git command crashed or was killed without cleaning up its lock file. The file `.git/index.lock` is normally created at the start of an operation and deleted at the end; if the process never finished, the lock file is left behind.

How to get out.

1. Confirm that no other git process is actually running:

```
ps aux | grep git
```

2. If nothing is running, delete the stale lock file:

```
rm -f .git/index.lock
```

3. Retry your original command.

How to avoid it. Do not kill git processes with `kill -9` (SIGKILL) — use `Ctrl-C` (SIGINT) which allows the process to clean up. If your IDE and terminal both run git on the same repository simultaneously, they may occasionally collide; let one finish before running the other.

Submodule out of sync / empty submodule directory

What it means. A subdirectory that should contain a submodule is empty, or the submodule is pinned to an older commit than what `.gitmodules` or the parent commit records.

Why it happens. The repository was cloned without `--recurse-submodules`, or the parent repository was updated to point at a newer submodule commit and you haven't run the update command yet.

How to get out.

1. Initialize and fetch any submodules that have never been populated:

```
git submodule update --init --recursive
```

2. After pulling new commits in the parent repository, bring submodules to the commit recorded by the parent:

```
git submodule update --recursive
```

3. To see which submodules are out of sync:

```
git submodule status
# lines starting with '-' are not initialized
# lines starting with '+' are checked out at a different commit than recorded
```

4. If a submodule URL changed (e.g. the upstream moved), sync the config and then update:

```
git submodule sync --recursive
git submodule update --init --recursive
```

How to avoid it. Clone with `git clone --recurse-submodules <url>`. Set the global option so future clones always recurse:

```
git config --global submodule.recurse true
```

See the *submodule* chapter for the full lifecycle.

HTTPS vs SSH authentication failures

What it means. Git fails with “authentication failed”, “Permission denied (publickey)”, or “could not read Username” when pushing or fetching.

Why it happens.

- **HTTPS:** Your stored credentials are out of date (password changed, token expired, or two-factor authentication enabled since you last pushed).
- **SSH:** The public key is not uploaded to the hosting service, or the wrong key is being offered (common when you have multiple SSH identities).

How to get out — HTTPS.

1. Generate a personal access token (PAT) with repo scope on GitHub/GitLab (account settings → developer settings → tokens).
2. Erase the cached credential and try again — git will prompt for the new token:

```
git credential reject <<EOF
protocol=https
host=github.com
EOF
git fetch
```

Or, on macOS, clear the keychain entry via Keychain Access and let git re-prompt.

How to get out — SSH.

1. Check which key is being offered:

```
ssh -vT git@github.com 2>&1 | grep "Offering"
```

2. If the wrong key is offered, specify the correct one in `~/.ssh/config`:

```
Host github.com
  IdentityFile ~/.ssh/id_ed25519_github
  IdentitiesOnly yes
```

3. Confirm the public key is uploaded to the hosting service (GitHub: Settings → SSH and GPG keys).
4. Test the connection:

```
ssh -T git@github.com
# Expected: "Hi <username>! You've successfully authenticated..."
```

How to avoid it. Prefer SSH with a key protected by `ssh-agent` for day-to-day work. For CI/CD environments where SSH keys are impractical, use short-lived HTTPS tokens scoped to the minimum required permissions.

“detected dubious ownership in repository”

What it means. Git refuses to operate on a repository because the directory is owned by a different user than the one running the git command.

Why it happens. Introduced in git 2.35.2 as a CVE fix. Common triggers:

- Running git as root in a directory owned by a regular user (e.g. `sudo git pull`).
- A Docker container running as a different UID than the files were created with.
- A shared filesystem where the repository was created by a different account than the one you’re using now.

How to get out.

1. The correct fix is to run git as the same user who owns the directory. Check who owns it:

```
ls -la /path/to/repo | head -3
```

2. If this is a trusted repository and you intentionally need to run git as a different user (e.g. in a container), mark the directory safe for your current user:

```
git config --global --add safe.directory /path/to/repo
```

Use `*` only in controlled environments (e.g. a dedicated CI container where all content is trusted):

```
git config --global --add safe.directory '*'
```

1. In Docker, the cleanest fix is to align UIDs:

```
RUN useradd -u 1001 appuser  
USER appuser
```

How to avoid it. Do not mix `sudo git` and regular user git on the same repository. In containers, make the UID of the running process match the UID that owns the repository files.